

PrimeTime Constraint Consistency Rules

Version R-2020.09, September 2020



Copyright and Proprietary Information Notice

© 2020 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

1. General Design Rules	10
Rule: CAP_0001	17
Rule: CAP_0002	19
Rule: CAP_0003	20
Rule: CAP_0004	21
Rule: CAP_0005	23
Rule: CAS_0001	25
Rule: CAS_0002	28
Rule: CAS_0003	31
Rule: CAS_0004	33
Rule: CGR_0001	35
Rule: CGR_0002	37
Rule: CGR_0003	38
Rule: CGR_0004	40
Rule: CGR_0005	41
Rule: CGR_0006	42
Rule: CGR_0007	44
Rule: CGR_0008	45
Rule: CGR_0009	46
Rule: CGR_0010	50
Rule: CLK_0001	53
Rule: CLK_0002	55
Rule: CLK_0003	56
Rule: CLK_0004	60
Rule: CLK_0005	63
Rule: CLK_0006	65
Rule: CLK_0007	66

Rule: CLK_0008	69
Rule: CLK_0009	72
Rule: CLK_0010	76
Rule: CLK_0011	78
Rule: CLK_0012	80
Rule: CLK_0013	85
Rule: CLK_0014	87
Rule: CLK_0015	90
Rule: CLK_0016	91
Rule: CLK_0017	94
Rule: CLK_0018	97
Rule: CLK_0019	101
Rule: CLK_0020	102
Rule: CLK_0021	108
Rule: CLK_0023	109
Rule: CLK_0024	113
Rule: CLK_0026	115
Rule: CLK_0027	118
Rule: CLK_0028	120
Rule: CLK_0029	123
Rule: CLK_0030	124
Rule: CLK_0031	129
Rule: CLK_0032	129
Rule: CLK_0033	130
Rule: CLK_0034	132
Rule: CLK_0035	134
Rule: CLK_0036	135
Rule: CLK_0037	138
Rule: CLK_0038	140
Rule: CLK_0039	141
Rule: CLK_0040	141
Rule: CLK_0041	142

Rule: CLK_0042	143
Rule: CLK_0043	144
Rule: CMP_0001	146
Rule: CMP_0002	149
Rule: CMP_0003	151
Rule: CNL_0001	152
Rule: CNL_0002	153
Rule: CNL_0003	155
Rule: CNL_0004	158
Rule: CNL_0005	161
Rule: CSL_0001	162
Rule: CSL_0002	164
Rule: CSL_0003	166
Rule: CSL_0004	168
Rule: CSL_0005	169
Rule: CSL_0006	172
Rule: CSL_0007	174
Rule: CTR_0002	175
Rule: CTR_0003	177
Rule: CTR_0004	178
Rule: CTR_0005	180
Rule: CTR_0006	181
Rule: CTR_0001	183
Rule: DES_0001	184
Rule: DES_0002	189
Rule: DES_0003	193
Rule: DES_0004	196
Rule: DRV_0001	198
Rule: DRV_0002	200
Rule: DRV_0003	203
Rule: DRV_0004	204
Rule: DRV_0005	206

Rule: EXC_0001	207
Rule: EXC_0002	208
Rule: EXC_0003	212
Rule: EXC_0004	214
Rule: EXC_0005	218
Rule: EXC_0006	220
Rule: EXC_0007	223
Rule: EXC_0008	225
Rule: EXC_0009	227
Rule: EXC_0010	228
Rule: EXC_0011	230
Rule: EXC_0012	232
Rule: EXC_0013	233
Rule: EXC_0014	234
Rule: EXC_0015	237
Rule: EXD_0001	240
Rule: EXD_0002	242
Rule: EXD_0003	245
Rule: EXD_0004	248
Rule: EXD_0007	250
Rule: EXD_0008	252
Rule: EXD_0009	254
Rule: EXD_0010	256
Rule: EXD_0011	258
Rule: EXD_0012	259
Rule: EXD_0013	261
Rule: EXD_0014	263
Rule: EXD_0015	266
Rule: HIER_001	268
Rule: LOOP_001	269
Rule: LOOP_002	271
Rule: NTL_0001	274

Rule: NTL_0002	276
Rule: NTL_0003	277
Rule: NTL_0004	279
Rule: NTL_0005	280
Rule: NTL_0006	281
Rule: OPN_0001	283
Rule: PRF_0001	285
Rule: TRN_0001	287
Rule: TRN_0002	290
Rule: UNC_0001	292
Rule: UNC_0002	294
Rule: UNC_0003	296
Rule: UNC_0004	298
Rule: UNC_0005	299
Rule: UNC_0006	300
Rule: UNT_0001	300
Rule: UNT_0002	301
Rule: UNT_0003	303

2. Block-To-Top Rules	304
Rule: B2T_CAS_0001	306
Rule: B2T_CAS_0002	309
Rule: B2T_CAS_0003	311
Rule: B2T_CLK_0001	314
Rule: B2T_CLK_0002	318
Rule: B2T_CLK_0003	322
Rule: B2T_CLK_0004	325
Rule: B2T_CLK_0005	328
Rule: B2T_CLK_0006	331
Rule: B2T_CLK_0007	335
Rule: B2T_CLK_0008	338
Rule: B2T_CLK_0010	342

Rule: B2T_CLK_0011	345
Rule: B2T_CLK_0012	348
Rule: B2T_CLK_0013	350
Rule: B2T_CLT_0001	353
Rule: B2T_CLT_0002	355
Rule: B2T_CLT_0003	357
Rule: B2T_DIS_0001	360
Rule: B2T_DIS_0002	363
Rule: B2T_DIS_0003	366
Rule: B2T_DIS_0004	369
Rule: B2T_EXC_0001	372
Rule: B2T_EXC_0002	375
Rule: B2T_EXC_0003	377
Rule: B2T_EXD_0001	381
Rule: B2T_EXD_0002	385
Rule: B2T_EXD_0003	387
Rule: B2T_OPC_0001	390
Rule: B2T_OPC_0002	393
Rule: B2T_OPC_0003	396
Rule: B2T_OPC_0004	399
Rule: B2T_TRL_0001	402
Rule: B2T_TRL_0002	405
Rule: B2T_TRL_0003	407
Rule: B2T_UNC_0001	410
Rule: B2T_UNC_0002	412
Rule: B2T_UNC_0003	414
Rule: B2T_UNC_0004	416
Rule: B2T_UNC_0005	418
Rule: B2T_UNC_0006	420
3. Constraint Comparison Rules	423
Rule: S2S_CAS_0001	424

Rule: S2S_CAS_0003	426
Rule: S2S_CLK_0001	429
Rule: S2S_CLK_0003	434
Rule: S2S_CLK_0004	437
Rule: S2S_CLK_0006	440
Rule: S2S_CLK_0007	441
Rule: S2S_CLK_0010	444
Rule: S2S_CLK_0012	447
Rule: S2S_CLT_0001	450
Rule: S2S_CLT_0003	452
Rule: S2S_CST_0001	455
Rule: S2S_CST_0003	457
Rule: S2S_DIS_0001	459
Rule: S2S_DIS_0003	462
Rule: S2S_EXC_0001	465
Rule: S2S_EXC_0003	468
Rule: S2S_EXD_0001	471
Rule: S2S_EXD_0003	474
Rule: S2S_UNC_0001	478
Rule: S2S_UNC_0003	480
Rule: S2S_UNC_0004	482
Rule: S2S_UNC_0006	484

1

General Design Rules

Rules highlighted in gray are disabled by default.

Rule ID	Severity	Message
CAP_0001	Error	Output or inout port port has zero or incomplete capacitance values.
CAP_0002	Error	Port port has negative pin_or_wire capacitance values.
CAP_0003	Warning	Port port has inconsistent pin_or_wire capacitance values.
CAP_0004	Warning	The capacitance constraint on object_type object is larger than the maximum capacitance design rule.
CAP_0005	Warning	The set_max_capacitance constraint on object_type object is larger than the maximum capacitance design rule in the library.
CAS_0001	Error	Net net has conflicting user case analysis values on load pins.
CAS_0002	Error	Pin or Port pin is driven by conflicting values. A value of zero will be used.
CAS_0003	Error	Pin or Port pin propagated value of type constant_type with value constant_value conflicts with the user set constraint of user_constraint_value .
CAS_0004	Error	Bidirectional pin pin has conflicting values on the load and driver side.
CGR_0001	Error	Clocks generated from the same master clock master_clock are declared as asynchronous.
CGR_0002	Error	Clock clock1 is generated directly or indirectly from the clock clock2 . They should not be declared as asynchronous.
CGR_0003	Error	Clock clock1 and clock clock2 are defined asynchronous with each other. There are clocks generated directly or indirectly from these clocks which are not defined as asynchronous.
CGR_0004	Error	Clock clock1 is missing an asynchronous definition with at least one clock in each of the following asynchronous clock pairs
CGR_0005	Error	Clock clock1 is generated directly or indirectly from the clock clock2 . They should not be declared as physically exclusive.

Rule ID	Severity	Message
CGR_0006	Warning	Clock clock1 and clock clock2 are defined physically exclusive with each other. There are clocks generated from these clocks which are not defined as physically exclusive.
CGR_0007	Warning	Multiple clocks are defined at pin/port pin_port . These clocks should be defined as exclusive/asynchronous.
CGR_0008	Warning	Multiple clocks defined at pin/port pin_port have false path set between them. For accurate signal integrity results use the set_clock_groups command.
CGR_0009	Error	Clocks clock1 and clock2 have valid timing paths between them, but their common base period is more than the limit times the clock period.
CGR_0010	Error	Clocks clock1 and clock2 might be interacting but their common base period is more than the limit times the clock period.
CLK_0001	Warning	Clock clock has count sources on inout ports.
CLK_0002	Warning	Generated clock clock has count sources on input or inout ports.
CLK_0003	Error	Generated clock clock is not expanded because it has no clock reaching its master source pin .
CLK_0004	Warning	Mismatch between generated clock definitions at pin_or_port and potential master clocks.
CLK_0005	Info	Virtual clock clock has the same name as a design object.
CLK_0006	Warning	Clock clock source source has a logic constant or case value.
CLK_0007	Error	Generated clock clock master source source has a logic constant or case value.
CLK_0008	Warning	Generated clock clock has paths from the sources of master clock master_clock to generated clock sources with differing sequential depth.
CLK_0009	Error	Generated clock clock is not expanded because master clock master_clock does not reach its master source pin .
CLK_0010	Error	Generated clock clock is not expanded because its master clock master_clock is not expanded.
CLK_0011	Error	Generated clock clock is not expanded because potential master clock master_clock is not expanded.
CLK_0012	Warning	Generated clock clock has non-unate sense for master clock master_clock at its master source.

Rule ID	Severity	Message
CLK_0013	Error	Generated clock clock ignores multiple rising edges of master clock master_clock because of user-defined duty cycle.
CLK_0014	Info	Clock clock is created on a pin instead of a port.
CLK_0015	Warning	Clock clock is created on a hierarchical pin.
CLK_0016	Error	Generated clock clock has no source latency path from its master clock master_clock .
CLK_0017	Error	Generated clock clock master source is not in the logic between the start of master clock master_clock and the start of the generated clock.
CLK_0018	Warning	Clock clock has non-unate logic in the clock network.
CLK_0019	Warning	Design has both propagated and ideal non-virtual clocks.
CLK_0020	Error	Generated clock clock has edge relationships with its master clock master_clock that cannot be satisfied. Only paths with path_sense sense exist from the master clock to source pin source_pin . An expected_sense sense is expected.
CLK_0021	Info	Clock clock is not used in this scenario.
CLK_0023	Warning	There are count clocks on source source with the same period and waveform.
CLK_0024	Warning	Register Clock pin pin has count clocks.
CLK_0026	Warning	Clock clock is used as data. One or more sources of the clock fans out to a register data pin or to a constrained primary output or inout port.
CLK_0027	Error	Conflicting set_clock_sense for clock clock at pin pin .
CLK_0028	Warning	The master clock for generated clock clock is ambiguous. There are multiple clocks on -source pin master_source .
CLK_0029	Warning	The last edge shift value for generated clock clock is not equal to the first edge shift.
CLK_0030	Warning	There is reconvergent logic in the network for clock clock .
CLK_0031	Warning	The waveform for clock clock has more than two edges.
CLK_0032	Error	Non-combinational generated clock clock has a source identical to its master source object.
CLK_0033	Error	Clock clock has ideal latency during post-layout analysis.
CLK_0034	Error	Clock clock has propagated latency during pre-layout analysis.

Rule ID	Severity	Message
CLK_0035	Warning	No clock-gating check inferred for the instance: cell clock pin: clk_pin enable pin: enable_pin lib cell: lib_cell clk name: clk_name
CLK_0036	Error	Generated clock clock (file: gclk_flr) has incorrect waveform inversion with its master clock master_clock (file: mclk_flr) that cannot be satisfied.
CLK_0037	Warning	There are other clock sources in the clock latency path from the sources of the master clock 'master_clock' to the source of generated clock 'clock'.
CLK_0038	Warning	Detects generated clocks with a sequential source network spanning block boundaries.
CLK_0039	Warning	Detects circular dependency in generated clock definition.
CLK_0040	Warning	Detects clock definition conflicts on the same net.
CLK_0041	Warning	The clock clock has a loop in the source latency network.
CLK_0042	Error	The user-applied case analysis overlaps with clock propagation.
CLK_0043	Warning	Creation of a clock 'clock' blocks propagation of clocks reaching to the specified point.
CMP_0001	Warning	Command command is unsupported by products . Command used in N locations.
CMP_0002	Warning	Option -option of command command is unsupported by products . Option used in N locations.
CMP_0003	Warning	Application variable sh_command_abbrev_mode is not set to the recommended value of <i>Command-Line-Only</i> .
CNL_0001	Warning	Inconsistent network latency values on object_type object .
CNL_0002	Warning	Input and output delays are defined relative to clock clock . This clock has incomplete network latency values required for those external delays even though its source ports or pins have more complete network latencies defined on them.
CNL_0003	Warning	Clock network latency is specified on object_type object but that object is not in any clock network
CNL_0004	Warning	Clock network latency is specified on object_type object with respect to clock clock but that object is not in the clock's network
CNL_0005	Warning	All register clock pins in the fanout of pin have no clock latency for clock clock .
CSL_0001	Warning	Inconsistent source latency values on object_type object .

Rule ID	Severity	Message
CSL_0002	Warning	Input or output delays are defined relative to clock clock . This clock has incomplete source latency values required for those external delays even though its source ports or pins have more complete source latencies defined on them.
CSL_0003	Warning	Clock source latency is specified on object_type object but that object is not a clock source.
CSL_0004	Warning	Generated clock clock master clock master_clock is not propagated and has zero or incomplete source latency values.
CSL_0005	Warning	Generated clock clock has source latency values less than that of master clock master_clock when the clock edge relationships are considered.
CSL_0006	Warning	Clock source latency is specified on object_type object with respect to clock clock but that object is not in the clock's network.
CSL_0007	Warning	Source latency defined on pin/port pin_port will overwrite the clock source latency for clock clock .
CTR_0001	Warning	Clock clock has inconsistent clock transition values.
CTR_0002	Warning	Clock clock has incomplete clock transition values.
CTR_0003	Warning	Propagated clock clock has ideal clock transition values.
CTR_0004	Warning	Virtual clock clock has clock transition values.
CTR_0005	Warning	Clock clock has no clock transition values in pre-layout.
CTR_0006	Warning	Clock clock has clock transition values in post-layout.
DES_0001	Warning	Register clock pin pin has no clock signal propagating to it.
DES_0002	Warning	All checks on the register clock pin pin are disabled due to reason .
DES_0003	Warning	Generated clock source latency register clock pin pin has no clock signal.
DES_0004	Warning	Register clock pin pin does not have a clock signal that can sensitize the register.
DRV_0001	Warning	Input or inout port port has no input transition or driving cell or drive resistance specified.
DRV_0002	Warning	Input or inout port port has incomplete input transition or driving cell or drive resistance specified.
DRV_0003	Warning	Input or inout port port has input transition or drive resistance instead of set_driving_cell .

Rule ID	Severity	Message
DRV_0004	Warning	Input or inout port port has inconsistent drive resistance values.
DRV_0005	Warning	Input or inout port port has inconsistent input transition values.
EXC_0001	Error	An exception has conflicting rise_or_fall and to_rise_or_fall options specified.
EXC_0002	Warning	An exception has some invalid start and endpoints specified.
EXC_0003	Warning	All -from objects for an exception are invalid path startpoints.
EXC_0004	Warning	All -to objects for an exception are invalid.
EXC_0005	Error	An exception will create startpoints or endpoints and break other timing paths through those pins.
EXC_0006	Warning	An exception does not specify any valid paths.
EXC_0007	Warning	An exception has incomplete values.
EXC_0008	Error	An exception is forcing startpoints or endpoints by breaking the clock network.
EXC_0009	Warning	Inconsistent exception ; min_delay must be less than max_delay .
EXC_0010	Info	An exception has no corresponding min_delay for a max_delay exception.
EXC_0011	Info	An exception has no corresponding max_delay for a min_delay exception.
EXC_0012	Info	An exception has incomplete values.
EXC_0013	Info	An exception has incomplete values.
EXC_0014	Warning	An exception is fully overridden by other exceptions.
EXC_0015	Info	An exception is partially overridden by other exceptions.
EXD_0001	Warning	Input or inout port port has no input delay specified.
EXD_0002	Warning	Input or inout port port has no clock-related input delay specified.
EXD_0003	Warning	Output or inout port port has no clock-related output delay specified.
EXD_0004	Warning	The input_or_output delay on object_type object_name has incomplete values.
EXD_0007	Warning	No clock signals exist at reference pin reference_pin for input_or_output delay on object_type object_name .

Rule ID	Severity	Message
EXD_0008	Warning	The input_or_output delay at pin object_name is forcing a start_or_end point that blocks paths through this pin.
EXD_0009	Warning	The input delay at object_type object_name has values larger than max_percent % of the relative clock's period.
EXD_0010	Warning	The output delay at object_type object_name has values larger than max_percent % of the relative clock's period.
EXD_0011	Warning	Some ports have no input delay or input delay without a relative clock. A default clock will be assumed for such ports because the timing_input_port_default_clock variable is set to true .
EXD_0012	Error	The input delay at object_type object_name has zero arrival window for min and max values.
EXD_0013	Info	The output delay at object_type object_name has inconsistent values.
EXD_0014	Warning	The input/output delay set on the combinational path from from_object_type from_object to to_object_type to_object is incorrect.
EXD_0015	Warning	The input delay at object_type object_name has inconsistent values.
HIER_001	Warning	Hierarchical pin pin has constraint constraint defined on flr .
LOOP_001	Warning	Arc from pin from_pin to to_pin disabled due to combinational loop.
LOOP_002	Warning	Arc from pin from_pin to to_pin belongs to a sequential loop.
NTL_0001	Warning	Output port port is unbuffered.
NTL_0002	Error	Net net has both strong and three-state drivers.
NTL_0003	Warning	Net net has potentially invalid multiple strong drivers.
NTL_0004	Info	Net net is a top-level feedthrough.
NTL_0005	Warning	Unresolved reference to reference_name .
NTL_0006	Warning	Net net has high fanout; fanout count is fanout_count .
OPN_0001	Warning	The constraint constraint set on object_typeobject has incomplete options options .
PRF_0001	Warning	There are count clock pins with more than limit clocks.
TRN_0001	Warning	The transition constraint on object_type object is larger than the maximum transition design rule.

Rule ID	Severity	Message
TRN_0002	Warning	The set_max_transition constraint on object_type object is larger than the maximum transition design rule in the library.
UNC_0001	Warning	Input or output delays are defined relative to clock clock . This clock has incomplete uncertainty values required for those external delays even though its source ports or pins have more complete uncertainties defined on them.
UNC_0002	Warning	Clock uncertainty is specified on object_type object but that object is not in any clock network.
UNC_0003	Warning	Clock uncertainty is incomplete or incorrect between two interacting clocks clock1 and clock2 .
UNC_0004	Warning	Inter-clock uncertainty is smaller than simple uncertainty for clock X.
UNC_0005	Warning	Inter-clock uncertainty is smaller than object based uncertainty at <i>pin/port</i> for clock X.
UNC_0006	Warning	Object-based uncertainty at <i>pin/port</i> is smaller than simple uncertainty for clock X.
UNT_0001	Error	Library library has no units defined.
UNT_0002	Error	Library library has incomplete units defined.
UNT_0003	Info	The libraries used by this design have inconsistent units.

Rule: CAP_0001

Output or inout port **port** has zero or incomplete capacitance values.

Severity: Error

Description

Output or inout ports should have non-zero pin capacitance values specified for min/max rise/fall combinations.

Risk Associated with Violation

When no output load is specified, the resulting delay calculation will be optimistic. To model realistic design scenarios, proper loads need to be specified for min/max delays and rise/fall transitions.

One Possible Scenario

Consider a scenario where the capacitance for an **out** output port is set in incompletely specified:

```
set_load 2 -max [get_ports out]
```

The minimum capacitive load is missing and, as a consequence, the minimum delay calculation for paths ending at this output is optimistic. This could make the design fail.

What Next

The "Violation Details" section in the GUI reports the pin capacitance values in a table. The location of the constraint (file name and line number) is also reported.

In addition, you can use the [report_port](#) command to get details on the violated port as shown:

```
ptc_shell> report_port [get_ports out]
```

```
*****
Report : port
Design : CAP_0001
Scenario: default
Version: ...
Date   : ...
*****
```

Port	Dir	Pin Cap		Wire Cap		Attributes
		Min rise/fall	Max rise/fall	Min rise/fall	Max rise/fall	
out		0.00/0.00	2.00/2.00	0.00/0.00	0.00/0.00	

Fix Suggestion

Realistic capacitance values should be specified for all combinations of min/max delays and rise/fall transitions.

In the preceding scenario, a constraint specifying the load value with the `-min` option can be added to address minimum delay calculations to this output.

```
set_load 1 -min [get_ports out]
```

See Also

[set_load](#)

[get_ports](#)

Rule: CAP_0002

Port **port** has negative **pin_or_wire** capacitance values.

Severity: Error

Description

The port mentioned in this violation has some negative capacitance values. This is most likely due to an incorrect constraint.

Risk Associated with Violation

The output load is not realistic and this could result in overly optimistic delay calculation. A consequence might be a failing chip even though static timing has a positive slack.

One Possible Scenario

Consider a scenario where the capacitance for an **out** output port is set in the following way:

```
set_load -2 [get_ports out]
```

The negative value for the load will trigger a **CAP_0002** violation and result in optimistic delay calculation for paths ending at port **out**.

What Next

The "Violation Details" section in the GUI reports the pin capacitance values in a table. The location of the constraint (file name and line number) is also reported.

In addition, you can use the [report_port](#) command to get details on the violated port as seen below:

```
ptc_shell> report_port
*****
Report : port
Design : CAP_0002
Scenario: default
Version: ...
Date : ...
*****
```

Port	Dir	Pin Cap		Wire Cap		Attributes
		Min	Max	Min	Max	
		rise/fall	rise/fall	rise/fall	rise/fall	
CLK	in	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	
in		0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00	
out		-2.00/-2.00	-2.00/-2.00	0.00/0.00	0.00/0.00	

Fix Suggestion

Specifying a nonnegative, realistic capacitance value for all combinations of min/max delays and rise/fall transition ensures correct timing delay calculation.

See Also

[set_load](#)

[get_ports](#)

Rule: CAP_0003

Port **port** has inconsistent **pin_or_wire** capacitance values.

Severity: Warning

Description

The capacitance values specified for the pin cap or the wire cap of a port are inconsistent. Each minimum value should be smaller than the corresponding maximum value.

Risk Associated with Violation

Minimum and maximum values are used in delay calculation to characterize the minimum and maximum corners. Having inconsistent capacitance values results in inaccurate timing calculations.

One Possible Scenario

Consider a scenario where the capacitance for an **in** input port is set in the following way:

```
set_load 4 -min -wire in
set_load 2 -max -wire in
```

The wire cap value for min is more than that for max. This triggers a **CAP_0003** violation. Inconsistent wire capacitances are provided for the **in** port in such a scenario; the minimal capacitance cannot be smaller than the maximum capacitance.

What Next

The "Violation Details" section displays the inconsistencies in capacitance values. It also points to the particular command in the SDC file where these values were specified. In the preceding scenario, the wire capacitance values would be displayed as shown in the following table.

Wire Capacitance

	Min	Max
Rise	4.0000	2.0000

Fall 4.0000 2.0000

Source File: ./scripts/constraints.tcl, line3: ./scripts/constraints.tcl, line4

Fix Suggestion

The reported capacitance values should be verified and modified such that the minimum capacitances are always smaller or equal to the maximum capacitances.

See Also

[set_load](#)

Rule: CAP_0004

The capacitance constraint on **object_type object** is larger than the maximum capacitance design rule.

Severity: Warning

Status: Disabled by default

Description

A capacitance constraint is being specified through the [set_load](#) command. However, the value specified in the command is larger than the limit specified by the [set_max_capacitance](#) command, the applicable library level maximum capacitance, or the pin level maximum capacitance attribute. The most restrictive limit is used to perform the check.

If no limit has been specified with the [set_max_capacitance](#) constraint or in the library, the constraint is compared to the **max_capacitance** property associated with this rule.

This rule does not perform design rule checking (DRC) of capacitance in the design. Instead, only the value in the constraint is checked against the design rule limit. A [set_load](#) constraint can pass this check, but the design might have maximum capacitance violations when design rule checking is performed in another tool. In addition, if multiple ports are driven by the same cell and each port has a [set_load](#) constraint, the check is performed on each constraint individually without summing the values to get a total capacitance.

The CAP_0004 rule is disabled by default and can be enabled with the `enable_rule CAP_0004` command before running the [analyze_design](#) command.

Risk Associated with Violation

The value in the [set_load](#) constraint can directly lead to a design rule check violation for maximum capacitance in another tool. If a cell drives a larger load than intended, this can result in slower signal transitions and higher power consumption in the design.

One Possible Scenario

Consider a library where the maximum capacitance value is set to 1.5 femtofarads:

```
library (example_lib) {  
  
...  
  
capacitive_load_unit(1.000000, "ff");  
  
...  
  
default_max_capacitance : 1.500000;  
  
...  
}
```

The constraint file for the design specifies a maximum capacitance of 2.1 library units (in this case femtofarads) on an output port of the design, as shown here:

```
set_load 2.1 [get_ports o]
```

The maximum capacitance set on the port is larger than the maximum allowed capacitance of 1.5 femtofarads from the library. A CAP_0004 violation will be issued.

What Next

The 'Violation Details' section provides a table of the capacitance values for rise/fall and min/max analysis, as well as the file name and line number of the [set_load](#) constraint. For the example scenario, the following table is created in the Information Pane.

	Min	Max
Rise	2.100000	2.100000
Fall	2.100000	2.100000

You can also use the [get_attribute](#) command to verify the related capacitance attribute on the object specified in the constraint. For the example scenario, you can verify one entry in the table of capacitance values by using the following command:

```
ptc_shell> get_attribute [get_ports o] pin_capacitance_max_fall  
  
2.100000
```

Fix Suggestion

You should specify capacitance constraints that are smaller than the maximum capacitance design rule to avoid maximum capacitance design rule check (DRC) violations in another tool.

Rule Property: **max_capacitance**

By default, the **max_capacitance** rule property is set to 500 femtofarads (fF). Any [set_load](#) constraint that is over this value will issue a CAP_0004 violation if there are no capacitance limits specified through the [set_max_capacitance](#) constraint or in the library. If no units are specified with the command, farads are assumed by the tool. You can change the threshold used for checking with the [set_rule_property](#) command.

```
set_rule_property max_capacitance 200fF CAP_0004
```

For the rule property value to have an effect, [analyze_design](#) must be run after the [set_rule_property](#) command is issued.

See Also

[set_load](#)

[set_max_capacitance](#)

[get_attribute](#)

[set_rule_property](#)

[analyze_design](#)

[enable_rule](#)

[disable_rule](#)

Rule: CAP_0005

The `set_max_capacitance` constraint on ***object_type object*** is larger than the maximum capacitance design rule in the library.

Severity: Warning

Status: Disabled by default

Description

A [set_max_capacitance](#) constraint has a value larger than the applicable max capacitance attribute in the related library. When design rule checking is performed in another tool, the more restrictive value is used. The value specified in the [set_max_capacitance](#) constraint will be redundant. This rule finds [set_max_capacitance](#) constraints with this issue.

If no limit has been specified in the library, the constraint is compared to the **max_capacitance** property associated with this rule.

The CAP_0005 rule is disabled by default and can be enabled by issuing the following command before running the [analyze_design](#) command:

```
enable_rule CAP_0005
```

Risk Associated with Violation

The [set_max_capacitance](#) constraint can be used to specify a capacitance value to be used during design rule checking in another tool. If the calculated load that a cell or port drives is larger than the limit specified, it is considered to have failed the design rule check. The most restrictive limit from the constraint or applicable library level max capacitance or pin level max capacitance is used to perform the check. If the intent of the constraint was to set a value more restrictive than the library, cells in the design may drive larger loads than intended, resulting in slower signal transitions and higher power consumption.

One Possible Scenario

Consider a library where the maximum capacitance values are set to 2 femtofarads, as shown here:

```
library (example_lib){  
  
    ...  
  
    capacitive_load_unit(1.000000, "ff");  
  
    ...  
    default_max_capacitance : 1.500000;  
    cell (AND2)  
  
        pin (Z) {  
  
            direction : "output";  
  
            max_capacitance : 2.000000;  
  
        }  
  
    ...  
}
```

The constraint file for the design specifies a maximum capacitance of 2.1 library units (in this case femtofarads) on an output pin of a cell in the design, as shown here:

```
set_max_capacitance 2.1 [get_pins ul/Y]
```

The maximum allowed capacitance specified on the pin by the constraint is larger than the maximum allowed capacitance of 2 femtofarads specified in the library. This causes a CAP_0005 violation to be issued.

What Next

The 'Violation Details' section provides the file name and line number of the [set_max_capacitance](#) constraint. You can also use the [get_attribute](#) command to verify the related max_capacitance attribute on the object specified in the constraint. For the example scenario, you can verify the attribute using the following command:

```
ptc_shell> get_attribute [get_pins ul/Y] max_capacitance 2.100000
```

Fix Suggestion

The [set_max_capacitance](#) constraint will only have an effect in design rule checking if it is more conservative than the related library maximum capacitance attribute. You should review the value specified for correctness, and either modify the value if it is incorrect or remove the constraint.

Rule Property: max_capacitance

By default, the **max_capacitance** rule property is set to 500 femtofarads (fF). Any [set_max_capacitance](#) constraint that is over this value causes a CAP_0005 violation to be issued if there are no capacitance limits specified in the library. If no units are specified with the command, farads are assumed by the tool. You can change the threshold used for checking with the [set_rule_property](#) command as shown:

```
set_rule_property max_capacitance 200fF CAP_0005
```

For the rule property value to have an effect, [analyze_design](#) must be run after the [set_rule_property](#) command is issued.

See Also

[set_load](#)

[set_max_capacitance](#)

[get_attribute](#)

[set_rule_property](#)

[analyze_design](#)

[enable_rule](#)

[disable_rule](#)

Rule: CAS_0001

Net **net** has conflicting user case analysis values on load pins.

Severity: Error

Description

Multiple conflicting case analysis values are specified on load pins of the net. This is a situation that cannot occur in normal circuit operation and may result in incorrect analysis. The specified case values are propagated forward for each conflicting load pin, even though they disagree with one another.

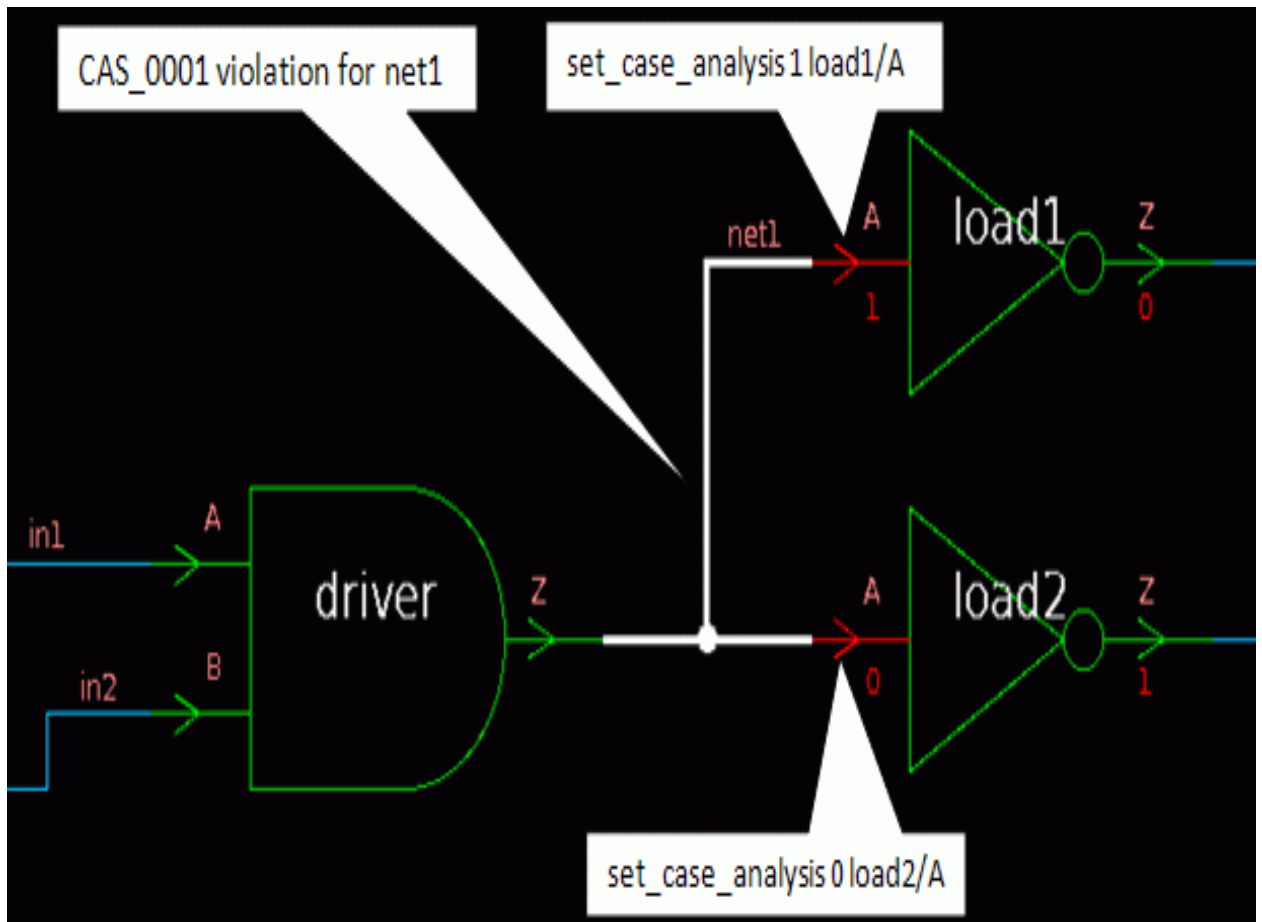
Risk Associated with Violation

A condition that is not possible in circuit operation will be analyzed. This can result in poor analysis coverage or in the analysis of false timing paths.

One Possible Scenario

Consider a scenario where a single hierarchical net is driven by a single driver, and connects to two input pins. A [set_case_analysis](#) of '0' is set on one of the input pins and a [set_case_analysis](#) of '1' is set on the other. Because these two pins are connected to the same net, this condition is not physically possible, and a **CAS_0001** rule violation will be issued.

Figure 1 *Physically Impossible Condition on a Pin*



What Next

The "Violation Details" section will give you the names of the net and its load pins, along with their case values. To find the drivers of the net use the `report_net -connections` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> report_net -connections [get_net net1]

*****
Report : net
        -connections
*****
Connections for net 'net1':
Driver Pins                                     Type
-----
driver/Z                                     Output Pin (AN2)

Load Pins                                     Type
-----
load2/A                                     Input Pin (IV)
load1/A                                     Input Pin (IV)
```

The report contains the driver pins for the net as well as its load pins.

Fix Suggestion

To resolve the issue, move the case value setting to the driver of the net with the appropriate value for this scenario.

For the example scenario, the command can be issued as follows:

```
set_case_analysis 1 driver/Z
remove_case_analysis load1/A
remove_case_analysis load2/A
```

All load pins of the net will have the case value of 1.

See Also

[set_case_analysis](#)

[report_net](#)

[remove_case_analysis](#)

Rule: CAS_0002

Pin or port **pin** is driven by conflicting values. A value of zero will be used.

Severity: Error

Description

The given pin or port is multi-driven. The drivers have conflicting case values. The value conflict is resolved to zero.

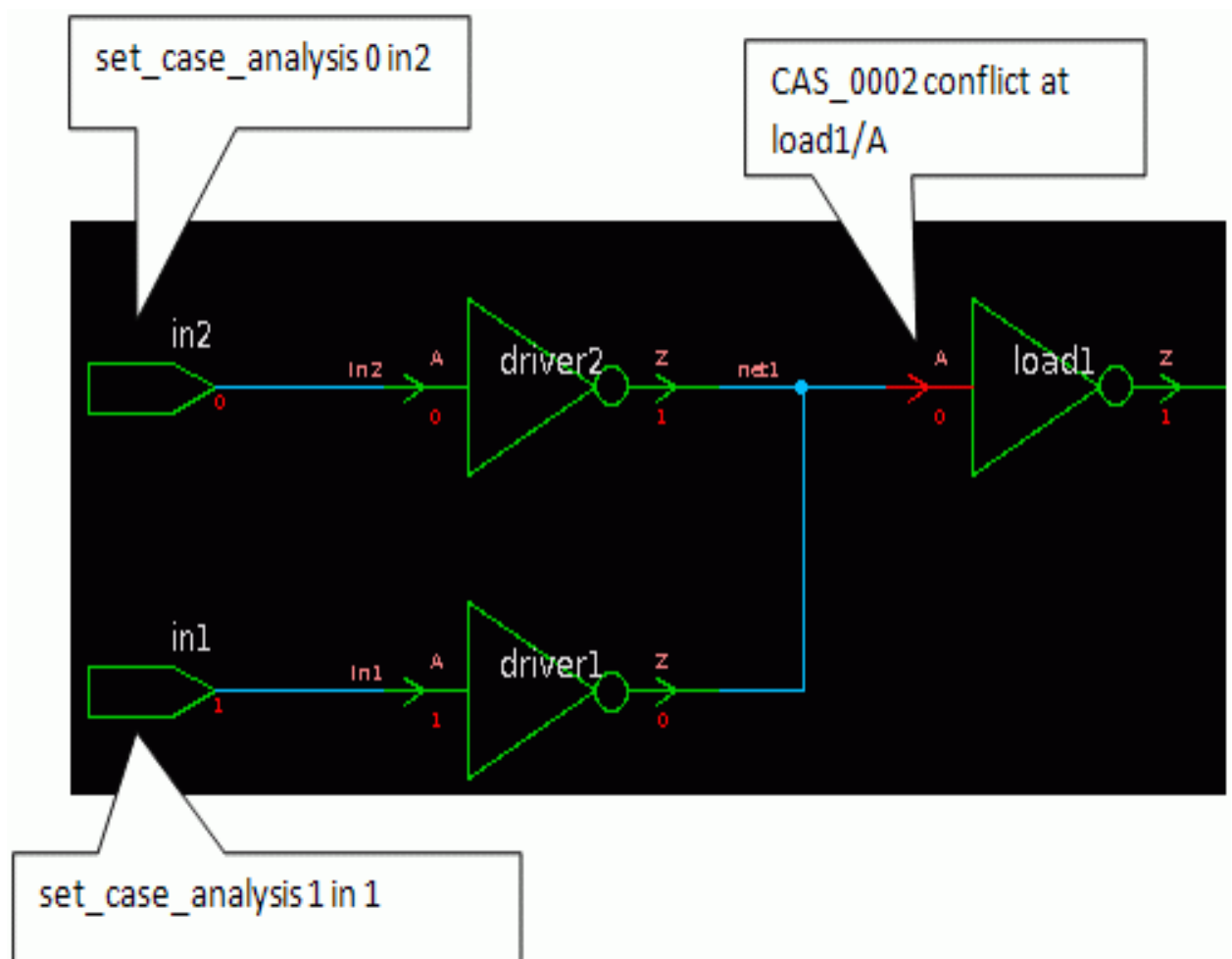
Risk Associated with Violation

Timing will be performed with case analysis settings that are physically impossible. This might result in less analysis coverage of the scenario, or in the analysis of false timing paths. It is also possible that the netlist has multiple strong drivers when three-state drivers were intended.

One Possible Scenario

Consider a scenario where a single net is connected to two strong drivers and one input pin. A case value of '0' propagates to one driver and a case value of '1' propagates to the other driver. A **CAS_0002** violation will be issued for the input pin connected to the net where the case analysis values conflict. The case value will be resolved to the value of '0'.

Figure 2 Conflicting Case Analysis Values



What Next

Use the `report_case_details -to conflict_pin` command to find the `set_case_analysis` statements that are causing the conflict. For the example scenario, the command can be issued as follows:

```
ptc_shell> report_case_details -to load1/A
```

Properties	Value	Pin/Port
from user case	0	load1/A (IV)
Case fanin report:		
Verbose Source Trace for pin/port load1/A:		
Path number: 1		
Path Ref #		
Value	Properties	Pin/Port
0	conflict	load1/A (IV)
2		
0	F()=A'	driver1/Z (IV)
3		
1	F()=A'	driver2/Z (IV)
Path number: 2		
Path Ref #		
Value	Properties	Pin/Port
0	F()=A'	driver1/Z (IV)
1		driver1/A (IV)
1	user case	in1 (in)
Path number: 3		
Path Ref #		
Value	Properties	Pin/Port
1	F()=A'	driver2/Z (IV)
0		driver2/A (IV)
0	user case	in2 (in)

The report shows that the case value '0' propagates to the **driver1/Z** pin from the input port **in1**, and the case value '1' propagates to the **driver2/Z** pin from the input port **in2**. However, the **driver1/Z** and **driver2/Z** pins are connected to the same net, resulting in a value conflict at **load1/A**.

Fix Suggestion

To resolve the issue, change the case settings as appropriate for the functional mode you are constraining.

For the example scenario, the following command can be issued to propagate a case value of '0' to the **load1/A** pin:

```
set_case_analysis 1 in2
```

Both strong drivers will have a propagated case value of '0', and there will be no conflict.

See Also

[set_case_analysis](#)

[report_case_details](#)

[remove_case_analysis](#)

Rule: CAS_0003

Pin or Port **pin** propagated value of type **constant_type** with value **constant_value** conflicts with the user set constraint of **user_constant_value**.

Severity: Error

Description

The given pin has a case analysis value set on it. There is a case value that propagates to this pin with a conflicting value. The case value set on this pin will be used.

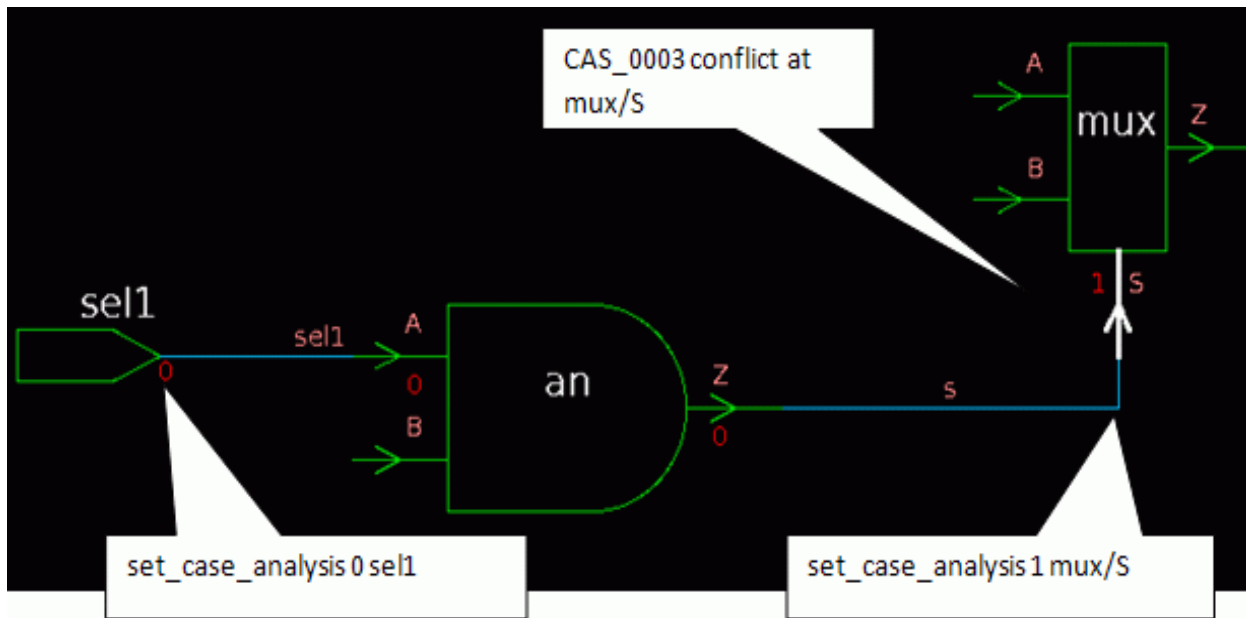
Risk Associated with Violation

Timing will be performed on a scenario of case analysis settings that are physically impossible. This may result in less analysis coverage of the scenario, or in the analysis of false timing paths.

One Possible Scenario

Consider a scenario where a [set_case_analysis](#) '1' is set on the select pin of a mux, and there are other [set_case_analysis](#) constraints on the fanin of that pin that propagate a '0' to that pin. A **CAS_0003** violation will issued, and the case value of '1' set on the select pin of the mux will be used.

Figure 3 Conflicting set_case_analysis Constraints



What Next

The “Violation Details” section will display the sources of the case value that propagated to the pin with a conflict. To see the logic between those sources and the conflict, use the **report_case_details -to <conflict>** command. For the example scenario, the command can be issued as follows:

```
ptc_shell> report_case_details -to mux/S
```

Properties	Value	Pin/Port
user case	1	mux/S (MUX21H)

Case fanin report:

Verbose Source Trace for pin/port mux/S:

Path number: 1

Path	Ref #	Value	Properties	Pin/Port
	1	user case		mux/S (MUX21H)
	0	F()=A B		an/Z (AN2)
	0			an/A (AN2)
	0	user case		sel1 (in)

The report shows that the case value '0' propagates to the output pin **an/Z** from the input port **sel1**, but that a conflicting case value of '1' has been set on the **mux/S** pin.

Fix Suggestion

To resolve the issue, change the case settings as appropriate for the functional mode you are constraining.

For the example scenario, to use the case value from the input port **sel1**, the case analysis on the select pin of the mux can be removed. The command can be issued as follows:

```
remove_case_analysis mux/S
```

See Also

[set_case_analysis](#)

[report_case_details](#)

[remove_case_analysis](#)

Rule: CAS_0004

Bidirectional pin **pin** has conflicting values on the load and driver side.

Severity: Error

Description

The case value for the drivers of the net connected to the bidirectional pin and the case value propagated through a cell to the bidirectional pin conflict. The value conflict is resolved to zero.

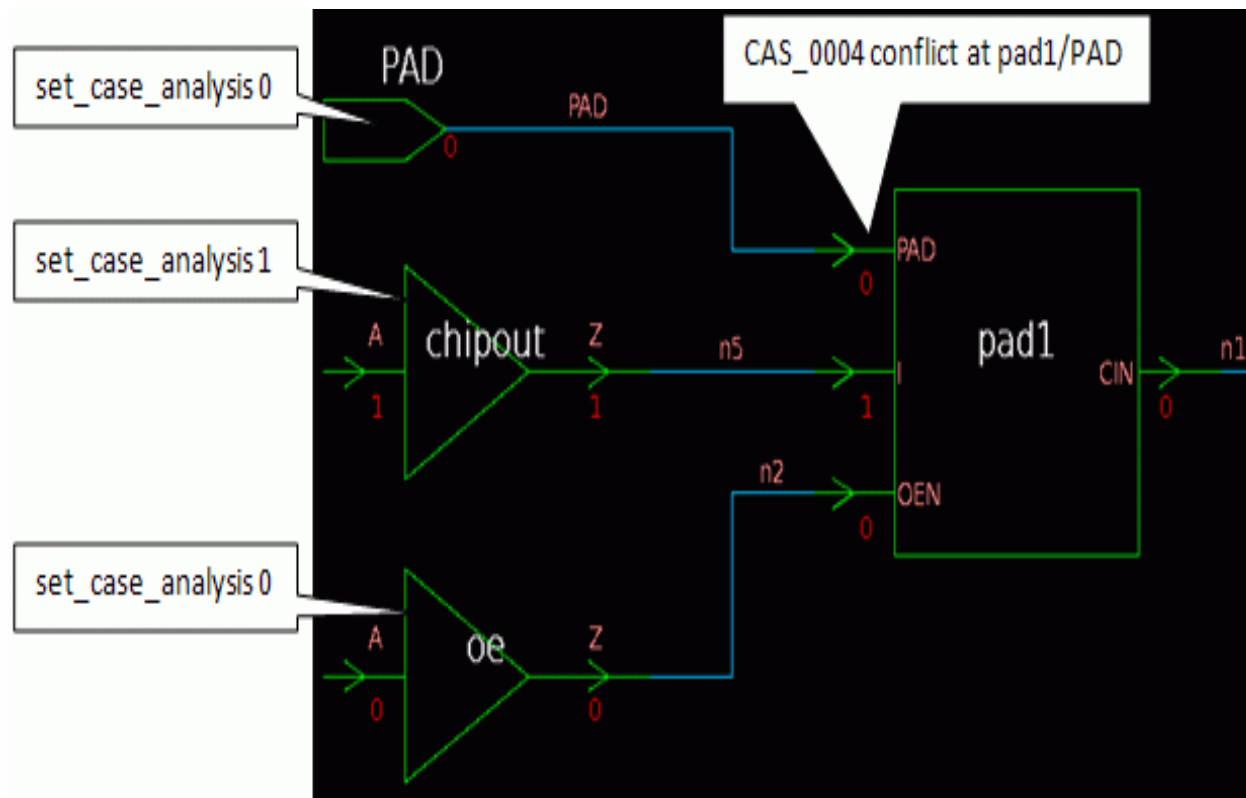
Risk Associated with Violation

Timing will be performed on a scenario of case analysis settings that are physically impossible. This may result in less analysis coverage of the scenario, or in the analysis of false timing paths.

One Possible Scenario

Consider a scenario with a PAD cell. The input pin pad1/oen and input pin pad1/I are configured such that the value at the input pin pad1/I propagates to the inout pin pad1/PAD. In this example, a value of '1' propagates from pad1/I to pad1/PAD. If the port PAD is has a set_case_analysis '0', then this value also propagates to pad1/PAD. The conflict is resolved to '0'. A **CAS_0004** violation is issued for pin pad1/PAD.

Figure 4 Conflict at in/out Pin of a PAD cell



What Next

The “Violation Details” section will display the sources of the case value that propagated to the pin with a conflict. To see the logic between those sources and the conflict, use [report_case_details -to <conflict pin>](#). For the example scenario, the command can be issued as follows:

```
ptc_shell> report_case_details -to pad1/PAD
```

Properties	Value	Pin/Port
from user case	0	pad1/PAD (pad1)

Case fanin report:

Verbose Source Trace for pin/port pad1/PAD:

Path number: 1

Path	Ref #	Value	Properties	Pin/Port
		0	conflict	pad1/PAD (pad1)
2		1		pad1/I (pad1)
3		0		pad1/OEN (pad1)

```
4          0      user case          PAD (inout)
```

Path number: 2

Path Ref #	Value	Properties	Pin/Port
1			pad1/I (pad1)

Path number: 3

Path Ref #	Value	Properties	Pin/Port
0			pad1/OEN (pad1)

Path number: 4

Path Ref #	Value	Properties	Pin/Port
0	user case		PAD (inout)

The report shows that the case value '0' propagates to the pad1/PAD pin from the input port PAD, but that a conflict exists between this value at the pad1/PAD pin.

Fix Suggestion

To resolve the issue, change the case analysis settings as appropriate for the functional mode you are constraining.

For the example scenario, the following command can be issued to remove the conflicting case analysis value from the input port PAD:

```
remove_case_analysis [get_port PAD]
```

The case value from pad1/I will be used.

See Also

[set_case_analysis](#)

[report_case_details](#)

[remove_case_analysis](#)

Rule: CGR_0001

Clocks generated from the same master clock **master_clock** are declared as asynchronous.

Severity: Error

Description

Clocks which are generated from the same master clock cannot be asynchronous with each other.

Risk Associated with Violation

Paths between the clocks declared as asynchronous will not be analyzed. In addition, for crosstalk delay, infinite windows analysis is performed on nets in aggressor/victim relationships between the clocks. This may result in overly conservative signal integrity analysis.

One Possible Scenario

Consider a scenario where two generated clocks with the same master clock are declared asynchronous:

```
create_clock -period 4 clk1
create_generated_clock -source clk1 -divide_by 2 clk2
create_generated_clock -source clk1 -divide_by 4 clk3
set_clock_groups -asynchronous -group {clk2} -group {clk3}
```

The two generated clocks, clk2 and clk3, cannot be asynchronous because they both have the same master clock, clk1, and a **CGR_0001** violation will be issued.

What Next

The "Violation Details" section will give you the names of the clocks which are generated from the same master clock, but declared as asynchronous. You can click on the clock name(s) to see the relevant clock definition, and the location of the problematic clock group definition is also provided.

Fix Suggestion

To resolve this issue, consider adjusting the clock grouping relationship of the two clocks. In the example above, the asynchronous relationship can be changed by removing the [set_clock_groups](#) command:

```
set_clock_groups -asynchronous -group {clk2} -group {clk3}
```

Alternatively, you might want to change the master/generated clock relationship such that the reported clocks are not generated from the same source.

See Also

[set_clock_groups](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock](#)

[report_clock_crossing](#)
[remove_clock_groups](#)
[remove_clock](#)
[remove_generated_clock](#)

Rule: CGR_0002

Clock **clock1** is generated directly or indirectly from the clock **clock2**. They should not be declared as asynchronous.

Severity: Error

Description

Clocks which are generated from one another cannot be asynchronous.

Risk Associated with Violation

Paths between the clocks declared as asynchronous will not be analyzed. In addition, for crosstalk delay, infinite windows analysis is performed on nets in aggressor/victim relationships between the clocks. This may result in overly conservative signal integrity analysis.

One Possible Scenario

Consider a scenario where a generated clock and its master clock are declared asynchronous:

```
create_clock -period 4 clk1
create_generated_clock -source clk1 -divide_by 2 clk2
set_clock_groups -asynchronous -group {clk1} -group {clk2}
```

The clock, clk2, is a generated clock with master clock, clk1. Because clk2 is generated from clk1, they cannot be asynchronous to each other, and a **CGR_0002** violation will be issued.

What Next

The "Violation Details" section will give you the names of the generated clock and the master clock that are declared as asynchronous. You can click on the clock name(s) to see the relevant clock definition, and the location of the problematic clock group definition is also provided.

Fix Suggestion

To resolve this issue, consider adjusting the clock grouping relationship of the two clocks. In the preceding above, the asynchronous relationship can be changed by removing the [set_clock_groups](#) command:

```
set_clock_groups -asynchronous -group {clk1} -group {clk2}
```

Alternatively, you might want to change the master/generated clock relationship of the reported clocks.

See Also

[set_clock_groups](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock -groups](#)

[report_clock_crossing](#)

[remove_clock_groups](#)

[remove_clock](#)

[remove_generated_clock](#)

Rule: CGR_0003

Clock **clock1** and clock **clock2** are defined asynchronous with each other. There are clocks generated directly or indirectly from these clocks which are not defined as asynchronous.

Severity: Error

Description

If two clocks are asynchronous, then all the clocks generated from these clocks should be asynchronous with each other. A generated clock does not inherit the asynchronous properties of its master clock, so it should be declared asynchronous to clocks that are asynchronous with its master clock.

Risk Associated with Violation

For clocks not declared asynchronous, paths between the clocks will be analyzed, possibly resulting in timing violations being reported for false paths. In addition, for crosstalk delay, the timing window relationship will be preserved on nets in aggressor/victim relationships between the clocks. This may result in optimistic signal integrity analysis.

One Possible Scenario

Consider a scenario where two master clocks are declared asynchronous, while no asynchronous relationship is specified between other clock pairs:

```
create_clock -period 4 clk1
create_clock -period 4 clk2
create_generated_clock -source clk1 -divide_by 2 clk3
create_generated_clock -source clk2 -divide_by 2 clk4
set_clock_groups -asynchronous -group {clk1} -group {clk2}
```

Two master clocks, clk1 and clk2 are asynchronous to each other. The two generated clocks clk3 and clk4 should also be asynchronous because this is the relationship between their master clocks. Since the generated clocks clk3 and clk4 do not have an asynchronous relationship, a **CGR_0003** violation will be issued.

What Next

The "Violation Details" section will give you the names of the clock pairs that are missing asynchronous clock declarations. You can click on the clock name(s) to see the relevant clock definition, and the location of the problematic clock group definition is also provided.

Fix Suggestion

To resolve this issue, consider declaring the clock pairs as asynchronous. In the example above, the correct asynchronous relationship can be declared by changing the [set_clock_groups](#) command as follows:

```
set_clock_groups -asynchronous -group {clk1 clk3} \
    -group {clk2 clk4}
```

Alternatively, you might change the master/generated clock relationship of the reported clocks.

See Also

[set_clock_groups](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock -groups](#)

[report_clock_crossing](#)

[remove_clock_groups](#)

[remove_clock](#)

[remove_generated_clock](#)

Rule: CGR_0004

Clock **clock1** is missing an asynchronous definition with at least one clock in each of the following asynchronous clock pairs.

Severity: Error

Description

Clocks which are not generated directly or indirectly from either of the two asynchronous clocks should have an asynchronous relationship with at least one of them. A clock can't be synchronous with two clocks that are not synchronous to each other.

Risk Associated with Violation

For clocks not declared asynchronous, paths between the clocks will be analyzed, possibly resulting in timing violations being reported for false paths. In addition, for crosstalk delay, the timing window relationship will be preserved on nets in aggressor/victim relationships between the clocks. This may result in optimistic signal integrity analysis.

One Possible Scenario

Consider a scenario where two clocks are declared asynchronous. Another clock that is not generated from either of the two clocks is not declared as asynchronous to either one:

```
create_clock -period 4 clk1
create_clock -period 4 clk2
create_clock -period 4 clk3
set_clock_group -asynchronous -group {clk1} -group {clk2}
```

The clocks, clk1 and clk2 are asynchronous to each other. The third clock, clk3, should have an asynchronous relationship to either clk1 or clk2. Because clk3 does not have an asynchronous relationship with either clk1 or clk2, a **CGR_0004** violation will be issued.

What Next

The "Violation Details" section will give you the names of the clocks pairs which are declared asynchronous. The reported clock should be declared asynchronous to at least one clock in each asynchronous clock pair. You can click on the clock name(s) to see the relevant clock definition, and the location of the problematic clock group definition is also provided.

Fix Suggestion

To resolve this issue, consider declaring the reported clock as asynchronous to at least one clock in each asynchronous clock pair.

See Also

[set_clock_groups](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock -groups](#)

[report_clock_crossing](#)

[remove_clock_groups](#)

[remove_clock](#)

[remove_generated_clock](#)

Rule: CGR_0005

Clock **clock1** is generated directly or indirectly from the clock **clock2**. They should not be declared as physically exclusive.

Severity: Error

Description

Clocks which are generated from one another cannot be physically exclusive.

Risk Associated with Violation

Paths between the clocks declared as physically exclusive will not be analyzed. An incomplete timing analysis may be performed on the design. In addition, for crosstalk delay, it is assumed that there is no possible crosstalk interaction on nets in aggressor/victim relationships between the clocks. This may result in optimistic signal integrity analysis.

One Possible Scenario

Consider a scenario where a generated clock and its master clock are declared physically exclusive:

```
create_clock -period 4 clk1
create_generated_clock -source clk1 -divide_by 2 clk2
set_clock_group -physically_exclusive -group {clk1} -group {clk2}
```

The generated clock, clk2, has clock, clk1, as its master. The two clocks cannot be physically exclusive because they are physically related to each other, and a **CGR_0005** violation will be issued.

What Next

The "Violation Details" section will give you the names of the related clocks that are declared as physically exclusive. You can click on the clock name(s) to see the relevant clock definition, and the location of the problematic clock group definition is also provided.

Fix Suggestion

To resolve this issue, consider adjusting the clock grouping relationship of the two clocks. In the example above, this can be achieved by removing the following command:

```
set_clock_group -physically_exclusive -group {clk1} -group {clk2}
```

Alternatively, you might want to change the master/generated clock relationship of the reported clocks.

See Also

[set_clock_groups](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock -groups](#)

[report_clock_crossing](#)

[remove_clock_groups](#)

[remove_clock](#)

[remove_generated_clock](#)

Rule: CGR_0006

Clock **clock1** and clock **clock2** are defined physically exclusive with each other. There are clocks generated from these clocks which are not defined as physically exclusive.

Severity: Warning

Description

If two clocks are physically exclusive, then all the clocks generated from these clocks should be physically exclusive with each other. A generated clock does not inherit the

physically exclusive properties of its master clock, so it should be declared physically exclusive to clocks that have this relationship to its master clock.

Risk Associated with Violation

For clocks not declared physically exclusive, paths between the clocks will be analyzed, possibly resulting in timing violations due to false paths. In addition, for crosstalk delay, the timing window relationships will be preserved on nets in aggressor/victim relationships between the clocks. This may result in pessimistic signal integrity analysis because physically exclusive clocks assume that there is no possible crosstalk interaction on nets with this relationship.

One Possible Scenario

Consider a scenario where two master clocks are declared physically exclusive, while no physically exclusive relationship is specified between clocks derived from the master clocks:

```
create_clock -period 4 clk1
create_clock -period 4 clk2
create_generated_clock -source clk1 -divide_by 2 clk3
create_generated_clock -source clk2 -divide_by 4 clk4
set_clock_groups -physically_exclusive -group {clk1} -group {clk2}
```

Two master clocks, clk1 and clk2, are physically exclusive. The two generated clocks, clk3 and clk4, should also be physically exclusive with each other because this is the relationship between their master clocks. Since the generated clocks clk3 and clk4 do not have a physically exclusive relationship, a CGR_0006 violation will be issued.

What Next

The "Violation Details" section will give you the names of the clocks pairs which should have been declared physically exclusive. You can click on the clock names to see the relevant clock definition, and the location of the problematic clock group definition is also provided.

Fix Suggestion

To resolve this issue, consider declaring the clock pairs as physically exclusive. In the example above, the correct physically exclusive relationship can be declared by changing the [set_clock_groups](#) command as follows:

```
set_clock_groups -physically_exclusive -group {clk1 clk3} \
    -group {clk2 clk4}
```

Alternatively, you might change the master/generated clock relationship of the reported clocks.

See Also

[set_clock_groups](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock -groups](#)

[report_clock_crossing](#)

[remove_clock_groups](#)

[remove_clock](#)

[remove_generated_clock](#)

Rule: CGR_0007

Multiple clocks or a clock with either [set_input_delay](#) or [set_output_delay](#) are defined at pin/port ***pin_port***. In both cases, (the defined multiple clocks or a defined clock with the reference clock), these clocks should be defined as exclusive/asynchronous.

Severity: Warning

Description

Clocks originating from the same source should be set as physically exclusive.

Risk Associated with Violation

The tools will conservatively analyze all potential coupling conditions between the clocks that are not declared asynchronous or physically exclusive. This may result in poor crosstalk analysis performance and over-conservative analysis result.

Possible Scenarios

Consider a scenario where two clocks are created on the same port, but no physically exclusive relationship is specified:

```
create_clock -per 4 -name clk1 clk1
create_clock -per 4 -name clk1_dup -add clk1
```

Consider another scenario where a clock is defined at a port and a [set_input_delay](#) is also specified, but no physically exclusive relationship is specified between the defined clock and the reference clock:

```
create_clock -per 5 -name jtag_clk jtag_clk  
set_output_delay 1.4 -clock mclk { jtag_clk }
```

What Next

The "Violation Details" section will give you the names of the clock pairs which should be declared as physically exclusive.

Fix Suggestion

To resolve this issue, consider declaring the clock given pairs as physically exclusive. In the examples above, this can be achieved with the following command:

```
set_clock_group -physically_exclusive -group {clk1} -group {clk1_dup}
```

See Also

[report_clock_crossing](#)

[create_clock](#)

[set_clock_groups](#)

[remove_clock_groups](#)

Rule: CGR_0008

Multiple clocks defined at pin/port **pin_port** have false path set between them. For accurate signal integrity results use the **set_clock_groups** command.

Severity: Warning

Description

Multiple clocks with false path relationships have been defined at the same source. It is recommended that a clock group relationship be defined in place of the individual false path exceptions. Defining a clock group relationship will disable the timing paths between clocks while allowing more accurate delay calculations in SI mode with PrimeTime.

Risk Associated with Violation

This violation may result in less accurate SI analysis results. Modeling the clocks interactions with [set_clock_groups](#) will allow the following nuances in SI delay calculations:

- Two clocks defined to be **logically exclusive** of each other have no logical timing paths between them. PrimeTime does not check the logical timing between the clocks, but PrimeTime SI still checks for possible crosstalk interaction between them.

- Two clocks defined to be **physically exclusive** of each other have no logical timing paths between them, and furthermore, are considered physically isolated from each other. PrimeTime does not check the logical timing between the clocks and PrimeTime SI assumes no possible crosstalk interaction between them.
- Two clocks defined to be **asynchronous** with each other have no timing relationship at all. PrimeTime does not check the logical timing between the clocks, but PrimeTime SI still checks for possible crosstalk interaction between them, using infinite arrival windows.

One Possible Scenario

Consider a scenario where two clocks are created on the same port, and [set_false_path](#) relationships are specified between the two clocks:

```
create_clock -per 4 -name clk1 clk1
create_clock -per 4 -name clk1_dup -add clk1
set_false_path -from clk1 -to clk1_dup
set_false_path -from clk1_dup -to clk1
```

What Next

The "Violation Details" section will give you the names of the clocks that are created on the same port with [set_false_path](#) relationships specified.

Fix Suggestion

To resolve this issue, consider using [set_clock_groups](#) to replace [set_false_path](#) between clocks. Below is an example of the command to achieve this:

```
set_clock_groups -physically_exclusive -group {clk1} -group {clk1_dup}
```

See Also

[report_clock_crossing](#)

[set_clock_groups](#)

[remove_clock_groups](#)

Rule: CGR_0009

Clocks **clock1** and **clock2** have valid timing paths between them, but their common base period is more than the limit times the clock period.

Severity: Error

Description

These two clocks should be mutually exclusive even though a valid timing path exists between them. The clocks can be either asynchronous, logically exclusive, or physically exclusive. A CGR_0009 rule violation is triggered when one of the following conditions is met:

- The common base period of those two clocks is greater than the **max_period_multiplier** property multiplied by the period of the larger of the two clock periods.
- The **min_period_multiplier** property multiplied by the period of the smaller of the two clock periods.

The common base period is the least common multiple between the two clock periods.

If you change the **skip_clocks_with_MCP** property, the tool will not issue CGR_0009 rule violations if there are clock-to-clock multicycle paths between the clocks. Use the [set_rule_property](#) command to change the value of properties.

Risk Associated with Violation

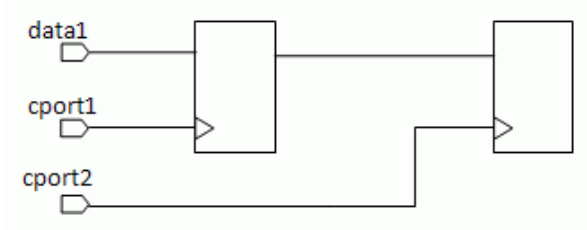
For clocks not defined as asynchronous in the [set_clock_groups](#) command, the timing paths between the two clocks will be analyzed, and that can result in large timing violations caused by false paths.

One Possible Scenario

Consider a scenario in which two functional mode constraints are merged into a single set of constraints. The first functional mode has clocks VCLK1, CLK1, and CLK2. The second functional mode has clocks VCLK1_test, CLK1_test, and CLK2_test. The two virtual clocks VCLK1 and VCLK1_test are missing from the clock group definition.

```
create_clock -name VCLK1 -period 3.1
create_clock -name VCLK1_test -period 11.07
create_clock -name CLK1 -period 3.1 [get_ports cport1] -add
create_clock -name CLK2 -period 6.2 [get_ports cport2] -add
create_clock -name CLK1_test -period 11.07 [get_ports cport1] -add
create_clock -name CLK2_test -period 22.14 [get_ports cport2] -add
set_clock_groups -group {CLK1 CLK2} -group {CLK1_test CLK2_test}
    -physically_exclusive
set_input_delay 1.23 -clock VCLK1 -add [get_port data1]
set_input_delay 4.23 -clock VCLK1_test -add [get_port data1]
```

Figure 5 *set_input_delay Virtual Clocks Omitted from Clock Group*



Because the clock group for VCLK1 and VCLK1_test has not been defined, **CGR_0009** violations will be issued:

CGR_0009: Clocks VCLK1 and CLK1_test have valid timing paths between them, but their common base period is more than the limit times the clock period.

CGR_0009: Clocks CLK1 and VCLK1_test have valid timing paths between them, but their common base period is more than the limit times the clock period.

What Next

The "Violation Details" section identifies the clocks that should be declared asynchronous and the associated clock periods. You can review the clock names for the relevant clock definitions or review the clock periods using the [report_clock](#) command. To review the paths between the clocks you can use the [analyze_paths](#) command:

```
ptc_shell> analyze_paths -from clock1 -to clock2 -path_type summary
ptc_shell> analyze_paths -from clock2 -to clock1 -path_type summary
```

Summary of Paths:

Dominant	Overridden	Count	Clocks		
Startpoint	Endpoint	Constraint	Constraints	(r,f,R,F)	Launch/ Capture
data1 (in)	ff1a/D (FD1)			(1,1,1,1)	VCLK1_test/ CLK1

Fix Suggestion

To resolve the scenario issue described previously, add the clocks, VCLK1 and VCLK1_test, to separate clock groups and define the clock pair as asynchronous, as shown in the following command:

```
set_clock_groups -group {CLK1 CLK2 VCLK1} \
-group {CLK1_test CLK2_test VCLK1_test} -asynchronous
```

Rule Property

max_period_multiplier

A **CGR_0009** violation is issued when the base period between the two clocks is greater than this property multiplied by the larger of the two clock periods. Increasing or decreasing the value of this property decreases or increases the number of **CGR_0009** violations. The default for this parameter is 101.

Note that the base period expansion is also affected by the **min_period_multiplier** property. The violation is issued only if the common base period exceeds the smaller of the following two values:

- max_period_multiplier times larger clock period
- min_period_multiplier times smaller clock period

min_period_multiplier

A **CGR_0009** violation is issued when the base period between the two clocks is greater than this property multiplied by the smaller of the two clock periods. Increasing or decreasing the value of this property decreases or increases the number of **CGR_0009** violations. The default for this parameter is 1000.

Note that the base period expansion is also affected by the **max_period_multiplier** property. The **min_period_multiplier** value should be set to a number greater than or equal to the **max_period_multiplier** value. If not, the **max_period_multiplier** value is used as the multiplication factor for smaller periods as well. The violation is issued only if the common base period exceeds the smaller of the following two values:

- max_period_multiplier times larger clock period
- min_period_multiplier times smaller clock period

skip_clocks_with_MCP

By default, a violation is issued for all pairs of interacting clocks that satisfy the period checks described previously. However, if there are MCPs between the two clocks, they might not cause timing violations even if the clocks are not defined as asynchronous or physically exclusive. You can avoid issuing violations in this situation by enabling the **skip_clocks_with_MCP** rule property.

Note:

It is recommended that the clocks be declared asynchronous or physically exclusive. This ensures that the paths between the two clocks are not timed and also takes care of signal integrity effects.

See Also

[set_clock_groups](#)

[report_clock](#)

[analyze_paths](#)

Rule: CGR_0010

Clocks **clock1** and **clock2** might be interacting, but their common base period is more than the **limit** times the clock period.

Severity: Error

Status: Disabled by default

Description

The two clocks are likely to be mutually exclusive but are not specified as such. The clocks can be asynchronous or physically exclusive. Note that this rule is an extension of the **CGR_0009** rule where all pairs of clocks that are not explicitly defined as asynchronous or physically exclusive are considered for base period expansion. This is because even if valid timing paths do not exist between a pair of clocks, they might still require base clock expansion for signal integrity analysis, therefore this rule is a part of the `primetime_si` ruleset. This rule is disabled by default. A common base period is the least common multiple between the two clock periods. This violation is issued if the common base period is larger than the smallest of the following two values:

- the **max_period_multiplier** property multiplied by the period of the larger of the two clock periods
- the **min_period_multiplier** property multiplied by the period of the smaller of the two clock periods

The value of the properties can be changed with the [set_rule_property](#) command.

Risk Associated with Violation

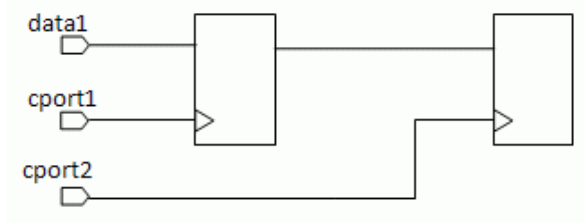
For clocks not defined as asynchronous in the [set_clock_groups](#) command, the timing paths between the two clocks will be analyzed, and that can result in large timing violations due to false paths. Also, for signal integrity analysis, the base period expansion might be clipped to a large limit and can lead to large runtimes. For clocks not defined as physically exclusive in the [set_clock_groups](#) command, the signal integrity analysis will still be considered which can lead to pessimistic timing results causing timing violations.

One Possible Scenario

Consider a scenario in which two functional mode constraints are merged into a single set of constraints. The first functional mode has clocks VCLK1, CLK1, and CLK2. The second functional mode has clocks VCLK1_test, CLK1_test, and CLK2_test. The two virtual clocks VCLK1 and VCLK1_test are missing from the clock group definition.

```
create_clock -name VCLK1 -period 3.1
create_clock -name VCLK1_test -period 11.07
create_clock -name CLK1 -period 3.1 [get_ports cport1] -add
create_clock -name CLK2 -period 6.2 [get_ports cport2] -add
create_clock -name CLK1_test -period 11.07 [get_ports cport1] -add
create_clock -name CLK2_test -period 22.14 [get_ports cport2] -add
set_clock_groups -group {CLK1 CLK2} -group {CLK1_test CLK2_test}
    -physically_exclusive
set_input_delay 1.23 -clock VCLK1 -add [get_port data1]
set_input_delay 4.23 -clock VCLK1_test -add [get_port data1]
```

Figure 6 *set_input_delay Virtual Clocks Omitted from Clock Group*



Because the clock group for VCLK1 and VCLK1_test is not defined, **CGR_0010** violations are issued:

CGR_0010: Clocks VCLK1 and CLK1_test might be interacting, but their common base period is more than the limit times the clock period.

CGR_0010: Clocks CLK1 and VCLK1_test might be interacting, but their common base period is more than the limit times the clock period.

What Next

The "Violation Details" section identifies the clocks that should be declared asynchronous along with their periods. You can review the clock name or names for the relevant clock definition or review the clock periods using the [report_clock](#) command.

Fix Suggestion

To resolve the scenario issue described previously, add the clocks, VCLK1 and VCLK1_test, to separate clock groups and define the clock pair as physically exclusive. The example syntax below shows the clock definition using the [set_clock_groups](#) command.

```
set_clock_groups -group {CLK1 CLK2 VCLK1} \  
-group {CLK1_test CLK2_test VCLK1_test} -physically_exclusive
```

The following clock group definition also eliminates the **CGR_0010** violation, but it can lead to pessimistic signal integrity results and should not be used.

```
set_clock_groups -group {CLK1 CLK2 VCLK1} \  
-group {CLK1_test CLK2_test VCLK1_test} -asynchronous
```

Rule Properties

max_period_multiplier

A **CGR_0010** violation is issued when the base period between the two clocks is greater than this property multiplied by the larger of the two clock periods. Increasing or decreasing the value of this property will decrease or increase the number of **CGR_0010** violations. The default for this parameter is 101.

Note that the base period expansion is also affected by the **min_period_multiplier** property. The violation is issued only if the common base period exceeds the smaller of the following two values:

- max_period_multiplier times larger clock period
- min_period_multiplier times smaller clock period

min_period_multiplier

A **CGR_0010** violation is issued when the base period between the two clocks is greater than this property multiplied by the smaller of the two clock periods. Increasing or decreasing the value of this property will decrease or increase the number of **CGR_0010** violations. The default for this parameter is 1000.

Note that the base period expansion is also affected by the **max_period_multiplier** property. The **min_period_multiplier** value should be set to a number greater than or equal to the **max_period_multiplier** value. If not, the **max_period_multiplier** value is used as the multiplication factor for the smaller period as well. The violation is issued only if the common base period exceeds the smaller of the following two values:

- max_period_multiplier times larger clock period
- min_period_multiplier times smaller clock period

See Also

[set_clock_groups](#)

[report_clock](#)

[analyze_paths](#)

Rule: CLK_0001

Clock **clock** has **count** sources on inout ports

Severity: Warning

Description

The circuit behavior of an inout port can be unpredictable if it has a clock signal driving it. If the port is functioning as an output while the clock is driving, bus contention occurs. If the clock is a three-state condition, any registers in the fanout of the clock can lose state.

Risk Associated with Violation

If the inout port is operating only in input mode for this scenario, there is no risk. It is also possible that the clock is defined on the incorrect port, or the port is defined with the wrong direction.

One Possible Scenario

Consider the scenario where a clock is defined on an inout port:

```
create_clock bidirClock -period 2
```

Since the port is bidirectional (or inout), and a clock has been defined on it, a **CLK_0001** rule violation will be issued.

What Next

Verify that the circuit is safe according to your design rules. Synopsys tools assume that the clock is valid. For the example scenario, you can verify the direction of the port with the [get_attribute](#) command:

```
get_attribute [get_port bidirClock] direction
```

You can also use the [analyze_clock_networks](#) command to view the clock network from the port of interest to assess how the clock propagates in your design. For the example scenario, the following command can be used:

```
ptc_shell> analyze_clock_networks -traverse_disabled -style full_expanded  
  \ -from [get_port bidirClock]
```

Clock Sense Abbreviations:

P - positive

Clock Network End Type Abbreviations:

REG - register

PORT - port

CG - clock_gating

Full report for clock: bidirClock - 2 partial path branches

Branch 0:

Branch

Level Info Sense Notes Port/Pin

0 P PORT, source

bidirClock (inout)

Branch 1:

Branch

Level Info Sense Notes Port/Pin

0 P source bidirClock (inout)

1 P CG ckin/A (BTS4)

2 P ckin/Z (BTS4)

3 P REG div2/CP (FD1)

The report shows that the clock is defined on an inout port and propagates to an internal register in the design.

Fix Suggestion

There are several possibilities for resolving this rule violation. If the inout port is only operating in input mode for the scenario of interest, no change may be needed. If the port has the wrong direction, the Verilog module related to the port can be updated. Last, if the clock is defined on the incorrect port, it can be moved to the correct port for your design. For the example scenario, based on knowledge of the design, the bidirClock port only acts as an input port for the design in this mode, so no change is needed.

See Also

[create_clock](#)

[get_attribute](#)

[analyze_clock_networks](#)

Rule: CLK_0002

Generated clock **clock** has **count** sources on input or inout ports.

Severity: Warning

Description

When creating a generated clock on an input or inout port, the generated clock will only derive its waveform characteristics from the master clock. Delay is derived from delay information at the port itself. Additionally, creating a generated clock to drive an inout port can cause unpredictable circuit behavior. If the port is functioning as an output while the clock is driving, bus contention occurs. If the clock is a three-state condition, any registers in the fanout of the clock can lose state.

Risk Associated with Violation

In post-layout conditions, a source latency for the generated clock cannot be calculated from the master clock. This may lead to inaccuracies in the timing checks related to the generated clock.

One Possible Scenario

Consider the scenario where a generated clock is defined on an input port:

```
create_clock CLK1 -period 2
create_generated_clock -source CLK1 -name gen_clk [get_ports CLK2]
```

Since the generated clock was defined on input port CLK2, a **CLK_0002** rule violation will be issued.

What Next

Verify that the circuit is safe according to your design rules. Synopsys tools assume that the generated clock is valid. For the example scenario, you can examine the generated clock definition by clicking on the name of the generated clock in the 'Violation Details'. You can use the [report_port](#) command to verify the direction of the port specified in the "Violation Details":

```
ptc_shell> report_port [get_ports CLK2]
```

Port	Dir	Pin Cap		Wire		Attributes
		Min rise/fall	Max rise/fall	Min rise/fall	rise/fall	
CLK2	in	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.	

The report shows that CLK2 is an input port, so the generated clock has been defined on an input port. You can also use the [analyze_clock_networks](#) command to view the clock network from the port or the clock network associated with the master clock to assess the clock network in your design and potentially identify an internal pin for the source of the generated clock.

Fix Suggestion

There are several possibilities for resolving this rule violation. If the waveform characteristics are the only thing required from the master clock, and the delay characteristics of the port are to be used for the generated clock, then no change may be needed.

If the port is an inout port and has the wrong direction, the Verilog module related to the port can be updated.

Last, if the generated clock is defined incorrectly on a port, it can be moved to the correct pin for your design. For the example scenario, based on knowledge of the design, the generated clock was incorrectly defined on the port and the constraint was changed to an internal pin of the design:

```
create_generated_clock -source CLK1 -name gen_clk [get_pins ff1/Q]
```

See Also

[create_clock](#)

[create_generated_clock](#)

[report_port](#)

[analyze_clock_networks](#)

[get_attribute](#)

Rule: CLK_0003

Generated clock **clock** is not expanded because it has no clock reaching its master source **pin**.

Severity: Error

Description

The pin or port given for the `-source` option of this generated clock has no clock, therefore, no master clock can be determined for it. A generated clock without a master clock has no waveform, and it is not propagated forward to register clock pins.

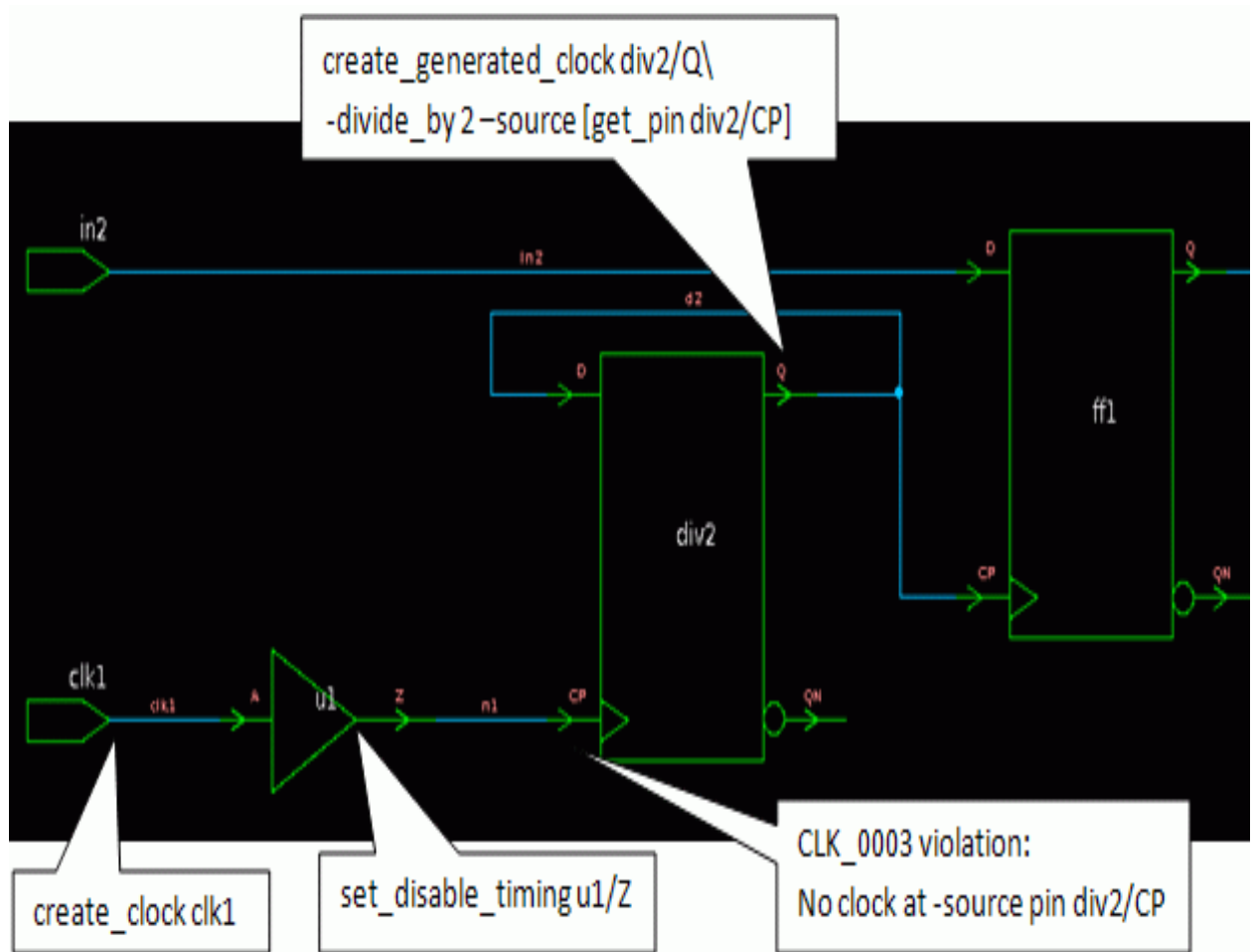
Risk Associated with Violation

The portion of the circuit that should be clocked by this generated clock is untested, resulting in incomplete timing analysis on the design.

One Possible Scenario

This example scenario has a clock divider, *div2*. The constraints have a generated clock definition at the output, *div2/Q*, with the *div2/CP* clock pin, as the **-source** pin. No **-master_clock** option has been given. A CLK_0003 rule violation will be issued if no clock propagates to the *div2/CP* pin. (A CLK_0028 violation will be issued if there are multiple clocks at *div2/CP*.) In this example scenario, there is no clock at the *div2/CP* pin because of the [set_disable_timing](#) constraint at the *U1/Z* pin.

Figure 7 Clock Divider With No -master_clock Option



What Next

Determine why there is no clock at the **-source** pin or if an incorrect **-source** pin was specified in the `create_generated_clock` command. If the **-source** pin is correct, then the clock may be missing because:

- The master clock has not been defined.
- The master clock is blocked from the **-source** pin by another constraint.

To determine whether the master clock has been blocked, issue the command:

```
analyze_clock_networks -through <-source pin> -traverse_disabled
```

For this example scenario, the command would be:

```
ptc_shell> analyze_clock_networks -through div2/CP -traverse_disabled
```

Example	End Pin/Port Count	End Type Clocks	Potential Constraints
---------	-----------------------	--------------------	--------------------------

div2/CP (FD1)	1	REG	set P0 BK_0
---------------	---	-----	-------------

Potential Clock Sets:

```
set P0 (1 clocks):
  clk1 - positive
```

Clock Blocking Constraints:

```
BK_0:
  DT#0 = set_disable_timing
    at pin: u1/Z
    constraints.tcl, line 3
```

The report shows that the master clock, **clk1**, is blocked by the [set_disable_timing](#) constraint at the **u1/Z**. pin. The file and line number of the [set_disable_timing](#) constraint is given.

Fix Suggestion

Add the missing master clock, remove the constraints blocking the correct master clock, or move the **-source** pin to the correct location.

For the preceding example scenario, the [set_disable_timing](#) constraint can be removed by issuing the command:

```
remove_disable_timing u1/Z
```

Or the [set_disable_timing](#) command can be removed from the constraint file.

Corresponding PrimeTime Error Message

Error Message: PTE-023

Severity: Warning

Description

The generated clock '%s' has not been expanded. Create or activate its master clock.

See Also

[create_generated_clock](#)

[analyze_clock_networks](#)

Rule: CLK_0004

Mismatch between generated clock definitions at *pin_or_port* and potential master clocks.

Severity: Warning

Description

At the given pin or port location, there should be a generated clock (specified with the **-add** and **-master_clock** options) for each clock signal meeting the following requirements:

- Present on the master sources
- Propagating through source latency paths to this pin or port

The violation details section lists the master clocks with missing generated clock statements at the reported pin or port location.

This rule looks for potential masters as follows:

1. All the pins provided through the **-source** option of all generated clocks on the violated pin or port.
2. Any clock blocked at the violated pin or port by the generated clock definitions.
3. Any register clock pin in the source latency paths of the existing generated clocks and in the fanout of the master clocks of those generated clocks.
4. Any generated clocks defined in the source latency path of the target generated clock with the same master clock. This portion of the rule check is controlled by the rule property **warn_on_generated_clocks_of_same_master**

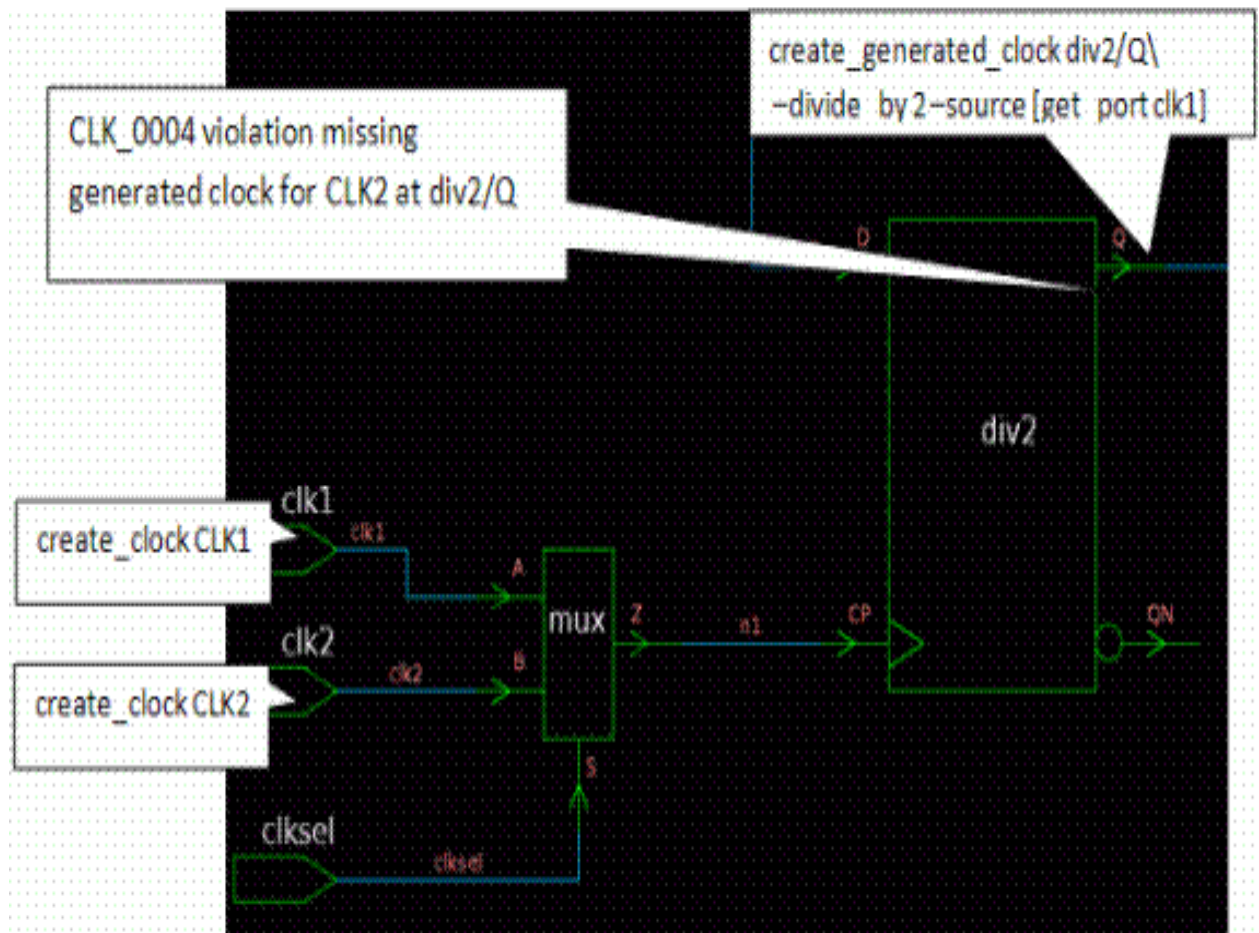
Risk Associated with Violation

The register clock pins in the fanout of the given pin will be missing some of the clock waveforms that are physically possible for the circuit. This may result in a circuit that has not been completely verified and potentially create unbalanced clock trees during clock tree synthesis.

One Possible Scenario

Consider a clock divider, *div2*, with two clocks, *CLK1* and *CLK2* propagating to the clock pin of the clock divider. Only one generated clock is created on the output of the clock divider with *CLK1 as its master clock*. Since two clocks are actually propagating to this location (*CLK1* and *CLK2*), a CLK_0004 rule violation will be issued for the *div2/Q* pin. The "Violation Details" section reports that potential *CLK2* master clock exists at *div2/CP*, but it is not used when creating a generated clock at *div2/Q*.

Figure 8 Only One Generated Clock Created on Output of Clock Divider



What Next

Examine the "Violation Details" section to find which generated clocks exist at the violated pin or port. This section also mentions which potential master clocks are missing from the generated clock statements.

To see the source latency paths of any of the existing generated clocks you can use the [analyze_clock_networks](#) command as seen here:

```
ptc_shell> analyze_clock_networks -to gclk1 -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source }
Design : CLK_0004
Scenario: default
```

```
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
  P - positive
  R - rise_triggered
Clock Network End Type Abbreviations:
  REG - register
  CS - clock_source

Source latency paths for generated clock: gclk1
from master clock: clk1
```

```
Branch 0:
      Branch
Level  Info      Sense Notes      Port/Pin
-----
      0          P      source      clk1 (in)
      1          P          mux/A (MUX21H)
      2          P          mux/Z (MUX21H)
      3          P      REG      div2/CP (FD1)
      4          P          div2/CP (FD1)
      5          R      CS      div2/Q (FD1)
```

You can also report what clocks are in the source latency of the register missing generated clocks using:

```
ptc_shell> analyze_clock_networks -to div2/CP -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source }
Design  : CLK_0004
Scenario: default
Version: ...
Date    : ...
*****
Clock Sense Abbreviations:
  P - positive
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: clk1

```
Branch 0:
      Branch
Level  Info      Sense Notes      Port/Pin
-----
      0          P      source      clk1 (in)
      1          P          mux/A (MUX21H)
      2          P          mux/Z (MUX21H)
```

```

3          P      REG      div2/CP (FD1)

Full report for clock: clk2

Branch 0:
  Branch
Level  Info      Sense Notes      Port/Pin
  0          P      source    clk2 (in)
  1          P          mux/B (MUX21H)
  2          P          mux/Z (MUX21H)
  3          P      REG      div2/CP (FD1)

```

Fix Suggestion

Add the missing generated clocks to the pin or port as shown here:

```

create_generated_clock DIV2/q -add \
    -name gclk2 \
    -source [get_port clk2] \
    -master_clock clk2 \
    -divide_by 2

```

Rule Property: warn_on_generated_clocks_of_same_master

This rule property controls whether violations for this rule are issued if there are generated clocks defined in the source latency path of the generated clock that have the same master clock. The default for this rule property is **true**, which means a violation for this rule is issued for this condition.

See Also

[create_generated_clock](#)

[analyze_clock_networks](#)

Rule: CLK_0005

Virtual clock **clock** has the same name as a design object.

Severity: Info

Description

A virtual clock was defined with the same name as a port, pin, or net, and might not be what was intended.

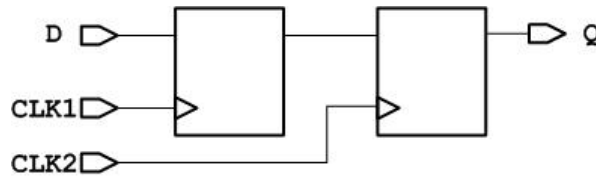
Risk Associated with Violation

This kind of virtual clock, if intended to be an actual clock in the design, can lead to unconstrained paths in the design and incorrect constraints. Some constraints related to the virtual clock can be ignored and result in unrealistic timing violations.

One Possible Scenario

Consider a simple scenario where the clock is mistakenly defined as a virtual clock.

Figure 9 Clock Mistakenly Defined as a Virtual Clock



```
create_clock -name CLK1 -period 2
create_clock -name CLK2 -period 1 [get_ports CLK2]
set_multicycle_path 2 -from [get_clocks CLK1] -to [get_clocks CLK2]
```

The 'CLK1' clock is a virtual clock that is not associated with any clock source port or pin. In this case, the following warning message is also issued:

```
Warning: Creating virtual clock named 'CLK1' with no sources. (UIC-021)
```

The design is unconstrained because there is no clock signal associated with the 'CLK1' port. In addition, the multicycle path exception from 'CLK1' to 'CLK2' is ignored because the 'CLK1' virtual clock is not associated with the 'CLK1' clock port.

Fix Suggestion

Change the virtual clock to a non-virtual clock or change the virtual clock name such that the clock name differs from an existing port, pin or net name in the design.

In the example scenario, the virtual clock can be made into a non-virtual clock as follows:

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
set_multicycle_path 2 -from [get_clocks CLK1] -to [get_clocks CLK2]
```

Now the design is properly constrained and the multicycle path exception will be applied correctly.

See Also

[create_clock](#)

Rule: CLK_0006

Clock **clock** source **source** has a logic constant or case value.

Severity: Warning

Description

A clock source has been defined at a port or pin that has a constant value due to a logic constant or [set_case_analysis](#) constraint. The constant value will suppress the propagation of the clock waveform.

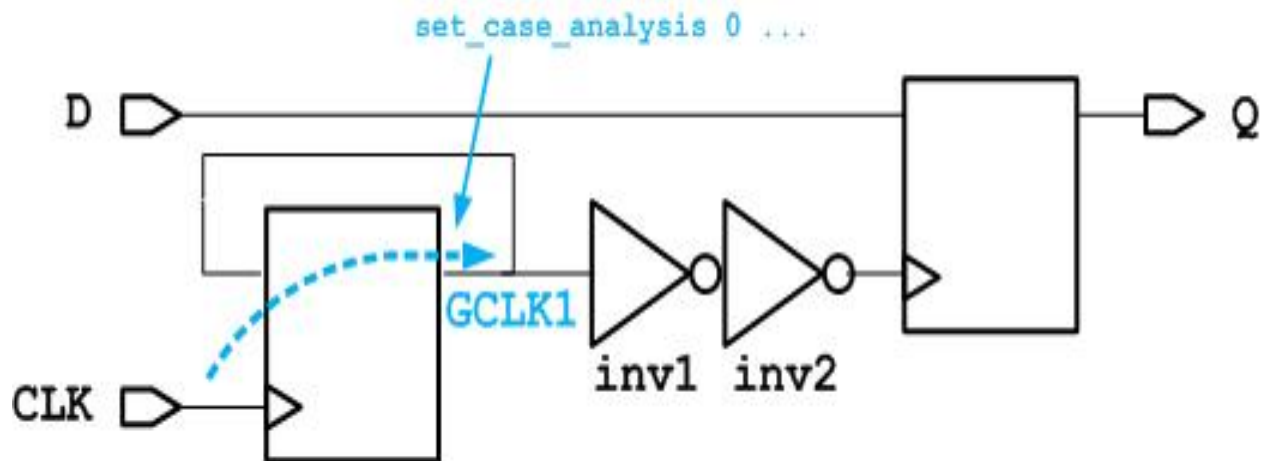
Risk Associated with Violation

A constant value at a clock source will deactivate the clock, and no timing analysis will be performed for the portion of design that is affected by the clock.

One Possible Scenario

Consider a simple scenario involving a generated clock at a register output. The generated clock 'GCLK1' is defined on the output pin of a register, and this pin's value is set to 0 by [set_case_analysis](#). This constant suppresses the propagation of the generated clock and deactivates timing analysis of the downstream logic affected by 'GCLK1'.

Figure 10 Generated Clock at Register Output



```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 -source [get_ports CLK] -divide_by 2
[get_pins ff1/Q]
set_case_analysis 0 [get_pins ff1/Q]
```

Because the generated clock is defined at a pin with a constant value, a CLK_0006 rule violation will be issued.

What Next

The violation message gives the name of the clock and the clock source pin. To further identify the root cause of the constant arriving at the source pin, use the [report_case_details](#) command. This command can show how logic constants and user case values propagate to the clock source pin.

For the example scenario, the command can be issued as follows:

```
ptc_shell> report_case_details -to [get_pins ff1/Q]
```

As can be seen in the following report excerpt, the clock source pin has a constant value of 0 due to a user-specified [set_case_analysis](#).

Properties	Value	Pin/Port

user case	0	ff1/Q

Fix Suggestion

If the intent is not to suppress the clock signal, remove the logic constant or the user-specified case value that caused the clock source pin to assume a constant value.

In the example scenario, this can be done by simply removing the [set_case_analysis](#) constraint:

```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 -source [get_ports CLK] -divide_by 2
[get_pins ff1/Q]
set_case_analysis 0 [get_pins ff1/Q]
```

This will allow generated clock 'GCLK1' to propagate.

See Also

[create_generated_clock](#)

[create_clock](#)

[set_case_analysis](#)

[report_case_details](#)

Rule: CLK_0007

Generated clock **clock** master source **source** has a logic constant or case value.

Severity: Error

Description

The master source (a port or a pin) of the reported generated clock has a constant value due to logic or [set_case_analysis](#) constraint. The constant value will suppress the propagation of the generated clock waveform.

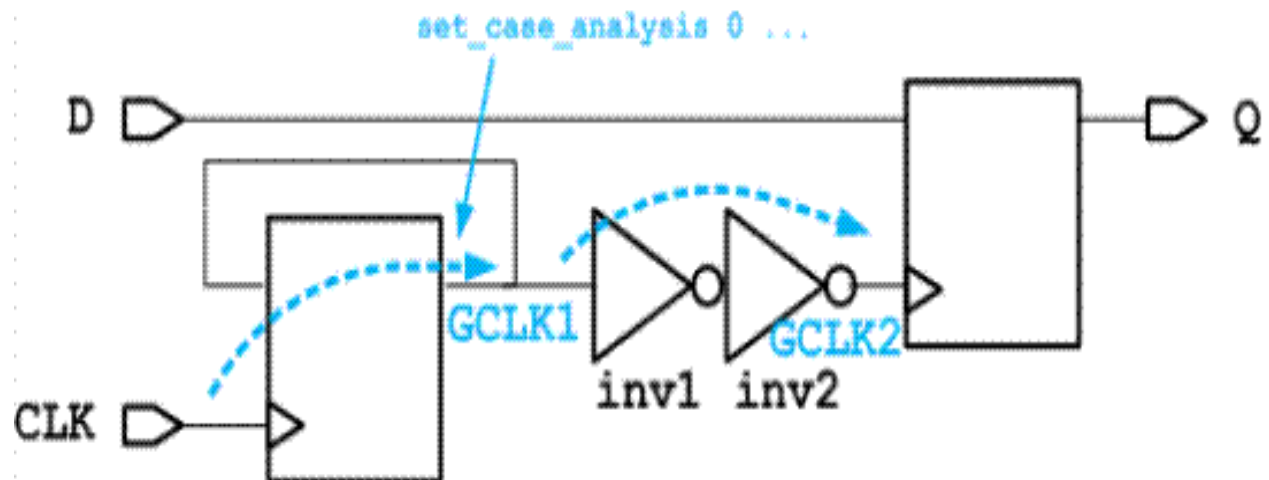
Risk Associated with Violation

Constant value at the master clock source will deactivate the generated clock and no timing analysis will be performed for the portion of design that is affected by the generated clock. This could result in an incompletely timed design.

One Possible Scenario

Consider a simple scenario involving two generated clocks. The first generated clock's ('GCLK1') source pin is defined at the output of a register whose value is set to 0 by [set_case_analysis](#). The second generated clock ('GCLK2') has its master clock source pin defined at the output of the first register. This constant master clock source suppresses the clock propagation and de-activates the timing analysis of the downstream logic affected by the generated clock 'GCLK2'.

Figure 11 Two Generated Clocks



```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 -source [get_ports CLK] -divide_by 2
[get_pins ff1/Q]
create_generated_clock -name GCLK2 -source [get_pins ff1/Q] -divide_by 1
[get_pins inv2/Z]
set_case_analysis 0 [get_pins ff1/Q]
```

What Next

The violation message gives the name of the clock and its master clock source pin. To identify further the root cause of the constant arriving at the master clock source pin, use the [report_case_details](#) command. This command shows how logic constants and user case values propagate to the master clock source pin.

For the example scenario, the command can be issued as follows:

```
ptc_shell> report_case_details ff1/Q
```

As can be seen in the report excerpt below, the clock source pin has a constant value of 0 due to user case.

Properties	Value	Pin/Port
user case	0	ff1/Q

To analyze how the constant value propagates to the source pin of the generated clock 'GCLK2', supply the generated clock source pin in the **-to** option of the [report_case_details](#) command:

```
ptc_shell> report_case_details -to inv2/Z
```

Properties	Value	Pin/Port
from user case	0	inv2/Z

Case fanin report:

Verbose Source Trace for pin/port inv2/Z:

Path number: 1

Path Ref # Value Properties Pin/Port

```

0  F()=A' inv2/Z
1  inv2/A
1  F()=A' inv1/Z
0  inv1/A
0  user case ff1/Q

```

Fix Suggestion

If the intent is not to suppress the generated clock signal, remove the logic constant or the user case that caused the master clock source pin to assume a constant value.

In the example scenario, this can be done by simply removing the [set_case_analysis](#) constraint:

```

create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 -source [get_ports CLK] -divide_by 2
[get_pins ff1/Q]

```

```
create_generated_clock -name GCLK2 -source [get_pins ff1/Q] -divide_by 1  
[get_pins inv2/Z]  
#set_case_analysis 0 [get_pins ff1/Q]
```

See Also

[create_generated_clock](#)

[create_clock](#)

[set_case_analysis](#)

[report_case_details](#)

Rule: CLK_0008

Generated clock **clock** has paths from the sources of master clock **master_clock** to generated clock sources with differing sequential depth.

Severity: Warning

Description

There are several possible latency paths reaching the generated clock. Not all the paths cross the same number of sequential elements and not all the paths may be needed for the latency calculation. The clock latency calculation may become too pessimistic. The generated clock may need to be modified to remove the pessimism.

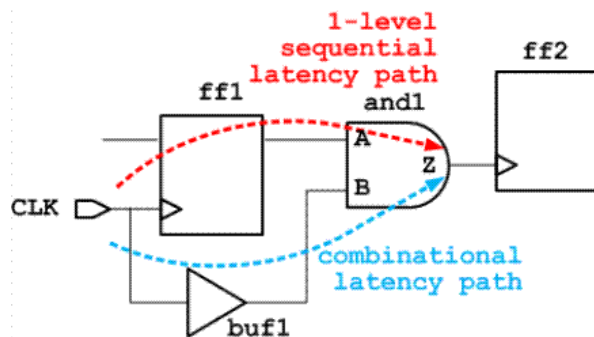
Risk Associated with Violation

If this violation is not addressed, the timing analysis results may be too pessimistic since the timing analysis tool has to account for every path. This can lead to unnecessary increased effort in optimization and timing closure.

One Possible Scenario

Consider a scenario where there are two latency paths from the master clock to a generated clock. One path involves a sequential depth of one and the other path is combinational. In this example, the design intent is to propagate clock through the combinational path and use the sequential path as the signal pin of clock gating logic. However, clock latency calculation involves both latency paths and the “worst” path is chosen for timing analysis. It is likely that the sequential path is chosen for hold latency calculation because a sequential path delay tends to be larger than a combinational path delay. This may lead to undesirable hold violations associated with the generated clock.

Figure 12 Two Latency Paths From Master Clock to Generated Clock



```
create_clock -name sys_clk -period 4 [ get_ports CLK ]
create_generated_clock -name gclk -divide_by 1 -source [get_ports CLK]
[get_pins and1/Z]
```

What Next

The violation message gives the name of the generated clock and the Violation Details section provides the location where the differing sequential depth branches merge. To further identify the source of the problem, use the [analyze_clock_networks](#) command to find out which latency paths are involved from the master clock to the generated clock. The generated clock name can be supplied in the `-to` option and `-style full` option can be used to display full details including:

- multiple paths (“branches”)
- logic levels
- clock sense (“positive”, “negative”, “rise_triggered”, “fall_triggered”)
- sequential CLK-to-Q paths (“REG”).

For the example scenario, the command can be issued as follows:

```
ptc_shell> analyze_clock_networks -to gclk -style full
```

The sequential depth can be obtained by counting the number of paths marked by “REG” in the report. In the following example, the branch 0 has a sequential depth of 1 as it involves one sequential cell and the branch 1 is a combinational path since there are no sequential cells in the path.

Source latency paths for generated clock: gclk
from master clock: sys_clk - 2 partial path branches

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0	S1	P	source	CLK (in)
1		P	REG	ff1/CP (FD1)
2		P		ff1/CP (FD1)
3		R		ff1/Q (FD1)
4		R	D and1/A (AN2)	
5	E1	P,R	CS and1/Z (AN2)	

Branch 1: from branch 0 reconverges to branch 0

Level	Branch Info	Sense	Notes	Port/Pin
1		P		buf1/A (B1I)
2		P		buf1/Z (B1I)
3		P	CG and1/B (AN2)	

Fix Suggestion

Different scenarios may require different approaches. The basic idea is to ensure that the latency path for the generated clock includes only the actual clock propagation path from the master clock to the generated clock. This may be done by using one of the following techniques:

- moving the generated clock source
- creating intermediate generated clocks
- using the [set_clock_sense](#)-stop_propagation command
- using the -combinational option to the [create_generated_clock](#) command

For the example scenario, this can be done by changing the generated clock “gclk” to “combinational”. This means that the latency path for “gclk” will include only combinational paths where the master clock propagates to the generated clock and no paths involving sequential elements.

The modification can be done as follows:

```
create_clock -name sys_clk -period 4 [ get_ports CLK ]
create_generated_clock -name gclk -combinational -source [get_ports CLK]
[get_pins and1/A]
```

See Also

[create_generated_clock](#)

[create_clock](#)

[analyze_clock_networks](#)

[set_clock_sense](#)

Rule: CLK_0009

Generated clock ***clock*** is not expanded because master clock ***master_clock*** does not reach its master source ***pin***.

Severity: Error

Description

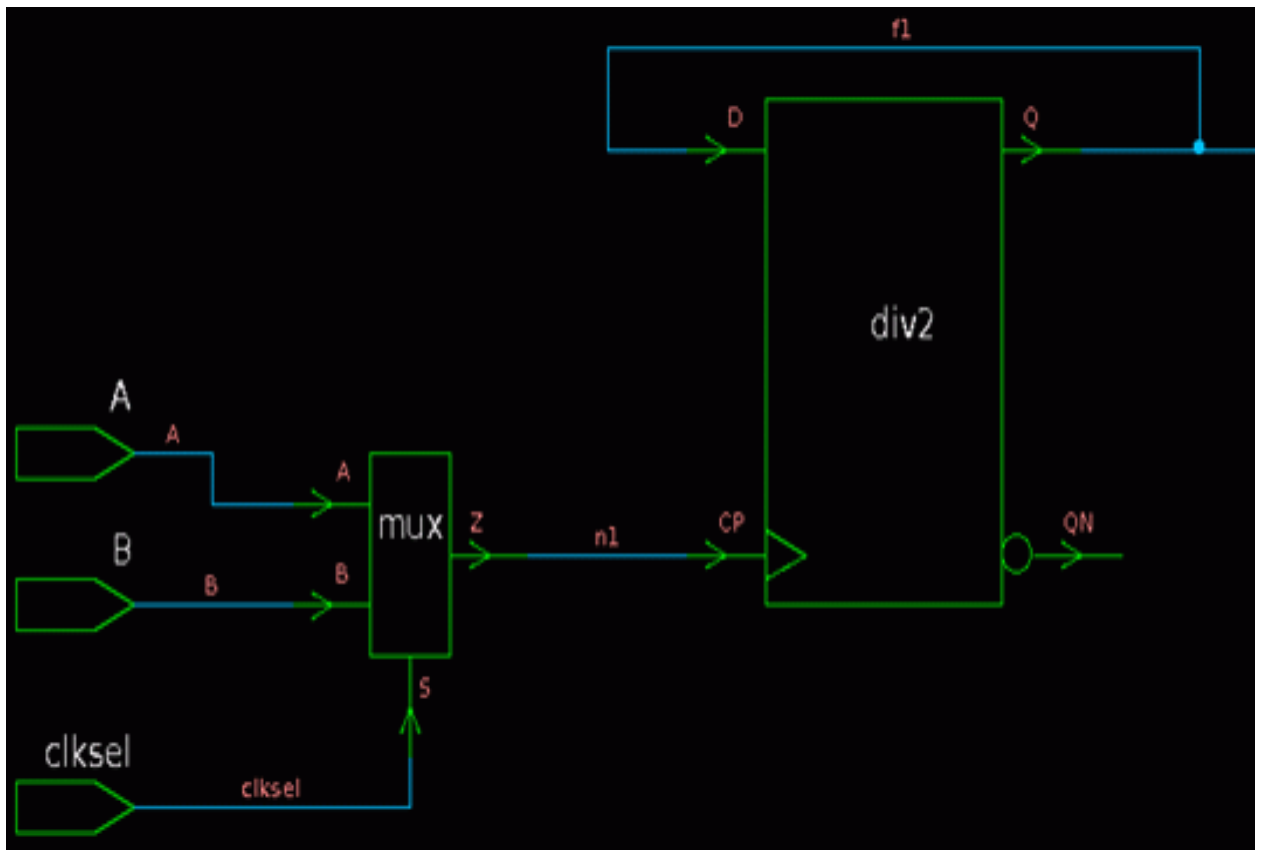
The source pin of this generated clock does not belong to the clock network of the given generated clock's master clock. The sense of the master clock at the -source pin is required to compute the waveform of a generated clock. No waveform will be created and, as a consequence, the generated clock cannot be propagated forward to register clock pins.

Risk Associated with Violation

The portion of the circuit that should be clocked by this generated clock is untested. This can result in potentially incorrect timing if the generated clock is supposed to be propagated to registers.

One Possible Scenario

Figure 13 Two Clocks Propagating to a Clock Divider Structure



In this example scenario, two clocks are propagating to some clock divider structure. In order to guarantee the proper timing analysis, generated clocks are created on the output of the clock dividers as seen in the sample script below.

```
create_clock [get_port A] -name clk1 -period 2
create_clock [get_port B] -name clk2 -period 3
create_generated_clock -name gclk1 -divide_by 2 div2/Q \
    -source [get_port B] -add -master_clock clk1
create_generated_clock -name gclk2 -divide_by 2 div2/Q \
    -source [get_port A] -add -master_clock clk2
```

However, some mistakes were introduced in those constraints, resulting in CLK_0009 violations:

- Generated clock “gclk1” with “clk1” as its master “clk1” but refers to port “B” as its source. This is incorrect; the source should be port “A”.

- Similarly, generated clocks “gclk2” should refer to port “B” as its source pin rather than port “A”.

What Next

- Check that the master_clock option is correct with respect to the source pin.
- Check that the source pin is correct with respect to the master clock.

You can use the [report_clock](#) command to achieve this as demonstrated below.

```
ptc_shell> report_clock
*****
Report : clock
Design : CLK_0009
Scenario: default
Version: ...
Date   : ...
*****
```

```
Attributes:
  p - Propagated clock
  G - Generated clock
  I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
clk1	2.00	{0 1}		{A}
clk2	3.00	{0 1.5}		{B}
gclk1			G	{div2/Q}
gclk2			G	{div2/Q}

Generated Clock	Master Source	Generated Clock	Master Source	Waveform Modification
gclk1	B	div2/Q	clk1	div(2)
gclk2	A	div2/Q	clk2	div(2)

This reports shows the source discrepancy. Port “A” is the source of clock “clk1”, but port “B” is the source of generated “gclk1”.

- If the master clock and the source pin seem correct, investigate why the master_clock doesn’t propagate to the source pin. Most likely, another constraint or a design structure is preventing proper clock sensitization:
- The master clock could be blocked from the source pin by another constraint. You can report such a situation using the command below. The **-traverse_disable** switch will allow constraint consistency to trace through disabled timing arcs and report potential clock sources.

```
analyze_clock_network -through <-source pin> \
-traverse_disabled
```

```
ptc_shell> analyze_clock_network -through [get_ports A]
-traverse_disabled
```

```
*****
```

```
Report : analyze_clock_networks
        -max_endpoints=1000
        -style short
        -end_types {register port clock_source }
        -traverse_disabled
```

```
Design : CLK_0009
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Clock Network End Type Abbreviations:
```

```
REG - register
```

```
PORT - port
```

```
CS - clock_source
```

Example End Pin/Port	End Type	Potential Clocks
div2/CP (FD1)	REG	set P0 BK_0
1		

```
Potential Clock Sets:
```

```
set P0 (1 clocks):
```

```
clk1 - positive
```

```
Clock Blocking Constraints:
```

```
BK_0:
```

```
STOP#0 = set_clock_sense -stop
```

```
at pin: b2/Z
```

```
./src/scripts/constraints.tcl, line 11
```

- The master clock could be blocked by an unresolved cell in the clock network. When there appears to be no path and you expect one, an unresolved cell in the path, or a cell with an incomplete model is a possibility. Use the [all_fanout](#) command to look for where the path is disconnected.

```
ptc_shell> all_fanout -from [get_ports A] {"A", "b2/A", "b2/Z"}
```

Fix Suggestion

Change the generated clock to correct the master_clock and -source pin/port or remove constraints that are blocking the master clock from the -source pin/port.

For the example scenario above, fix the -source values:

```
create_clock [get_port A] -name clk1 -period 2
```

```
create_clock [get_port B] -name clk2 -period 3
create_generated_clock -name gclk1 -divide_by 2 div2/Q \
    -source [get_port A] -add -master_clock clk1
create_generated_clock -name gclk2 -divide_by 2 div2/Q \
    -source [get_port B] -add -master_clock clk2
```

Corresponding PrimeTime Error Message

Error Message: PTE-023

Severity: Warning

Description

The generated clock '%s' has not been expanded. Create or activate its master clock.

See Also

[create_generated_clock](#)

[analyze_clock_networks](#)

[report_clock](#)

Rule: CLK_0010

Generated clock **clock** is not expanded because its master clock **master_clock** is not expanded.

Severity: Error

Status: Disabled by default

Description

A generated clock cannot be activated; the specified master_clock is not expanded.

Risk Associated with Violation

The portion of the circuit that should be clocked by this generated clock is unconstrained. This is likely to result in incorrect timing for the design.

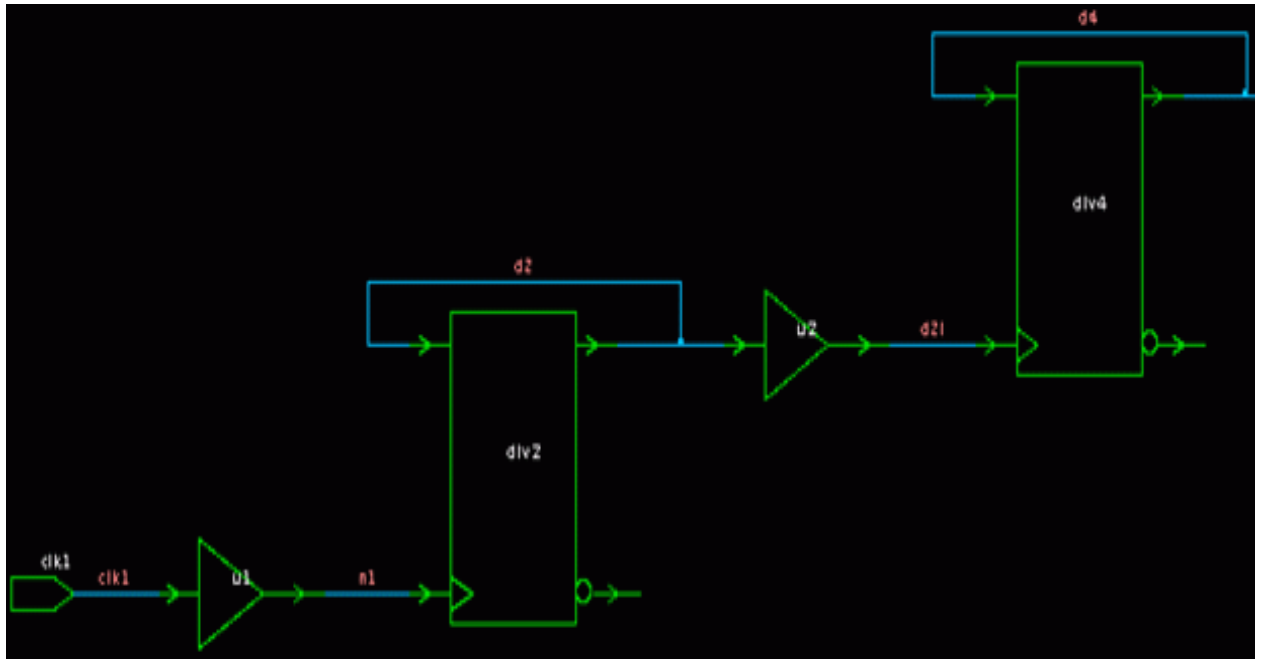
One Possible Scenario

Consider the following scenario involving two cascading generated clocks. The second generated clock “d4” specifies the first generated clock “d2” as its master clock. However, no clock has been created at the port clk1, and thus the generated clock “d2” cannot be activated. This results in a [CLK_0003](#) violation being flagged on “d2” and no waveform can be computed for generated clock “d2”. This [CLK_0003](#) violation in turn creates a

CLK_0010 violation for “d4” since a valid waveform would need to be present on “d2” to compute “d4”.

```
create_generated_clock -name d2 -divide_by 2 div2/Q -source div2/CP
create_generated_clock -name d4 -divide_by 2 div4/Q \
    -source div2/Q -master_clock d2 -add
```

Figure 14 Two Cascading Generated Clocks



What Next

Investigate and fix the violation [CLK_0003](#) or [CLK_0009](#) related to the master clock. When this is done, the **CLK_0010** violation should be addressed as a side effect.

Fix Suggestion

See the violation for the master clock. ([CLK_0003](#) or [CLK_0009](#)). For the example scenario above, fix the **-source** values:

```
create_clock clk1 -period 2
```

Corresponding PrimeTime Error Message

Error Message: PTE-023

Severity: Warning

Description

The generated clock '%s' has not been expanded. Create or activate its master clock.

See Also

[create_generated_clock](#)

[analyze_clock_networks](#)

[report_clock](#)

[CLK_0003](#)

[CLK_0009](#)

Rule: CLK_0011

Generated clock **clock** is not expanded because potential master clock **master_clock** is not expanded.

Severity: Error

Description

A generated clock cannot be activated if only unexpanded master clocks drive the master source. Fixing the issue with the unexpanded master clocks resolves this violation.

This rule differs from the [CLK_0010](#) rule because the **-master_clock** option for the [create_generated_clock](#) command was not used, therefore the unexpanded master clock appears as a potential master clock.

Risk Associated with Violation

The portion of the circuit clocked by this generated clock is unconstrained. No timing analysis will be performed for the portion of the circuit that is associated with this generated clock.

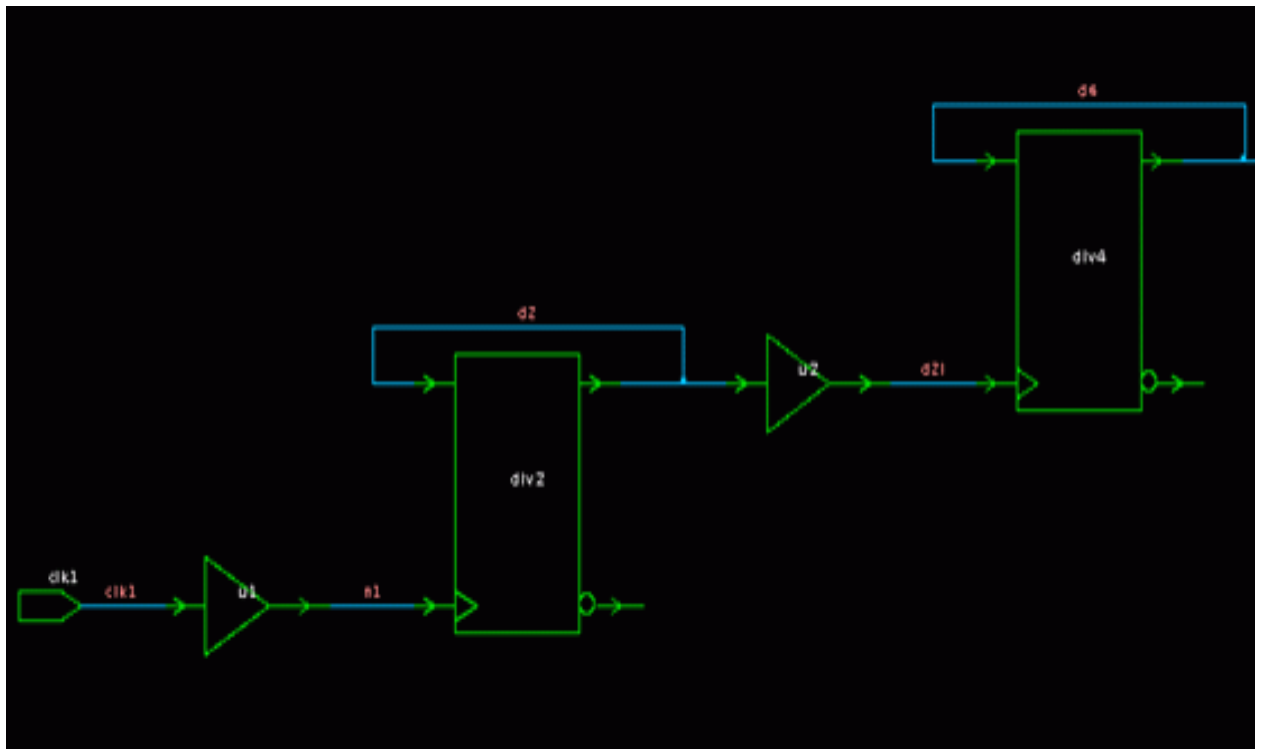
One Possible Scenario

Consider the following scenario involving two cascaded generated clocks. The generated clock **d4** specifies the generated clock **d2** as its master clock. The generated clock **d2** cannot be activated because no clock has been created at the port **clk1**. This results in a [CLK_0003](#) violation being issued for **d2**. A **CLK_0011** rule violation will be issued for **d4** because its definition does not contain the **-master_clock** option, and no waveform can be computed for it due to the error at **d2**.

```
create_generated_clock -name d2 -divide_by 2 [get_pins div2/Q] \  
    -source [get_pins div2/CP]
```

```
create_generated_clock -name d4 -divide_by 2 [get_pins div4/Q] \  
-source [get_pins div4/CP]
```

Figure 15 Two Cascaded Generated Clocks



What Next

Investigate the [CLK_0003](#), [CLK_0009](#), [CLK_0010](#), or **CLK_0011** rule violation related to the master clock.

Fix Suggestion

Fix the [CLK_0003](#), [CLK_0009](#), [CLK_0010](#), or **CLK_0011** rule violation related to the master clock. For the preceding example scenario, specify the **-source** values for the potential master clock:

```
create_clock [get_ports clk1] -period 2
```

Corresponding PrimeTime Error Message

Error Message: PTE-023

Severity: Warning

Description

The generated clock '%s' has not been expanded. Create or activate its master clock.

See Also

[create_generated_clock](#)

[create_clock](#)

[analyze_clock_networks](#)

[report_clock](#)

[CLK_0003](#)

[CLK_0009](#)

[CLK_0010](#)

Rule: CLK_0012

Generated clock **clock** has non-unate sense for master clock **master_clock** at its master source.

Severity: Warning

Description

The waveform of a generated clock is dependent on the sense at its master source. When the sense is non-unate, only the positive sense of the master clock is used. If both clock senses need to be considered, create additional generated clocks to avoid incorrect or pessimistic timing analysis. The choice of clock sense can be made explicit by adding the [set_clock_sense](#) command at the non-unate location.

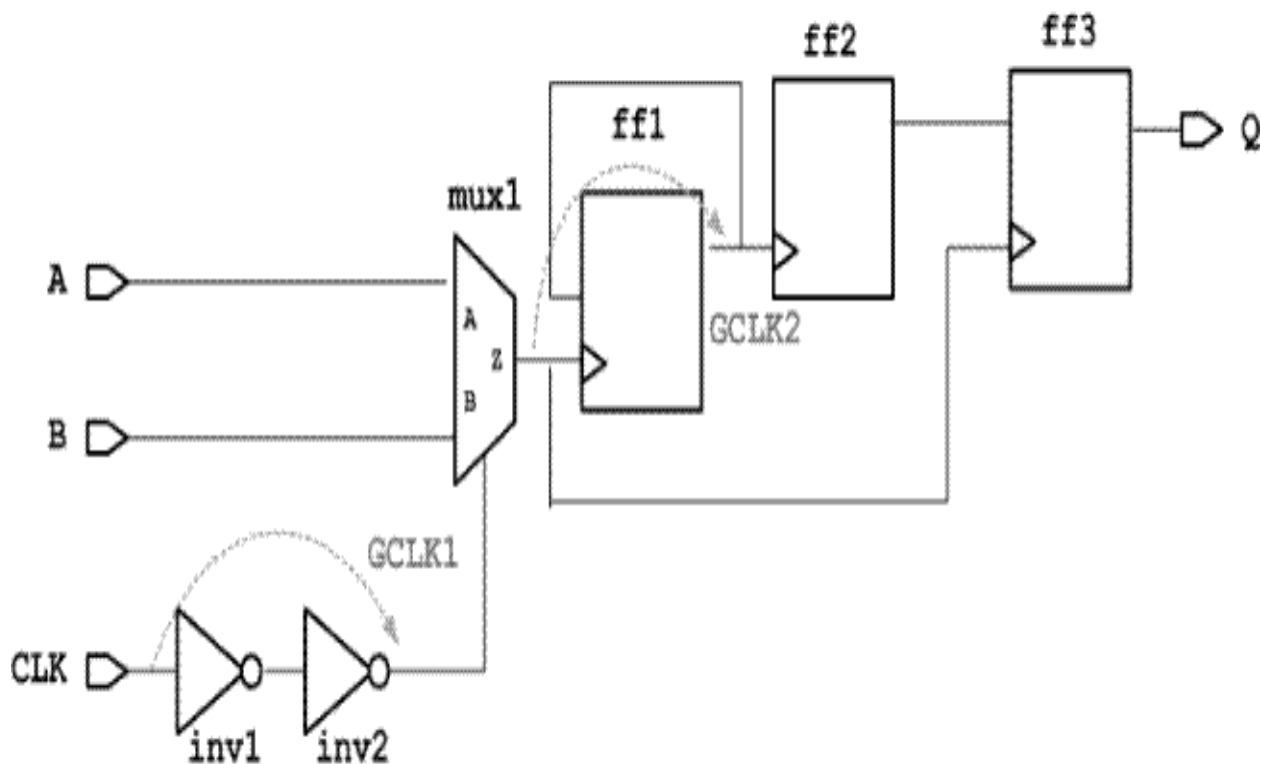
Risk Associated with Violation

If this violation is not addressed, the resulting timing analysis may be incorrect or pessimistic depending on how the generated clock interacts with the master clock. Incorrect results may be obtained if the expected clock sense is negative but only the positive clock sense propagates. Incorrect clock sense may lead to a half cycle shift in clock waveforms. Pessimistic result may arise if there are interactions with the negative sense of the master clock and the positive sense of the generated clock. Pessimistic timing can lead to increased optimization and timing closure efforts.

One Possible Scenario

Consider a scenario involving two generated clocks. The second generated clock **GCLK2** is essentially derived from the first generated clock **GCLK1**. The path from **GCLK1** source pin to **GCLK2** source pin involves a MUX selector pin to MUX output pin path.

Figure 16 Two Generated Clocks with MUX Selector Pin in the Path



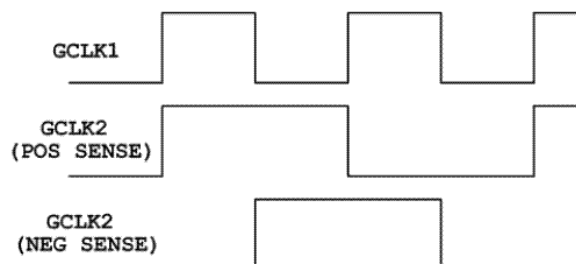
Depending on the logic values at the inputs of the MUX, the clock sense may become positive or negative through the MUX. As a result, both positive and negative sense of **GCLK1** is available at the master source pin of **GCLK2** but only the positive sense will be used for the waveform and latency calculation of **GCLK1**. When timing results are analyzed at register 'ff3/D', the following cases are possible due to the interaction between **GCLK1** and **GCLK2**:

Launching Clock Sense (GCLK2)	Capturing Clock Sense (GCLK1)
Case 1 Positive	Negative
Case 2 Positive	Positive
Case 3 Negative	Negative
Case 4 Negative	Positive

Due to the way **GCLK2** is defined, case 3 and case 4 will not be considered. As a result, an incorrect timing result may be obtained if a negative clock sense propagation is

expected (case 3). As can be seen from the waveform diagram below, the positive sense waveform of **GCLK2** differs from the negative sense waveform by a half a cycle.

Figure 17 GCLK1 Waveforms



At the same time, a pessimistic timing may arise since case 1 may not be possible due to the mutually exclusive nature of the MUX: $Z = A S' + B S$

A	B	Clock Sense at mux1/Z
0	0	Clock is inactive due to 0
0	1	Positive
1	0	Negative
1	1	Clock is inactive due to 1

```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 [get_pins inv2/Z] \
    -source [get_ports CLK] -combinational
create_generated_clock -name GCLK2 [get_pins ff1/Q] \
    -source [get_pins mux1/Z] -divide_by 2
set_propagated_clock [all_clocks]
```

What Next

The Violation Details section provides the names of the generated clock and the non-unate master clock as well as the master source pin. There are multiple ways to analyze the problem. One way is to use the [report_clock](#) command to view the clock waveform for **GCLK2** to see if the clock edges are correct. As can be seen from the report below, only the positive clock sense is propagated for **GCLK2**.

Clock	Period	Waveform	Attrs	Sources
CLK	2.00	{0 1}	p	{CLK}
GCLK1	2.00	{0 1}	p, G	{inv2/Z}
GCLK2	4.00	{0 2}	p, G	{ff1/Q}

Another way to analyze this problem is to use the `clocks_with_sense` attribute. This attribute provides, for each pin or port, information on clock and its sense for all its associated clocks. The `clocks_with_sense` attribute can be queried as follows:

```
ptc_shell> get_attribute [get_pins mux1/Z] clocks_with_sense
{ { "GCLK1" "negative" } { "GCLK1""positive" } }
ptc_shell> get_attribute [get_pins ff2/CP] clocks_with_sense
{ { "GCLK2" "positive" } }
```

The attribute shows that there are both positive and negative senses of the master clock **GCLK1** at the master source pin of **GCLK2** but only the positive sense of **GCLK2** propagates to register 'ff2'.

Alternatively, the [analyze_clock_networks](#) command can be used to find out how clock senses change along the relevant clock path. The `-style full` option provides full details including:

- multiple paths ("branches")
- logic levels
- clock sense ("positive", "negative", "rise_triggered", "fall_triggered")
- sequential CLK-to-Q paths ("REG")

The generated clock name can be supplied as the `-to` option and the master source pin to the `-through` option as follows:

```
ptc_shell> analyze_clock_networks -to GCLK2 -through mux1/Z -style full
```

The "Sense" column from the [analyze_clock_networks](#) output provides details on clock sense propagation along the clock path. For the example scenario, [analyze_clock_networks](#) shows that there is only positive sense of **GCLK2** at the MUX output 'mux1/Z'.

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	inv2/Z (IV)
1		P		mux1/S (MUX21H)
2		P		mux1/Z (MUX21H)
3		P	REG	ff1/CP (FD1)
4		P		ff1/CP (FD1)
5		R	CS	ff1/Q (FD1)

Fix Suggestion

If both the positive clock sense and the negative clock sense need to be considered, create additional generated clocks to fix incorrect clock waveforms or remove unrealistic pessimism.

For the example scenario, this can be done as follows:

```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1_POS [get_pins mux1/Z] \
    -source [get_ports CLK] \
    -combinational -add -master_clock CLK
create_generated_clock -name GCLK1_NEG [get_pins mux1/Z] \
    -source [get_ports CLK] \
    -combinational -add -master_clock CLK -invert
create_generated_clock -name GCLK2_POS [get_pins ff1/Q] \
    -source [get_pins mux1/Z] \
    -divide_by 2 -add -master_clock GCLK1_POS
create_generated_clock -name GCLK2_NEG [get_pins ff1/Q] \
    -source [get_pins mux1/Z] \
    -divide_by 2 -add -master_clock GCLK1_NEG
set_clock_groups -logically_exclusive -group {GCLK1_POS GCLK2_POS} \
    -group {GCLK1_NEG GCLK2_NEG}
set_propagated_clock [all_clocks]
```

Note that the positive sense and negative sense of both generated clocks are declared to be mutually exclusive.

If it is known that only one clock sense is possible, the pessimism can be reduced even further by using the [set_clock_sense](#) command. In the example scenario, if the input conditions A = 1 and B = 0 can never happen, then the negative clock sense can be eliminated by using

```
set_clock_sense -positive mux1/Z
```

Then, the resulting timing analysis will not include any violations at register 'ff3/D' that are due to a launching clock associated with the negative sense of **GCLK1** and a capturing clock with the negative sense of **GCLK2**.

See Also

[create_generated_clock](#)

[create_clock](#)

[analyze_clock_networks](#)

[get_attribute](#)

[set_clock_sense](#)

[report_clock](#)

Rule: CLK_0013

Generated clock **clock** ignores multiple rising edges of master clock **master_clock** due to a user-defined duty cycle in the generated clock definition.

Severity: Error

Description

A generated clock specified with the **-multiply_by** option and a user-defined duty cycle, creates its waveform using the first rising edge and the period of the master clock. The master clock has multiple rising edges defined in its period, though the additional edges are ignored by this generated clock.

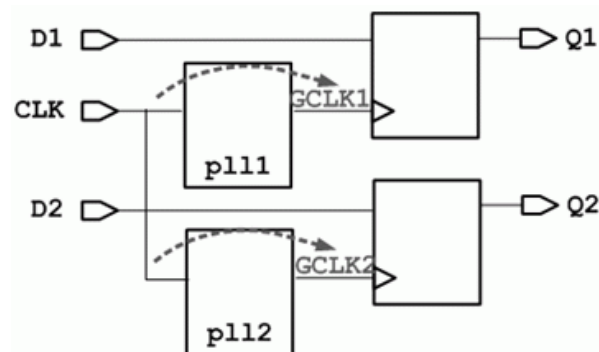
Risk Associated with Violation

The generated clock waveform may be incorrect, especially for the falling edge, since this is defined by the [create_generated_clock](#) command using the **-duty_cycle** option. Timing analysis will be performed, but the results are based on an incorrect generated clock because the master clock has multiple rising edges.

One Possible Scenario

Consider the two frequency divider clocks produced by phase lock loop (PLL) circuits. Both of the generated clocks **GCLK1** and **GCLK2** are defined using the **-multiply_by 2** option with a master clock that has multiple rise and fall edges within a single period. **GCLK1** has an additional duty cycle constraint of 40%.

Figure 18 Two Frequency Divider Clocks Produced by Phase Lock Loop (PLL) Circuits

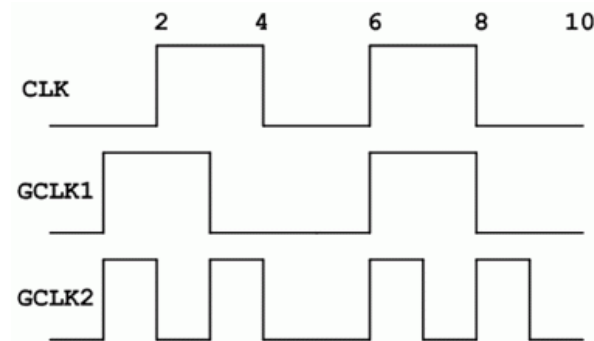


```
create_clock -name CLK -period 10 -waveform {2 4 6 8} \  
[get_ports CLK]  
create_generated_clock -name GCLK1 -source [get_ports CLK] \  
-multiply_by 2 -duty_cycle 40 [get_pins pll1/CLK_OUT]
```

```
create_generated_clock -name GCLK2 -source [get_ports CLK] \  
    -multiply_by 2 [get_pins pll2/CLK_OUT]
```

The master clock **CLK** has multiple rising edges at time units 2 and 6. The generated clock, **GCLK1**, will only use the first rising edge to create its waveform due to the **-duty_cycle** option constraint in its specification. **GCLK2** does not use the **-duty_cycle** option but is a **-multiply_by 2** clock derived from the master clock **CLK** and has the same number of rising or falling edges.

Figure 19 CLK With Multiple Rising Edges, **GCLK1** With Duty Cycle Constraint, **GCLK2** is **-multiply_by2** Clock of CLK



What Next

The “Violation Details” section provides the waveforms details for the generated clock and its master clock.

```
Master Clock  
Period: 10.0000  
Waveform: {2.0000 4.0000 6.0000 8.0000}  
Generated Clock  
Period: 5.0000  
Waveform: {1.0000 3.0000}
```

Check the master clock and the generated clock waveforms for correctness. The waveforms for clocks in the design can be obtained by using the [report_clock](#) command. For the example scenario, the command can be issued as follows for the clocks of interest:

```
ptc_shell> report_clock [get_clocks {CLK GCLK1 GCLK2}]
```

Attributes:

p - Propagated clock
G - Generated clock
I - Inactive clock

Clock	Period	Waveform	Attrs	Sources
CLK	10.00	{2 4 6 8}	{CLK}	
GCLK1	5.00	{1 3}	G {pll1/CLK_OUT}	
GCLK2	5.00	{1 2 3 4}	G {pll2/CLK_OUT}	

Generated Clock	Master Source	Generated Clock	Master Modification	Waveform
GCLK1	CLK	pll1/CLK_OUT	CLK div(2)	
GCLK2	CLK	pll2/CLK_OUT	CLK div(2)	

Fix Suggestion

If the generated clock waveform needs to be reshaped with the duty cycle option, consider changing the master clock waveform definition to have a single rising edge and a single falling edge. This will affect the shape of the master clock, and it may also affect the clock waveform of other generated clocks that use the same master clock. In the example scenario, this can be done by removing the last two edges from the master clock waveform:

```
create_clock -name CLK -period 10 -waveform {2 4} [get_ports CLK]
```

Alternatively, if the generated clock does not need a modified duty cycle, the **-duty_cycle** option can be removed from the generated clock definition.

See Also

[create_generated_clock](#)

[create_clock](#)

[report_clock](#)

Rule: CLK_0014

Clock **clock** is created on a pin instead of a port.

Severity: Info

Description

A clock has been defined on an internal pin. Any clock defined on a pin will ignore incoming slews and latencies up to that pin which can impact timing accuracy. This rule will not be triggered if the pin has no incoming timing arcs.

The presence of a **CLK_0014** violation is preceded by a **UIC-019** message when reading the SDC constraints.

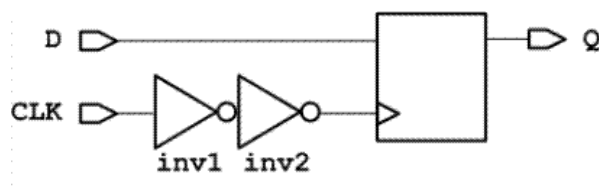
Risk Associated with Violation

Ignoring incoming slews or latencies on clock paths may lead to optimistic timing results resulting in slower chip operation or even failure.

One Possible Scenario

Consider a simple scenario where the clock is defined at pin 'inv2/Z' that lies on the path from the clock port 'CLK' to the register. Since the clock source is at the inverter output pin 'inv2/Z', any clock slews or latencies from the port 'CLK' to the pin 'inv2/Z' will be ignored during clock latency calculation. This may lead to optimistic timing results on the flip-flop and might cause the chip to fail.

Figure 20 Clock Defined on an Internal Pin



```
create_clock -name CLK1 -period 2 [get_pins inv2/Z]
```

What Next

The violation message gives the name of the clock and the Violation Details section provides the location of the clock source pin. To identify further the clock latency path associated with the clock, you can use the [analyze_clock_networks](#) command. The **-full** switch on the report command can be used to get detailed information related to that clock path such as:

- logic levels
- clock sense ("positive", "negative", "rise_triggered", "fall_triggered")

For the preceding example, the violation can be analyzed as follow:

```
ptc_shell> analyze_clock_networks -from CLK1 -style full
```

From the report excerpt below, you can see that the clock source latency does not include the path from the clock port to the inverter output 'inv2/Z'.

Full report for clock: CLK1

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	inv2/Z (IV)
1		P	REG	ff1/CP (FD1)

Fix Suggestion

There are two possible approaches to address this violation. One approach is to move the clock definition directly to the input port. In the example scenario, this can be done by:

```
create_clock -name CLK1 -period 2 [get_ports CLK]
```

Running [analyze_clock_networks](#) with the modified constraint will show that the clock latency path now includes the path from the clock port to the register including the two inverters (as seen below).

Full report for clock: CLK1

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK (in)
1		P		inv1/A (IV)
2		N		inv1/Z (IV)
3		N		inv2/A (IV)
4		P		inv2/Z (IV)
5		P	REG	ff1/CP (FD1)

Another approach is to create a generated clock as the output of the inverter 'inv2'. A generated clock will automatically inherit the latency and slew data from its master clock. In the example scenario, this can be done by first creating a master clock at the port and then driving a generated clock at 'inv2/Z' from the master clock:

```
create_clock -name CLK1 -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 [get_pins inv2/Z] \
    -combinational -source [get_ports CLK]
```

Running [analyze_clock_networks](#) with the modified constraint will show that the clock latency path includes the path from the clock port to the register including the two inverters (as seen below). Note that 'inv2/Z' is now marked as clock source "CS" since this is a generated clock source pin.

```
ptc_shell> analyze_clock_networks -from CLK1 -style full
Full report for clock: CLK1
```

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK (in)
1		P		inv1/A (IV)
2		N		inv1/Z (IV)
3		N		inv2/A (IV)
4		P	CS	inv2/Z (IV)

See Also

[create_generated_clock](#)

[create_clock](#)

[analyze_clock_networks](#)

Rule: CLK_0015

Clock **clock** is created on a hierarchical pin.

Severity: Warning

Description

Defining a clock on a hierarchical pin breaks the net parasitics in a way that reduces the accuracy of latency calculation. Because a hierarchical pin does not have a set physical location, the latency calculation will be arbitrary.

Risk Associated with Violation

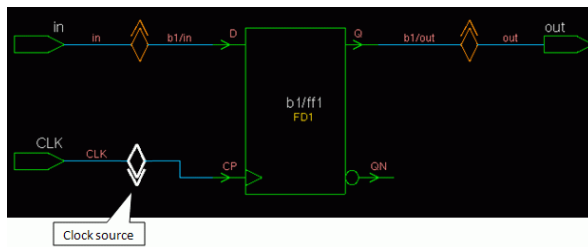
Creating clocks on hierarchical pins will result in potentially incorrect latency calculation and, as a consequence, incorrect timing analysis.

One Possible Scenario

The following clock definition shows that a clock source has been defined on a hierarchical pin.

```
create_clock -period 10 -name hier_clock b1/CLK
```

Figure 21 Clock Source Defined on a Hierarchical Pin



The physical location at which this hierarchical pin exists is arbitrary. In such a scenario **CLK_0015** rule violation will be given.

What Next

The [all_fanin](#) command can be used to find out which driving cell leaf pins correspond to this hierarchical pin. In the above example this command would be:

```
all_fanin -to b1/CLK -flat -pin_levels 1 {"b1/CLK", "CLK"}
```

Fix Suggestion

Consider moving this clock definition to the driving leaf pins. The location of the cell leaf pin will now be known and the latency will be computed in an accurate fashion.

See Also

[create_clock](#)

[all_fanin](#)

Rule: CLK_0016

Generated clock **clock** has no source latency path from its master clock **master_clock**.

Severity: Error

Description

There is no valid path from the source of the master clock to the source of the generated clock due to:

- no topological path (possibly due to an unresolved reference or a black box)
- disabled logic
- the path is blocked by another clock source
- a [set_clock_sense-stop](#) command in the path to the source of the generated clock

The "Violation Details" section provides reasons for the lack of a source latency path.

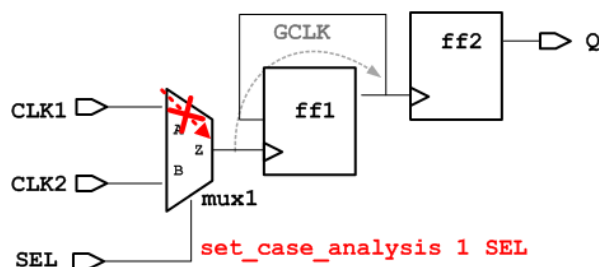
Risk Associated with Violation

The generated clock will not be expanded, and registers clocked by the generated clock will not be constrained. Timing analysis will not occur for paths related to the registers, and the timing results will be incomplete for the design.

One Possible Scenario

Consider the following scenario where the path from the master clock to the generated clock is blocked by a `set_case_analysis` command on the select pin of a MUX. The case analysis specified on the select pin of the MUX allows clock **CLK2** to propagate, but does not allow clock **CLK1** to propagate. The generated clock **GCLK** specifies clock **CLK1** as its master clock.

Figure 22 Blocked Path to the Generated Clock



```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
create_generated_clock -name GCLK -divide_by 2 \
    -source [get_pins ff1/CP] -master_clock CLK1 \
    -add [get_pins ff1/Q]
set_case_analysis 1 [get_ports SEL]
```

Since the case analysis on the select pin of the MUX propagates the clock **CLK2**, there is no latency path from **CLK1** to **GCLK**, and a **CLK_0016** rule violation will be issued.

What Next

Examine the "Violation Details" section for reasons the latency path is missing between the master clock source and the source of the generated clock.

If the reason reported is no topological path from the master clock to the generated clock, check if there are any unresolved references (black boxes) in the design. Black boxes are flagged with [LNK-805](#) warning message of the form:

```
Warning: Unable to resolve reference to 'BBOX' in 'top'. (LNK-805)
```

If the reason for the missing clock latency path is because it is blocked, disabled, or the clock is not propagating, use the [analyze_clock_networks](#) command to understand the root cause of the missing latency path. In the example scenario, this debug command can be issued by supplying the generated clock in the **-to** option along with the **-traverse_disabled** and **-style full** options. The **-traverse_disabled** option allows clock path traversal on disabled paths, and the **-style full** option provides full details of the clock path including:

- logic levels
- clock sense (positive, negative, rise_triggered, fall_triggered)
- sequential CLK-to-Q paths (REG)

The command and its output are shown below:

```
ptc_shell> analyze_clock_networks -to [get_clocks GCLK] \
-traverse_disabled -style full
```

```
Source latency paths for generated clock: GCLK
from master clock: CLK1
```

```
Branch 0:
```

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK1 (in)
1		[P]		mux1/A (MUX21H)
2		[P]	CARC#0	mux1/Z (MUX21H)
3		[P]	REG	ff1/CP (FD1)
4		[P]		ff1/CP (FD1)
5		[R]	CS	ff1/Q (FD1)

```
Clock Blocking Constraints:
```

```
CARC#0 = case_value_disabled_arc
at positive_unate arc from: mux1/A
to: mux1/Z
```

The output from the [analyze_clock_networks](#) command shows the clock latency path is disabled due to a [set_case_analysis](#) command that disables the timing arc from **mux1/A** to **mux1/Z**.

Fix Suggestion

If black boxes are blocking the clock paths, remove them by reading in additional library or design files. If the clock path is disabled/blocked, remove the disabling condition, or redefine the generated clock so that the path from the master clock to the generated clock is not disabled/blocked. For the example scenario, the [set_case_analysis](#) command can be removed:

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
```

```
create_clock -name CLK2 -period 2 [get_ports CLK2]
create_generated_clock -name GCLK -divide_by 2 \
    -source [get_pins ff1/CP] -master_clock CLK1 \
    -add [get_pins ff1/Q]
#set_case_analysis 1 [get_ports SEL]
```

Alternatively, the generated clock **GCLK** can be derived from clock **CLK2** instead of **CLK1**:

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
create_generated_clock -name GCLK -divide_by 2 \
    -source [get_pins ff1/CP] -master_clock CLK2 \
    -add [get_pins ff1/Q]
set_case_analysis 1 [get_ports SEL]
```

Corresponding PrimeTime Error Message

Error Message: PTE-075

Severity: Warning

Description

Generated clock '%s' has no path to its master clock.

See Also

[create_generated_clock](#)

[create_clock](#)

[analyze_clock_networks](#)

[report_disable_timing](#)

[set_clock_sense](#)

Rule: CLK_0017

The master source of generated clock **clock** is not in the fanin of the generated clock source pin.

Severity: Error

Description

The master source of the generated clock, as given by the **-source** option to the [create_generated_clock](#) command, is not in the fanin cone of the generated clock source pin. The generated clock will still propagate, but there will be issues with the latency of the generated clock. In addition, the sense of the master clock used to create the generated

Risk Associated with Violation

One Possible Scenario

```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 [get_pins ff1/Q] \
    -divide_by 2 -source [get_pins inv1/Z] \
    -master_clock CLK -add
set propagated_clock [all_clocks]
```

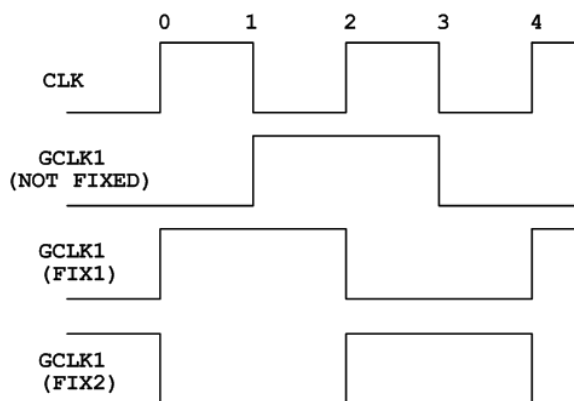
1. The resulting clock waveform is wrong. The generated clock waveform is neither a fully inverted nor a fully non-inverted divide-by-2 waveform of the original master clock waveform. As shown in the diagram below, the rising edge of the master clock is at

time 0 and the falling edge at time 1. The rising edge of the generated clock will be at time 1 and the falling edge at time 3, instead of 0 and 2 or 2 and 4.

2. Zero source clock latency will be assumed for the generated clock so the clock network delay from ff1/D to ff1/Q will not be included in timing check at ff3/D. This will lead to pessimistic setup timing results at ff3/D.

For this scenario, the PrimeTime tool will issue a **UITE-461** error message to alert users to this situation. However, not all violations of this rule will cause a **UITE-461** error message in PrimeTime.

Figure 24 Master Clock and Generated Clock Waveforms



What Next

Use the [report_clock](#) command to compare the master clock and generated clock waveforms to make sure that the generated clock waveforms are correct. For the example scenario, the command and its output are as follows:

```
ptc_shell> report_clock
Attributes:
  p - Propagated clock
  G - Generated clock
  I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
CLK	2.00	{0 1}	p	{CLK}
GCLK1	4.00	{1 3}	p, G	{ff1/Q}

The report output shows that the generated clock has a rising edge at time 1 and a falling edge at time 3.

In addition, look for other violations related to the generated clock involving edge relations. In the example scenario, a violation of [CLK_0020](#) will also be reported:

Generated clock **GCLK1** has edge relationships with its master clock **CLK** that cannot be satisfied. Only paths with 'rise_triggered' sense exist from the master clock to source pin **ff1/Q**. A 'fall_triggered' sense is expected.

Fix Suggestion One

Consider changing the **-source** pin or port of the generated clock so that it lies in the fanin cone of the generated clock source pin. In the example scenario, the **-source** option can be changed to **CLK** from **inv1/Z**.

```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 [get_pins ff1/Q] \
    -divide_by 2 -source [get_ports CLK] \
    -master_clock CLK -add
set_propagated_clock [all_clocks]
```

The generated clock waveform now has a rising edge at 0 and a falling edge at 2. See **GCLK1 (FIX1)** in the timing diagram.

Fix Suggestion Two

Alternatively, the generated clock can be inverted without changing the **-source** option. This will also remove the current and [CLK_0020](#) violations in the example scenario.

```
create_clock -name CLK -period 2 [get_ports CLK]
create_generated_clock -name GCLK1 [get_pins ff1/Q] \
    -divide_by 2 -source [get_pins inv1/Z] \
    -master_clock CLK -add -invert
set_propagated_clock [all_clocks]
```

The generated clock waveform will have a rising edge at 2 and a falling edge at 4. See **GCLK1 (FIX2)** in the timing diagram.

See Also

[create_generated_clock](#)

[create_clock](#)

[report_clock](#)

Rule: CLK_0018

Clock **clock** has non-unate logic in the clock network.

Severity: Warning

Description

There is non-unate logic in the clock network. The locations where the clock network becomes non-unate are given in the details of this warning. Both the positive and negative sense of the clock will be propagated. If both clock senses are possible, but not simultaneously, consider creating generated clocks that are mutually exclusive to remove unrealistic pessimism. If only one clock sense can propagate, consider adding a [set_clock_sense](#) constraint to remove the pessimism.

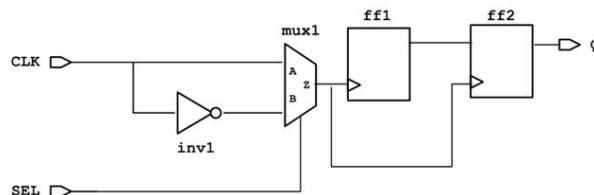
Risk Associated with Violation

If this violation is not addressed, the resulting timing analysis may be pessimistic because both the positive sense and negative sense of the clock will be assumed to propagate simultaneously. In practice, only one clock sense may propagate, or both clock senses may propagate, but not simultaneously. Pessimistic timing results can lead to increased effort in optimization and timing closure.

One Possible Scenario

Consider a scenario where a MUX has both a clock signal and an inverted clock signal as inputs. If the mode of operation of the circuit is not specified by setting a case analysis value on the select pin of the MUX, both the positive and negative sense of the clock CLK will be propagated.

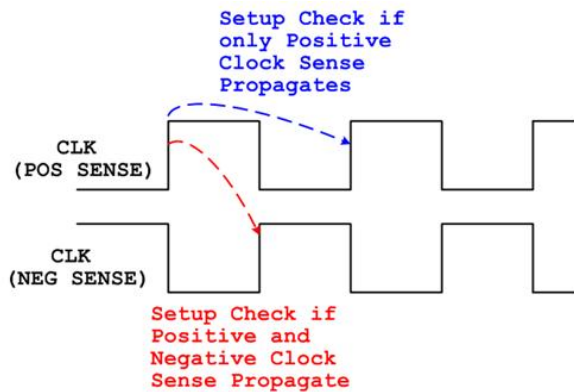
Figure 25 *Timing Checks Depend on Clock Sense Propagation*



```
create_clock -name CLK -period 2 [get_ports CLK]
```

Timing analysis will assume that both positive and negative clock senses can propagate simultaneously as shown in the waveform diagram below.

Figure 26 Timing Violation



Timing violations will be reported at register 'ff2/D' that are due to a launching clock associated with the positive sense of 'CLK' and a capturing clock with the negative sense of 'CLK' and vice versa.

What Next

The "Violation Details" section provides the name of the clock and the non-unate pins. There are several ways to analyze the root cause of the non-unateness. One way is to use the [analyze_clock_networks](#) command to investigate how clock senses change along the relevant clock path. The **-style full** option provides full details including:

- multiple paths ("branches")
- logic levels
- clock sense (positive, negative, rise_triggered, fall_triggered)
- sequential CLK-to-Q paths (REG)

The clock name can be supplied as the **-from** option and one of the non-unate pins to the **-through** option as follows:

```
ptc_shell> analyze_clock_networks -from CLK -through mux1/Z -style full
```

The 'Sense' column from the [analyze_clock_networks](#) output provides details on the clock sense change along the clock path. For the example scenario, the report shows that there are both positive and negative clock senses at the MUX output 'mux1/Z'.

```
Branch 0:
      Branch
Level  Info      Sense Notes      Port/Pin
-----
      0          P      source      CLK
      1          P          mux1/A (MUX21H)
      2  S1      P,N          mux1/Z (MUX21H)
```

3 P,N REG ff1/CP (FD1)

Another way to analyze the root cause of non-unate clocks is to use the **clocks_with_sense** attribute. This attribute provides, for each pin or port, information on a clock and its sense for all clocks associated with the object. The **clock_with_sense** attribute can be queried as follows:

```
ptc_shell> get_attribute [get_pins mux1/A] clocks_with_sense \
             get_attribute [get_pins mux1/A] clocks_with_sense \
             { { "CLK" "positive" } }

ptc_shell> get_attribute [get_pins inv1/Z] clocks_with_sense \
             get_attribute [get_pins inv1/Z] clocks_with_sense \
             { { "CLK" "negative" } }

ptc_shell> get_attribute [get_pins mux1/Z] clocks_with_sense \
             get_attribute [get_pins mux1/Z] clocks_with_sense \
             { { "GCLK" "negative" } { "GCLK" "positive" } }
```

Fix Suggestion

If the positive clock sense and the negative clock sense cannot propagate simultaneously, create two mutually exclusive generated clocks at the non-unate pins to remove the unrealistic pessimism. If only one clock sense can propagate, consider adding a [set_clock_sense](#) constraint to remove the pessimism, or a [set_case_analysis](#) constraint to propagate the desired sense of the clock for the mode of operation.

For the example scenario, case analysis can be specified on the MUX select pin to propagate the positive edge clock:

```
set_case_analysis 0 [get_pins mux1/S]
```

Now, only one sense of clock 'CLK' will arrive at the register clock pins 'ff1/CP' and 'ff2/CP'.

Corresponding PrimeTime Error Message

Error Message: PTE-070

Severity: Warning

Description

A non-unate path in clock network detected. Propagating both inverting and noninverting senses of clock '%s' from pin '%s'.

See Also

[create_generated_clock](#)

[create_clock](#)
[analyze_clock_networks](#)
[get_attribute](#)
[set_clock_sense](#)
[set_case_analysis](#)

Rule: CLK_0019

Design has both propagated and ideal non-virtual clocks.

Severity: Warning

Description

The design has some propagated clocks and some ideal clocks. For pre-layout analysis, ideal clocks are usually appropriate. For post-layout analysis, propagated clocks should be used. It is not common for some clocks to be pre-layout and some to be post-layout. Note that virtual clocks are ignored for this check and so are the clocks defined at output ports.

Risk Associated with Violation

Ideal clocks in post-layout analysis will lead to incorrect timing results because only specified clock latencies are considered. The clock latencies given with [set_clock_latency](#) may be larger or smaller than the actual latencies. Incorrect timing results may lead to slower chip operation or even failure. Propagated clocks in pre-layout analysis may add unintended pessimism or optimism to timing results because the clock network will not have been implemented with proper drive strength and skew.

One Possible Scenario

Consider a post-layout design where all the clocks need to be propagated. In the constraint file, there is a [set_propagated_clock](#) command that is applied to all the clocks.

```
create_clock -name CLK1 -period 10 [get_ports CLK1]
create_clock -name CLK2 -period 20 [get_ports CLK2]
set_propagated_clock [all_clocks]
...
create_clock -name CLK3 -period 30 [get_ports CLK3]
```

However, the clock definition for 'CLK3' appears after the [set_propagated_clock](#) command, so 'CLK3' becomes ideal instead of propagated.

What Next

The "Violation Details" section provides a list of ideal clocks and a list of propagated clocks. Consider making either one of the clock lists empty by changing clocks from ideal to propagated or vice versa. Note that virtual clocks and clocks defined at output ports are not included in these lists.

Fix Suggestion

For post-layout analysis, declare the ideal clocks shown in the Violation Details section as propagated clocks. For the example scenario, this can be done by moving the [set_propagated_clock](#) command after the last clock definition as in:

```
create_clock -name CLK1 -period 10 [get_ports CLK1]
create_clock -name CLK2 -period 20 [get_ports CLK2]
...
create_clock -name CLK3 -period 30 [get_ports CLK3]
set_propagated_clock [all_clocks]
```

For pre-layout analysis, consider changing the propagated clocks to ideal clocks to remove unintended pessimism. This can be done by removing [set_propagated_clock](#) commands.

See Also

[create_generated_clock](#)

[create_clock](#)

[set_propagated_clock](#)

[report_clock](#)

Rule: CLK_0020

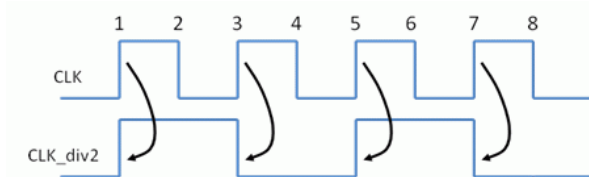
Generated clock **clock** has edge relationships with its master clock **master_clock** that cannot be satisfied. Only paths with **path_sensesense** exist from the master clock to source pin **source_pin**. An **expected_sensesense** is expected.

Severity: Error

Description

A generated clock definition implies certain relationships between the master clock source and the generated clock source. For example, a simple **-divide_by 2** generated clock implies that a rising master clock edge can cause both a rising and falling generated clock edge (see Figure 1 below).

Figure 27 Relation Between a Clock and its divide_by_2 Generated Clock



A generated clock definition specifies how the master clock waveform, as it exists at the **-source** pin (from the [create_generated_clock](#) command), is altered.

- If the **-source** pin given is one of the master clock sources, then the waveform modifier (**-divide_by**, **-multiply_by**, **-edges**, **-combinational**, **-invert**) specifies how the master clock waveform is modified to become the generated clock waveform.
- If the **-source** pin is a clock network pin, the master clock waveform may be inverted at that pin. In the inverted case, that generated clock modifier specifies how the inverted master clock waveform is altered to become the generated clock waveform.

The **CLK_0020** rule looks at the source latency paths of the generated clock and determines whether the specified relationship to its master exists. If the relationship cannot be proven, a **CLK_0020** violation is triggered for that particular generated clock. In addition, a zero propagated latency is used for the edges of the generated clock where no source latency path is found.

This rule violation correlates to the PrimeTime message **UITE-461**.

Risk Associated with Violation

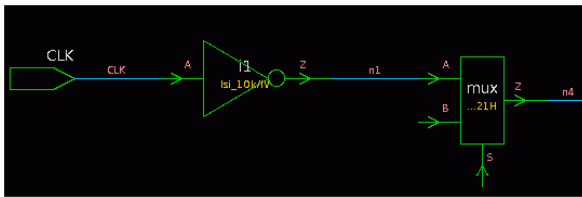
The generated clock waveform might be incorrect, the needed source latency paths might be disabled for some reason, and the propagated latency might not be able to be computed. These problems can lead to incorrect or unexpected timing results.

One Possible Scenario

Consider a scenario where an inverted clock reaches the input of a multiplexer. A generated clock (with a **-combinational** modifier) is added to the output of the MUX. The clock port (**CLK**) is defined as the **-source** pin of that generated clock, as shown here:

```
create_clock -name clk1 -period 2 [get_ports CLK]
create_generated_clock -name g1 mux/Z -source [get_ports CLK] \
    -combinational -master_clock clk1 -add
```

Figure 28 Inverted Clock Reaches the Input of a Multiplexer



These constraints specify that the generated clock, g1, has the same waveform as its master clock, clk1. The tool expects that there is a positive unate path from the port CLK to the output of the mux. This means that a rising edge on CLK should produce a rising edge on g1 and a falling edge on CLK should produce a falling edge on g1. However, when constraint consistency does the clock analysis, it concludes that only negative unate paths exist. This will trigger a **CLK_0020** violation as follows:

```
CLK_0020: Generated clock g1 has edge relationships with its master
clock
that cannot be satisfied. Only paths with negative sense exist from
the master clock to source pin mux/Z. A positive sense is expected.
```

What Next

The relationship between the generated clock and its master clock can be seen using the [report_clock](#) command. In the preceding example, the report shows that the waveform of **g1** is the same as the waveform of the master clock, **clk1**.

```
ptc_shell> report_clock {clk1 g1}
```

```
*****
Report : clock
Design : CLK_0020
Scenario: default
Version: ...
Date : ...
*****
```

```
Attributes:
p - Propagated clock
G - Generated clock
I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
clk1	2.00	{0 1}		{CLK}
g1	2.00	{0 1}	G	{mux/Z}
Generated Clock	Master Source	Generated Source	Master Clock	Waveform Modification

g1 CLK mux/Z clk1 div(1)

The source latency paths to the generated clock can be analyzed with the [analyze_clock_networks](#) command. In order to get the latency path to the generated clock and make sure all the data is displayed, the following options should be used: “**-to <generated_clock> -style full**”

This report also shows the sense of the master clock at each pin in the path.

```
ptc_shell> analyze_clock_networks -to g1 -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source }
Design : CLK_0020
Scenario: default
Version: ...
Date   : ...
*****
```

Clock Sense Abbreviations:

P - positive
N - negative

Clock Network End Type Abbreviations:

CS - clock_source

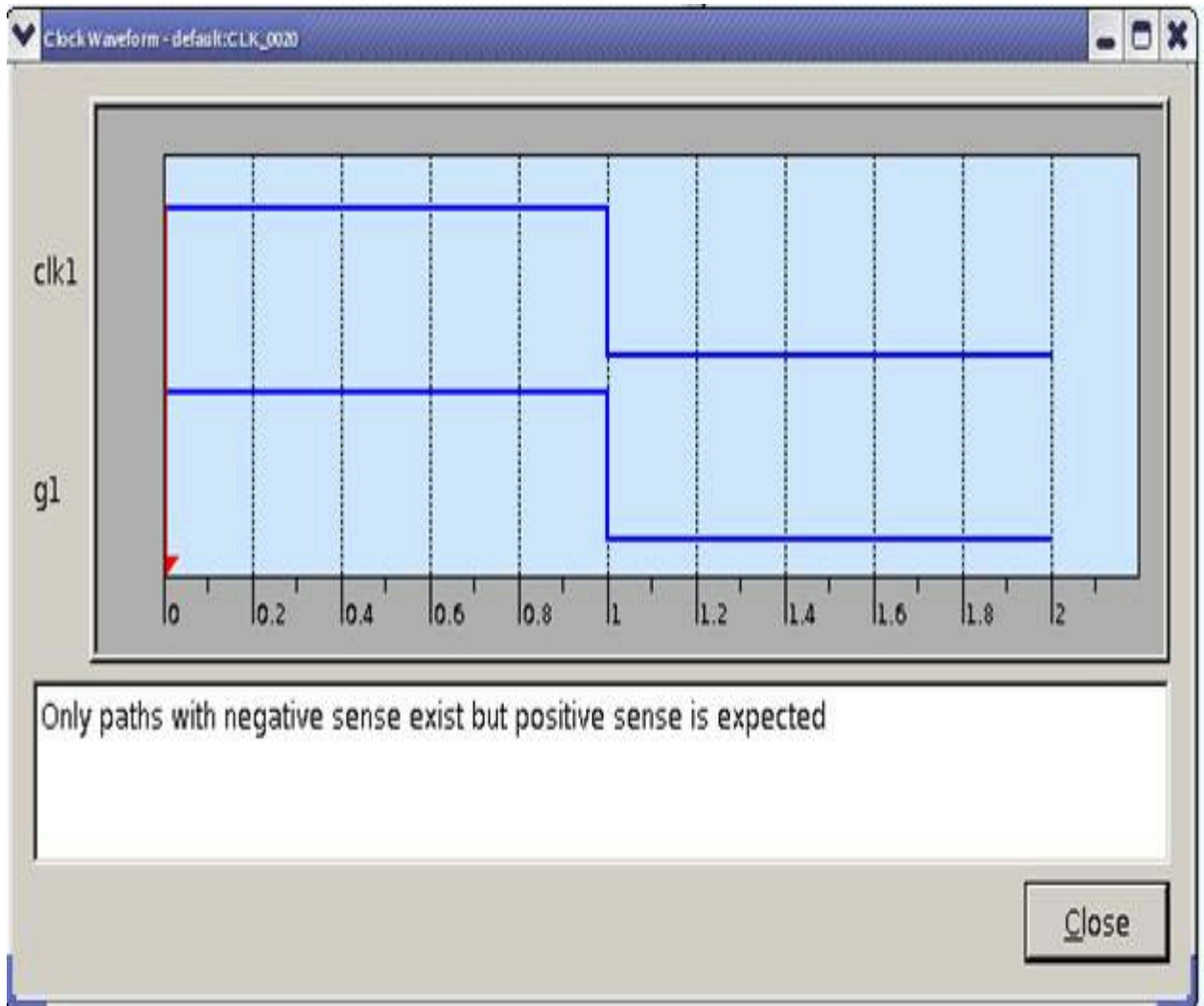
Source latency paths for generated clock: g1
from master clock: clk1

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK (in)
1		P		i1/A (IV)
2		N		i1/Z (IV)
3		N		mux/A (MUX21H)
4		N	CS	mux/Z (MUX21H)

From the report above, you can see that the generated clock (identified with the “CS” = Clock Source in the report) only has a negative sense relationship with its master (identified with the “source” word in the report).

You can also see the waveform for both master and generated clocks by clicking the waveform hyperlink on the top of the Info Pane window. The waveform will point to what edge cannot be met with a red marker. Below is an example of this waveform capability:



Fix Suggestion

This violation usually indicates some errors specifying the generated clocks. You should alter the generated clock definition so the waveform matches the existing source latency paths.

In the preceding example, this can be done by either:

- Adding the `-invert` option to the `create_generated_clock` command

```
create_generated_clock -name g1 mux/Z -source [get_ports CLK] \  
    -invert -combinational -master_clock clk1 -add
```

- Changing the `-source` to a pin after the master clock is inverted

```
create_generated_clock -name g1 mux/Z -source [get_pins mux/A] \
    -combinational -master_clock clk1 -add
```

The first corrected [create_generated_clock](#) command will lead to the following waveform as seen by the [report_clock](#) command. Note that the g1 waveform is inverted. The other generated clock statement will produce the same waveform.

```
ptc_shell> report_clock {clk1 g1}
```

```
*****
Report : clock
Design : CLK_0020
Scenario: default
Version: ...
Date : ...
*****
```

```
Attributes:
p - Propagated clock
G - Generated clock
I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
clk1	2.00	{0 1}		{CLK}
g1	2.00	{1 2}	G	{mux/Z}

Generated Clock	Master Source	Generated Source	Master Clock	Waveform Modification
g1	CLK	mux/Z	clk1	div(1), combinational, inv

Note: In the case of complex clock generation circuits, clock modifiers such as **divide_by**, **invert**, or **multiply_by** might not be enough to correctly specify your generated clocks. In these cases, the **-edge** option should be used.

Corresponding PrimeTime Error Message

Error Message: UITE-461

Severity: Error

Description

Generated clock '%s' '%s' is not satisfiable%

See Also

[create_generated_clock](#)

[analyze_clock_networks](#)

[report_clock](#)

Rule: CLK_0021

Clock **clock** is not used in this scenario.

Severity: Info

Description

A clock defined in the constraints is not used in any scenario. A clock is classified as unused if all the following conditions are met:

- no register clock pins receive the clock's signal
- no input or output delay is defined relative to the clock
- no generated clock has the clock specified as its master clock
- the clock does not propagate to an output/inout port

Risk Associated with Violation

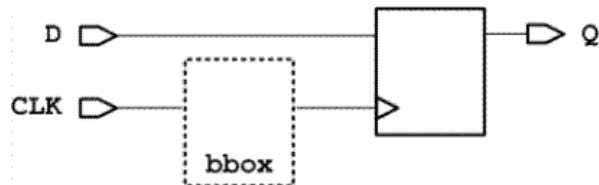
Unused clocks might be a symptom of an incorrectly or incompletely constrained design. This will also have a memory or runtime adverse effect in a static timing analysis tool since the clock still has to be accounted for.

One Possible Scenario

One possible scenario demonstrating this violation involves black boxes or unresolved references associated with a clock path. Such black boxes prevent the clock signal from propagating to register clock pins or output/inout ports and are flagged with [LNK-805](#) warning message of the form:

```
Warning: Unable to resolve reference to 'BBOX' in 'top'. (LNK-805)
```

Figure 29 *Unresolved References Associated With Clock Path*



Fix Suggestion

Check if there are any black boxes that are blocking the clock paths. It is a good idea to remove all the black boxes by reading in additional library or design files. A cell is

considered a black box if a definition for the cell is not found in any of the Verilog or library technology files.

The clock might be blocked from all registers with case settings or other constraints. If it is correct in this scenario for the clock not to be used, the clocks can be safely removed as they do not affect the function of the design.

See Also

[create_generated_clock](#)

[create_clock](#)

[set_input_delay](#)

[set_output_delay](#)

Rule: CLK_0023

There are **count** clocks on source **source** with the same period and waveform.

Severity: Warning

Description

More than one clock is defined on the same source, with identical periods and waveforms. This can result in increased runtime and memory usage. Consider combining the clocks unless they are needed for other constraint reasons such as clock latency pessimism removal. By default, this violation will be issued for generated clocks only if they have identical periods, waveforms and master clocks. If the **exclude_different_primary_masters** rule property is set to **false**, then generated clocks with identical periods and waveforms will be reported to have this violation even if their master clocks are different.

Risk Associated with Violation

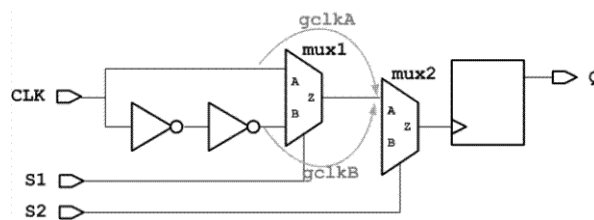
If this violation is not addressed, the timing analysis runtime and memory usage may increase without any reduction in clock latency pessimism. Failure to remove clock latency pessimism may lead to unrealistic setup and/or hold timing violations.

One Possible Scenario A

Consider a scenario where there are two generated clocks with the same generated clock source but with different master clock source pins. The master clock source pin for 'gclkA' is specified at 'mux1/A', and the master source pin for 'gclkB' at 'mux1/B'. However, for the purpose of clock latency calculation, the clock latency path begins from the point where the master clock originates, not where the master source pin is specified in the generated clock. Thus, generated clocks 'gclkA' and 'gclkB' have

- identical clock periods
- identical clock waveforms
- identical master clocks
- identical clock latency paths that start from the port 'CLK' and end at pin 'mux2/A'

Figure 30 Generated Clocks With Same Generated Clock Source; Different Master Clock Source Pins

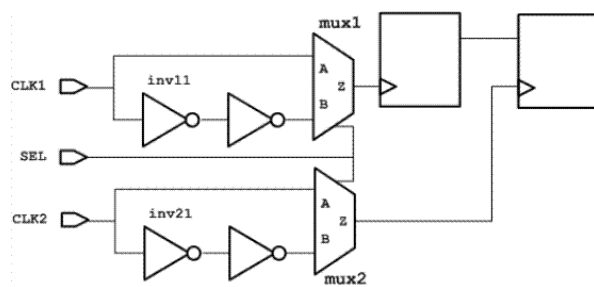


```
create_clock -name sysclk -period 4 [get_ports CLK]
create_generated_clock -name gclkA -divide_by 1 \
    -source [get_pins mux1/A] -master sysclk \
    [get_pins mux2/A] -add
create_generated_clock -name gclkB -divide_by 1 \
    -source [get_pins mux1/B] -master sysclk \
    [get_pins mux2/A] -add
```

One Possible Scenario B

Another possible scenario involves multiple clock MUXs controlled by the same select line. Since some clock latency paths are mutually exclusive, the pessimism in latency calculation can be removed by declaring some clocks as logically or physically exclusive. Consider two clock MUXs controlled by one common select line as follows:

Figure 31 Multiple Clock MUXs Controlled by Same Select Line



```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
create_generated_clock [get_pins mux1/Z] -name CLK1_SHORT \
```

```

        -source [get_pins mux1/A] \
        -divide_by 1 -add -master_clock [get_clocks CLK1]
create_generated_clock [get_pins mux1/Z] -name CLK1_LONG \
        -source [get_pins inv11/A]
        -divide_by 1 -add -master_clock [get_clocks CLK1]
create_generated_clock [get_pins mux2/Z] -name CLK2_SHORT \
        -source [get_pins mux2/A]
        -divide_by 1 -add -master_clock [get_clocks CLK2]
create_generated_clock [get_pins mux2/Z] -name CLK2_LONG \
        -source [get_pins inv21/A]
        -divide_by 1 -add -master_clock [get_clocks CLK2]
set_propagated_clock [all_clocks]
set_clock_groups -logically_exclusive \
        -group {CLK1_SHORT CLK2_SHORT} \
        -group {CLK1_LONG CLK2_LONG}

```

Two generated clocks are created at the output of each clock MUX. The long and short versions of each clock are declared as mutually exclusive by using [set_clock_groups](#). However, both the long and the short versions of the generated clocks have identical periods, waveforms, master clocks and latencies because the paths from the master clock source pin to the generated clock source pin are identical. As a result **CLK_0023** violation is issued and no pessimism in clock latency is removed despite an increase in runtime and memory usage.

What Next

The "Violation Detail" section provides the name of the clock that is being used as data as well as the clock source locations for the clock. To get more details on the data signals, use the [analyze_clock_networks](#) command.

```
ptc_shell> analyze_clock_networks -end_types data -from CLK -style end
```

End	Pin/Port	End Type	Clocks

ff2/d	(FD1)	D	set 0

```

Clock Sets:
  set 0 (1 clocks):
    CLK - positive, negative

```

As can be seen from the [analyze_clock_networks](#) command output, there is a data path that originates from the clock port 'CLK' and terminates at the register data pin 'ff2/D'.

Fix Suggestion

It is best to remove unnecessary generated clocks unless they are needed for purposes like removing pessimism in clock latency calculation. For example scenario A, the extra generated clock 'gclkB' can be removed without any impact on timing results.

For example scenario B, the duplicate generated clocks need to be changed to unique generated clocks in order to remove clock latency pessimism. This can be done by moving the generated clock source pins to the input pins of clock MUXs. This will create two unique generated clocks from each master clock such that the short (long) latency path traverses the path from the clock port to pin 'A' ('B') of the clock MUX. Now that correct latencies are associated with each generated clock, there will be fewer setup and/or hold violations to fix. The generated clock modification can be done as follows:

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
create_generated_clock [get_pins mux1/A] -name CLK1_SHORT \
    -source [get_ports CLK1] \
    -divide_by 1 -add -master_clock [get_clocks CLK1]
create_generated_clock [get_pins mux1/B] -name CLK1_LONG \
    -source [get_ports CLK1] -divide_by 1 \
    -add -master_clock [get_clocks CLK1]
create_generated_clock [get_pins mux2/A] -name CLK2_SHORT \
    -source [get_ports CLK2] \
    -divide_by 1 -add -master_clock [get_clocks CLK2]
create_generated_clock [get_pins mux2/B] -name CLK2_LONG \
    -source [get_ports CLK2] -divide_by 1 \
    -add -master_clock [get_clocks CLK2]
set_propagated_clock [all_clocks]
set_clock_groups -logically_exclusive \
    -group {CLK1_SHORT CLK2_SHORT} \
    -group {CLK1_LONG CLK2_LONG}
```

Rule Property: **exclude_different_primary_masters**

By default, the **exclude_different_primary_masters** rule property is set to **true**. The rule property can be set to **false** as follows:

```
set_rule_property exclude_different_primary_masters false CLK_0023
```

If the rule property is set to **false**, more **CLK_0023** violations will be reported because generated clocks with identical periods and waveforms will be reported to have this violation even if their master clocks are different.

See Also

[create_generated_clock](#)

[create_clock](#)

[analyze_clock_networks](#)

[set_clock_groups](#)

[set_propagated_clock](#)

Rule: CLK_0024

Register Clock pin *pin* has *count* clocks.

Severity: Warning

Status: Disabled by default

Description

This rule is disabled by default because it can give a large number of violations that the user may not be concerned about. It can be used to investigate further a [PRF_0001](#) rule violation. The number of clocks allowed at a register clock pin without triggering a violation is set with the **max_clocks_per_register** property on this rule.

The [PRF_0001](#) rule gives at most one violation. It gives the number of registers that exceed the **max_clocks_per_register** property of the [PRF_0001](#) rule. The performance of many tools is negatively impacted by having many clocks propagating to a single clock pin.

Risk Associated with Violation

There is a risk that the timing analysis will run slower and take more memory for a set of constraints with many clocks propagating to the same registers. There is also a risk that the design is not constrained for the functional modes as expected. The large number of clocks could also be due to an error in the case analysis settings for clock definitions in the design.

One Possible Scenario

You have a set of constraints that is meant to represent one functional mode for a design. It is expected that no register has more than one clock for this set of constraints. If any register has more than one clock, then for this example design there are other constraints missing that select the correct clock for this mode. To check the number of clocks reaching clock pin registers in the design, you would include the following commands in your run script:

```
enable_rule CLK_0024
set_rule_property max_clocks_per_register 1 CLK_0024
```

During [analyze_design](#), a **CLK_0024** violation will be issued if any clock pin has more than one clock.

What Next

The “Violation Details” section provides the names of the clocks at the register that has more than the maximum number of clocks. To check the clock network to the register in the violation, use the [analyze_clock_networks](#) command. The clock pin can be supplied in

the **-to** option and the **-style full** option can be used to display the full details of the clock network. For the example scenario, the command and its output are shown below.

```
ptc_shell> analyze_clock_networks -to [get_pins ff1/CP] -style full
Clock Sense Abbreviations:
  P - positive
  N - negative
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: CLK1

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK1 (in)
1		P		mux1/A (MUX21H)
2		P		mux1/Z (MUX21H)
3		P	REG	ff1/CP (FD1)

Full report for clock: CLK2

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK2 (in)
1		P		inv12/A (IV)
2		N		inv12/Z (IV)
3		N		mux1/B (MUX21H)
4		N		mux1/Z (MUX21H)
5		N	REG	ff1/CP (FD1)

The report shows that **CLK1** and an inverted version of **CLK2** are reaching the **ff1/CP** clock pin through the mux **mux1**.

Fix Suggestion

If you are using this rule to check that only one clock reaches each register, then you need to add additional case analysis values such that only one clock propagates through the design's various clock selector logic. For the example scenario, the violation can be resolved by deciding to propagate only the positive sense clock **CLK1**. This can be achieved with the following command:

```
set_case_analysis 0 [get_ports SEL]
```

If you are using this rule to determine whether merging several functional modes has created a large number of registers with more than **max_clocks_per_register**, you might want to turn off this rule and just use the **PRF_0001** rule.

Rule Property: max_clocks_per_register

The default of the rule property, **max_clocks_per_register**, can be queried using the following command:

```
get_rule_property max_clocks_per_register CLK_0024
```

It can be set to a different value using the [set_rule_property](#) command:

```
set_rule_property max_clocks_per_register 10 CLK_0024
```

The number of violations reported by this rule is dependent on the setting of this property. If it is set to a large limit, then pins below that limit will not be reported as violations.

See Also

[PRF_0001](#)

[analyze_clock_networks](#)

[get_rule_property](#)

[set_rule_property](#)

[analyze_design](#)

[disable_rule](#)

[enable_rule](#)

Rule: CLK_0026

Clock **clock** is used as data. One or more sources of the clock fans out to a register data pin or to a constrained primary output or inout port.

Severity: Warning

Description

The clock is used as data in the design. This can cause multiple problems during optimization, clock tree synthesis, and testing. Clock signals have a high fanout. When they are used as data this can cause large transition time violations in data paths which cannot be fixed by optimization tools. Also, if the data paths are not identified the clock tree synthesis runtime and quality can be adversely impacted. When the clock signals drive the data pin of registers, a race condition is created between the data pin and the clock pin of the registers. The race conditions may lead to the creation of poor test patterns or low test coverage.

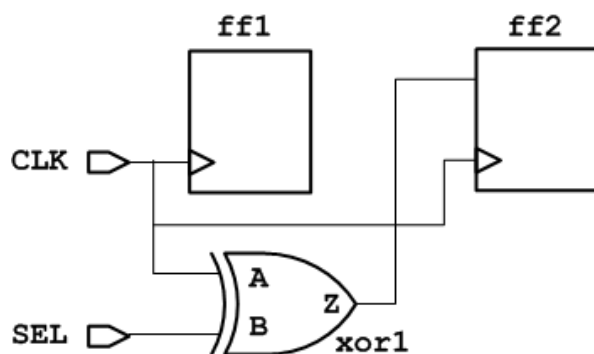
Risk Associated with Violation

Clocks used as data may cause large transition time violations which cannot be fixed during optimization; this can increase runtime or degrade quality during clock tree synthesis. It can also result in poor test pattern generation or low test coverage for design testing.

One Possible Scenario

Consider a scenario where a clock signal drives register data pins and register clock pins. At register 'ff2' the clock signal 'CLK' appears as data at the data pin and as clock at the clock pin.

Figure 32 Clock Signal Driving Register Data Pins and Register Clock Pins



```
create_clock -name CLK -period 1 [get_ports CLK]
set_output_delay -clock CLK 0.5 [all_outputs]
```

What Next

The "Violation Details" section provides the name of the clock that is being used as data and the clock source locations. Use the [analyze_clock_networks](#) command to get more detail about the data signals as shown in the report below:

```
ptc_shell> analyze_clock_networks -from [get_clocks CLK] -end_type data

*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style short
        -end_types {data}
Design : CLK_0006
Scenario: default
Version: ...
Date : ...
*****
Clock Network End Type Abbreviations:
    D - data

End
```

Example	End Pin/Port	Type	Clocks	Count
ff2/D	(FD1)	D	set 0	1

Clock Sets:
set 0 (1 clocks):
CLK - positive, negative

The report generated using the [analyze_clock_networks](#) command shows there is a data path that originates from the clock port **CLK** and terminates at the register data pin **ff2/D**. For more detail about this particular data path, use the **-to [get_pins ff2/D]** option, the **-style full** option, and the **-end_types data** option as shown below:

```
ptc_shell> analyze_clock_networks -from [get_clocks CLK] -end_type data
-to [get_pins ff2/D] -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {data}
Design : CLK_0006
Scenario: default
Version: ...
Date : ...
*****
```

```
Clock Sense Abbreviations:
P - positive
P,N - positive, negative
Clock Network End Type Abbreviations:
D - data
```

Full report for clock: CLK

Branch 0:

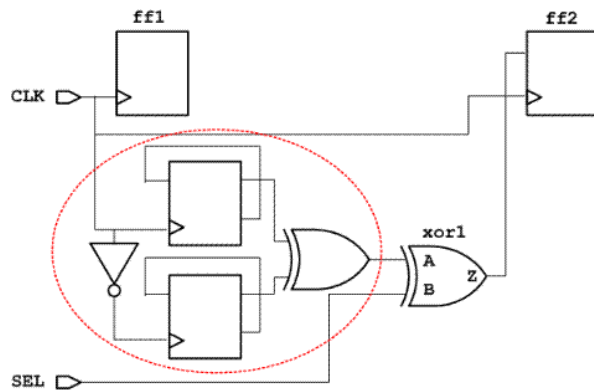
Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK (in)
1		P		xor1/A (EO)
2		P,N		xor1/Z (EO)
3		P,N	D	ff2/D (FD1)

The report shows a data path from the port **CLK** through the XOR gate **xor1** to the register data pin **ff2/D**.

Fix Suggestion

Check if the clock needs to be used as both data and clock. If possible, modify the design so the clock is not used as data. If the clock must be used as both clock and data, consider using clock isolation logic so the data paths and clock paths are separated. A simple clock isolation solution using a two-phase shifted clock divider is shown below:

Figure 33 Simple Clock Isolation Logic Using Two Phase-Shifted Clock Dividers



See Also

[create_clock](#)

[analyze_clock_networks](#)

Rule: CLK_0027

At pin, there is a conflicting set_clock_sense for **clock**.

Severity: Error

Description

There is a [set_clock_sense](#) constraint at the given pin and the requested clock sense does not exist at the pin for the given clock. This may be due to a conflicting [set_clock_sense](#) constraint on a previous pin, or it may be because the only clock paths to that pin are not of the sense specified.

Risk Associated with Violation

The [set_clock_sense](#) constraint for the pin will be ignored because the requested clock sense does not exist. The timing analysis performed on the design will not use the specified clock sense. This could affect the timing results for the design.

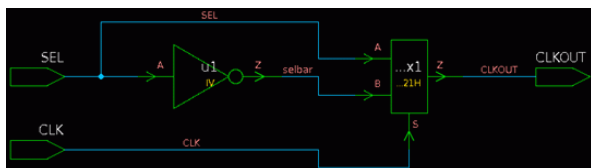
One Possible Scenario

Consider a scenario where a clock signal propagates from a select pin to an output pin of a mux. The clock sense may become positive or negative through the mux, based on the logic values at the inputs of the mux.

Port SEL value Clock Sense at mux1/Z

0	Positive
1	Negative

Figure 34 Clock Signal Propagates From Select Pin to Output Pin of MUX



```
create_clock -name CLK -period 2 [get_ports CLK]
set_case_analysis 0 [get_ports SEL]
set_clock_sense -negative mux1/Z
```

What Next

The “Violation Details” section provides the name of the clock and the location of the [set_clock_sense](#) setting. It will also show the requested sense and the available sense of the clock. Use the [analyze_clock_networks](#) command to find out how the clock sense changes along the relevant clock path. The **-style full** option provides full details, including:

- multiple paths (branches)
- logic levels
- clock sense (positive, negative, rise_triggered, fall_triggered)
- sequential CLK-to-Q paths (REG)

The clock name can be supplied as the **-from** option and the [set_clock_sense](#) pin to the **-through** option. Also include the **-traverse_disabled** option to see conflicting clock senses in the potential clock network. For the example scenario, the command would be issued as follows:

```
ptc_shell> analyze_clock_networks -traverse_disabled -from
[get_clocks{CLK}] \
                                -through [get_pins {mux1/Z}] -style full
*****
Report : analyze_clock_networks
        -max_endpoints=1
        -style full
        -end_types {register port clock_source }
        -traverse_disabled
Design : ...
Scenario: ...
Version: ...
```

```
Date      : ...
*****
Clock Sense Abbreviations:
  P - positive
Potential senses detected with -traverse_disabled:
  [N] - potential negative
Clock Network End Type Abbreviations:
  PORT - port

Full report for clock: CLK

Branch 0:
      Branch
Level  Info      Sense  Notes      Port/Pin
-----
      0          P      source    CLK (in)
      1          P          mux1/S (MUX21H)
      2          P [N]    IGSNS#0    mux1/Z (MUX21H)
      3          P [N]    PORT      CLKOUT (out)

Clock Blocking Constraints:
  IGSNS#0 = ignored set_clock_sense
    at pin: mux1/Z
```

The report shows that there is a potential negative clock at the output pin of the mux, but that is not the clock sense that is propagating.

Fix Suggestion

Change the constraints so that the conflicting clock sense is resolved.

For the example scenario, it is not necessary to select the clock sense if the case value is used. The [set_clock_sense](#) command can be removed from the golden constraint file to resolve the violation.

See Also

[set_clock_sense](#)

[analyze_clock_networks](#)

Rule: CLK_0028

The master clock for generated clock **clock** is ambiguous. There are multiple clocks on -source pin **master_source**.

Severity: Warning

Description

The generated clock has been created without a **-master_clock** or **-add** options. The **-source** pin has more than one potential master clock leading to an ambiguous definition. Constraint consistency will, by default, pick the master clock with the least numbers of generated clocks as the primary master. If the potential master clocks tie for the number of levels, then the first created master clock will be chosen.

Risk Associated with Violation

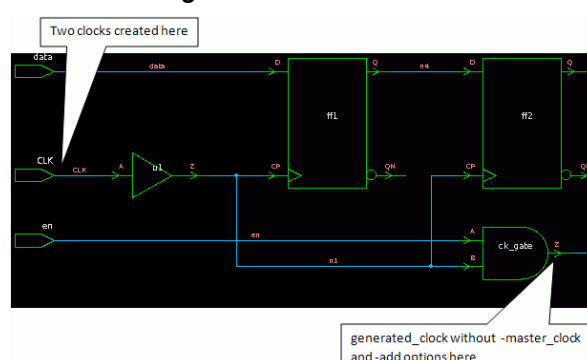
Because the clock definition is ambiguous, there is a non negligible level of unknown associated with that clock definition. Not all tools in the flow may pick the same master clock and, even if they do, the constraints may be altered and the master clocks order changed in the constraint file changing the master clock for this generated clock. The waveform computed for this generated clock is not dependable and can potentially lead to incorrect timing analysis.

One Possible Scenario

In the example below, a generated clock **CLK_gate** has been added at the output of a clock gating cell. The generated clock does not have a **-master_clock** specified, and the **-source** port has two clocks defined on it. This leads to an ambiguous clock definition. Constraint consistency will select the first clock created at the port, **CLK_SCAN** as the master clock of generated clock **CLK_gate**. This example also has an additional generated clock (properly specified) at the same location with the **-master_clock** and **-add** options.

```
create_clock -name CLK_SCAN -period 5 [get_port CLK] -add
create_clock -name CLK_NORM -period 2.3 [get_port CLK] -add
create_generated_clock -name CLK_gate -combinational \
    -source [get_port CLK] ck_gate/Z
create_generated_clock -name CLK_gate2 -combinational \
    -source [get_port CLK] ck_gate/Z \
    -master_clock CLK_NORM -add
```

Figure 35 Ambiguous Clock Definition



What Next

You can use the [report_clock](#) command to see the master clock and waveform for the generated clock.

For this example scenario, the command would be:

```
ptc_shell> report_clock
```

Attributes:

```
p - Propagated clock
G - Generated clock
I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
CLK_NORM	2.30	{0 1.15}	p	{CLK}
CLK_SCAN	5.00	{0 2.5}	p	{CLK}
CLK_gate	5.00	{0 2.5}	p, G	{ck_gate/Z}
CLK_gate2	2.30	{0 1.15}	p, G	{ck_gate/Z}

Generated Clock	Master Source	Generated Clock	Master Modification	Waveform
CLK_gate combinational	CLK	ck_gate/Z	CLK_SCAN	div(1),
CLK_gate2 combinational	CLK	ck_gate/Z	CLK_NORM	div(1),

Fix Suggestion

Remove the ambiguity when creating the generated clock by using the **-master clock** and **-add** options to the [create_generated_clock](#). This will allow a correct and dependable computation of the clock waveforms.

For the preceding example scenario:

```
create_generated_clock -name CLK_gate -combinational \
    -source [get_port CLK] ck_gate/Z \
    -master_clock CLK_SCAN -add
```

Corresponding PrimeTime Error Message

Error Message: PTE-004

Severity: Warning

Description

The master clock for generated clock clock is ambiguous. There are multiple clocks on the -source pin master_source.

See Also

[create_generated_clock](#)
[report_clock](#)

Rule: CLK_0029

The last edge shift value for generated clock **clock** is not equal to the first edge shift.

Severity: Warning

Description

The generated clock is specified with the **-edges** and **-edge_shift** options. The last edge specifies when the generated waveform repeats. Normally, the shift value for this last edge should be the same as the first shift value, resulting in a generated clock period equal to the difference between the last edge and the first edge.

Risk Associated with Violation

Specifying a different shift for the last edge may result in an edge relationship between the master clock and generated clock that changes from period to period. Clock generation circuitry is typically not designed to generate clocks of this type.

One Possible Scenario

Consider a scenario where a generated clock is created using edges and the timing of the edges is modified with the **-edge_shift** option. The last edge shift value is **1.1** time units and the first edge shift value is **0.2** time units in the generated clock definition:

```
create_clock -period 10 [get_ports CLK]
create_generated_clock -source CLK ffl/Q -name genc \
    -edges {1 1 3} -edge_shift {0.2 0.6 1.1}
```

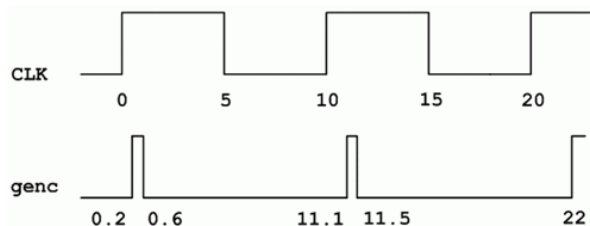
Because the first and last edge shift values are different, the tool issues a **CLK_0029** rule violation.

What Next

The violation message will give you the name of the generated clock that has different first and last edge shift values.

Examine the edge relationships between the clocks to verify whether or not the edge relationship between the master clock and generated clock remains the same between periods. For the example scenario, the first two periods of the master clock and generated clock are shown as follows:

Figure 36 Edge Relationship Between Master Clock and Generated Clock



For the first period, the time difference between the rising edge of the master clock and generated clock is 0.2 time units because $(0.2 - 0 = 0.2)$. For the second period, the time difference between the rising edge of the master clock and generated clock is 1.1 time units because $(11.1 - 10 = 1.1)$. The difference in the edge shift values causes the relationship between the rising and falling edges of the master clock and generated clock to change with each period of the clock.

Fix Suggestion

If the edge relationship between the master clock and generated clock remains the same between periods, the violation can be waived. If the edge relationship varies, consider modifying the **-edge_shift** specification so that the first and last edge shift values match.

See Also

[create_generated_clock](#)

[create_clock](#)

Rule: CLK_0030

There is reconvergent logic in the network for clock **clock**.

Severity: Warning

Description

There is some reconvergent logic in the clock network of the reported clock. This might impact the accuracy of timing analysis.

Risk Associated with Violation

Reconvergent paths may result in pessimistic timing results due to non-unate or reconvergent clock network. Non-unate clock network is discussed in more details for rule [CLK_0018](#). Reconvergent paths may lead to pessimistic timing results because static timing analysis tools can use one latency path for setup analysis and a different latency path for hold analysis even though both latency paths share a common point from which all downstream sequential elements are driven. There are methods to remove such

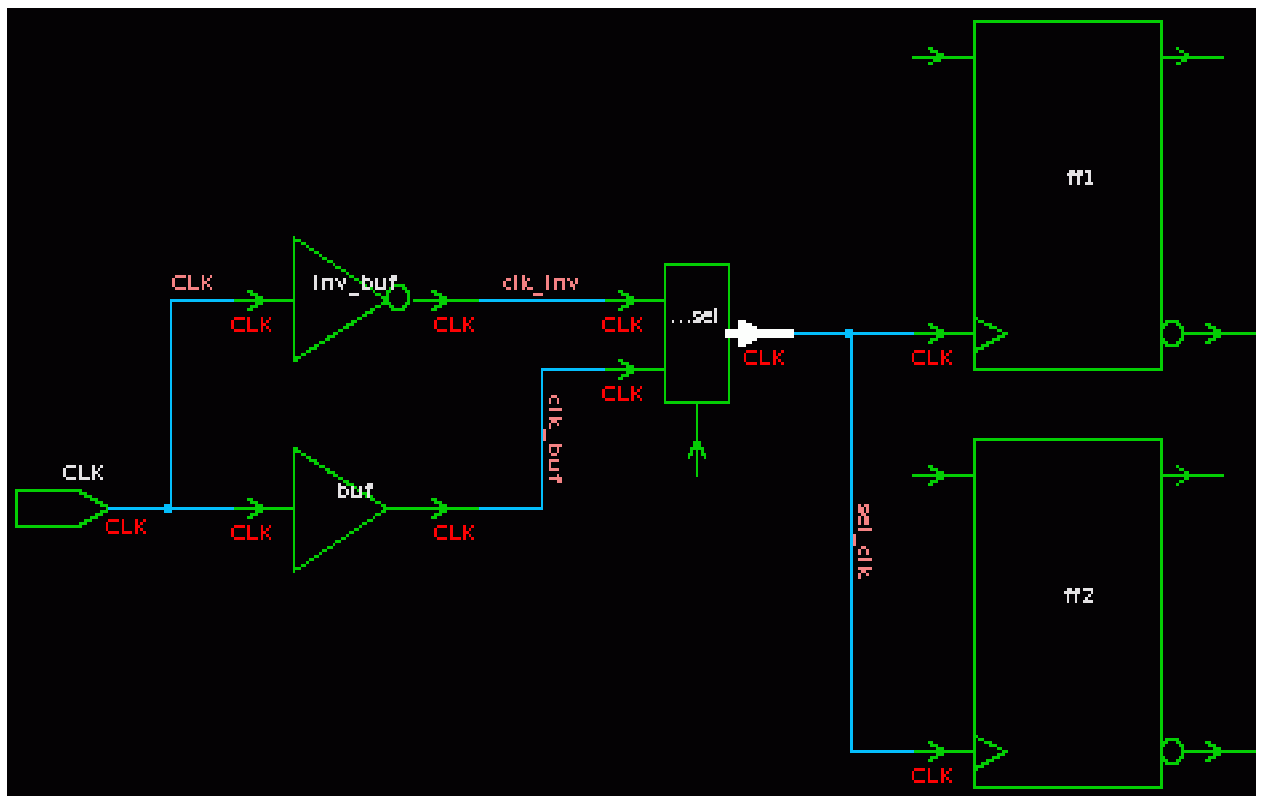
pessimism like Clock Reconvergence Pessimism Removal (CRPR) but such methods lead to increased runtime or memory usage in timing analysis.

One Possible Scenario A: Reconvergent paths create non-unate clock sense

Consider the following design with a clock defined at the input port 'CLK':

```
create_clock -period 10 CLK
```

Figure 37 Reconvergent Paths Creating Non-Unate Clock Sense

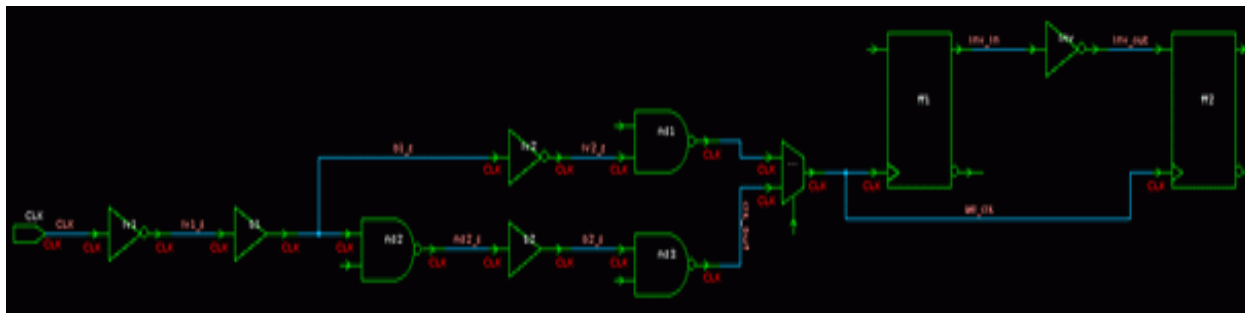


This example shows a clock reconvergence leading to a non-unate clock situation at the output of the MUX. Static timing analysis tools will allow both the negative and the positive sense of CLK to propagate simultaneously through the MUX. This is physically not possible and will lead to pessimistic timing results. Such pessimism can be removed by defining two generated clocks on the inputs of the MUX and declaring them to be mutually exclusive using clock groups.

One Possible Scenario B: Clock reconvergence creates pessimism in timing analysis

Consider the example shown in the schematic below. The clock path is sharing a common path before splitting into separate branches and reconverging at the MUX output. This can create pessimistic timing results.

Figure 38 Clock Reconvergence Creating Pessimism in Timing Analysis



What Next

The "Violation Details" section lists all the reconvergent points. Use the [analyze_clock_networks](#) command through each of the reconvergent point with **-style full** option to get more details.

For **Scenario A**, the clock network report appears as follows:

```
ptc_shell> analyze_clock_networks -through clk_sel/Z -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source }
Design : UNC_0001
Scenario: default
Version: ...
Date : ...
*****
```

Clock Sense Abbreviations:

```
P - positive
N - negative
P,N - positive, negative
```

Clock Network End Type Abbreviations:

```
REG - register
```

Full report for clock: CLK - 3 partial path branches

Branch 0:

Level	Info	Sense	Notes	Port/Pin
0	S2	P	source	CLK (in)

1		P		inv_buf/A (IV)
2		N		inv_buf/Z (IV)
3		N		clk_sel/A (MUX21H)
4	S1,E2	P,N		clk_sel/Z (MUX21H)
5		P,N	REG	ff1/CP (FD1)

Branch 1: from branch 0

Level	Info	Sense	Notes	Port/Pin
5		P,N	REG	ff2/CP (FD1)

Branch 2: from branch 0 reconverges to branch 0

Level	Info	Sense	Notes	Port/Pin
1		P		buf/A (B1I)
2		P		buf/Z (B1I)
3		P		clk_sel/B (MUX21H)

The point of reconvergence is indicated by the “E2” label. Notice the “P, N” information displayed in the report at that place. This indicates that both the Positive and Negative sense of the clock are propagating simultaneously here, hence the non-unate clock.

For **Scenario B**, you can also use the [analyze_clock_networks](#) command to visualize where the branches and reconvergence pins are in the design. For the preceding design, the report appears as follows:

```
*****
Report : analyze_clock_networks
Design : UNC_0001
Scenario: default
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
Clock Network End Type Abbreviations:
  REG - register
  CG - clock_gating

Full report for clock: CLK - 2 partial path branches

Branch 0:
  Branch
Level  Info      Sense Notes      Port/Pin
-----
0      0          P      source    CLK (in)
1      1          P          iv1/A (IV)
2      2          N          iv1/Z (IV)
3      3          N          b1/A (B1I)
```

4	S1	N		b1/Z (B1I)
5		N		iv2/A (IV)
6		P		iv2/Z (IV)
7		P	CG	nd1/B (ND2)
8		N		nd1/Z (ND2)
9		N		clk_sel/A (MUX21H)
10	E1	N		clk_sel/Z (MUX21H)
11		N	REG	ff1/CP (FD1)

Branch 1: from branch 0 reconverges to branch 0

Level	Info	Sense	Notes	Port/Pin
5		N	CG	nd2/A (ND2)
6		P		nd2/Z (ND2)
7		P		b2/A (B1I)
8		P		b2/Z (B1I)
9		P	CG	nd3/A (ND2)
10		N		nd3/Z (ND2)
11		N		clk_sel/B (MUX21H)

You can identify the reconvergence point with its “E1” label, but in that case there is no non-unate clock (as opposed to the scenario A design).

Fix Suggestion

For **Scenario A**, if the reconvergence is correct then specify generated clocks at the reconvergent points and declare them as mutually exclusive. Here is an example:

```
ptc_shell> create_generated_clock -invert -multiply_by 1 \
-name inv_clk -source CLK clk_sel/A
ptc_shell> create_generated_clock -multiply_by 1 -name buf_clk \
-source CLK clk_sel/B
ptc_shell> set_clock_groups -physically_exclusive \
{inv_clk} -group {buf_clk}
```

For **Scenario B**, you might want to use Clock Reconvergence Pessimism Recovery (CPRP) techniques. This can be done in by using the **timing_remove_clock_reconvergence_pessimism** variable as in:

```
ptc_shell> set_app_var timing_remove_clock_reconvergence_pessimism true
```

Rule Property: enable_async_endtype

By default, the **enable_async_endtype** rule property is set to false. If it set to true, the reconvergence clock connected to asynchronous pins (like SD/CD) is reported. But by default, it will not report CLK_0030 for reconvergence logic present on asynchronous pins.

See Also

[create_clock](#)

[analyze_clock_networks](#)
[create_generated_clock](#)
[set_clock_groups](#)

Rule: CLK_0031

The waveform for clock **clock** has more than two edges.

Severity: Info

Description

Most waveforms have a single pair of rise and fall edges. Some tools do not support waveforms with four or more edges.

Risk Associated with Violation

There may be design tools that do not support clock definitions with more than two edges in their specification. All Galaxy tools support such a clock definition.

One Possible Scenario

Consider a clock definition with more than one pair of rise and fall edges.

```
create_clock -period 10 CLK -waveform {0 1 3 4}
```

All Galaxy tools support more than two edges in the waveform specification. When a clock has more than two edges in its definition, the tool issues a **CLK_0031** violation.

What Next

Check to see if clock specifications of this type are supported by all of the design tools in your flow. If they are, this violation can be waived.

See Also

[create_clock](#)

Rule: CLK_0032

Non-combinational generated clock **clock** has a source identical to its master source object.

Severity: Error

Description

The reported generated clock has the same source as its master source. Since the generated clock is not combinational, this is an incorrect statement.

Risk Associated with Violation

The constraint describes a condition that is not physically possible in a real design. This will most likely result in incorrect timing analysis since the clock relationship is erroneous.

One Possible Scenario

Consider a scenario where a generated clock is declared as divide-by-2 from its master source, also acting as its source pin:

```
create_generated_clock -name GCLK -source ffl/CP -divide_by 2 ffl/CP
```

What Next

The “Violation Details” reports the pin acting as both the source and master source pin for the generated clock.

Fix Suggestion

There are two different approaches to solving this issue depending where the error was made in the generated clock definition:

- You can declare the generated clock to have a combinational relationship with the master clock
- You can move the master source pin such that the latency path satisfies the master/generated clock relationship.

For example, in the preceding scenario, changing the generated clock declaration to the follow would solve the issue:

```
create_generated_clock -name GCLK -source ffl/CP -combinational
```

See Also

[create_generated_clock](#)

[analyze_clock_networks](#)

Rule: CLK_0033

Clock **clock** has ideal latency during post-layout analysis.

Severity: Error

Status: Disabled by default

Description

All clocks should be propagated during post-layout analysis. Virtual clocks and clocks defined on output ports are excluded from this check.

Risk Associated with Violation

Ideal clocks in post-layout analysis may lead to inaccurate timing results because clock latencies are not considered. This can lead to missed timing violations in the design.

One Possible Scenario

Consider a post-layout design where it is desired that all clocks should be propagated. In the constraint file, there is a `set_propagated_clock` command that is applied to all the clocks.

```
create_clock -name CLK1 -period 1 [get_ports CLK1]

set_propagated_clock [all_clocks]
create_clock -name CLK2 -period 2 [get_ports CLK2]
```

However, the clock definition for **CLK2** appears after the `set_propagated_clock` command, so **CLK2** does not get propagated. As a result, a **CLK_0033** violation will be issued by the tool for **CLK2**.

What Next

The violation message will include the name of the clock that is not propagated. Virtual clocks and clocks defined on output ports are not included in this check.

You can also obtain a report of the clocks in your design and whether they are propagated with the `report_clock` command. For the example scenario, the report can be obtained as follows:

```
ptc_shell> report_clock

*****
Report : clock
Design : ...
Scenario: ...
Version: ...
Date : ...
*****

Attributes:
p - Propagated clock
G - Generated clock
I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
CLK1	1.00	{0 0.5}	p	{CLK1}

```
CLK2      2.00      {0 1}      {CLK2}
```

The report shows that **CLK1** is a propagated clock, while **CLK2** is an ideal clock.

Fix Suggestion

For post-layout analysis, determine if the specified clock should be made propagated and include it in the collection of clocks to be propagated. For the example scenario, this can be done by moving the [set_propagated_clock](#) command in the constraint file so that it occurs after the last clock definition:

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
set_propagated_clock [all_clocks]
```

See Also

[set_propagated_clock](#)

[create_clock](#)

[create_generated_clock](#)

[report_clock](#)

Rule: CLK_0034

Clock **clock** has propagated latency during pre-layout analysis.

Severity: Error

Status: Disabled by default

Description

All clocks should be ideal during pre-layout analysis. Virtual clocks and clocks defined on output ports are excluded from this check.

Risk Associated with Violation

Propagated clocks in pre-layout analysis will have their latency calculated with cell delays only, since no detailed parasitics are present in the design for interconnect delay. This may lead to inaccurate timing results.

One Possible Scenario

Consider a pre-layout design with two clocks, **CLK1** and **CLK2**. In the constraint file, there is a [set_propagated_clock](#) command that is applied to clock **CLK2**.

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
```

```
create_clock -name CLK2 -period 2 [get_ports CLK2]
set_propagated_clock [get_clocks CLK2]
```

As a result, **CLK1** will be ideal, but **CLK2** is propagated. When this rule is enabled, a **CLK_0034** violation will be issued for **CLK2** because it is propagated.

What Next

The violation message will include the name of the clock that is propagated. Virtual clocks and clocks defined on output ports are not included in this check.

You can also obtain a report of the clocks in your design and whether they are propagated with the [report_clock](#) command. For the example scenario, the report can be obtained as follows:

```
ptc_shell> report_clock

*****
Report : clock
Design : ...
Scenario: ...
Version: ...
Date : ...
*****

Attributes:
p - Propagated clock
G - Generated clock
I - Inactive clock

Clock      Period    Waveform    Attrs    Sources
-----
CLK1       1.00      {0 0.5}
CLK2       2.00      {0 1}      p      {CLK2}
```

The report shows that **CLK2** is a propagated clock, while **CLK1** is an ideal clock.

Fix Suggestion

For pre-layout analysis, consider removing the propagated clock declaration and replacing it with the [set_clock_latency](#) command. This command can be removed later when post-layout data is available for the design. For the example scenario, the [set_propagated_clock](#) command can be removed and replaced with a [set_clock_latency](#) command.

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
set_clock_latency 2.36[get_clocks CLK2]
```

See Also

[set_propagated_clock](#)

[remove_propagated_clock](#)
[set_clock_latency](#)
[create_clock](#)
[create_generated_clock](#)
[report_clock](#)

Rule: CLK_0035

No clock-gating check inferred for the instance: **cell** clock pin: **clk_pin** enable pin: **enable_pin** lib cell: **lib_cell** clk name: **clk_name**.

Severity: Warning

Description

The enable pin specified converges with a clock network, but no clock gating check can be inferred. The library cell logic does not create a high or low clock gating check, and there is no [set_clock_gating_check-high](#) or **-low** or [set_disable_clock_gating_check](#) constraint at this location.

Constraint consistency can infer the clock gating check level (high or low) if the gate logic can be determined to be equivalent to logic AND or logic OR. Otherwise, the you need to specify the check level to enable clock gating checks on the cell.

Risk Associated with Violation

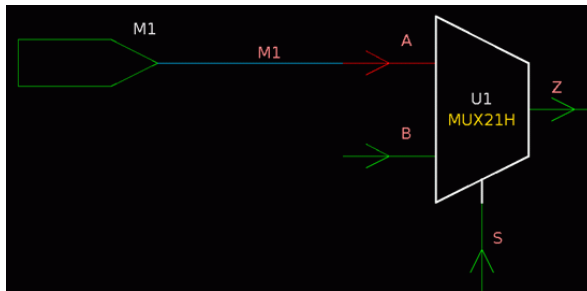
The clock gating checks will not be created because the clock gating check level cannot be inferred. Any timing violations related to the clock gating signal cannot be detected and will not be reported.

One Possible Scenario

Consider a scenario shown in Figure 1, where a clock is created on the clock port **M1**, and a clock gating timing check cannot be inferred on the **MUX21H** cell since it does not have the logic AND or logic OR:

```
create_clock -period 5 [get_ports M1] -name m1
```

Figure 39 Unable to Infer Clock Gating Check



What Next

The violation message provides the name of the cell where a clock gating check cannot be inferred, along with the corresponding clock pin, enable pin, and library cell.

Fix Suggestion

To enable the clock gating check on this cell, use the [set_clock_gating_check](#) command with the **-high** or **-low** option to specify the check level information. If clock gating checks are not required, you can remove this violation by disabling the clock gating checks for this cell with the [set_disable_clock_gating_check](#) command.

See Also

[set_clock_gating_check](#)

[set_disable_clock_gating_check](#)

[report_clock_gating_check](#)

Rule: CLK_0036

Generated clock **clock** (file: **gclk_flr**) has incorrect waveform inversion with its master clock **master_clock** (file: **mclk_flr**) that cannot be satisfied.

Severity: Error

Description

There are edge triggered cells in the paths between the master clock and the generated clock that might involve waveform inversion. The specified clock waveform cannot be computed under the current conditions. It is likely that your generated clock definition has either:

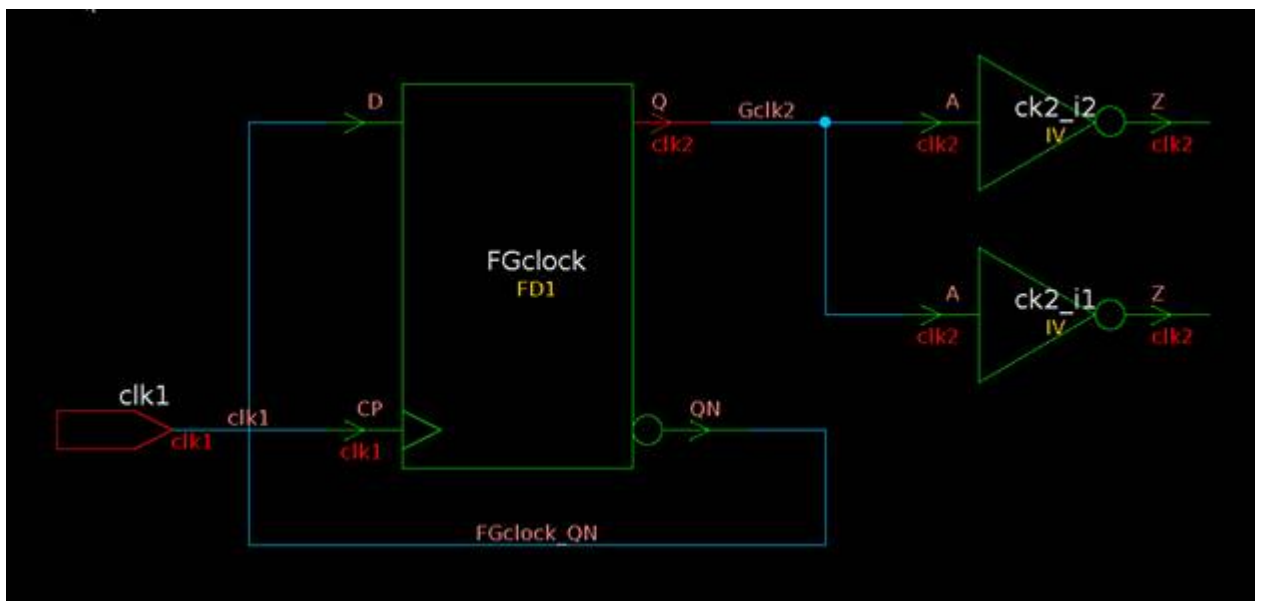
- a missing or an incorrect **-invert** option
- a missing or an incorrect inverting **-edges** option

This violation will not be issued for generated clocks that already have [CLK_0020](#) or [CLK_0008](#) violations.

Risk Associated with Violation

If the generated clock definition incorrectly specifies waveform inversion, all timing paths affected by the generated clock can receive the wrong waveform. This can affect timing results of those paths and can negatively affect runtime and accuracy during timing optimization and timing closure. When performing this analysis, constraint consistency looks at inversions created with the *-invert* switch as well as inversions created by the *-edges* definitions.

One Possible Scenario



```
create_clock
-period 10 -name clk1 [get_ports clk1]
create_generated_clock
-divide_by 2 -name clk2 \
-source [get_ports clk1] \
-master [get_clocks clk1] \
[get_pins FGclock/Q] -add -invert
```

In the preceding example, the generated clock, `clk2`, is defined with a waveform inversion on the `create_clock` command. However, this doesn't match the connectivity in the actual design (the waveform should not be inverted). This results in constraint consistency issuing a **CLK_0036** violation.

What Next

The Violation Details section shows whether the *-invert* option is missing or incorrectly specified. For the above example, it shows the following information:

Generated clock 'clk2' is specified with the *-invert* option but inversion is not expected.

The Debugging Help section provides the command you can use to further to identify the cone of logic between the master clock and the generated clock source pin. For the preceding example, it has information about the following command, which also generates the schematic.

```
ptc_shell> analyze_clock_networks -through [get_ports {clk1}]
-to [get_clocks {clk2}] -max_endpoints 1 -style full -end_types
{clock_source} -nosplit
```

```
*****
```

```
Report : analyze_clock_networks
```

```
-max_endpoints=1
```

```
-style full
```

```
-end_types {clock_source}
```

```
-no_split
```

```
Design : B2T_CLK_0003
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Clock Sense Abbreviations:
```

```
P - positive
```

```
R - rise_triggered
```

```
Clock Network End Type Abbreviations:
```

```
REG - register
```

```
CS - clock_source
```

```
Source latency paths for generated clock: clk2
```

```
from master clock: clk1
```

```
Branch 0:
```

Branch			
Level	Info	Sense Notes	Port/Pin

0	P	source	clk1 (in)
1	P	REG	FGclock/CP (FD1)
2	P		FGclock/CP (FD1)
3	R	CS	FGclock/Q (FD1)

What Next

Check the generated clock definition and the design and redefine the generated clock with the *-invert* option added or removed. In the example scenario previously described, the *-invert* option should be removed to match the clk2 waveform to the design.

report_clock

analyze_clock_networks

CLK_0020

CLK_0008

There are other clock sources in the clock latency path from the sources of the master clock 'master_clock' to the source of generated clock 'clock'.

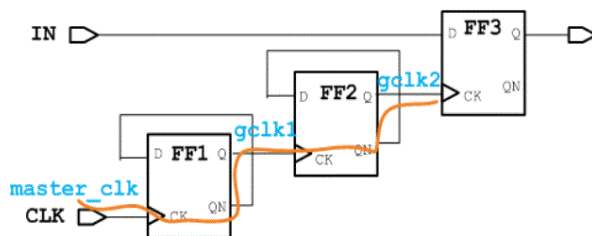
Severity: Warning

Enabled by default

There are clock sources that launch clocks different from the current generated clock or the master of current generated clock in the paths between the master clock and the generated clock. You might want to redefine the generated clock in terms of the clocks that are present in these paths instead of the current master clock.

When clock latency paths from a master clock source to a generated clock source contain pins which are the sources of other clocks, static timing tools might perform traversals involving both sequential logic and other clock sources, causing analysis complications (such as missing inferred clock gating checks) or increased runtime.

Consider the clock generation structure described in the schematic below and the following clock definitions. The clock paths are highlighted in orange in the schematic:



```
create_clock -name master_clk -period 10 -waveform [list 0 5] [get_ports CLK]

create_generated_clock -add -name gclk1 -master master_clk-source [get_ports CLK] -divide_by 2 [get_pins FF1/Q]

create_generated_clock -add -name gclk2 -master master_clk-source [get_ports CLK] -divide_by 4 [get_pins FF2/Q]
```

In this example, both generated clocks *gclk1* and *gclk2* are specified with respect to the same master clock *master_clk*. However, the generated clock, *gclk1*, is created between the clocks *master_clk* and *gclk2*. As a consequence, *gclk2* should be expressed in terms of *gclk1* instead of in terms of *master_clk*. Constraint consistency issues a **CLK_0037** violation whenever this situation is detected in the design.

What Next

The Violation Details section displays additional information related to the reported generated clock source latency. Most importantly, it identifies clock sources in the clock source latency path. In the example above, the violation details show the following message:

The following clock source pins occur in the source latency paths between the master clock (*master_clk*) and generated clock (*gclk2*):

* FF1/Qp

The Debugging Help section gives the command that can be used to further identify the cone of logic between the master clock and the reported generated clock source pin.

```
ptc_shell> analyze_clock_networks -through [get_ports CLK] -to [get_clocksgclk2] -max_endpoints 1 -style full -end_types {clock_source} -nosplit
```

Fix Suggestion

The recommended practice is to define the generated clocks in the design with respect to the intermediate clock source present in the source latency paths instead of the current master clock. In the above example, the constraints can be re-written as follows:

```
create_clock -name master_clk -period 10 -waveform [list 0 5] [get_ports CLK]
create_generated_clock -add -name gclk1 -master master_clk -source [get_ports CLK] -divide_by 2 [get_pins FF1/Q]
create_generated_clock -add -name gclk2 -master gclk1 -source [get_pins FF1/Q] -divide_by 2 [get_pins FF2/Q]
```

See Also

[report_clock](#)

[analyze_clock_networks](#)

[CLK_0008](#)

Rule: CLK_0038

Detects generated clocks with a sequential source network spanning block boundaries.

Severity: Warning

Status: Disabled by default

Description

The generated clock can be defined inside a block, whereas the source latency network of the generated clock could be crossing the block boundary where the generated clock is defined. This kind of scenario can lead to problems during the hierarchical analysis.

This rule uses the **set_hierarchical_config** command to define the blocks and the instances in the current design. This block information is used by this rule to identify the blocks that are of interest to the user. Only the generated clocks defined in these blocks are considered for the source latency path identification by the **analyze_design** command.

```
set_hierarchical_config -block block_name -instances instances list
```

Risk Associated with Violation

The source latency numbers for the generated clock might be different in the block-level analysis and the top-level analysis. This can lead to inaccurate results in hierarchical designs.

What Next

The Violation Details section displays additional information related to the reported generated clock source latency. Most importantly, it identifies clock sources, the master source, and the master clock for the reported generated clock.

Use the **analyze_clock_networks** command to identify the clock source latency path.

Fix Suggestion

The recommended practice is to define generated clocks within the same block as the master clock. The **analyze_clock_networks** command is used to identify the source latency path which crosses the block boundaries.

Example command:

```
ptc_shell> analyze_clock_networks -from [get_pins CLK] -to [get_clocks generated_clock]  
-max_endpoints 1 -style full -end_types {clock_source} -nosplit
```

See Also

[analyze_clock_networks](#)

[analyze_design](#)

[set_hierarchical_config](#)

Rule: CLK_0039

Detects circular dependency in generated clock definition.

Severity: Warning

Enabled by default

Description

This rule detects loops of generated clocks in which there is a circular dependency of the generated clock to the master clock.

Corresponding PrimeTime Error Message

Error Message: PTE-024

Severity: Error

Description: The following generated clocks '%s' form a loop.

See Also

[report_clock](#)

[analyze_clock_networks](#)

[CLK_0037](#)

Rule: CLK_0040

Detects clock definition conflicts on the same net.

Severity: Warning

Status: **Enabled by default**

Description

This rule detects the clock definition conflicts on the same net. This is most useful when you have a hierarchical design, and a clock definition at the block level and another at the top level. When performing a flat analysis, the clocks might be defined with different definitions on the same net. This rule helps in identifying such problems.

Risk Associated with Violation

It is not physically possible to have different clock waveforms in the same net.

What Next

The Violation details section displays additional details about the clocks with conflicting definitions. It shows all of the details, such as where the clock is defined, its master clock, and the SDC browser link. Most importantly, it lists all of the net segments in a hierarchical design that can lead to the conflict.

Fix Suggestion

Different scenarios might require different approaches. The basic idea is to ensure that there are no conflicting clock definitions on the same hierarchical net segments.

See Also

[analyze_design](#)

Rule: CLK_0041

The clock **clock** has a loop in the source latency network.

Severity: Warning

Status: Enabled by default

Description

This rule detects the loop in the source latency network of the clock. This warning means that this network contains at least one loop where a path feeds back into itself.

Risk Associated with Violation

Source latency numbers might not be accurate because the loop could be broken at an arbitrary place. This can lead to inconsistent values.

What Next

The Violation details section shows the clocks that have a loop in the source latency network. It also shows a SDC browser link that can point to the source file where the clock is defined.

Fix Suggestion

Use the **analyze_clock_networks** command to identify the latency paths of the clock that are involved in the loop.

Corresponding PrimeTime Error Message

Error Message: PTE-103

Severity: Warning

Description

Loop(s) were detected in the generated source latency network of clock %s. PrimeTime is prevented from computing the source latency of this clock with complete accuracy.

See Also

[analyze_clock_networks](#)

[analyze_design](#)

Rule: CLK_0042

The user-applied case analysis overlapse with clock propagation.

Severity: Error

Description

This violation occurs when clock propagation overlaps with user-applied case analysis.

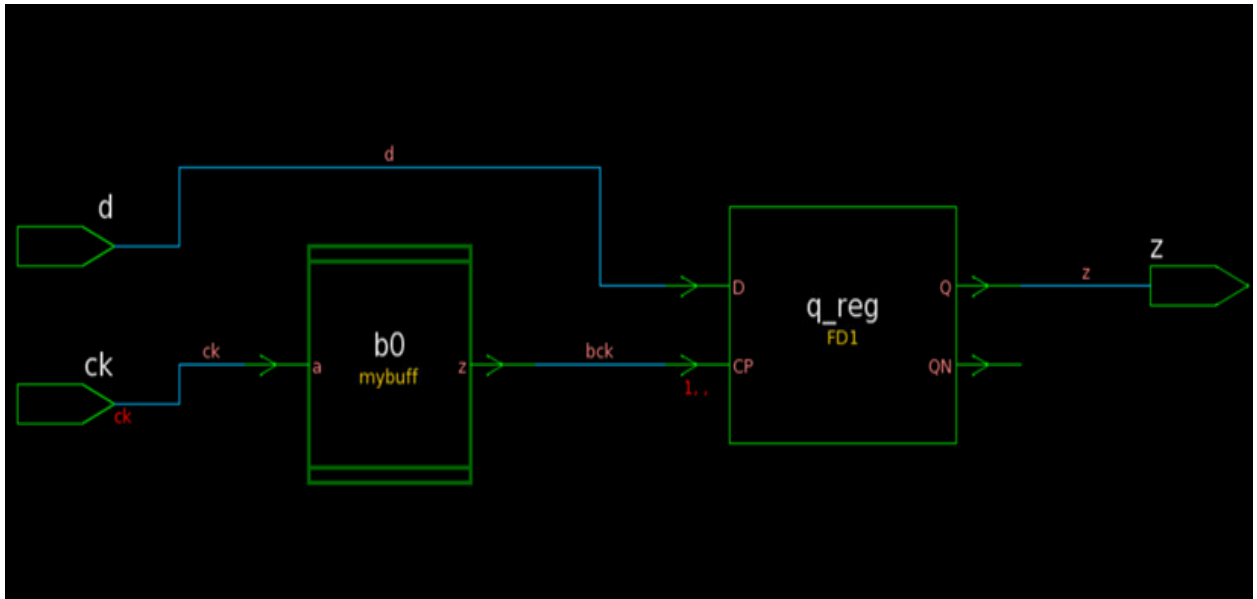


The screenshot shows a window titled "[default:CLK_0042 #1] ...". It contains three links at the top: "Debugging Help", "Fix Suggestion", and "Online Help". The main text reads: "Violation 1: The user-applied case analysis on 'b0/z' overlaps with clock propagation." Below this is a section titled "Violation Details" with a sub-header "Clocks:" followed by a bulleted list containing the link "ck". Another section titled "Debugging Help" contains the text: "check if the set case analysis on pin/port is stopping propagation of clocks reaching that point". The final section is titled "Fix Suggestion" and contains the text: "Remove set case analysis from pin/port".

Risk Associated with Violation

There is a risk that an unwanted case will stop the clock propagation and might lead to a missing path

One Possible Scenario



```
create_clock -name ck -period 10 -waveform {0 5} [get_ports ck]
set_case_analysis [get_pins b0/z]
```

What Next

Check if the case value is wanted. Remove the case value if it is an incorrect set.

Fix Suggestion

Use the `remove_case_analysis` command on the port/pin to remove the case value.

See Also

[set_case_analysis](#)

[remove_case_analysis](#)

Rule: CLK_0043

Creation of a clock 'clock' blocks the propagation of clocks reaching to the specified point.

Severity: Warning

Description

This violation is given when the creation of clock on a port/pin is blocking the propagation of clocks reaching until that point.



The screenshot shows a window titled "[default:CLK_0043 #1] ...". It contains three tabs: "Debugging Help", "Fix Suggestion", and "Online Help". The main text area displays the following information:

- Violation 1:** Creation of clock '[ck1](#)' blocks the propagation of clocks reaching '[b0/z](#)'.
- Violation Details**
- Clocks:**
 - [ck](#)
- Debugging Help**

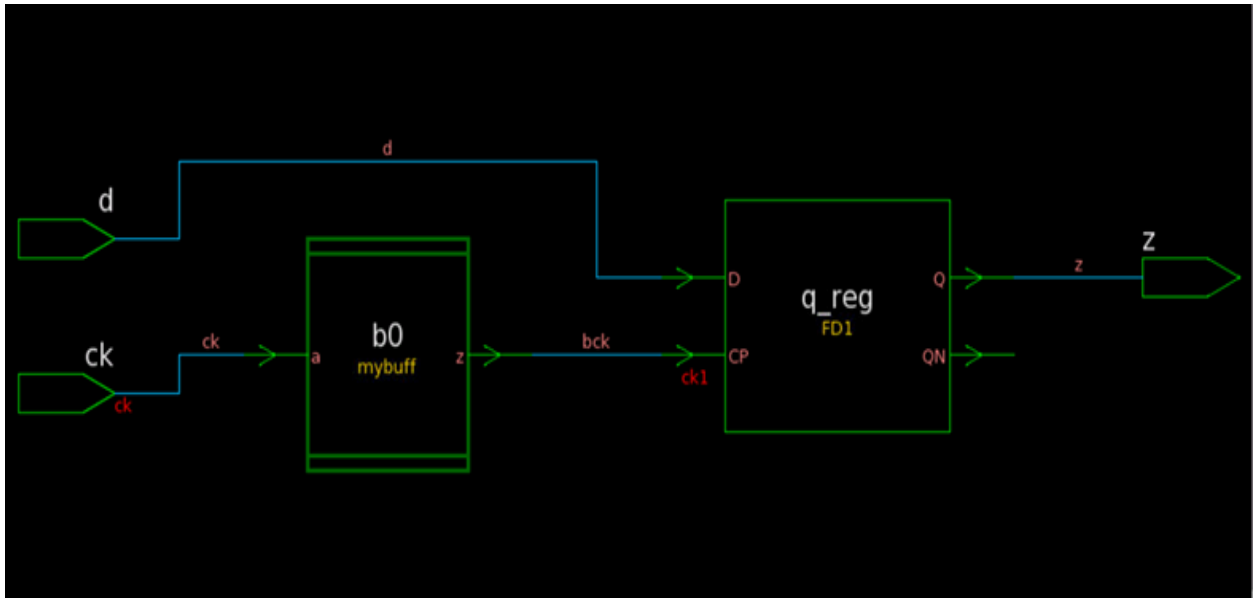
Check if the create_clock or create_generated_clock command on pin/port is blocking any clock reaching till that point.
- Fix Suggestion**

If propagation of any blocked clock has to continue then create generated clock for all such clocks.

Risk Associated with Violation

The clock created on the propagating path of other clocks will lead to all the other clocks stop propagation. If this is unintended it might lead to incorrect clock definition across the design.

One Possible Scenario



```
create_clock -name ck -period 10 -waveform {0 5} [get_ports ck]
create_clock -name ck1 -period 20 -waveform {0 10} [get_pins b0/z]
```

In this scenario, clock ck1 is created on the path of clock ck. It should use `create_generated_clock -master_clock ck -source [get_pins b0/a] -divided_by 2 -name ck1` to define the clock.

What Next

Check if this is wanted and change the clock definition accordingly.

Fix Suggestion

If the propagation of any blocked clock has to continue, then create a generated clock for all such clocks.

See Also

[create_clock](#)

[create_generated_clock](#)

Rule: CMP_0001

Command '**command**' is unsupported by **products**. Command used in **N** locations.

Severity: Warning

Status: Disabled by default

Description

This rule checks your scripts and constraint files for any command not supported across the Synopsys Galaxy Products (IC Compiler, Design Compiler, and PrimeTime). The purpose of this rule is to make sure that the commands in your constraints are supported by all the tools in your design flow.

Risk Associated with Violation

If a command is used with constraint consistency scripts but is not supported in one of the other Galaxy products, there is a risk that the constraint analysis might be inaccurate. In this case a **CMP_0001** violation is flagged and one can make sure the command is correct and/or take steps to correct it.

What Next

Constraint consistency knows about what command is supported in what tool by pointing at their relevant install tree. If constraint consistency is installed in a common tree with the other Galaxy products, then the list of commands supported by the versions of the products installed at the Synopsys root will be used.

Constraint consistency can also be configured to use different versions of PrimeTime/ Design Compiler/IC Compiler when performing the **CMP_0001** rule checks. This can be achieved by setting the variables below to point to the relevant install trees:

```
set_app_var pt_synopsys_root /u/release/primetime/C-2009.06
set_app_var icc_synopsys_root /u/release/icc/B-2008.09-SP4
set_app_var dc_synopsys_root /u/release/synthesis/B-2008.09-SP5
```

If the variables for other product roots are not set and those products are not installed in the current Synopsys root, constraint consistency is configured to use information from the most recent release of each product (at the time the constraint consistency executable you are using was released).

Fix Suggestion

Organize your constraints such as tool specific commands are separate from your design constraints.

If you do not wish to check your constraints for compatibility across the Galaxy Products, you can turn off the rule to prevent further occurrence of this violation. Below are two ways to do so:

1. Disabling the individual rule

```
ptc_shell> disable_rule CMP_0001
```

2. Disabling the “compatibility” ruleset. This will disable all rules belonging to that ruleset.

```
ptc_shell> disable_rule [get_ruleset compatibility]
```

If you are not sure what rules belong to the compatibility ruleset, you can report them using the [report_rule](#) command as seen below:

```
ptc_shell> report_rule [get_rulesets compatibility]
```

```
*****
Report : rule
Version: ...
Date   : ...
*****
```

Rule	Severity	Status	Message
CMP_0001	warning	disabled	Command ''command'' is unsupported by 'products'. Command used in 'N' locations.
CMP_0002	warning	disabled	Option '-option' of command ''command'' is unsupported by 'products'. Option used in 'N' locations.
CMP_0003	warning	disabled	Application variable, sh_command_abbrev_mode is not set to the recommended value of 'Command-Line-Only'.

You can also retrieve the list of all rulesets currently available in constraint consistency using the [get_rulesets](#) command:

```
ptc_shell> get_rulesets {"compatibility", "postlayout", \
                        "prelayout", "primetime_si", \
                        "starc", "timing_correlation"}
```

See Also

[disable_rule](#)

[enable_rule](#)

[pt_synopsys_root](#)

[dc_synopsys_root](#)

[icc_synopsys_root](#)

[get_rulesets](#)

[report_rule](#)

Rule: CMP_0002

Option '*-option*' of command '*command*' is unsupported by *products*. Option used in *N* locations.

Severity: Warning

Status: Disabled by default

Description

This rule checks your scripts and constraint files for any command option not supported across the Synopsys Galaxy Products (IC Compiler, Design Compiler and PrimeTime). The purpose of this rule is to make sure that all the options for the commands in your constraints are supported by all the tools in your design flow.

Risk Associated with Violation

If an option for a given command is used with constraint consistency scripts but is not supported in one of the other Galaxy products, there is a risk that the constraint analysis might be inaccurate. In this case a **CMP_0002** violation is flagged and one can make sure the command is correct and/or take steps to correct it.

What Next

Constraint consistency knows about what command option is supported in what tool by pointing at their relevant install tree. If constraint consistency is installed in a common tree with the other Galaxy products, then the list of commands and their related options supported by the versions of the products installed at the Synopsys root will be used.

constraint consistency can also be configured to use different versions of PrimeTime/ Design Compiler/IC Compiler when performing the **CMP_0002** rule checks. This can be achieved by setting the variables below to point to the relevant install trees:

```
set_app_var pt_synopsys_root /u/release/primetime/C-2009.06
set_app_var icc_synopsys_root /u/release/icc/B-2008.09-SP4
set_app_var dc_synopsys_root /u/release/synthesis/B-2008.09-SP5
```

If the variables for other product roots are not set and those products are not installed in the current Synopsys root, constraint consistency is configured to use information from the most recent release of each product (at the time the constraint consistency executable you are using was released).

Fix Suggestion

Organize your constraints so tool specific commands are separate from your design constraints.

If you do not wish to check your constraints for compatibility across the Galaxy Products, you can turn off the rule to prevent further occurrence of this violation. Below are two ways to do so:

1. Disabling the individual rule

```
ptc_shell> disable_rule CMP_0002
```

2. Disabling the “compatibility” ruleset. This will disable all rules belonging to that ruleset.

```
ptc_shell> disable_rule get_ruleset compatibility
```

If you are not sure what rules belong to the compatibility ruleset, you can report them using the [report_rule](#) command as seen below:

```
ptc_shell> report_rule [get_rulesets compatibility]
```

```
*****
```

```
Report : rule
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Rule Severity Status Message
```

```
-----  
CMP_0001 warning disabled Command ''command'' is unsupported by  
'products'. Command used in 'N' locations.
```

```
CMP_0002 warning disabled Option '-option'' of command ''command'' is  
unsupported by 'products'. Option used in 'N' locations.
```

```
CMP_0003 warning disabled Application variable, sh_command_abbrev_mode is  
not set to the recommended value of 'Command-Line-Only'.
```

You can also retrieve the list of all rulesets currently available in constraint consistency using the [get_rulesets](#) command:

```
ptc_shell> get_rulesets {"compatibility", "postlayout",  
                        "prelayout", "primetime_si",  
                        "starc", "timing_correlation"}
```

See Also

[disable_rule](#)
[enable_rule](#)
[pt_synopsys_root](#)
[dc_synopsys_root](#)
[icc_synopsys_root](#)
[get_rulesets](#)
[report_rule](#)

Rule: CMP_0003

Application variable **sh_command_abbrev_mode** is not set to the recommended value of 'Command-Line-Only'.

Severity: Warning

Status: Disabled by default

Description

Using command abbreviations in constraint scripts can cause compatibility issues between different tools and/or between future versions of the same tool. Using the value of 'Command-Line-Only' for the [sh_command_abbrev_mode](#) application variable will cause the tool to only accept command abbreviations from the interactive shell.

Even though this implies your scripts use non-abbreviated commands, it provides a more accurate checking mechanism for command/options compatibility between the various Galaxy tools.

Risk Associated with Violation

A command, non-ambiguous for a given tool, abbreviated in a script may not be unambiguous in another tool or in a future version of that tool.

A non-ambiguous abbreviated command will be ignored in the script since the tool will not be able to fully comprehend it. This is very likely to result in differences in behavior whenever a given script is used in several Galaxy products and, as a consequence, constraint consistency will flag the situation with a **CMP_0003** rule violation.

In addition to the **CMP_0003** violation, a CMD-006 user message is issued by constraint consistency whenever an ambiguous command is used.

For example the abbreviated command "**get_cloc**" may be a unique abbreviation in Design Compiler, but will give the following error when the same abbreviation is used in PrimeTime:

```
Error: ambiguous command 'get_cloc' matched 2 commands:
```

```
(get_clock_network_objects, get_clocks) (CMD-006)
```

In Design Compiler, a clock will be created and used for synthesis and optimization. In PrimeTime, the command being ambiguous, no clock will be created. This will result in very different timing behaviors between Design Compiler and PrimeTime.

Fix Suggestion

Set the [sh_command_abbrev_mode](#) command to the recommended value below. This will generate error messages whenever you have abbreviated commands in your script. You should then fix any abbreviated commands in your scripts by providing the full, non-abbreviated, command.

```
set_app_var sh_command_abbrev_mode Command-Line-Only
```

See Also

[sh_command_abbrev_mode](#)

Rule: CNL_0001

Inconsistent clock network latency values on ***object_type object***.

Severity: Warning

Description

The clock network latency values are inconsistent. Each min value should be smaller than the corresponding max value.

Risk Associated with Violation

By definition, min/max values are used for minimum/maximum operating conditions. It is inconsistent with the intent of having the different analysis modes to have the max clock network latency to be less than the min clock network latency. This could lead to incorrect delay calculation and analysis.

One Possible Scenario

Consider a scenario where the clock network latency is set on a clock **CLK** as follows:

```
set_clock_latency -rise -min 11 [get_clock CLK]
set_clock_latency -fall -min 2 [get_clock CLK]
set_clock_latency -rise -max 3 [get_clock CLK]
set_clock_latency -fall -max 4 [get_clock CLK]
```


The **-rise -min** value is larger than the **-rise-max** value which is inconsistent. It should be the smallest value. For this situation, the tool will issue a **CNL_0001** rule violation.

What Next

The “Violation Details” section of the rule violation shows a table for the source latency values defined at the object. For the scenario described above, the table reported is shown below:

	Min	Max
Rise	11.0000	3.0000
	0	
Fall	2.0000	4.0000

You can also view the related constraint definitions in the Constraint Browser by clicking on the links in the “Network latency specified at:” section of the “Violation Details.”

Fix Suggestion

Changing the constraints to have consistent min/max clock network latency values will fix this violation. For the example scenario, changing the **-rise -min** value to be less than the **-rise -max** value will fix the violation.

```
set_clock_latency -rise -min 1 [get_clock CLK]
```

See Also

[set_clock_latency](#)

Rule: CNL_0002

Input/output delays are defined relative to clock **clock**. This clock has incomplete network latency values required for those external delays even though its source ports/pins have more complete network latencies defined on them.

Severity: Warning

Description

Since the network latency values for the clock are incomplete, no network latency will be applied for paths with from/to points which have input or output delay relative to this clock. The network latency set on the clock sources is not applied on these paths. This could result in incorrect timing analysis. Network latency should be applied to the clock object.

Risk Associated with Violation

Clock network latency values applied on a clock source pin/port will not be used when delays need to be calculated on paths that have ports with external delay settings on them.

Only latency values applied on clock objects are used for path delay calculations, and if these are not specified or are incomplete, zero values are assumed. This could affect both setup and hold timing calculations if it is assumed that the clock network latency values specified on the clock source pin/port will be used instead.

One Possible Scenario

Consider a scenario, where a clock is defined on an input port:

```
create_clock -period 10 [get_ports CLK]
```

Clock network latencies are defined on both the clock object and the clock source port.

```
set_clock_latency -min 3 [get_clock CLK]
set_clock_latency 4 [get_ports CLK]
```

An input delay is specified for a port relative to this clock:

```
set_input_delay 5 -clock CLK -add_delay [get_ports INPUT]
```

The input delay set on the **INPUT** port has been defined with respect to clock **CLK**, but since no network latency is defined on the clock object for the max condition, the clock network latency for maximum delay calculations will be zero. The network latency of **4** applied on the **CLK** port object will not be used. For this situation, a **CNL_0002** rule violation is issued to warn the user that the clock network latency applied on the port object will not be used for clocks specified in external delay definitions.

What Next

The “Violation Details” section shows a table for the network latency values: one for the clock and one for its sources. For the scenario described above, the tables reported in the “Violation Details” section would be:

Network latency values on the clock

	Min	Max
Rise	3.0000	--
Fall	3.0000	--

Network latency values on source ‘CLK’

	Min	Max
Rise	4.0000	4.0000
Fall	4.0000	4.0000

You can also view the related constraint definitions in the Constraint Browser by clicking on the link in the “Network latency specified at:” section associated with each table in the “Violation Details.”

Fix Suggestion

Modify the constraints so that a complete set of clock network latency values is specified. For the example scenario, adding network latency for the clock object max condition as follows will fix the violation:

```
set_clock_latency -max 4 [get_clock CLK]
```

If an incomplete network latency definition is the desired analysis for the from/to ports having external delays with respect to this clock, then no fix is required.

See Also

[set_clock_latency](#)

[set_input_delay](#)

[set_output_delay](#)

[create_clock](#)

Rule: CNL_0003

Clock network latency is specified on ***object_type object*** but that object is not in any clock network.

Severity: Warning

Description

Clock network latency should only be specified on clocks, pins, or ports in a clock network.

Risk Associated with Violation

The clock network latency specified for an object that is not in a clock network will be ignored. This could affect both setup and hold timing of the design.

One Possible Scenario

Consider the circuit below where clock network latency has been specified on an intermediate pin **inv/Z** in the clock network. The [set_case_analysis](#) command is specified on the clock network before this pin. When the case value is propagated, the clock no longer reaches the pin **inv/Z**, and the clock latency specified on the pin is ignored. As a result, the tool will issue a **CNL_0003** rule violation.

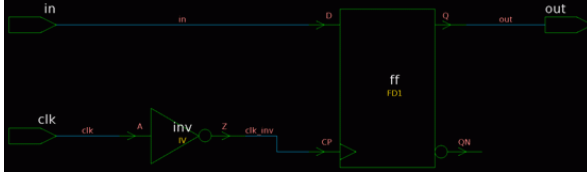


Figure 1: Clock Network Latency Specified on Intermediate Pin

```
create_clock -period 10 [get_clocks clk]
set_case_analysis 0 [get_pins inv/A]
set_clock_latency 1.3 [get_pins inv/Z]
```

What Next

The “Violation Details” section provides a link with the file name and line number to the clock network latency specification.

This violation can be caused by a pin or port that may be part of a clock network, but another constraint is blocking the clock. Use the **potential_clocks** attribute to check if the pin or port could be in a clock network, but was blocked by the [set_case_analysis](#) or [set_disable_timing](#) command specification or some other constraint.

For the example scenario, the value of the **potential_clocks** attribute is as follows:

```
ptc_shell> get_attribute [get_pins inv/Z] potential_clocks {"clk"}
```

If the **potential_clocks** attribute returns a clock, you can use the [analyze_clock_networks](#) command with the **-traverse_disabled** option to find which constraint blocked this clock from reaching the pin or port. For the example scenario, the command can be issued as follows:

```
ptc_shell> analyze_clock_networks -traverse_disabled -from [get_clocks
clk] \
```

```
-through [get_pins inv/Z] -style full
```

```
*****
```

```
Report : analyze_clock_networks
```

```
-max_endpoints=1000
```

```
-style full
-end_types {register port clock_source }
-traverse_disabled
Design : ...
Scenario: default
Version: ...
Date : ...
*****
Clock Sense Abbreviations:
P - positive
Potential senses detected with -traverse_disabled:
[P] - potential positive
[N] - potential negative
Clock Network End Type Abbreviations:
REG - register
Full report for clock: clk
Branch 0:
Branch
Level Info Sense Notes Port/Pin
-----
0 P source clk (in)
1 [P] CASE#0 inv/A (IV)
2 [N] CASE#0 inv/Z (IV)
3 [N] REG, CASE#0
ff/CP (FD1)
Clock Blocking Constraints:
CASE#0 = case_value_source
```

```
Sources of Case Values
```

```
Value: 0 at Pin: inv/A
```

```
test.tcl, line 2
```

The report shows that the [set_case_analysis](#) constraint is blocking the clock at the pin **inv/A**.

If the object is not in a potential clock network, then there may be an error in specifying the pin, or there is a missing clock definition. An error in the pin specification could be caused if the wrong pin is specified, or if a wildcard is used in the [set_clock_latency](#) command, and it returns more pins than expected.

Fix Suggestion

If the pin or port is in a potential clock network, check if the blockage is correct for this scenario. If it is correct, the violation can be waived or the [set_clock_latency](#) command can be altered to remove the object associated with the violation.

If the pin or port is not part of a clock network, the latency specification should be moved to other pins or ports that are part of the clock network so that it is not ignored.

See Also

[set_clock_latency](#)

[create_generated_clock](#)

[create_clock](#)

[get_attribute](#)

[report_attribute](#)

[set_case_analysis](#)

[set_disable_timing](#)

[analyze_clock_networks](#)

Rule: CNL_0004

Clock network latency is specified on ***object_type object*** with respect to clock ***clock*** but that object is not in the clock's network.

Severity: Warning

Description

Clock network latency should only be specified on pins or ports which are part of the specified clock network.

Risk Associated with Violation

The clock network latency specified for an object that is not in a clock's network will be ignored. This could affect both setup and hold timing of the design.

One Possible Scenario

Consider the scenario below where a design contains two clocks. The clock network latency specification is specified on pin **ff1/CP** with respect to clock **clk2** instead of **clk1**. Since the **ff1/CP** pin is not in the **clk2** network, this clock network latency specification is ignored, and the tool will issue a **CNL_0004** rule violation.

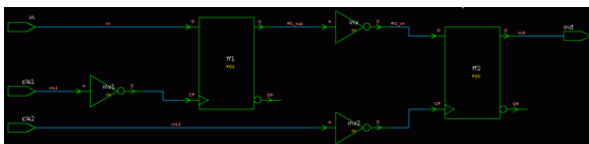


Figure 1: Design Containing Two Clocks

```
create_clock -period 10 [get_clocks clk1]
create_clock -period 10 [get_clocks clk2]
set_clock_latency 1.3 [get_pins ff1/CP] -clock clk2
```

What Next

The “Violation Details” section provides a link with the file name and line number to the clock network latency specification.

This violation can be due to an incorrect clock that is specified with the [set_clock_latency](#) command. Query the **clocks** attribute on the pin or port to determine which clocks reach the pin or port. If a different set of clocks reaches this pin or port than the one specified in the constraint, then check to see if one of those clocks should be defined in the clock network latency command.

For the example scenario, the **clocks** attribute can be queried as follows:

```
ptc_shell> get_attribute [get_pins ff1/CP] clocks {"clk1"}
```

If the contents of the **clocks** attribute are not the expected ones, then the clock of interest could potentially be blocked from reaching the pin or port. The **potential_clocks** attribute associated with the object will help determine which clocks could reach the pin or port. For the example scenario, the attribute can be queried as follows:

```
ptc_shell> get_attribute [get_pins ff1/CP] potential_clocks
```

```
Warning: Attribute 'potential_clocks' does not exist on pin  
'ff1/CP' (ATTR-3)
```

For the example scenario, the clock **clk1** arrives at the **ff1/CP** pin, but there are no other potential clocks for that pin. Further debug can be completed as described below to identify if there are other potential clocks that exist on the object or if the expected clock is not in the **clocks** or **potential_clocks** attributes query. Resolution of the violation is discussed in the “Fix Suggestion” section below.

If the result for the **potential_clocks** attribute query is the intended clocks to be used in the [set_clock_latency](#) command, use the [analyze_clock_networks](#) command with the **-traverse_disabled** option to see why the clocks were blocked. The report for the [analyze_clock_networks](#) command will include any [set_case_analysis](#) or [set_disable_timing](#) results or other constraints blocking the clock from reaching the object. An example of the [analyze_clock_networks](#) command used to identify potential clocks is shown below:

```
ptc_shell> analyze_clock_networks -traverse_disabled \  
          -from [get_clocks <potential_clock_name>] \  
          -through [get_pins <pin_name>] \  
          -style full
```

Fix Suggestion

Check to see if the clock name in the **-clock** option of the network latency is correctly specified. In the example scenario, the network latency needs to be specified relative to the clock **clk1**.

```
set_clock_latency 1.3 [get_pins ff1/CP] -clock clk1
```

If the pin or port is in a potential clock network, see whether the blockage is correct for this scenario. If it is correct, the violation can be waived, or the [set_clock_latency](#) command can be altered to remove the object associated with the violation.

See Also

[set_clock_latency](#)

[create_clock](#)

[create_generated_clock](#)

[get_attribute](#)

[report_attribute](#)

[set_case_analysis](#)

[set_disable_timing](#)

analyze_clock_networks

Rule: CNL_0005

All register clock pins in the fanout of **pin** have no clock latency for clock **clock**.

Severity: Warning

Description

During pre-layout analysis, all clocks should have an ideal clock network latency.

Risk Associated with Violation

In pre-layout designs, path delays are not finalized. The propagated clock network latency may not reflect the real latency from the placement, clock-tree synthesis, and routing stages. Missing a realistic latency at the register clock pins may lead to inaccurate timing results.

One Possible Scenario

Consider the scenario shown in Figure 1 where clock signals reach the register clock pin **U2/CP**.

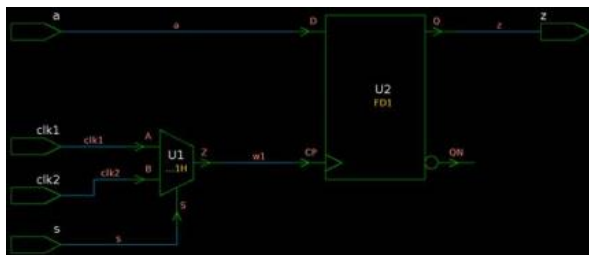


Figure 1: No clock network latency is specified on register clock pin

If the constraints are applied as shown below:

```
create_clock -period 10 [get_clocks clk1]
create_clock -period 5 [get_clocks clk2]
set_clock_latency -clock clk1 2.0 [get_pins U1/Z]
```

No clock network latency is defined at the register clock pin with respect to the clock **clk2**. This results in a CNL_0005 violation indicating that no clock latency has been defined for the fanout of port **clk2**.

What Next

The violation message reports the name of the clock and the pin where the ideal clock network latency can be set. In addition, the More Info section of the message lists up to 20 register clock pins affected by the missing clock network latency.

Fix Suggestion

Specify a realistic clock network latency value at the given pin or at the fanout of the pin with respect to the reported clock. For the scenario above, specify the clock network latency at the output of the multiplexer:

```
set_clock_latency -clock clk2 1.2 [get_pins U1/Z]
```

See Also

[set_clock_latency](#)

[remove_clock_latency](#)

Rule: CSL_0001

Inconsistent source latency values on ***object_type object***.

Severity: Warning

Description

The source latency values defined on the reported object are inconsistent. In order to correctly specify the latency on this object, each “early” value should be smaller than its corresponding “late” value and each “min” value should be smaller than its corresponding “max” value.

Risk Associated with Violation

By definition early/late and min/max values are used for minimum/maximum operating conditions. Having the late/max values being less than the early/min does not allow for proper analysis of the different operating corners of the design. If left as is, this could lead to incorrect delay calculation and analysis.

One Possible Scenario

Consider a scenario where the clock source latency is set on a clock port ‘CLK’ as follows:

```
set_clock_latency -source -early -rise -min 11 CLK
set_clock_latency -source -early -fall -min 2 CLK
set_clock_latency -source -early -rise -max 3 CLK
set_clock_latency -source -early -fall -max 4 CLK

set_clock_latency -source -late -rise -min 6 CLK
```

```
set_clock_latency -source -late -fall -min 7 CLK
set_clock_latency -source -late -rise -max 8 CLK
set_clock_latency -source -late -fall -max 9 CLK
```

The **-early -min -rise** value is larger than the other **rise** values. This is most likely not the expected behavior for this design and should be investigated. The combination of **-early** and **-min** should probably have the smallest latency here.

Constraint consistency will flag a **CSL_0001** violation to bring the problem to the designer's attention.

What Next

The Violation Detail section displays a table containing the various source latency values defined on the reported object. In the above scenario, this is what that table would look like:

	Early	Late
Min Rise	11.0000 0	6.0000
Min Fall	2.0000	7.0000
Max Rise	3.0000	8.0000
Max Fall	4.0000	9.0000

The various places in your SDC file where those values are defined are also reported in this section. One can easily find out from the table where the inconsistency is and then use the SDC hyperlinks to navigate to the exact SDC command where the latency is defined.

Fix Suggestion

Using realistic and consistent data for min/max and early/late latency values will address this problem.

In the example scenario above, changing the **-early -rise -min** value to be less than the other set of rise values would fix the problem.

```
set_clock_latency -source -early -rise -min 1 CLK
```

See Also

[set_clock_latency](#)

Rule: CSL_0002

Input or output delays are defined relative to clock **clock**. This clock has incomplete source latency values required for those external delays even though its source ports or pins have more complete source latencies defined on them.

Severity: Warning

Description

Some input and/or output delays are relative to a clock with incompletely specified source latency. Because of this, paths starting or ending at ports relative to this clock will not have source latency either. The source latency data specified on the clock source (for example the port where a clock is specified) will not be carried over to those paths. This could create incorrect timing results during static timing analysis.

Risk Associated with Violation

Only latency values applied on clock objects are used for path delay calculations and if these are not specified or are incompletely specified, zero delay values are assumed. The clock source latency values applied on a clock source pin/port will not be used when delays need to be calculated on paths which have ports with external delay settings on them.

This could lead to optimistic delay calculations if the user assumed that the clock source (pin or port) latency values will be used in place of the clock object latency values.

One Possible Scenario

Consider the following scenario where a clock is defined on an input port

```
create_clock -period 10 CLK
```

Clock source latencies are defined on both the clock object and the clock source port.

```
set_clock_latency -source -min 3 [get_clock CLK]  
set_clock_latency -source 4 [get_port CLK]
```

An input delay is specified to a port relative to this clock:

```
set_input_delay 4 -clock CLK -add IN
```

The input delay set on the **IN** port has been defined with respect to clock **CLK**, but since no source latency is defined on the clock object for the max condition, the clock source latency for maximum delay calculations will be zero. The source latency applied on the

port object **CLK** (with a value of 4) will not be used in place of the clock object source latency here.

Constraint consistency will flag a **CSL_0002** violation in this case to notify the user that incomplete and/or missing latency values on a clock object could be a problem.

What Next

The “Violation Detail” section displays a table containing the source latency values both for the clock and its sources. In the above scenario, the table would look like this:

Source latency values on the clock

	Early	Late
Min Rise	3.0000	3.0000
Min Fall	3.0000	3.0000
Max Rise	--	--
Max Fall	--	--

Source latency values on source ‘CLK’

	Early	Late
Min Rise	4.0000	4.0000
Min Fall	4.0000	4.0000
Max Rise	4.0000	4.0000
Max Fall	4.0000	4.0000

Fix Suggestion

It is recommended that clock latency be completely defined. This would help STA tools computing accurate timing reports.

In the example scenario presented above, one easy fix would be to add a -max latency value on the clock object as seen below:

```
set_clock_latency -source -max 4 [get_clock CLK]
```

See Also

[set_clock_latency](#)

[set_input_delay](#)

[set_output_delay](#)

Rule: CSL_0003

Clock source latency is specified on ***object_type object*** but that object is not a clock source.

Severity: Warning

Description

Source latency should only be specified on clocks or on clock source pins or ports.

Even though the [set_clock_latency](#) command itself does not allow latency values to be set on a non clock source pin or port, if a clock is removed or redefined to a new location, its latency values still persist at the outdated clock source pin or port. This is usually not the designer's intention when moving clocks.

Risk Associated with Violation

When a clock is redefined to another clock source in the design, the clock latency values set on its original clock source (pin or port) do not get copied over to the new source location. The clock source latency is ignored and has no value.

This is a situation which frequently arises when as a result of merging different constraint scripts together.

One Possible Scenario

Consider the scenario below where we move a clock from source port CLK to source port CL. In the original clock definition, a source latency value is specified for the clock.

```
create_clock -period 10 -name clk1 [get_port CLK]
set_clock_latency -source -max 3 [get_port CLK]
```

One can report the current clocks and latency values specified in the design using the [report_clock](#) and [report_clock-skew](#) commands. Here are the current values:

```
ptc_shell> report_clocks
```

Clock	Period	Waveform	Attrs	Sources
clk1	10.00	{0 5}		{CLK}

You can see in the report above that the clock "clk1" is defined on port "CLK".

```
ptc_shell> report_clocks -skew
```

Latency		Min Condition Source Latency				Max Condition Source			
Object	Rel_clk	Early_r	Early_f	Late_r	Late_f	Early_r	Early_f	Late_r	
Late_f									
CLK		3.00	3.00	3.00	3.00	-	-	-	

You can see from the report above that clock source latency values exist on port “CLK”, which is also the source of clock “clk1”. Now if one moves the clock “clk1” definition to a different port, using the command below, the clock latency data will not be moved along.

```
create_clock -period 10 -name clk1 [get_port CL]
```

```
ptc_shell> report_clock
```

Clock	Period	Waveform	Attrs	Sources
clk1	10.00	{0 5}		{CL}

The source of clock “clk1” has been move from port “CLK” to pot “CL”.

```
ptc_shell> report_clock -skew
```

Latency		Min Condition Source Latency				Max Condition Source			
Object	Rel_clk	Early_r	Early_f	Late_r	Late_f	Early_r	Early_f	Late_r	
Late_f									
CLK		3.00	3.00	3.00	3.00	-	-	-	

From the report above, however, the clock latency is still specified on port “CLK”. No new latency values are available on port “CL”.

What Next

The designer should examine the script and make sure the clock redefinition was intentional. If this is the case, any latency specification should also be manually transferred to the new clock source and the old latency values should be removed.

Fix Suggestion

Remove unnecessary latency values and re-apply the old values to the new clock source.

In the sample scenario described above, adding the source latency at port ‘CL’ will address the problem.

```
set_clock_latency -source -max 3 [get_clock CL]
```

In order to remove the violation, the unnecessary clock source latency value (which was specified on port CLK) should be removed. For the example scenario the command would look like this:

```
remove_clock_latency -source [get_port CLK]
```

See Also

[set_clock_latency](#)

[remove_clock_latency](#)

[create_clock](#)

[report_clock](#)

Rule: CSL_0004

Generated clock **clock** master clock **master_clock** is not propagated and has zero or incomplete source latency values.

Severity: Warning

Description

The reported clock's master is not propagated. It is expected that realistic source latency values be provided to represent the delay to the clock source. Constraint consistency could not find those latency values.

Risk Associated with Violation

Since no source latency values are specified, timing analysis tools will assume a zero delay value. This might result in incorrect timing analysis.

One Possible Scenario

Consider a scenario where a generated clock **gen_clock** is defined with respect to a **master_clock**.

```
create_clock -period 10 master_clock
create_generated_clock -name gen_clock \
    -source master_clock gen/Q -divide_by 2
```

If the **master_clock** is defined as ideal and not propagated, the generated clock **gen_clock** will have zero source latency values. Constraint Analyzer flags a **CSL_0004** in

such a case to warn the designer that there are potentially missing latency data on those clocks.

What Next

The “Violation Details” section reports whether the clock latency values are undefined, have a value of zero, or are incomplete.

Fix Suggestion

There are two different possible ways to address this issue depending on the situation. For the example scenario described above, here are the two different approaches to solve this problem:

1. The master clock was supposed to be propagated but somehow the command was never issued. In this case, propagating the clock will address the issue and the generated clock will inherit the proper latency values.

```
set_propagated_clock master_clock
```

2. The master clock is ideal. In this case, some realistic latency values should be provided in order to ensure correct timing analysis.

```
set_clock_latency -source 3 [get_clock gen_clock]
```

See Also

[create_clock](#)

[create_generated_clock](#)

[set_clock_latency](#)

[remove_clock_latency](#)

[create_clock](#)

[set_propagated_clock](#)

Rule: CSL_0005

Generated clock **clock** has source latency values less than that of master clock **master_clock** when the clock edge relationships are considered.

Severity: Warning

Description

The source latency of a generated clock is automatically computed when all the various clocks in the design are propagated. When you specify the source latency values for the generated clock, those values should be larger than the source latency of the corresponding master clock. The latency is created by taking into account the edge relationship between the master and the generated clocks.

Risk Associated with Violation

A generated clock source is further away from the master clock source and therefore has more latency than its master clock. Specifying smaller source latency values than the master clock leads to overly optimistic results.

One Possible Scenario

Consider a scenario where a generated clock 'gen_clock' is defined with respect to a **master_clock**. The generated clock has a **divide_by 2** relationship with respect to its master without sense inversion.

```
create_clock -period 10 -name master_clock  
create_generated_clock -name gen_clock -source master_clock gen/Q -  
divide_by 2
```

Clock source latencies are manually defined as follows for both the master clock and its related generated clock:

```
set_clock_latency -source -rise 4 [get_clocks master_clock]  
set_clock_latency -source -fall 1 [get_clocks master_clock]  
set_clock_latency -source -rise 2 [get_clocks gen_clock]  
set_clock_latency -source -fall 2 [get_clocks gen_clock]
```

When we look at the edge relationship between the master clock and its related generated clock, as shown in Figure 1 below, you can see the rising edges of the master clock triggers both the rising and the falling edges of the generated clock.

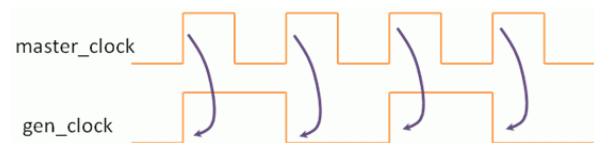


Figure 1: Edge Relationship Between Master Clock and Related Generated Clock

As consequence, on the generated clock side, the latency (for both rise and fall) should be at least as large as the “rising edge” latency of the master (four in the figure above). If the latency values for the generated clock are less, a **CSL_0005** violation is issued.

What Next

The “Violation Details” section reports the various latency data in a table for both the master and the generated clocks.

Master Clock Source Latency

	Early	Late
Min Rise	4.0000	4.0000
Min Fall	1.0000	1.0000
Max Rise	4.0000	4.0000
Max Fall	1.0000	1.0000

Generate Clock Source Latency

	Early	Late
Min Rise	2.0000	2.0000
Min Fall	2.0000	2.0000
Max Rise	2.0000	2.0000
Max Fall	2.0000	2.0000

Fix Suggestion

To achieve the correct source latency for the generated clock the edge relationships between both the master and the generated clocks should be considered.

In the above example, the source latency computation for the generated clock is at least four units (the master clock latency) for both the rise and fall data. In general, the source latency computation of the generated clock is the total of the master clock source latency and any latency between the master clock and the generated clock source; this includes all gates within the path. As an example, the following value would solve the **CSL_0005** violation:

```
set_clock_latency -source -rise 5 [get_clock gen_clock]
set_clock_latency -source -fall 5 [get_clock gen_clock]
```

See Also

[create_clock](#)

[create_generated_clock](#)

[set_clock_latency](#)

[report_clock](#)

Rule: CSL_0006

Clock source latency is specified on **object_type object** with respect to clock **clock** but that object is not in the clock's network.

Severity: Warning

Description

The source latency specification is incorrect, the port or pin reported using in the [set_clock_latency](#) command is not part of the clock network specified through the **-clock** option.

If the referenced object is a valid clock source, the **CSL_0006** rule verifies that the source latency is specified on an object belonging to the clock network of **-clock**.

For constraint consistency, the [set_clock_latency](#) command generates a [UIC-002](#) error message if the referenced object is not a proper clock source.

Risk Associated with Violation

Since the object where the source latency is specified does not belong to the related clock network, the source latency value will be ignored. This could result in incorrect clock network delay calculations and overly optimistic results.

One Possible Scenario

Consider a scenario shown in Figure 1 below, with two different clock networks **CLK1** and **CLK2**. Each of those clock networks originates respectively from ports **PORT_CLK1** and **PORT_CLK2**.

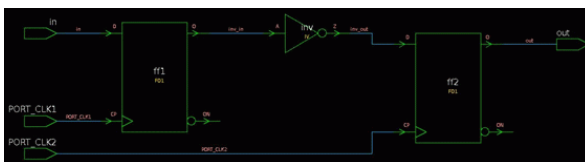


Figure 1: One Possible Scenario

```
create_clock -period 10 [get_ports PORT_CLK1] -name CLK1
create_clock -period 10 [get_ports PORT_CLK2] -name CLK2
set_clock_latency 2 -source [get_ports PORT_CLK1] -clock CLK2
```

The [set_clock_latency](#) command specifies the source latency for clock network **CLK2** at port **CLK1**. Since **CLK1** is not in the clock network **CLK2**, this latency will be ignored. A **CSL_0006** violation is issued as a warning.

What Next

The “Violation Details” section reports the file name and the line number where the [set_clock_latency](#) command was issued.

Additional information about the clock sources in the fanout of a port is available by using the [analyze_clock_networks](#) command. You can verify whether **CLK2** is part of the clock network starting at port **PORT_CLK1** as shown in the example below:

```
ptc_shell> analyze_clock_networks -from [get_ports PORT_CLK1]
```

```
*****
```

```
Report : analyze_clock_networks
        -max_endpoints=1000
        -style short
        -end_types {register port clock_source }
Design : CSL_0006
Scenario: default
Version: ...
Date : ...
*****
```

```
Clock Network End Type Abbreviations:
```

```
REG - register
PORT - port
CS - clock_source
```

Example	End Pin/Port	End Type	Clocks
ff1/CP (FD1)		REG	set 0
1			

```
Clock Sets:
```

```
set 0 (1 clocks):
    CLK1 - positive
```

```
1
```

The result by using the [analyze_clock_networks](#) command shows the only clock source found starting from **PORT_CLK1** is clock **CLK1**.

Fix Suggestion

The source latency specification needs to be revised so the clock and its relative clock source match.

In the above scenario, if the requirement is to set the clock latency on the port **PORT_CLK1** with respect to the clock **CLK1**, the revised specification would be:

```
set_clock_latency -source 5 [get_ports PORT_CLK1] -clock CLK1
```

See Also

[create_clock](#)

[set_clock_latency](#)

[analyze_clock_networks](#)

Rule: CSL_0007

Source latency defined on pin/port ***pin_port*** will overwrite the clock source latency for clock ***clock***.

Severity: Warning

Description

The clock source latency should be specified on either the clock or the clock definition point, but not both at one time. Doing so results in overwriting the clock source latency of the clock and could create incorrect timing results during static timing analysis.

Risk Associated with Violation

Since the source latency value is changed, it can potentially lead to unexpected and incorrect analysis results, if ignored.

One Possible Scenario

Consider the following scenario where a clock is defined on an input port:

```
create_clock -name clk1 -period 10 -waveform { 0 5 } [get_ports {clkp}]
```

A clock source latency is defined on the clock object:

```
set_clock_latency -source -min 55 -dynamic 11 [get_clocks {clk1}]
```

```
set_clock_latency -source -max 55 -dynamic 11 [get_clocks {clk1}]
```

A clock source latency is also defined on the clock definition point of **clk1**:

```
set_clock_latency -source -clock clk1 1000 [get_ports clkp] -early
```

```
set_clock_latency -source -clock clk1 2000 [get_ports clkp] -late
```

This clock source latency on the **clkp** port will overwrite the source latency of **clk1**. PrimeTime constraint consistency flags a **CSL_0007** in such a case to warn the designer that the source latency of the clock object has been changed so that any unexpected results can be avoided.

What Next

The “Violation Details” section displays the clock for which the source latency has been overwritten, along with the clock definition.

Fix Suggestion

You should check the source latencies applied on the clock definition point and remove the incorrect ones.

In the example scenario presented previously, one easy fix is to remove the source latency applied on the port **clkp** as follows:

```
remove_clock_latency -source -clock clk1 [get_ports {clkp}]
```

See Also

[set_clock_latency](#)

[remove_clock_latency](#)

[report_clock](#)

Rule: CTR_0002

Clock **clock** has incomplete clock transition values.

Severity: Warning

Description

The reported clock has incomplete transition data. Whenever specifying transition time, all four transition types need to be added: **min_rise**, **min_fall**, **max_rise**, and **max_fall**.

Risk Associated with Violation

If only partial clock transition information is provided, the resulting timing analysis might be pessimistic in some corners.

The definition of transition time on a clock overrides the standard way of computing transition on register pins. This is used only for ideal clock, during pre-layout timing analysis. Post-layout timing analysis should use the propagated clocks for more accurate timing results. If no clock transition is specified for an ideal clock, two different techniques can be used for computing timing:

- 0 delay is assumed if the [timing_ideal_clock_zero_default_transition](#) variable is **true**
- Standard delay is computed otherwise as for any other pin in the design

It is very unlikely that only some transitions are estimated and overridden by a user, while others are left to the default behavior of the tool. This is why a **CTR_0002** violation is flagged whenever partial transition times are specified for a given clock.

One Possible Scenario

Consider a scenario where a clock is created on the clock port, and only **-min** clock transitions are specified:

```
create_clock -period 10 CLK
set_clock_transition -min 4 CLK
```

Specifying a minimum rise/fall transition on the clock network is equivalent to estimate than when the clock tree is done, there will be a 4 ns transition on this register clock pin. It will then be very unlikely that the maximum rise/fall will be 0 ns. As a consequence, there is a possible timing inaccuracy with the current clock specification and the **CTR_0002** will bring this to the user's attention.

You can report the transition values on any clock in the design using the [report_clock](#) command as seen below:

```
ptc_shell> report_clock -skew CLK

*****
Report : clock_skew
Design : CTR_0002
Scenario: default
Version: ...
Date   : ...
*****
```

Object	Min Rise Transition	Min Fall Transition	Max Rise Transition	Max Fall Transition
CLK	4.00	4.00	-	-

What Next

The violation message will report the clock with incomplete transitions. The “Violation Details” section will give you the transition details currently set on that clock.

Fix Suggestion

You should review the current clock specification and add the missing clock transition values. In the above example, one needs to specify **-max** transition value for the clock **CLK** to be completely specified.

See Also

[set_clock_transition](#)
[report_clock](#)

Rule: CTR_0003

Propagated clock **clock** has ideal clock transition values.

Severity: Warning

Description

Even though the reported clock is propagated, it has some transition values specified through the [set_clock_transition](#) command. These user defined transitions time will be ignored and the propagated clock transition will be used for delay calculation instead.

Risk Associated with Violation

The propagated transition times will be used instead of the user specified clock transition values. This is the expected behavior for a propagated clock. If the intent was to use the user defined clock transition values, the clock should not be propagated.

One Possible Scenario

Consider a scenario where a clock is created on the clock port, and is propagated:

```
create_clock -period 10 CLK
set_propagated_clock CLK
set_clock_transition 3 CLK
```

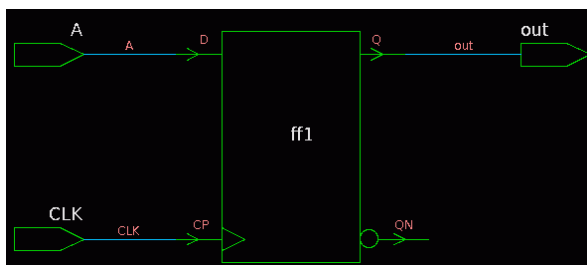


Figure 1: Missing Input Delay

The transition times used at the register clock pin **ff1/CP** will be the results of propagating the clock **CLK** through the net **CLK**. These transition time will *not* be the ideal transition times specified in the SDC file through the [set_clock_transition 3 CLK](#) command.

What Next

The violation message reports what propagated clock also has ideal transitions specified. The “Violation Details” section reports more detailed information related to these transition times.

The transition times for a specific clock can also be reported using the [report_clock](#) command as seen below:

```
ptc_shell> report_clock -skew CLK

*****

Report : clock_skew
Design : CTR_0003
Scenario: default
Version: ...
Date   : ...
*****
```

Object	Min Rise Transition	Min Fall Transition	Max Rise Transition	Max Fall Transition
CLK	3.00	3.00	3.00	3.00

Fix Suggestion

Ideal clock transition times should not be specified on propagated clocks. If the clock is indeed a propagated clock and that the intent is to use propagated values, the [set_clock_transition](#) should be removed from the SDC file so as not to cause confusion.

If the intent is to use ideal clock transition in place of the propagated transitions, then the clock should not be propagated. This can be achieved in two different ways:

- By not having [set_propagated_clock](#) in the constraint file
- By using the [remove_propagated_clock](#) command

See Also

[set_propagated_clock](#)
[remove_propagated_clock](#)
[report_clock](#)
[set_clock_transition](#)

Rule: CTR_0004

Virtual clock **clock** has clock transition values.

Severity: Warning

Description

The virtual clock reported by the violation has some clock transition values. The [set_clock_transition](#) command is intended to be used with non-virtual, ideal clocks only. Virtual clocks are used to specify the behavior of your block with the outside environment.

Risk Associated with Violation

There is no risk associated with the violation, but the specified transitions will be ignored.

One Possible Scenario

Consider a scenario in which a virtual clock is created, then clock transitions are specified on the virtual clock:

```
create_clock -period 10 -name v1
set_clock_transition 4 [get_clocks v1]
set_input_delay 2 -clock v1 [get_ports myInputPort]
```

What Next

The Violation message reports the virtual clock with transitions specified. The “Violation Details” section displays table containing the transition details. You can also use the [report_clock](#) command to get information related to a specific clock as shown below:

```
ptc_shell> report_clock [get_clocks v1]

*****

Report : clock

Design : CTR_0004

Scenario: default

Version: ...

Date : ...

*****

Attributes:

p - Propagated clock

G - Generated clock

I - Inactive clock
```

```
Clock Period Waveform Attrs Sources
-----

v1 10.00 {0 5} {}

ptc_shell> report_clock [get_clocks v1] -skew
*****

Report : clock_skew
Design : CTR_0004
Scenario: default
Version: ...
Date : ...
*****

Min Rise Min Fall Max Rise Max Fall
Object Transition Transition Transition Transition
-----

v1 4.00 4.00 4.00 4.00
```

Fix Suggestion

If virtual clock used only to specify the timing interface outside of the block, no specific step need to be taken. The clock transition values will be ignored though and you might want to take them out of your constraint file.

If the clock is also used for timing purpose inside the block, you need to make that clock a real clock (as opposed to a virtual one) by giving it a source.

See Also

[create_clock](#)

[report_clock](#)

Rule: CTR_0005

Clock **clock** has no clock transition values in pre-layout.

Severity: Warning

Status: Disabled by default

Description

A clock transition should be specified on all non-virtual clocks for pre-layout analysis.

Risk Associated with Violation

If a clock transition time is not specified, zero transition times will be used for analysis. This can lead to less accurate timing results during pre-layout analysis. This less accurate timing can result in poor optimization/implementation results prior to clock tree synthesis.

One Possible Scenario

Consider the scenario below, where a clock is created, but no clock transition is specified on the clock:

```
create_clock -period 10 [get_ports CLK]
```

The clock **CLK** has no transition time specified in the constraints for the design. As a result, a **CTR_0005** rule violation will be issued if this rule is enabled.

What Next

The violation message will include the name of the clock with the missing transition.

Fix Suggestion

To avoid the violation, you can specify a transition for the clock using the [set_clock_transition](#) command. In the example scenario, a rise transition time of 0.38 time units and a fall transition time of 0.25 time units can be specified for the clock **CLK** as follows:

```
set_clock_transition 0.38 -rise [get_clocks CLK]
```

```
set_clock_transition 0.25 -fall [get_clocks CLK]
```

See Also

[set_clock_transition](#)

[create_clock](#)

Rule: CTR_0006

Clock **clock** has clock transition values in post-layout.

Severity: Warning

Status: Disabled by default

Description

A clock transition should not be specified on clocks in post-layout analysis. For virtual clocks that have a clock transition specified, a [CTR_0004](#) rule violation is issued. The [CTR_0004](#) rule is a check performed on virtual clocks only.

No ideal clocks or clock transitions should be present in post-layout analysis. The [CLK_0033](#) rule can also be enabled to verify there are no ideal clocks present in the constraints.

Risk Associated with Violation

In post-layout analysis, calculated clock transition values will be used for timing analysis. The transition specified by the [set_clock_transition](#) command will be ignored. If SPICE simulation values are being used to override the clock network timing analysis in post-layout, the [set_annotated_transition](#) command should be used.

One Possible Scenario

Consider the scenario below, where a clock is created and propagated for post-layout analysis, and then the [set_clock_transition](#) command is applied to the clock:

```
create_clock -period 10 [get_ports CLK]
set_propagated_clock [get_clocks CLK]
set_clock_transition 4 [get_clocks CLK]
```

The clock **CLK** has a transition time **4**, specified in the constraints for the design. As a result, a **CTR_0006** rule violation will be issued if this rule is enabled.

What Next

The violation message will include the name of the clock that has an input transition defined. The "Violation Details" section will list the specified transition values for the clock.

Fix Suggestion

To resolve the violation, remove the [set_clock_transition](#) command related to the clock from the constraints. If SPICE simulation values are being used to override the calculated clock network timing, use the [set_annotated_transition](#) command on the pins in the clock network.

See Also

[CTR_0004](#)

[CLK_0033](#)

[set_clock_transition](#)

[remove_clock_transition](#)

[set_annotated_transition](#)

Rule: CTR_0001

Clock **clock** has inconsistent clock transition values.

Severity: Warning

Description

Some attributes in the clock definition are inconsistent. The **min_rise** attribute should be less than the **max_rise** one; and the **min_fall** attribute should be less than the **max_fall** one.

Risk Associated with Violation

The specified clock transition is inconsistent with real design data. This can potentially lead to incorrect analysis results if left unchanged.

One Possible Scenario

Consider a scenario where a clock is created on the clock port, and incorrect clock transitions are specified:

```
create_clock -period 10 CLK
set_clock_transition -min 4 CLK
set_clock_transition -max 2 CLK
```

The minimum transition time specified for the clock is larger than the maximum transition time. This is most likely a mistake in the clock definition and a **CTR_0001** violation will be triggered under those conditions.

What Next

The violation message will give you the clock with incorrect transitions values. The “Violation Details” section will give more details as to what exactly was specified for each of the clock transition attributes.

You can report the current transition time applied on any clock in your design through the [report_clock](#) command as seen below:

```
ptc_shell> report_clock -skew

*****
Report : clock_skew
Design : CTR_0001
Scenario: default
Version: ...
Date   : ...
```

Object	Min Rise Transition	Min Fall Transition	Max Rise Transition	Max Fall Transition
-----	-----	-----	-----	-----
CLK	4.00	4.00	2.00	2.00

Fix Suggestion

The clock specification should be reviewed and updated such that the minimum transition time is smaller or equal to the maximum transition time. This will ensure a more accurate timing analysis.

See Also

[set_clock_transition](#)

[report_clock](#)

Rule: DES_0001

Register clock pin *pin* has no clock signal propagating to it.

Severity: Warning

Description

The clock pin does not have a clock signal propagating to it. The design is not fully constrained. A register clock pin is any pin that is a reference to a timing check or has a sequential timing arc from it. Special cases of pins that are not clocked are identified in rule [DES_0003](#).

Risk Associated with Violation

The design is not fully constrained. Setup or hold checks cannot be properly performed at the register, and the paths to and from the register are not constrained. As a result, timing checks for the paths related to the register cannot be performed.

One Possible Scenario

Consider a scenario where a scan register is clocked by either the functional clock **CLK** or the scan clock **TCLK** through a MUX. A case analysis value is specified to select the MUX input of the functional clock **CLK**; however, the clock specification for **CLK** is missing from the constraints. As a result, no clock signal reaches the clock pin of the scan register **ff1/CP**. Since **ff1** is not clocked, a **DES_0001** rule violation will be issued by the tool.

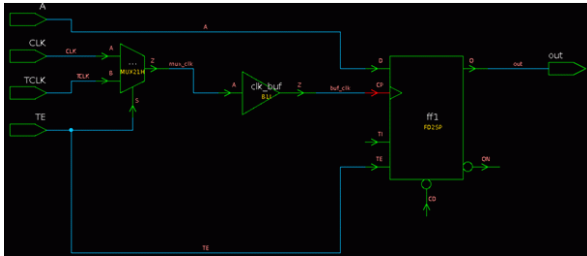


Figure 1: No Clock Signal Reaches Clock Pin of Scan Register ff1/CP

```
create_clock -period 30 [get_ports TCLK]
set_case_analysis 0 [get_ports TE]
```

What Next

The violation message identifies the clock pin that does not have a clock reaching it.

The “Debugging Help” section provides context specific suggestions for investigating the violation to determine why the clock pin does not have a clock signal reaching it.

There are several possible causes for this violation. Some common possibilities are that the clock is blocked from reaching the pin due to a constraint blocking it, that a clock definition is missing, or that a generated clock is not expanded.

Use the **potential_clocks** attribute to see if the pin is in a clock network, but was blocked by a [set_case_analysis](#), [set_disable_timing](#), or some other constraint.

For the example scenario, the attribute can be queried as follows:

```
ptc_shell> get_attribute [get_pins ff1/CP] potential_clocks
{"TCLK"}
```

If the **potential_clocks** attribute returns a clock, then use the [analyze_clock_networks](#) command with the **-traverse_disabled** option to find which constraint blocked the clock from reaching the pin. For the example scenario, the command can be issued as follows:

```
ptc_shell> analyze_clock_networks -from [get_clocks TCLK] \
-through [get_pins ff1/CP] -traverse_disabled -style full
```

Report : analyze_clock_networks

-max_endpoints=1

-style full

```
-end_types {register port clock_source}
```

```
-traverse_disabled
```

```
Design : ...
```

```
Scenario: ...
```

```
Version : ...
```

```
Date : ...
```

```
*****
```

```
Clock Sense Abbreviations:
```

```
P - positive
```

```
Potential senses detected with -traverse_disabled:
```

```
[P] - potential positive
```

```
Clock Network End Type Abbreviations:
```

```
REG - register
```

```
Full report for clock: TCLK
```

```
Branch 0:
```

```
Branch
```

```
Level Info Sense Notes Port/Pin
```

```
-----  
0 P source TCLK (in)
```

```
1 P clk_mux/B (MUX21H)
```

```
2 [P] CARC#0 clk_mux/Z (MUX21H)
```

```
3 [P] clk_buf/A (B1I)
```

```
4 [P] clk_buf/Z (B1I)
```

```
5 [P] REG ff1/CP (FD2SP)
```

```
Clock Blocking Constraints:
```

```
CARC#0 = case_value_disabled_arc
```

```
at positive_unate arc from: clk_mux/B
```

```
to: clk_mux/Z
```

The report shows that a case analysis command is disabling the timing arc allowing the scan clock **TCLK** to propagate through the MUX to the scan register clock pin **ff1/CP**.

If the object is not in a potential clock network, then there may be a missing clock definition. The [analyze_unclocked_pins](#) command can be used to investigate whether a missing clock definition on a port exists or not. For the example scenario, the following command can be used:

```
ptc_shell> analyze_unclocked_pins [get_pins ff1/CP]
```

```
*****
```

```
Report : analyze_unclocked_pins
```

```
Design : ...
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Analyze Register Clock Pins that do not have clocks assigned
```

```
Summary
```

```
-----
```

```
Register Clock Pin Status Number of Pins
```

```
-----
```

```
Clocked 0
```

```
Unclocked 1
```

```
Case disabled clock pin 0
```

```
Register behavior disabled 0
```

```
-----
```

```
Total register clock pins 1
```

```
Possible Startpoints for Missing Clock Definitions
```

```
Number
```

```
Pin/Port Startpoints unclocked Reason a Startpoint
```

CLK 1 input port

Clocks are blocked from register clock pins at the following locations:

Number

Clock Blocking Pin unclocked Reason

TCLK clk_mux/Z 1 case propagation

The report shows that the scan register clock pin **ff1/CP** also has a possible missing clock definition at the input port **CLK**.

If a generated clock is not expanded, review the violations in the [CLK_0003](#), [CLK_0009](#), [CLK_0010](#), or [CLK_0011](#) rules for the reasons the generated clock is not expanded.

Fix Suggestion

For most cases, constraints may be missing for the design or contain an error.

- If a clock signal is blocked from reaching the register clock pin, consider adjusting the constraints so the clock can propagate to the clock pin.
- If the register clock pin is not in any clock's potential clock network, evaluate whether a clock definition is missing from the constraints.
- If a generated clock is not being expanded, consider adjusting the constraints so that the clock can be expanded and propagate to the clock pin.

For the example scenario, a clock definition for the clock, **CLK**, needs to be added to the constraints for the functional mode of the design:

```
create_clock -period 10 [get_ports CLK]
create_clock -period 30 [get_ports TCLK]
set_case_analysis 0 [get_ports TE]
```

See Also

[DES_0003](#)

[CLK_0003](#)

[CLK_0009](#)

[CLK_0010](#)

[CLK_0011](#)

[analyze_clock_networks](#)

[analyze_unclocked_pins](#)

[create_clock](#)

[create_generated_clock](#)

[set_case_analysis](#)

[set_disable_timing](#)

Rule: DES_0002

All checks on the register clock pin *pin* are disabled due to *reason*.

Severity: Warning

Status: Disabled by default

Description

All checks on the reported register clock pin are disabled because of one of the following reasons:

- A case value in the design is reaching this particular pin. This could be the results of a [set_case_analysis](#) command or a tied signal in the netlist itself
- This clock pin cannot launch any valid path because relevant timing arcs are disabled

Risk Associated with Violation

The design is not fully constrained. Setup and hold checks cannot be correctly performed at the register. In addition, the paths to and from the register are not constrained. As a result, no timing checks related to this register can be done.

One Possible Scenario

Consider a scenario shown in Figure 1 and the related constraints below. This example demonstrates the two types of reasons that can trigger a DES_0002 warning.

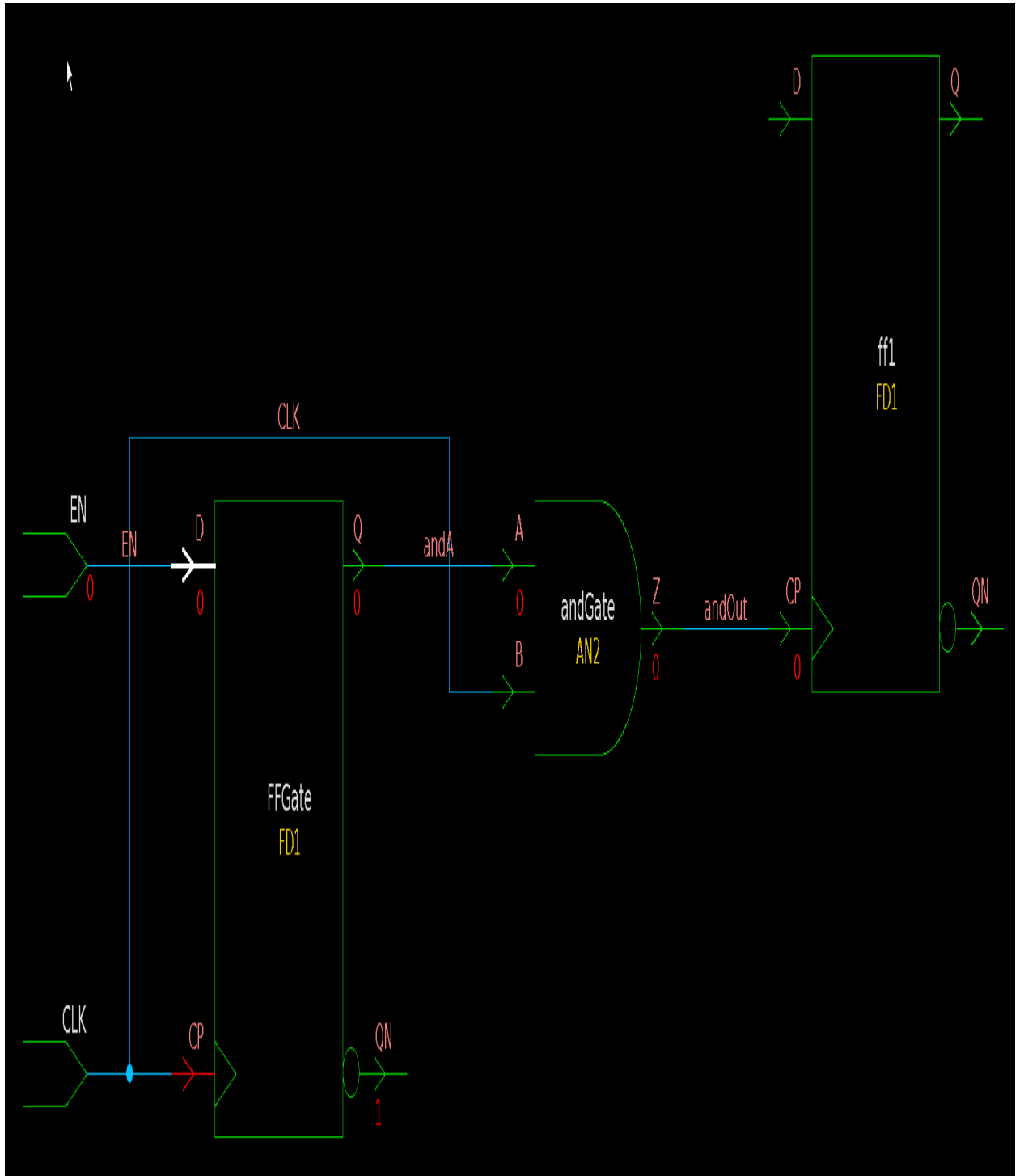


Figure 1

Constraint set:

```
enable_rule DES_0002
create_clock -period 10 CLK
set_case_analysis 0 [get_ports EN]

set_app_var case_analysis_sequential_propagation always
set_app_var case_analysis_propagate_through_icg true
```

This design and its associated constraints create two different DES_0002 violations: one for each register.

The ff1/CP pin is flagged because the constant value applied at port EN propagates through the register (FFGate) and the clock gating cell (andGate). This sets a constant value of 0 directly on the register clock pin, disabling this pin. All the checks on pin are also disabled.

The FFGate/CP pin also gets reported as a DES_0002 violation. The only valid path out of the register is through andGate/A. However, this is a constant pin because of the case value placed on port EN. As a consequence, there is no valid path that this register can launch and all the checks on the register clock pin are disabled.

What Next

In the GUI, the violation details section of the info pane window will let you know whether the violation is caused by a constant or some other disabled timing arcs. If further debugging is required, the next steps to take will vary depending of the cause of the violation.

Constant Register Pin:

You can use the [report_case_details](#) command to learn where the constant value is created in the design. Below is an example of such a report:

```
ptc_shell> report_case_details -to [get_pins ff1/CP]

*****
Report : case_details
      -to
      -max_objects = 1000
Design : DES_0002
Version: ...
Date   : ...
*****
Properties      Value      Pin/Port
-----
from user case      0      ff1/CP (FD1)

Case fanin report:
Verbose Source Trace for pin/port ff1/CP:
Path number: 1
Path Ref #      Value      Properties      Pin/Port
```

0		ff1/CP (FD1)
0	F()=A B	andGate/Z (AN2)
0		andGate/A (AN2)
0	F(): BST func	FFGate/Q (FD1)
0		FFGate/D (FD1)
0	user case	EN (in)

Disabled Check Arcs:

In the case of disabled check arcs, you can use the [analyze_unclocked_pins](#) command to verify the cause of the problem. A register clock pin with disabled check arcs should appear as “Register Behavior Disabled” in this report.

```
ptc_shell> analyze_unclocked_pins -verbose FFGate/CP
```

```
*****
```

```
Report : analyze_unclocked_pins
        -verbose
```

```
Design : DES_0002
```

```
Version: ...
```

```
Date   : ...
```

```
*****
```

```
Analyze Register Clock Pins that do not have clocks assigned
```

Summary

Register Clock Pin Status	Number of Pins
Clocked	0
Unclocked	0
Case disabled clock pin	0
Register behavior disabled	1

Total register clock pins	1

You can also report the various timing arcs on the cell and search for the ones marked with a ‘c’. The ‘c’ flag reports arcs disabled because of case-analysis. See the example below for the design presented earlier:

```
ptc_shell> report_disable_timing -all_arcs [get_cells FFGate]
```

```
*****
```

```
Report : disable_timing
```

```
Design : DES_0002
```

```
Version: ...
```

```
Date   : ...
```

```
*****
```

Attributes

- c - case-analysis
- C - Conditional arc
- d - default conditional arc

f - false net-arc
l - loop breaking
L - db inherited loopbreaking
m - mode
p - propagated constant
u - user-defined
U - User-defined library arcs

Cell or Port	From	To	Sense	Flag	Reason
FFGate Q = 0, QN = 1	CP	QN	rising_edge	c	D = 0,
FFGate Q = 0, QN = 1	CP	Q	rising_edge	c	D = 0,
FFGate	CP	CP	clock_pulse_width_high enabled		
FFGate	CP	CP	clock_pulse_width_low enabled		
FFGate Q = 0, QN = 1	CP	D	setup_clk_rise	c	D = 0,
FFGate Q = 0, QN = 1	CP	D	hold_clk_rise	c	D = 0,

Fix Suggestion

Verify that the constraints are correctly set in the design. If you find this warning is unexpected, adjust the case constant values in your design.

See Also

[set_case_analysis](#)

[report_case_details](#)

[analyze_unclocked_pins](#)

[report_disable_timing](#)

Rule: DES_0003

The register clock pin **pin** that is part of a generated clock source latency path does not receive a valid clock signal.

Severity: Warning

Description

The register clock pin has no clock signal propagating to it. This register is part of a generated clock source latency path.

Risk Associated with Violation

The setup and hold checks on the register will not be analyzed. The source latency through this nonclocked register can be properly timed as compared to those registers given in rule [DES_0001](#).

One Possible Scenario

Consider the scenario shown in Figure 1, where the register, **ff2**, is in the generated clock's source latency path, but no clock signal reaches the **ff2** clock pin.

```
create_clock -period 10 [get_ports CLK]
create_generated_clock -name gclk -divide_by 4 \
    -master_clock [get_clocks CLK] \
    -source [get_ports CLK] U1/Z -add -invert
```

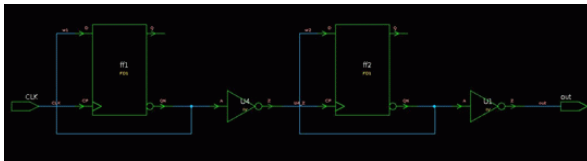


Figure 1: No Clock Signal Reaches the ff2 Clock Pin

What Next

The hold constraint for **ff2** will not be analyzed because there is no clock signal at **ff2/D**. You can review the source latency path of a generated clock using the [analyze_clock_networks](#) command.

```
ptc_shell> analyze_clock_networks -to [get_clock gclk] -style full
```

```
*****
```

```
Report : analyze_clock_networks
```

```
-max_endpoints=1000
```

```
-style full
```

```
-end_types {register port clock_source}
```

```
Design : DES_0003
```

```
Scenario: default
```

```
Version : ...
```

```
Date : ...
```

```
*****
```

```
Clock Sense Abbreviations:
```

P - positive

R - rise_triggered

Clock Network End Type Abbreviations:

REG - register

CS - clock_source

Source latency paths for generated clock: gclk

from master clock: CLK

Branch 0:

Branch

Level Info Sense Notes Port/Pin

0 P source CLK (in)

1 P REG ff1/CP (FD1)

2 P ff1/CP (FD1)

3 R ff1/QN (FD1)

4 R U4/A (IV)

5 R U4/Z (IV)

6 R REG ff2/CP (FD1)

7 R ff2/CP (FD1)

8 R ff2/QN (FD1)

9 R U1/A (IV)

10 R CS U1/Z (IV)

Fix Suggestion

To resolve this violation, consider creating generated clocks whenever the latency path is going through registers. To address the issue for this particular example, specify two generated clocks as shown below:

```
create_clock -period 10 [get_ports CLK]
create_generated_clock -name ff1_QCLK -divide_by 2 \
```

```
-master_clock [get_clocks CLK] \  
-source [get_ports CLK] -add  
create_generated_clock -name gclk -divide_by 2 \  
-master_clock [get_clocks ffl_QCLK] \  
-source [get_pins ffl/QN] U1/Z -add -invert
```

See Also

[analyze_clock_networks](#)

[create_generated_clock](#)

[create_clock](#)

[DES_0001](#)

Rule: DES_0004

Register clock pin *pin* does not have a clock signal that can sensitize the register.

Severity: Warning

Description

The clock pin has a clock signal. However, the clock signal does not have the required edge that can sensitize the register.

Risk Associated with Violation

The design is not fully constrained. Setup or hold checks cannot be properly performed at the register, and the paths to and from the register are not constrained. As a result, timing checks for the paths related to the register cannot be performed.

What Next

The violation message identifies the register clock pin that has a clock reaching it but the clock signal cannot sensitize the register.

The “Debugging Help” section provides context specific suggestions for investigating the violation to determine why the clock signal reaching the register clock pin cannot sensitize the register.

You can get more details about the clock and the clock pin by:

1. Using the [analyze_clock_networks](#) command

```
ptc_shell> analyze_clock_networks -style full -to chkbblock/CLKW  
*****  
Report : analyze_clock_networks  
-max_endpoints=1000  
-style full
```

```
-end_types {register port clock_source async_pin}
Design : test
Scenario: default
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
P - positive
FF - fall_to_fall
Clock Network End Type Abbreviations:
REG - register
CG - clock_gating
Full report for clock: clk
Branch 0:
Branch
Level  Info      Sense Notes      Port/Pin
-----
0      P         source      clk_in (in)
1      P         CG         prevblock/CLK
2      FF        COND#0     prevblock/GCLK
3      FF        REG         chkbblock/CLK
Clock Blocking Constraints:
COND#0 = conditional_arc
at positive_unate arc from: prevblock/CLK
to: prevblock/GCLK
```

2. Checking the **clocks_with_sense** attribute of the clock pin (which determines which clocks and with what edges reach the pin)

```
ptc_shell> get_attribute [get_pin chkbblock/CLK ] clocks_with_sense
{ { "clk" "fall_to_fall" } }
```

3. Checking the **is_rise_edge_triggered_clock_pin** and **is_fall_edge_triggered_clock_pin** attributes on the clock pin (which determine whether the clock pin is rise edge triggered or fall edge triggered)

```
ptc_shell> get_attribute [get_pin chkbblock/CLK ]
is_rise_edge_triggered_clock_pin
true
ptc_shell> get_attribute [get_pin chkbblock/CLK ]
is_fall_edge_triggered_clock_pin
false
```

See Also

[DES_0001](#)

[analyze_clock_networks](#)

[create_clock](#)

Rule: DRV_0001

Input or inout port **port** has no input transition or driving cell or drive resistance specified.

Severity: Warning

Description

The reported input or inout port is missing drive information. To represent the timing data accurately during static timing analysis calculations, all input and inout ports need complete drive data. The complete drive data includes input transition, driving cell and drive resistance. This can be accomplished by using the [set_input_transition](#), [set_driving_cell](#), and [set_drive](#) commands.

Risk Associated with Violation

During optimization, the drive of an input port is used to calculate the timing delay to gates driven by that port. The *resistance* is the output drive strength of the cell at the port, a high drive strength (resistance) means less drive capability and longer delays. Thus, a *resistance* of 0 is infinite drive, or no delay between the ports to all its connections; a specification of 0 resistance will result in overly optimistic delay calculations. The *resistance* must be in unit consistent with the technology library used during optimization. In addition to the drive resistance, the transition time should also be specified on that port.

A drive resistance of 0 is unrealistic and can result in optimistic delay calculation numbers.

The current drive resistance can be reported using the [report_port-drive](#) command as shown below:

```
ptc_shell> report_port [get_ports A] -drive
```

```
*****
```

```
Report : port
```

```
-drive
```

```
Design : DRV_0001
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Resistance (min/max)
```

```
Input Port Rise Fall
```

A -- --

What Next

In order to ensure the quality of your delay calculation, a realistic drive resistance should be specified. The `set_driving_cell` command is more convenient and accurate than `set_drive` for describing the drive capability of a port. The `set_drive` command removes any corresponding rise or fall driving cell attributes on the specified ports, whereas `set_driving_cell` will pick those values from the library itself.

The violation message will give you the input ports without driving cell or drive resistance specified. You should use `set_driving_cell` on those ports as demonstrated below.

```
ptc_shell> set_driving_cell -lib_cell AN2 \ -pin Z -library pt_lib
[all_inputs]
```

```
ptc_shell> report_port [get_ports A] -drive
```

```
*****
```

```
Report : port
```

```
-drive
```

```
Design : DRV_0001
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Resistance (min/max)
```

```
Input Port Rise Fall
```

A -- --

```
Driving Cell
```

```
Input Port Type Cell Mult Clock Attrs
```

A min_rise pt_lib/AN2/Z

```
A min_fall pt_lib/AN2/Z  
A max_rise pt_lib/AN2/Z  
A max_fall pt_lib/AN2/Z
```

Fix Suggestion

Specify the input resistance of the port using the [set_driving_cell](#) command.

See Also

[set_drive](#)

[set_input_transition](#)

[set_driving_cell](#)

[report_port](#)

Rule: DRV_0002

Input or inout port **port** has incomplete input transition or driving cell or drive resistance specified.

Severity: Warning

Description

The reported input or inout port is missing drive information. To represent the timing data accurately during static timing analysis calculations, all input and inout ports need complete drive data. The complete drive data includes input transition, driving cell and drive resistance. This can be accomplished by using the [set_input_transition](#), [set_driving_cell](#), and [set_drive](#) commands.

Risk Associated with Violation

During optimization, the drive of an input port is used to calculate the timing delay to gates driven by that port. The *resistance* is the output resistance of the cell that drives the port, such that higher drive strength (resistance) means less drive capability and longer delays. Thus, a *resistance* of 0 is infinite drive, or no delay between the ports and all that is connected to them. The *resistance* must be in unit consistent with the technology library used during optimization.

There are four different parameters controlling the drive of an input/inout port:

- **min_rise**
- **max_rise**

- **min_fall**
- **max_fall**

A value of 0 in any of those parameters is unrealistic and can result in optimistic delay calculation numbers.

One Possible Scenario

Consider a scenario where the drive of port A is specified as follow:

```
create_clock -period 10 [get_ports CLK]
set_input_transition -fall 4 [get_ports A]
```

When reporting the drive information using the [report_port-drive](#) command, you can see that the drive data is incomplete (as seen with "--/--" in the report). This situation potentially creates inaccurate static timing analysis calculation and a **DRV_0002** is issued in constraint consistency.

```
ptc_shell> report_port -drive [get_ports A]
```

```
*****
```

```
Report : port
```

```
-drive
```

```
Design : DRV_0002
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Resistance (min/max)
```

```
Input Port Rise Fall
```

```
-----
```

```
A -- --
```

```
Transition (min/max)
```

```
Input Port Rise Fall Related Clock
```

```
-----
```

```
A --/-- 4.00/4.00
```

What Next

The violation message reports which part has incomplete transition/driving cell/driving resistance.

The drive specifications need to be complete in order to get the most accurate timing calculation. Although it is possible to specify them through a combination of [set_drive](#) and [set_input_transition](#), it is best to use the [set_driving_cell](#) command and to choose a relevant cell from your current target library.

In the example above, this can be achieved using the following command:

```
ptc_shell> set_driving_cell -lib_cell AN2 \ -pin Z -library pt_lib  
[all_inputs]
```

```
ptc_shell> report_port [get_ports A] -drive
```

```
*****
```

```
Report : port
```

```
-drive
```

```
Design : DRV_0001
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Resistance (min/max)
```

```
Input Port Rise Fall
```

```
A -- --
```

```
Driving Cell
```

```
Input Port Type Cell Mult Clock Attrs
```

```
A min_rise pt_lib/AN2/Z
```

```
A min_fall pt_lib/AN2/Z
```

```
A max_rise pt_lib/AN2/Z
```

```
A max_fall pt_lib/AN2/Z
```

Fix Suggestion

You need to specify complete input transition/driving cell/driving resistance on input ports.

See Also

[set_input_transition](#)

[set_drive](#)

[set_driving_cell](#)

[report_port](#)

Rule: DRV_0003

Input or inout port **port** has input transition or drive resistance instead of **set_driving_cell**.

Severity: Warning

Status: Disabled by default

Description

In order to obtain a realistic input waveform for crosstalk analysis, it is important to use a driving cell on the port. When specifying this data with a combination of [set_input_transition](#) and [set_drive](#), PrimeTime SI will not be able to model the driver characteristics accurately.

Risk Associated with Violation

In order to calculate accurately the effect of ports that act as aggressors, a realistic waveform is needed. In the absence of [set_driving_cell](#), PrimeTime SI will use an ideal driver model, resulting in overly pessimistic crosstalk analysis. This may produce more crosstalk violations in your design.

One Possible Scenario

Consider a scenario where an input transition is specified on input port A instead of [set_driving_cell](#):

```
create_clock -period 10 [get_ports CLK]
set_input_transition -fall 4 [get_ports A]
```

The [report_port](#) command below shows that there is no driving cell data available for port A.

```
ptc_shell> report_port -drive [get_ports A]
```

```
*****
Report : port
       -drive
Design : DRV_0002
Scenario: default
Version: ...
Date   : ...
*****
```

Input Port	Resistance (min/max)		
	Rise	Fall	
A	--	--	

Input Port	Transition (min/max)		Related Clock
	Rise	Fall	
A	--/--	4.00/4.00	

This will result in an ideal driver model being used for crosstalk analysis in PrimeTime SI, and as a result, a **DRV_0003** violation is issued.

What Next

The violation message reports the port without a driving cell specified. In order to address this issue, a carefully selected driving cell should be chosen for the port.

Fix Suggestion

You need to specify a driving cell on the port. For example, if the input port is going to be driven by an AN2 cell, you can fix the violation by adding the [set_driving_cell](#) command to your constraints:

```
set_driving_cell -lib_cell AN2 -pin Z -library pt_lib [get_ports A]
```

See Also

[set_driving_cell](#)

[report_port](#)

[set_input_transition](#)

Rule: DRV_0004

Input or inout port **port** has inconsistent drive resistance values.

Severity: Warning

Description

The drive resistance values on the reported port are inconsistent. The values need to meet the following requirements:

- The min fall value needs to be smaller than or equal to its related max fall value
- The min rise value needs to be smaller than or equal to its related max rise value

Risk Associated with Violation

By definition, minimum and maximum values are used for minimum and maximum operating conditions. It is inconsistent with the intent of having different analysis modes to have the max resistance value be less than the min resistance value. This could lead to incorrect delay calculation and analysis.

One Possible Scenario

Consider a scenario where inconsistent resistance values are specified on port A:

```
create_clock -period 10 [get_ports CLK]
set_drive -min 4 [get_ports A]
set_drive -max 2 [get_ports A]
```

In this example, the following situations are present:

- **min_fall > max_fall**
- **min_rise > max_rise**

The values for the drive resistance are inconsistent and will trigger a **DRV_0004** violation in the design.

What Next

The violation message reports the port with inconsistent resistance values. The “Violation Details” section in the GUI has a table with the current **-min/-max** resistance values.

Fix Suggestion

Changing the **-min** drive resistance to be less than or equal to the **-max** drive resistance will fix the violation. For the above example, the **set_drive-min** constraint could be modified to a resistance of **1** to fix the violation:

```
set_drive -min 1 [get_ports A]
```

See Also

[set_drive](#)

Rule: DRV_0005

Input or inout port **port** has inconsistent input transition values.

Severity: Warning

Description

The input transition values on the reported port are inconsistent. The values need to meet the following requirements:

- The min fall value needs to be smaller than or equal to its related max fall value
- The min rise value needs to be smaller than or equal to its related max rise value

Risk Associated with Violation

By definition, minimum or maximum values are used for minimum or maximum operating conditions. It is inconsistent with the intent of having different analysis modes to have the max input transition value be less than the min transition value. This could lead to incorrect delay calculation and analysis.

One Possible Scenario

Consider a scenario where inconsistent input transition values are specified on port A:

```
create_clock -period 10 CLK
set_input_transition -min 3 [get_ports A]
set_input_transition -max 2 [get_ports A]
```

In this example, the following situations are present:

- **min_fall > max_fall**
- **min_rise > max_rise**

Those values for the input transition are inconsistent and will trigger a **DRV_0005** violation in the design.

What Next

The violation message reports the port with inconsistent input transition values. The “Violation Details” section in the GUI has a table with the current **-min/-max** input transition values.

Fix Suggestion

Changing the **-min** input transition to be less than or equal to the **-max** input transition will fix the violation. For the above example, the [set_input_transition-min](#) constraint could be modified to an input transition of **1** to fix the violation:

```
set_input_transition -min 1 [get_ports A]
```

See Also

[set_input_transition](#)

[report_port](#)

Rule: EXC_0001

An **exception** has conflicting *rise_or_fall* and *to_rise_or_fall* options specified.

Severity: Error

Description

The specified endpoint cannot simultaneously be both rising and falling. The **-rise/-fall** options refer to a rising/falling value at the end point. As a result, the **-rise_to** option conflicts with the **-fall** option and vice versa. The exception will be ignored.

Risk Associated with Violation

The exception will be ignored because of ambiguity due to the conflicting options. Ignored exceptions might result in undesired analysis of the design, resulting in larger optimization and/or timing closure effort. Multicycle paths in the design might be analyzed with the wrong timing, or timing violations might be reported for false paths. Maximum or minimum path delays will be checked against the library specified setup or hold time of the capturing clock.

One Possible Scenario

Consider a scenario where a constraint is specified with a conflicting **-rise** and **-fall_to** option:

```
set_multicycle_path 2 -rise -from [get_pins ff1/CP] \  
-fall_to [get_pins ff2/D]
```

This exception has conflicting values specified for the ff2/D end point. The **-fall_to** option specifies a falling edge at this point, and the **-rise** option specifies a rising edge at this point. Because both transitions cannot occur simultaneously, the exception will be ignored, and an **EXC_0001** rule violation will be issued.

What Next

The "Violation Details" section in the GUI contains the design objects in the exception and the location of the constraint conflicts.

Fix Suggestion

Revise the exception to remove the inconsistency. If the exception is not revised, it will be ignored. For the following example, the exception could be applied to both transitions at the endpoint, the **-rise/-fall** or **-rise_to/-fall_to** options need to be removed.

The example below shows possible ways to resolve the conflict:

- If the desired analysis at the end point is for a rising edge, it can be re-written in one of two ways:

```
set_multicycle_path 2 -rise -from [get_pins ff1/CP] \  
    -to [get_pins ff2/D]
```

Or:

```
set_multicycle_path 2 -from [get_pins ff1/CP] \  
    -rise_to [get_pins ff2/D]
```

- Similarly, if the desired analysis at the end point is for a falling edge, it can be re-written in one of two ways:

```
set_multicycle_path 2 -fall -from [get_pins ff1/CP] \  
    -to [get_pins ff2/D]
```

Or:

```
set_multicycle_path 2 -from [get_pins ff1/CP] \  
    -fall_to [get_pins ff2/D]
```

See Also

[set_false_path](#)

[set_multicycle_path](#)

[set_min_delay](#)

[set_max_delay](#)

Rule: EXC_0002

An **exception** has both valid and invalid startpoints and endpoints specified.

Severity: Warning

Description

The **-from** objects are not valid timing path startpoints, and the **-to** objects are not valid timing path endpoints.

Valid startpoints are:

- Input or bidirectional ports
- Register clock pins
- Transparent latch D-pins
- Internal pins that have a [set_input_delay](#) constraint set
- Internal pins that have a [set_min_delay](#) or [set_max_delay](#) constraint set with the **-from**, **-rise_from**, or **-fall_from** option
- A clock source

Valid endpoints are:

- Output or bidirectional ports
- Pins constrained by a setup/hold/recovery/removal timing check including:
- Register data pins
- Register preset and clear pins
- Clock gating enable pins
- Internal pins that have a **set_output_delay** constraint set
- Internal pins that have a [set_min_delay](#)/[set_max_delay](#) constraint set with the **-to**, **-rise_to**, or **-fall_to** option

The "Violation Details" section of the GUI shows the **from** or **to** objects that are invalid. The reasons are as follows:

- Non-startpoint – The pin is internal and is not a timing path startpoint
- Non-endpoint – The pin is internal and is not a timing path endpoint
- Endpoint – The pin or port is used as a **-from** object and is a timing path endpoint
- Startpoint – The pin or port is used as a **-to** object and is a timing path startpoint
- Case disabled – The pin or port has been disabled by [set_case_analysis](#)
- Check disabled – Timing check at the pin or port has been disabled
- Sequential disabled – A sequential arc at the timing path startpoint is disabled
- Node disabled – A valid endpoint or startpoint is disabled
- Hierarchical pin – The pin is on a hierarchical boundary

- Not a start cell – The cell used as a **-from** object does not have any valid timing path startpoints
- Not an end cell – The cell used as a **-to** object does not have any valid timing path endpoints

Only false path ([set_false_path](#)) and multicycle path ([set_multicycle_path](#)) exceptions are included in this check.

Risk Associated with Violation

The invalid startpoints and endpoints will be ignored, and the exception will take effect only on the valid timing points. The result is poor analysis or relaxed timing requirements being applied to the design.

One Possible Scenario

Consider a scenario where wildcards are used to specify that all paths that end at register pins with the name 'D' in a design are false:

```
set_false_path -to [get_pins */D]
```

The name 'D' could also be a pin name for other gates in the library that may not be valid timing path endpoints. In this scenario, the false path exception will not be applied to 'gate/D' because this is a combinational cell, and it is not a valid endpoint. An **EXC_0002** rule violation will be issued.

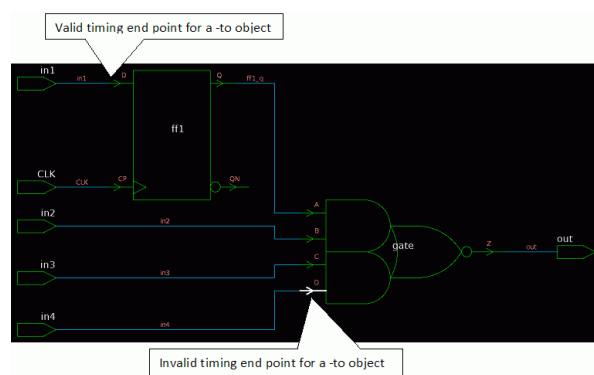


Figure 1: Valid and Invalid Endpoints Specified

What Next

Use the [analyze_paths](#) command to find out why the objects are disabled or invalid. This command uses **from/through/to** options just like the [set_false_path](#) and [set_multicycle_path](#) commands. In addition, use the **-traverse_disabled** option for disabled paths, and the **-unconstrained** option to check why objects are not valid startpoints or endpoints. The following example shows how the command can be used:

```
ptc_shell> analyze_paths -to [get_pins {ff1/D gate/D}] -path_type full
-traverse_disabled -unconstrained
```

Summary of Paths:

Clocks		Dominant	Overridden	Path
Startpoint	Endpoint	Constraint	Constraints	Count
Launch/Capture				
in4 (in)	gate/D (AO2)			2
unclocked/unclocked				
in1 (in)	ff1/D (FD1)	F0		2
unclocked/CLK				

Exception Information:

ID	Type	Setup	Hold	Source
F0	false_path	FALSE	FALSE	/u/gcamgr/test_data/scripts/constraints.tcl, line 3

Full report of all pins in the paths

Level	Pin	Attr
	Constraints	
1	in1 (in)	Start
1	in4 (in)	Start
2	gate/D (AO2)	
2	ff1/D (FD1)	End F0

The [analyze_paths](#) report shows that the gate/D pin is an input of the combinational cell AO2 and does not have a [set_output_delay](#), [set_min_delay](#), or [set_max_delay](#) constraint applied to it. Therefore, it is not a valid endpoint for the command [set_false_path -to \[get_pins */D\]](#).

Fix Suggestion

Revise the exceptions to specify only valid startpoints and endpoints.

Exceptions containing wildcard may lead to unintended false paths or multicycle paths being applied to the design. For example, false paths originally intended only for D-type flip-flops may be applied more broadly when new sequential elements are added to the design.

For the example scenario shown above, the false path can be specified as:

```
set_false_path -from ff1/D
```

See Also

[set_false_path](#)

[set_multicycle_path](#)

[analyze_paths](#)

Rule: EXC_0003

All **-from** objects for an **exception** are not valid path startpoints.

Severity: Warning

Description

All of the **-from** objects are not valid path startpoints. This exception will be ignored.

Valid startpoints are:

- Input or bidirectional ports
- Register clock pins
- Transparent latch D-pins
- Internal pins that have a [set_input_delay](#) constraint set
- Internal pins that have a [set_min_delay/set_max_delay](#) constraint set with the **-from**, **-rise_from**, or **-fall_from** option
- A clock source

The "Violation Details" section in the GUI shows the **-from** objects that are not valid. The reasons are as follows:

- Non-startpoint – The pin is internal and is not a timing path startpoint
- Endpoint – The pin or port is used as a **-from** object and is a timing path endpoint
- Case disabled – The pin or port has been disabled by [set_case_analysis](#)
- Check disabled – Timing check at the pin or port has been disabled
- Sequential disabled – A sequential arc at the timing path startpoint is disabled
- Node disabled – A valid startpoint is disabled
- Hierarchical pin – The pin is on a hierarchical boundary
- Non-start cell – The cell used as a **-from** object does not have any valid timing path startpoints

Only false path ([set_false_path](#)) and multicycle path ([set_multicycle_path](#)) exceptions are included in this check.

Risk Associated with Violation

The exception will be completely ignored. Ignored exceptions may lead to larger optimization and/or timing closure effort.

One Possible Scenario

Consider the following command used to disable a path between two registers:

```
set_false_path -from [get_pins ff1/Q] -to [get_pins ff2/D]
```

Even though the **-to** objects are correct, this exception is entirely ignored because the **-from** object is invalid, making the path specification ambiguous. The **EXC_0003** rule violation will be issued.

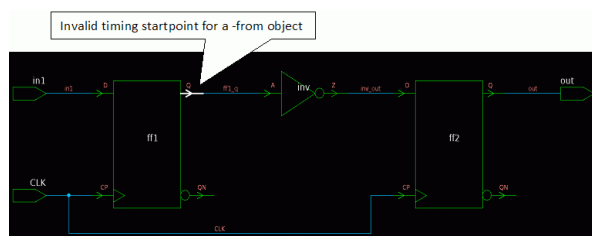


Figure 1: Ambiguous Path Specification

What Next

Use the [analyze_paths](#) command to find out why the objects are disabled or invalid. This command uses **-from**, **-through**, and **-to** options just like the [set_false_path](#) and [set_multicycle_path](#) commands. In addition, use the **-traverse_disabled** option for disabled paths, and the **-unconstrained** option to check why objects are not valid startpoints or endpoints. The following example shows how the command can be used:

```
ptc_shell> analyze_paths -from [get_pins {ff1/Q}] -to [get_pins {ff2/D}]  
-path_type full -traverse_disabled -unconstrained
```

Summary of Paths:

		Dominant	Overridden	Path
Clocks				
Startpoint	Endpoint	Constraint	Constraints	Count
Launch/Capture				
ff1/Q (FD1)	ff2/D (FD1)			2
unclocked/CLK				

Full report of all pins in the paths

Level	Pin	Constraints	Attr
1	ff1/Q	(FD1)	
2	inv/A	(IV)	
3	inv/Z	(IV)	
4	ff2/D	(FD1)	End

The [analyze_paths](#) report shows that the ff1/Q pin is not a valid startpoint for the path. The **-from** option for the constraint should be modified so that the exception is not ignored.

Fix Suggestion

Revise the exceptions to specify only valid startpoints.

For the example, set the false path to specify the clock pin of the register as the startpoint:

```
set_false_path -from [get_pins ff1/CP] -to [get_pins ff1/D]
```

See Also

[set_false_path](#)

[set_multicycle_path](#)

[analyze_paths](#)

Rule: EXC_0004

All **-to** objects for an **exception** are invalid timing path endpoints.

Severity: Warning

Description

All of the **-to** objects are not valid path end points. This exception will be ignored.

Valid endpoints are

- Output or bidirectional ports
- Pins constrained by a setup/hold/recovery/removal timing check including:
- Register data pins
- Register preset and clear pins
- Clock gating enable pins
- Internal pins that have a [set_output_delay](#) constraint set

- Internal pins that have a [set_min_delay/set_max_delay](#) constraint set with the **-to**, **-rise_to**, or **-fall_to** option

The "Violation Details" section in the GUI shows the **-to** objects that are invalid. The reasons are as follows:

- Non-endpoint – The pin is internal and is not a timing path endpoint
- Startpoint – The pin or port is used as a **-to** object but is a timing path startpoint
- Case disabled – The pin or port has been disabled by [set_case_analysis](#)
- Check disabled – The timing check at the pin or port has been disabled
- Sequential disabled – A sequential arc at the timing path startpoint is disabled
- Node disabled – A valid endpoint is disabled
- Hierarchical pin – The pin is on a hierarchical boundary
- Not an end cell – The cell is used as a **-to** object and does not have any valid timing path endpoints

Only false path ([set_false_path](#)) and multi-cycle path ([set_multicycle_path](#)) exceptions are included in this check.

Risk Associated with Violation

The exception will be ignored, resulting in timing violations due to false paths or stricter timing requirements. Ignored exceptions may lead to larger optimization and/or timing closure effort.

One Possible Scenario

Consider a scenario where a path is disabled between two registers with the constraint:

```
create_clock -period 10 [get_ports CLK]
set_false_path -from [get_pins ff1/CP] -to [get_pins inv/Z]
```

Even though the **-from** object is correct, this exception will be ignored because the **-to** object is invalid, which makes the path specification ambiguous. As a result, the **EXC_0004** rule violation will be issued.

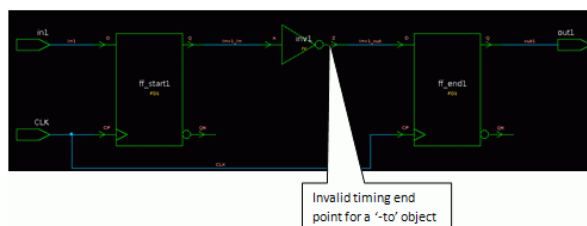


Figure 1: Path Disabled Between Registers Using Constraint; Path Specification Ambiguous

Another Possible Scenario

Consider a scenario where the correct startpoint and endpoint are specified for disabling a path between two registers with the following constraints:

```
create_clock -period 10 [get_ports CLK]
set_case_analysis 0 [get_pins ff_end2/CP]
set_false_path -from [get_pins ff_start1/CP] -to [get_pins ff_end2/D]
```

Even though both the **-from** and the **-to** objects are correct, this exception will be ignored. The reason is that the timing checks on the **-to** object are disabled by a [set_case_analysis](#) constraint on the **ff_end2** register's clock pin **CP**. The exception cannot be applied, and an **EXC_0004** rule violation will be issued.

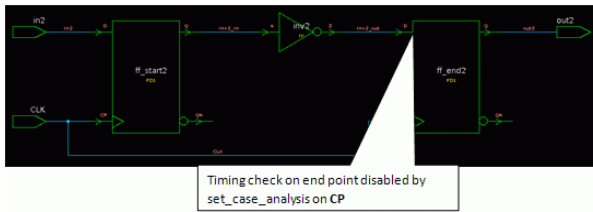


Figure 2: Path Disabled Between Registers Using Constraint; Path Specification Disabled

What Next

The “Violation Details” section in the GUI shows the reason the **-to** objects are invalid.

For the first example scenario above, the “Violation Details” section provides the following information:

[default:EXC_0004 #1] All -to objects for a 'false_path exceptio

Exception		
	Values	Reason
Type	false_path	
Values	Hold	Setup
	Rise	yes yes
	Fall	yes yes
From	ff1/CP	
To	inv/Z	non-endpoint
Source File	/u/zinmgr/test_data/rule_ref_desi gns/EXC_0004/src/scripts/constrai nts.tcl, line 3	

Figure 3: First Scenario Violation Details

The **Reason, non-endpoint**, is included in the table, which shows that the specified **-to** point is not a valid endpoint, and the [set_false_path](#) constraint was ignored.

For the second scenario, the “Violation Details” section provides the following information:

[default:EXC_0004 #2] All -to objects for a 'false_path' exception at /

Violation Details			
Exception			
	Values		Reason
Type	false_path		
Values		Hold Setup	
	Rise	yes yes	
	Fall	yes yes	
From	ff_start2/CP		
To	ff_end2/D		check disabled
Source File	/u/zinmgr/test_data/rule_ref_designs/ EXC_0004/src/scripts/constraints.tcl, line 7		

Figure 4: Second Scenario Violation Details

The **Reason, check disabled**, is highlighted in the table, which means that the timing check at the specified **-to** point is disabled, and the [set_false_path](#) constraint was ignored.

To understand why the timing check is disabled at the **-to** point, use [report_disable_timing-all_arcs](#) on the **-to** cell, which is the endpoint specified in the constraint. This command may be useful for the case disabled, check disabled, sequential disabled, and node disabled **Reason** provided in the “Violation Details” section. For the example scenario, the command and its report are shown below:

```
ptc_shell> report_disable_timing -all_arcs [get_cells ff_end2]
```

```
*****
Report : disable_timing
Design : ...
*****
```

```
Attributes
  c - case-analysis
  C - Conditional arc
  d - default conditional arc
  f - false net-arc
  l - loop breaking
  L - db inherited loopbreaking
  m - mode
  p - propagated constant
  u - user-defined
  U - User-defined library arcs
```

Cell or Port	From	To	Sense	Flag	Reason
ff_end2	CP	QN	rising_edge	c	CP = 0
ff_end2	CP	Q	rising_edge	c	CP = 0
ff_end2	CP	CP	clock_pulse_width_high	c	CP
ff_end2	CP	CP	clock_pulse_width_low	c	CP
ff_end2	CP	D	setup_clk_rise	c	CP = 0
ff_end2	CP	D	hold_clk_rise	c	CP = 0

The report shows that all the timing checks on the **ff_end2** register were disabled due to a case analysis condition (flag column) that sets the **CP** pin on **ff_end2** to 0 (reason column).

Fix Suggestion

It is best to revise the exceptions to specify only valid endpoints.

For the first example scenario, the correct way to specify the false path is to specify the data pin of the register as the endpoint:

```
set_false_path -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

For the second example scenario, the user should verify the [set_case_analysis](#) constraint. If it is correct, the false path specification definition will need to be revisited. If it is incorrect, the case analysis command can be removed from the constraints.

See Also

[set_false_path](#)

[set_multicycle_path](#)

[report_disable_timing](#)

[set_case_analysis](#)

Rule: EXC_0005

An **exception** will create startpoints or endpoints and break other timing paths through those pins.

Severity: Error

Description

A maximum or minimum delay exception forces the path startpoints or endpoints, because the **-from** or **-to** values are not typical timing path. This will break other paths from passing

through these points. The 'Violation Details' located in the GUI shows the **-from** or **-to** pins that have been forced to be startpoints or endpoints.

Risk Associated with Violation

This exception will disable all other paths that would have passed through these points.

One Possible Scenario

Consider a scenario where a maximum delay exception is set on a timing path as follows:

```
set_max_delay 2 -from [get_pins ff_start/Q] -to [get_pins ff_end1/D]
```

The constraint specifies the 'Q' pin of a register which is not a valid timing startpoint. Any other paths that pass through this pin will be disabled. The path from ff_start to ff_end2 will be disabled because a new timing startpoint is created at ff_start/Q. In addition, the path from ff_start/CP to ff_end1 is blocked by the startpoint ff_start/Q. This exception will cause a EXC_0005 violation to be issued.

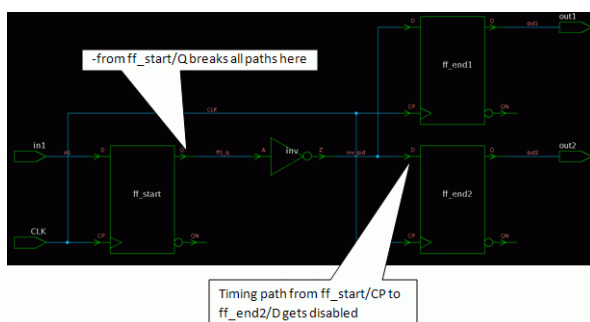


Figure 1: Invalid Timing Startpoint

What Next

Use the [analyze_paths](#) command to find out why the constraints are invalid startpoints or endpoints. This command uses **-from**, **-through**, or **-to** options just like the [set_max_delay](#) and [set_min_delay](#) commands. The command example is shown below:

```
ptc_shell> analyze_paths -unconstrained -traverse_disabled -path_type  
full -from [get_pin ff_start/CP] -to [get_pin ff_end1/D]
```

Summary of Paths:

		Dominant	Overridden	Path
Clocks				
Startpoint	Endpoint	Constraint	Constraints	Count
Launch/Capture				
ff_start/Q (FD1)	ff_end1/D (FD1)	E0		2
unclocked/CLK				

```
ff_start/CP (FD1)  ff_end1/D (FD1)      E0,MD#0      2 CLK/CLK
```

Exception Information:

ID	Type	Setup	Hold	Source
----	------	-------	------	--------

E0	max_min_delay	8.0000	--	/u/gcamgr/test_data/scripts/constraints.tcl, line 3
----	---------------	--------	----	---

Disabled Object Information:

```
MD#0 = set_min/max_delay
      at pin: ff_start/Q
      /u/gcamgr/test_data/scripts/constraints.tcl, line 3
```

Full report of all pins in the paths

Level Pin		Attr	
Constraints			
1	ff_start/CP (FD1)	Start	
2	ff_start/Q (FD1)	Start	E0
3	inv/A (IV)		
4	inv/Z (IV)		
5	ff_end1/D (FD1)	End	E0

The [analyze_paths](#) report shows that the path from the register, ff_start/CP to register, ff_end1/D has been disabled due to the startpoint created at the register, ff_start/Q by the [set_max_delay](#) command.

Fix Suggestion

Revise the exceptions to specify only valid startpoints and endpoints.

For the above example, the command specifies the maximum delay for the path by using the clock pin of the register, ff_start/CP, as the startpoint:

```
set_max_delay 2 -from [get_pins ff_start/CP] -to [get_pins ff_end1/D]
```

See Also

[set_max_delay](#)

[set_min_delay](#)

[analyze_paths](#)

Rule: EXC_0006

An **exception** does not specify any valid paths.

Severity: Warning

Status: Disabled by default

Description

The exception described in the 'Violation Details' section located in the GUI does not match any constrained timing paths in the design.

Risk Associated with Violation

The exception will have no effect on tool performance. If the exception is specified incorrectly, unintended paths may be analyzed.

One Possible Scenario

Consider a scenario where a user sets a false path exception as follows:

```
set_false_path -from [get_pins ff_start1/CP] -to [get_pins ff_end2/D]
```

A path between these two points does not exist in the design. Such a situation can arise either from a typographical error or because the path is disabled. For these cases, an **EXC_0006** violation is issued.

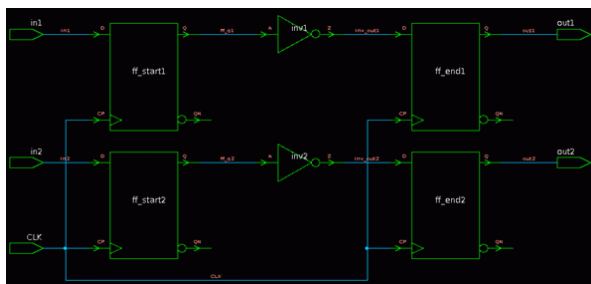


Figure 1: Missing Path

What Next

Use [analyze_paths](#) **-traverse_disabled** with the **-from**, **-through**, or **to** option to determine the reason for unconstrained timing paths. The command example is shown below:

```
ptc_shell> analyze_paths -unconstrained -traverse_disabled -path_type  
full -through [get_pin ff_start1/CP]
```

```
*****
```

```
Report : analyze_paths
```

```
*****
```

```
Summary of Paths:
```

	Dominant	Overridden	Path
Clocks			

Startpoint Launch/Capture	Endpoint	Constraint	Constraints	Count
ff_start1/CP (FD1) unclocked	ff_start1/QN (FD1)			2 CLK/
ff_start1/CP (FD1) CLK	ff_end1/D (FD1)			2 CLK/
CLK (in) CLK	ff_start1/CP (FD1)			2 CLK/

Full report of all pins in the paths

Level	Pin	Attr
Constraints		
1	CLK (in)	Start
1	ff_start1/CP (FD1)	Start
2	ff_start1/Q (FD1)	
3	inv1/A (IV)	
4	inv1/Z (IV)	
5	ff_end1/D (FD1)	End
2	ff_start1/QN (FD1)	
2	ff_start1/CP (FD1)	End

The [analyze_paths](#) report shows that no path exists between the register ff_start1/CP and the register, ff_end2/D.

Fix Suggestion

Revise the exceptions to specify only valid paths.

For the example above, a path exists between the register, ff_start1/CP and the register, ff_end1/D. If this path is a false path, the constraint can be updated as follows; otherwise the constraint can be removed.

For the example scenario, a path exists between ff_start1/CP and ff_end1/D. If this path is a false path, the constraint can be updated as follows:

```
set_false_path -from [get_pins ff_start1/CP] -to [get_pins ff_end1/D]
```

If this is not a false path, the constraint can be removed entirely.

See Also

[set_false_path](#)

[set_multicycle_path](#)

[set_max_delay](#)

[set_min_delay](#)

[analyze_paths](#)

Rule: EXC_0007

An **exception** has incomplete values.

Severity: Warning

Description

Both rise and fall values should be specified for maximum delay and minimum delay exceptions.

This check is performed only with the **-rise** and **-fall** options. There is no check if the specification is incomplete or if the **-rise_from**, **-fall_from**, **-rise_to**, or **-fall_to** options are used in the maximum or minimum delay exception.

Risk Associated with Violation

The path(s) with the missing transition are checked if they meet the library specified setup or hold time. This may be an undesired analysis for unspecified transitions.

One Possible Scenario

Consider a scenario where a [set_max_delay](#) exception is specified with the **-rise** flag, but no constraint is included for the **-fall** flag:

```
set_max_delay -rise 8 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

The maximum delay exception forces a startpoint and endpoint for the rising path delays. Since no maximum delay constraint is set for the falling path; the path will be checked against the setup constraint for the capturing clock of register, ff2. In cases where a maximum or minimum delay constraint is specified for one transition and not the other, an **EXC_0007** rule violation will be issued..

What Next

Use [analyze_paths](#) command with the **-from**, **-through**, or **-to** option to check timing paths that are in violation. The example command is shown below:

```
ptc_shell> analyze_paths -traverse_disabled -unconstrained -path_type  
full -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

```
*****
```

```
Report : analyze_paths
```

```
*****
```

```
Summary of Paths:
```

	Dominant	Overridden	Path
Clocks			

Startpoint Launch/Capture	Endpoint	Constraint	Constraints	Count
ff1/CP (FD1) CLK	ff2/D (FD1)	E0		2 CLK/

Exception Information:

ID	Type	Setup	Hold	Source
E0	max_min_delay	rise=8.0000,fall=---	--	/u/gcamgr/test_data/scripts/constraints.tcl, line 3

Full report of all pins in the paths

Level	Pin	Attr	
Constraints			
1	ff1/CP (FD1)	Start	E0
2	ff1/Q (FD1)		
3	inv/A (IV)		
4	inv/Z (IV)		
5	ff2/D (FD1)	End	E0

The [analyze_paths](#) report shows there are two paths between the register, ff1/CP, and the register, ff2/D. The [set_max_delay](#) command (from the "One Possible Scenario" section above) shows the exception specification of the path with a maximum delay of 8 for the rising path, and 0 delay for falling path.

Fix Suggestion

Provide the minimum and maximum delays, and rise and fall values for an exception.

For the example above, there are two ways the violation can be removed:

- If different fall and rise constraints need to be set, then add another [set_max_delay](#) constraint with the **-fall** flag:

```
set_max_delay -rise 8 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

```
set_max_delay -fall 9 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

- If the fall constraint is the same as the rise constraint, then the **-rise** flag can be removed from the existing constraint:

```
set_max_delay 8 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

Or a **-fall** constraint can be added with the same delay as the rise constraint:

```
set_max_delay -rise 8 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

```
set_max_delay -fall 8 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```


See Also

[set_min_delay](#)

[set_max_delay](#)

[analyze_paths](#)

Rule: EXC_0008

An **exception** is forcing startpoints or endpoints by breaking the clock network.

Severity: Error

Description

A maximum or minimum delay exception is forcing startpoints or endpoints. The **-from** or **-to** options specified are not timing startpoints or endpoints causing a break at the clock network. The 'Violation Details' section located in the GUI lists the pins that have been affected.

Risk Associated with Violation

This exception will break the clock network. An incomplete clock network may result in inaccurate timing analysis on the design.

One Possible Scenario

Consider a scenario where a maximum delay constraint is set on an invalid startpoint:

```
set_max_delay 8 -from [get_pins inv/Z] -to [get_pins ff1/CP]
```

The inverter, inv/Z pin is not a valid timing startpoint; the [set_max_delay](#) command will create a startpoint at the specified pin and prevent the clock from propagating any further. This exception will cause an **EXC_0008** violation to be issued.

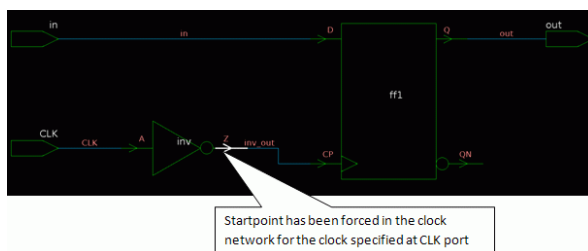


Figure 1: Invalid Timing Startpoint Preventing Clock Propagation

What Next

Use the [analyze_clock_networks](#) command to find out which clocks networks are broken as a result of the exceptions. The command example is shown below:

```
ptc_shell> analyze_clock_networks -through [get_pins inv/Z] -style full
-traverse_disabled
```

Clock Sense Abbreviations:

P - positive

Potential senses detected with -traverse_disabled:

[N] - potential negative

Clock Network End Type Abbreviations:

REG - register

Full report for clock: CLK

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	CLK (in)
1		P		inv/A (IV)
2		[N]	MD#0	inv/Z (IV)
3		[N]	REG, MD#1	ff1/CP (FD1)

Clock Blocking Constraints:

MD#0 = set_min/max_delay

at pin: inv/Z

/u/gcamgr/test_data/scripts/constraints.tcl, line 3

MD#1 = set_min/max_delay

at pin: ff1/CP

The [analyze_clock_networks](#) report shows the path from the CLK port to the register, ff1/CP, has been disabled due to the startpoint created at the inverter, inv/Z.

Fix Suggestion

Revise the exceptions to specify only valid startpoints and endpoints.

For the example above, specify the maximum delay for the path using the clock source as the startpoint. Also, check your clock network to see if the maximum or minimum delay constraint can be removed. The example command is shown below:

```
set_max_delay 8 -from [get_ports CLK] -to [get_pins ff1/CP]
```

See Also

[set_max_delay](#)

[set_min_delay](#)

[analyze_clock_networks](#)

Rule: EXC_0009

Inconsistent **exception**; *min_delay* must be less than *max_delay*.

Severity: Warning

Description

Values for [set_max_delay](#) and [set_min_delay](#) must be consistent. The min rise delay must be less than the max rise delay. Similarly, the min fall delay must be less than the max fall delay.

Setup analysis on a design uses the worst-case operating condition, which produces the largest delay, to ensure that the setup time for the slowest path to each end point in the design is met. Similarly, hold analysis uses the best-case operating condition, which produces smallest delay, to ensure that the hold time for the fastest path to each end point in the design is met. It is expected that the max delay will always be larger than the min delay.

Risk Associated with Violation

If the min delay value is not smaller than the max delay, the min (hold) and max (setup) analysis will be performed on possibly incorrect values, resulting in optimistic timing.

One Possible Scenario

Consider a scenario where a max delay and min delay constraint are set on the same startpoint and endpoint, and the min delay is larger than the max delay:

```
set_max_delay 2 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

```
set_min_delay 6 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

The min delay is set to 6 library timing units, and the max delay is set to 2 library timing units. Because the min delay is larger than the max delay, this will cause an EXC_0009 violation to be issued.

What Next

The 'Violation Details' section will give you the locations of the related [set_max_delay](#) and [set_min_delay](#) constraints with the problem.

Fix Suggestion

It is best to revise the exceptions with appropriate values for correct timing analysis. For the example scenario, the violation can be resolved by updating the constraints so that the max delay value is larger than the min delay:

```
set_max_delay 3 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

```
set_min_delay 2 -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

Make sure the values used make sense for your design.

See Also

[set_max_delay](#)

[set_min_delay](#)

Rule: EXC_0010

An **exception** has no corresponding **min_delay** for a **max_delay** exception.

Severity: Info

Description

A path constrained with a [set_max_delay](#) is also expected to have a corresponding [set_min_delay](#) exception.

Risk Associated with Violation

If the path specified in the from/through/to region of the [set_max_delay](#) is constrained through other means, then there is no risk associated with this violation. However, if the path is not constrained with a library check or a [set_min_delay](#) then the minimum timing on this path will not be checked.

The purpose of this violation is to raise the awareness of the user that they could have potentially missed specifying a constraint.

One Possible Scenario

Consider a scenario where a user sets a max delay constrain on a combinational path of the design.

```
set_max_delay 8 -from [get_ports in] -to [get_ports out]
```

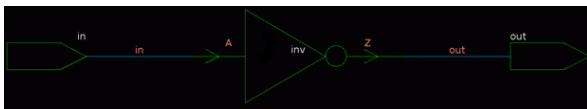


Figure 1: Unconstrained Minimum Delay

Because this design has a purely combinational path and you did not specify input and output delays, its minimum delay is unconstrained. As a consequence, that minimum delay is never checked during timing analysis and could results in unexpected timing analysis.

Constraint consistency flags an **EXC_0010** violation to make sure that the missing [set_min_delay](#) does not cause some paths to go unchecked by analysis.

What Next

Use the [analyze_paths](#) command on the from/through/to region to see if the path is unconstrained. If the report shows that the launch/capture clocks are ‘unlocked’ then the path is not fully constrained and the **EXC_0010** should be taken into consideration.

```
*****
Report : analyze_paths
Design : EXC_0010
Version: ...
Date   : ...
*****
Path Endpoints:
                                Dominant   Overridden   Path
Clocks                        Constraint  Constraints   Count
Endpoint                      Launch/Capture
-----
out (out)                      E0                        2      unlocked/
unlocked

Exception Information:
ID      Type                      Setup   Hold    Source
-----
E0      max_min_delay      8.0000  --      ./EXC_0010/src/scripts/
constraints.tcl, line 2
```

The “Violation Details” section reports what constraints have actually been applied on that path and which ones are missing. The following is the example report corresponding to the previous scenario:

Type	max_min_delay		
Values		Min	Max
	Rise	--	8.0000
	Fall	--	8.0000
From	in		
To	out		
Source File	./src/scripts/constraints.tcl, line 2		

Fix Suggestion

Specifying a [set_min_delay](#) constraint for the from/through/to corresponding to the [set_max_delay](#) constraint would eliminate this rule violation.

See Also

[set_max_delay](#)

[set_min_delay](#)

[analyze_paths](#)

Rule: EXC_0011

An **exception** has no corresponding **max_delay** for a **min_delay** exception.

Severity: Info

Description

A path constrained by [set_min_delay](#) is also expected to have a corresponding [set_max_delay](#) exception.

Risk Associated with Violation

If the path specified in the from/through/to region of the [set_min_delay](#) is constrained through other means, then there is no risk associated with the **EXC_0011** violation. However, if the path is not constrained with a library check or a [set_max_delay](#) then the maximum timing on this path will not be checked.

The purpose of this violation is to raise awareness of a potentially missing constraint in the design.

One Possible Scenario

Consider a scenario where a user sets a min delay constrain on a combinational path of the design.

```
set_min_delay 2 -from [get_ports in] -to [get_ports out]
```

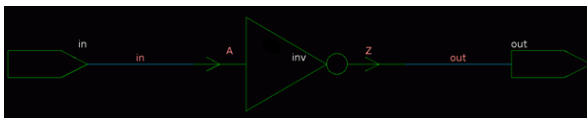


Figure 1: Unconstrained Maximum Delay

Because this design has a purely combinational path and you have not specified input and output delays, its maximum delay is unconstrained. As a consequence, the maximum delay is never checked during timing analysis. Constraint consistency flags an **EXC_0011**

violation to make sure that the missing [set_max_delay](#) does not cause some paths to go unchecked by analysis.

What Next

Use [analyze_paths](#) on the from/through/to specification to see if the path is indeed unconstrained. If the report shows that the launch/capture clocks are 'unclocked' then the path is unconstrained and the source of the **EXC_0011** needs to be investigated.

```
*****
Report : analyze_paths
Design : EXC_0011
Version: ...
Date   : ...
*****
Path Endpoints:
```

	Dominant	Overridden	Path
Clocks			
Endpoint	Constraint	Constraints	Count Launch/Capture
out (out)	E0		2
unclocked/unclocked			

Exception Information:

ID	Type		Setup	Hold	Source
E0	max_min_delay	--	8.0000		./EXC_0011/src/scripts/constraints.tcl, line2

The "Violation Details" section also reports what constraints are currently applied on that path and what is missing. The following is an example of what you would get for the previous scenario.

Type	max_min_delay		
Values		Min	Max
	Rise	2.0000	--
	Fall	2.0000	--
From	<u>in</u>		
To	<u>out</u>		
Source File	./src/scripts/constraints.tcl, line 2		

Fix Suggestion

Specifying a [set_max_delay](#) constraint for the from/through/to corresponding to the [set_min_delay](#) constraint would eliminate this rule violation.

See Also

[set_max_delay](#)

[set_min_delay](#)

[analyze_paths](#)

Rule: EXC_0012

An **exception** has incomplete values.

Severity: Info

Status: Disabled by default

Description

An incomplete false path exception was specified. In most cases all combinations of rise/fall and setup/hold should be false if any of them are false. The violation detail section highlights which value is missing.

Risk Associated with Violation

If the current exception truly captures the intent of the designer and that not all the rise/fall and setup/hold are false for the specified path, this violation can be safely ignored.

However, it is not a common practice that the user would want the path to be false only for a subset of those.

Fix Suggestion

It is best to re-express the false path exception for all the rise/fall and setup/hold combinations.

The current set of exceptions can be reported with the [report_exceptions](#) command as seen below:

```
ptc_shell> report_exceptions -from [get_pins {ff1/CP}] -to [get_pins
{ff2/D}]

*****

Report : exceptions
Design : EXC_0012
Scenario: default
Version: ...
Date : ...

*****

# ./src/scripts/constraints.tcl, line 3
set_false_path -setup -rise -from [get_pins {ff1/CP}] -to [get_pins {ff2/
D}]
```

See Also

[set_false_path](#)

[report_exceptions](#)

Rule: EXC_0013

An **exception** has incomplete values.

Severity: Info

Status: Disabled by default

Description

An incomplete multicycle path exception was specified. In most cases all combinations of rise/fall and setup/hold should be specified if any of them are multicycle. The violation detail section will highlight which combination value is missing.

Risk Associated with Violation

If the current exception truly captures the intent of the designer and that not all combinations of the rise/fall and setup/hold have a multicycle condition, this violation can be safely ignored. However, it is not a common practice that the user would want the path to have a multicycle value only for a subset of those.

Fix Suggestion

It is best to re-express the multicycle path exception for all the rise/fall and setup/hold combinations.

The currently set exceptions can be reported using the [report_exceptions](#) command as seen below:

```
ptc_shell> report_exceptions -from [get_pins {ff1/CP}] -to [get_pins
{ff2/D}]
*****

Report : exceptions
Design : EXC_0013
Scenario: default
Version: ...
Date : ...
*****
#####
## Exceptions that are dominant for at least one path:
# ./src/scripts/constraints.tcl, line 3
set_multicycle_path 2 -setup -rise -from [get_pins {ff1/CP}] -to
[get_pins {ff2/D}]
```

See Also

[set_multicycle_path](#)

[report_exceptions](#)

Rule: EXC_0014

An **exception** is fully overridden by other exceptions.

Severity: Warning

Status: Disabled by default

Description

The exception is fully overridden by at least one other exception or a clock group. It will be ignored for the purpose of constraint analysis since its behavior is fully covered by the overriding exception.

Risk Associated with Violation

This exception will have no impact within constraint consistency. However, the specified constraints can negatively impact the runtime and memory performance of other timing analysis tools.

One Possible Scenario

Consider a scenario where both a false path and a multicycle path exception are defined as follows:

```
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
set_multicycle_path -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

The false path exception fully overrides the multicycle exception; the tool will issue the **EXC_0014** violation to report that the multicycle path exception is ignored.

This rule is disabled by default but is enabled using the [enable_rule](#) command as shown below:

```
ptc_shell> enable_rule [get_rule EXC_0014]
```

The tool uses a set of precedence rules to determine if an exception overrides another one. In case of a conflict, the timing exceptions use the following order of precedence:

- [set_false_path](#)
- [set_max_delay/set_min_delay](#)
- [set_multicycle_path](#)

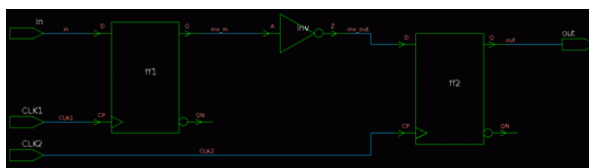


Figure 1: Multicycle Path Exception is Ignored

What Next

You can use the [analyze_paths](#) command with the relevant **-from**, **-through**, or **-to** option to determine what other exceptions or clock groups have precedence.

For the example above, you can use the [analyze_paths](#) command on the multicycle path exception to understand why exception path is ignored, as shown below:

```
ptc_shell> analyze_paths -from [get_pins ff1/CP] -to [get_pins ff2/D]
```

```
*****
```

```
Report : analyze_paths
```

```
-path_type end
```

```
-max_endpoints = 1000
```

```
Design : EXC_0014
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Path Endpoints:
```

```
Dominant Overridden Count Clocks
```

```
Endpoint Constraint Constraints (r,f,R,F) Launch/Capture
```

```
-----  
ff2/D (FD1) F0 M1 (1,1,1,1) CLK1/CLK2
```

```
Exception Information:
```

```
ID Type Setup Hold Source
```

```
-----  
F0 false_path FALSE FALSE ./src/scripts/constraints.tcl, line 4
```

```
M1 multicycle_path 2 0 ./src/scripts/constraints.tcl, line 5
```

Fix Suggestion

If the overridden exception is redundant, remove the exception from the constraint specification to avoid unnecessary runtime and memory impact on the analysis tools.

To remove redundant exceptions automatically, use the following command:

```
ptc_shell> redirect -file dominant.tcl {report_exception -dominant}
```

The file now contains only the exceptions that are dominant for the design. An exception is considered dominant if altering it in any way changes the timing behavior of the design.

In the example described above, the resulting dominant exceptions are shown below:

```
*****

Report : exceptions
Design : EXC_0014
Scenario: default
Version: ...
Date : ...

*****

#####
Exceptions that are dominant for at least one path:

# ./src/scripts/constraints.tcl, line 4
set_false_path -from [get_clocks {CLK1}] -to [get_clocks {CLK2}]
```

See Also

[set_false_path](#)

[set_multicycle_path](#)

[set_max_delay](#)

[set_min_delay](#)

[report_exceptions](#)

[analyze_paths](#)

[enable_rule](#)

Rule: EXC_0015

An **exception** is partially overridden by other exceptions.

Severity: Information

Status: Disabled by default

Description

The exception given in the "More Information" section is partially overridden by other exceptions or clock groups. This means that the exception is dominant on some paths

selected by the –from/-through/-to options of the exception but on some other paths, the exception is overridden by a higher priority exception.

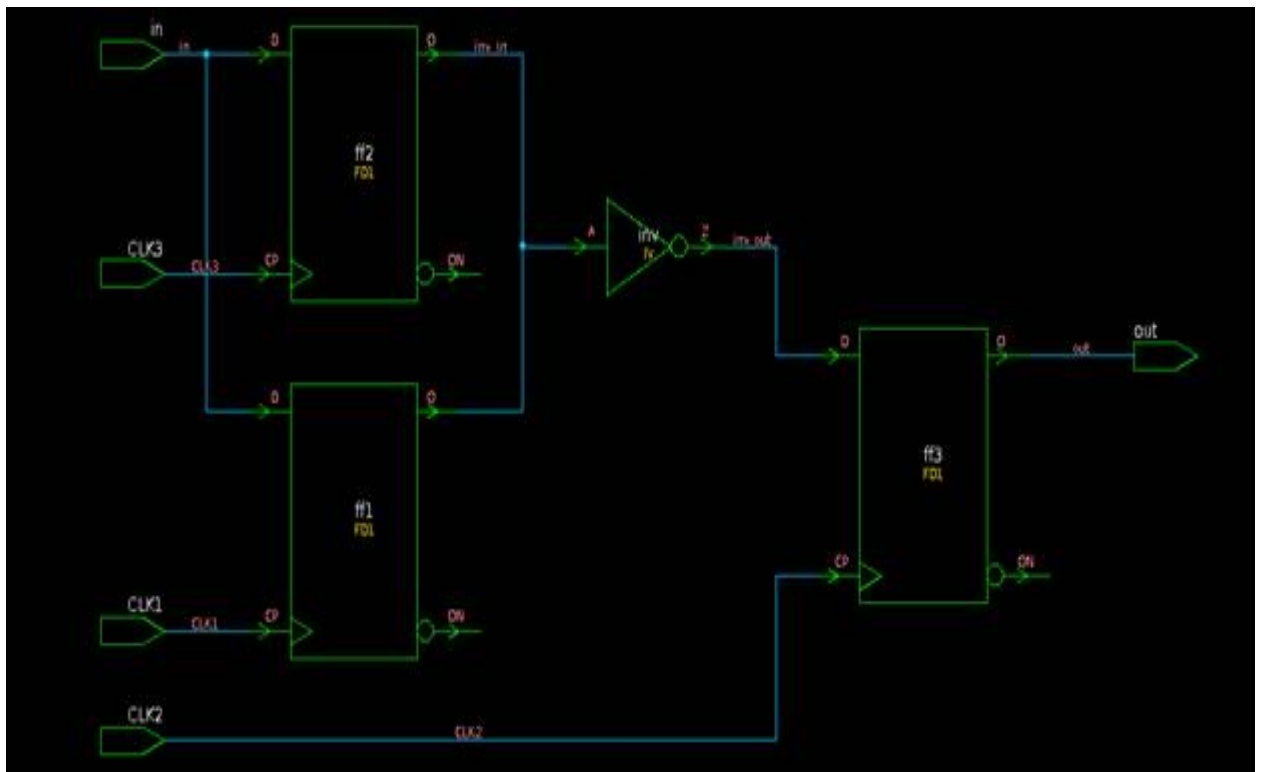
Use the [analyze_paths](#) command on the from/through/to specification to determine what other exceptions/clock groups have precedence over this one for certain endpoints.

Risk Associated with Violation

There is no impact on the timing analysis results. However, the runtime and memory performance of timing analysis tools or optimization tools might be negatively impacted unless the exceptions are rewritten to remove the overlapping segments.

One Possible Scenario

Consider a scenario where you set a **set_false_path** and a **set_multicycle_path** exception as follows:



```
create_clock -period 10 CLK1
create_clock -period 10 CLK2
create_clock -period 10 CLK3
set_false_path -from [get_clocks CLK1] -to [get_clocks CLK2]
```

```
set_multicycle_path 2 -from [get_pins {ff1/CP ff2/CP}] -to {get_pins ff3/D}
```

In this case, the false path exception partially overrides the multicycle exception because the false path takes precedence for the path between ff1/CP to ff3/D. However, the multicycle exception is dominant on the path between ff2/CP to ff3/D. The tool gives an EXC_0015 information message for such a scenario. This rule is kept disabled by default since it incurs slightly higher runtime, but you can enable it by using the **enable_rule** command. The exception precedence rules are explained in detail in the PrimeTime Suite User Guide. Generally, in case of a conflict, the timing exceptions have the following order of priority from highest to lowest: **set_false_path** > **set_max_delay/set_min_delay** > **set_multicycle_path**.

What Next

Use the [analyze_paths](#) command on the from/through/to specification to determine what other exceptions/clock groups have precedence over this one.

For this example, the [analyze_paths](#) command is shown as follows:

```
analyze_paths -from [get_pins {ff1/CP ff2/CP}] -to [get_pins ff3/D]
```

```
*****
Report : analyze_paths
Design : EXC_0015
...
*****
```

Path Endpoints:

```
Dominant Overridden Count Clocks
Endpoint Constraint Constraints (r,f,R,F) Launch/Capture
```

```
ff3/D (FD1) M1 (1,1,1,1) CLK3/CLK2
ff3/D (FD1) F0 M1 (1,1,1,1) CLK1/CLK2
```

Exception Information:

```
ID Type Setup Hold Source
```

```
F0 false_path FALSE FALSE ./EXC_0015/src/scripts/constraints.tcl, line 4
```

```
M1 multicycle_path 2 0 ./EXC_0015/src/scripts/constraints.tcl, line 5
```

The `analyze_paths` output shows that the multicycle path *M1* is overridden by false path *F0* for the endpoint *ff3/D* for one path that is launched by *CLK1* and captured by *CLK2*.

Fix Suggestion

The partially over-ridden exception can be rewritten to help reduce the runtime of the analysis tools. In the preceding example, the multicycle exception can be rewritten as:

```
set_multicycle_path 2 -from [get_pins ff2/CP] -to [get_pins ff3/D]
```

See Also

[set_false_path](#)

[set_multicycle_path](#)

[set_max_delay](#)

[set_min_delay](#)

[report_exceptions](#)

[analyze_paths](#)

[enable_rule](#)

Rule: EXD_0001

Input or inout port **port** has no input delay specified.

Severity: Warning

Description

The reported input port is not a clock source. The port does not have an input delay value, and it also has paths from it which are not disabled or false paths. In order to enable proper constraining of paths from the port, a realistic input delay needs to be provided for all data input ports.

There are properties for this rule that affect whether a violation is issued.

The **suppress_violations_for_false_paths** property controls whether a violation is issued or suppressed based on whether all of the paths from the port are false. The default value for this property is **true**, which means that a violation will not be issued if all paths from the port are false.

The **suppress_violations_for_min_max_delays** property controls whether a violation is issued or suppressed based on whether all paths from the port have [set_min_delay](#) and [set_max_delay](#) constraints. The default value for this property is **false**, which means a violation will be issued if the port is not a clock source and has no input delay specified, irrespective of whether the [set_min_delay](#) and [set_max_delay](#) constraints are set on paths originating from the port.

Risk Associated with Violation

Realistic input delay value relative to a clock is required in order to perform accurate delay calculation in the design. Without proper input delays, the delay calculation might lead to incorrect setup or hold check results at the path endpoint.

One Possible Scenario

Consider a scenario where a clock is created on port CLK, and a related data port A with no input delays

```
create_clock -period 10 CLK
```

The lack of input delay on port “A” will impact the accuracy of any path originating from port “A”.

What Next

The violation message will give you the input port which has no input delay specified. You can get information related to that port using the [report_port](#) command as seen below.

```
ptc_shell> report_port -input_delay [get_ports A]
```

```
*****
Report : port
        -input_delay
Design : EXD_0001
Scenario: default
Version: ...
Date   : ...
*****
```

Input Port	Input Delay				Related	
	Min		Max		Related	Related
	Rise	Fall	Rise	Fall	Clock	Pin

A	--	--	--	--	--	--

Fix Suggestion

To resolve this issue, consider specifying input delay for each non-clock source input port. This input delay should also be relative to a clock in the design.

For example, in the scenario described above, you might set use:

```
set_input_delay -clock CLK 10 A
```

Rule Property: suppress_violations_for_false_paths

The default value for this rule property is **true**, which means a violation is issued only if the port is not a clock source, is missing a [set_input_delay](#) constraint, and has paths that are non-disabled and non-false. When this rule property is set to **false**, a violation is issued if the port fails the check and has paths from it that are false or non-disabled. The number of violations for this rule can increase when the rule property is set to its non-default value, since ports will be reported that have all false paths from them.

Rule Property: `suppress_violations_for_min_max_delays`

This rule property considers whether paths from the port are constrained by `set_min_delay` and `set_max_delay` commands when performing its check. The default value for this rule property is **false**, which means violations are issued if the port is not a clock source and has no input delay, irrespective of whether the paths from the port are constrained by the `set_min_delay` and `set_max_delay` commands. When this rule property is set to **true**, the violation is issued if the port fails the check and has paths that do not have `set_min_delay` and `set_max_delay` exceptions.

See Also

[set_input_delay](#)

[set_max_delay](#)

[set_min_delay](#)

[report_port](#)

Rule: EXD_0002

Input/inout port **port** has no clock-related input delay specified.

Severity: Warning

Description

The reported port has a valid input delay value but it is not relative to a specific clock in the design. Input ports that have paths which are not disabled or false paths and are not clock sources should have an input delay relative to a clock. This enables proper constraining of timing paths originating from the port.

There are properties for this rule that affect whether a violation is issued.

The **`suppress_violations_for_false_paths`** property controls whether a violation is issued or suppressed based on whether all of the paths from the port are false. The default value for this property is **true**, which means that a violation will not be issued if all paths from the port are false.

The **`suppress_violations_for_min_max_delays`** property controls whether a violation is issued or suppressed based on whether all paths from the port have `set_min_delay` and `set_max_delay` constraints. The default value for this property is **false**, which means a violation will be issued if the port is not a clock source and has no input delay specified, irrespective of whether the `set_min_delay` and `set_max_delay` constraints are set on paths originating from the port.

Risk Associated with Violation

Input delay value relative to a clock is required in order to perform accurate delay calculation in the design. When no such clock information is provided with a [set_input_delay](#) command, the delay calculation tool will assume one of the following:

- For combinational designs, the delay will be considered relative to time 0
- For sequential designs, the delay will be relative to a newly created clock. The period of this clock will be determined by considering the sequential cells in the transitive fanout of the reported port

Leaving this violation in the design might results in erroneous hold and/or setup violations in the design. It is a good practice to always specify a relative clock for any input delay.

One Possible Scenario

Consider a scenario, where a clock is created on the clock port, while port A has no input delay specified with respect to clock CLK:

```
create_clock -period 10 CLK
create_clock -period 13 CLK2
set_input_delay 10 A
```

If this input “A” fans to several registers clock pins which receive either clock “CLK” or clock “CLK2”, the lack of input delay with respect to CLK will impact setup/hold checks performed at those registers.

What Next

The violation message will give you the input port which has no input delay specified. You can use [analyze_paths](#) to determine what clock launches and captures from the relevant port. In the case of an unspecified relative clock for input delay, you should see the launch clock being reported as “unclocked”.

```
ptc_shell> analyze_path -from [get_ports A]

*****
Report : analyze_paths
Design : EXD_0002
Version: ...
Date   : ...
*****
Path Endpoints:
```

Endpoint	Dominant Constraint	Overridden Constraints	Path Count	Clocks Launch/Capture
ff1/D (FD1)			2	unclocked/CLK

You can also use the [report_port](#) command to verify whether or not a relative clock is specified for a particular port. Below is an example showing a missing relative clock (“—”instead of clock name in the report).

```
ptc_shell> report_port -input_delay [get_ports A]
```

```
*****
Report : port
        -input_delay
Design : EXD_0002
Scenario: default
Version: ...
Date   : ...
*****
```

Input	Port	Input Delay		Max	Related	
		Min			Related	
	Rise	Fall	Rise	Fall	Clock	Pin
A	10.00	10.00	10.00	10.00	--	--

Fix Suggestion

To resolve this issue, consider specifying input delay for each non-clock source input port.

For example, in the scenario described above, you might set input delay on port A with respect to clock CLK by using the following command:

```
set_input_delay -clock CLK 10 A
```

Reporting the path originating from port A will now lead to a known launch clock as seen below:

```
ptc_shell> analyze_path -from [get_ports A]
```

```
*****
Report : analyze_paths
Design : EXD_0002
Version: ...
Date   : Thu ...
*****
```

Path Endpoints:

Endpoint	Dominant	Overridden	Path	Clocks
Capture	Constraint	Constraints	Count	Launch/
ff1/D (FD1)			2	CLK/CLK

Rule Property: suppress_violations_for_false_paths

The default value for this rule property is **true**, which means a violation is issued only if the port is not a clock source, is missing a [set_input_delay](#) constraint, and has paths that are non-disabled and non-false. When this rule property is set to **false**, a violation is issued if the port fails the check and has paths from it that are false or non-disabled. The number

of violations for this rule can increase when the rule property is set to its non-default value, since ports will be reported that have all false paths from them.

Rule Property: **suppress_violations_for_min_max_delays**

This rule property considers whether paths from the port are constrained by the [set_min_delay](#) and [set_max_delay](#) commands when performing its check. The default value for this rule property is **false**, which means violations are issued if the port is not a clock source and has no clock-related input delay, irrespective of whether the paths from the port are constrained by the [set_min_delay](#) and [set_max_delay](#) commands. When this rule property is set to **true**, the violation is issued if the port fails the check and has paths that do not have [set_min_delay](#) and [set_max_delay](#) exceptions.

See Also

[set_input_delay](#)

[set_max_delay](#)

[set_min_delay](#)

[report_transitive_fanout](#)

[report_port](#)

[analyze_paths](#)

Rule: EXD_0003

Output or inout port **port** has no clock-related output delay specified.

Severity: Warning

Description

The reported output or inout port is not a clock signal and has paths which are not disabled or false. The port should have an output delay relative to a specific clock in the design. This warning violation is issued for one of the reasons described below:

- No output delay is defined for this output on input port
- An output delay is defined, but its relative clock (through the **-clock** option) is not defined

Paths to such ports are unconstrained unless the [set_max_delay](#) command is specified to the output or inout port.

There are rule properties for this rule that affect whether a violation is issued.

The **suppress_violations_for_false_paths** rule property controls whether a violation is issued or suppressed based on whether all of the paths to the port are false. The default value for this rule property is true, which means a violation will not be issued if all paths to the port are **false**.

The **suppress_violations_for_min_max_delays** rule property controls whether a violation is issued or suppressed based on whether all paths to the port have [set_min_delay](#) and [set_max_delay](#) constraints. The default value for this rule property is **false**, which means a violation will be issued if the port is not a clock signal and has no output delay or clock-related output delay specified, irrespective of whether the [set_min_delay](#) and [set_max_delay](#) constraints are set on paths converging to the port.

Risk Associated with Violation

Without proper definition of output delays, the path(s) ending at this output port cannot be properly timed. This may therefore lead to incorrect results in timing analysis.

One Possible Scenario

Consider a scenario where a clock is created on the clock port, while port “out” has no output delay specified with respect to clock CLK:

```
create_clock -period 10 CLK
```

The lack of output delay on port “out” will leave any path through port “out” unconstrained

Adding an output delay without specifying the related clock will still leave the path unconstrained. Even though the command below specifies an output delay for port out, it is not enough to constrain the path for timing analysis.

```
set_output_delay 2 [get_ports out]
```

What Next

The violation message reports which output port has no output delay specified (or an incompletely specified delay). Use the [report_port](#) command to get more detailed information related to output delays on a specified port.

If no output delay has been specified at all, you would get the report seen below:

```
ptc_shell> report_port -output_delay [get_ports out]
```

```
*****
Report : port
-output_delay
Design : EXD_0003
Scenario: default
Version: ...
Date   : ...
```

```
*****
```

Output Port	Output Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		

out	--	--	--	--	--	--

If an output delay is specified but its related clock information is missing, the report would look like this:

```
ptc_shell> report_port -output_delay [get_ports out]
```

```
*****
```

```
Report : port
        -output_delay
Design : EXD_0003
Scenario: default
Version: ...
Date   : ...
```

```
*****
```

Output Port	Output Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		
-----	-----					
out	2.00	2.00	2.00	2.00	--	--

Fix Suggestion

To resolve this issue, consider specifying output delay for each non-clock output port with respect to clocks whose clock network may reach the output port.

For example, in the scenario described above, you might set output delay on port out with respect to clock CLK by:

```
set_output_delay -clock CLK 10 [get_ports out]
```

Rule Property: suppress_violations_for_false_paths

The default value for this rule property is **true**, which means a violation is only issued if the port is not a clock signal, is missing a [set_output_delay](#) or clock-related [set_output_delay](#) constraint, and has paths that are non-disabled and non-false. When this rule property is set to **false**, a violation is issued if the port fails the check and has paths to it that are false or non-disabled. The number of violations for this rule can increase when the rule property is set to its non-default value, since ports will be reported that have all false paths to them.

Rule Property: suppress_violations_for_min_max_delays

This rule property considers whether paths to the port are constrained by [set_min_delay](#) and [set_max_delay](#) commands when performing its check. The default value for this rule property is **false**, which means violations are issued if the port is not a clock signal and has no output delay or clock-related output delay, irrespective of whether paths to the port are constrained by the [set_min_delay](#) and [set_max_delay](#) commands. When this rule property is set to **true**, a violation is issued if the port fails the check and has paths that do not have [set_min_delay](#) and [set_max_delay](#) exceptions.

See Also

[set_input_delay](#)

[set_output_delay](#)

[set_max_delay](#)

[set_min_delay](#)

[report_port](#)

Rule: EXD_0004

The *input_or_output* delay on ***object_type object_name*** has incomplete values.

Severity: Warning

Description

The input and output delay constraint specification should include all four values: **min_rise**, **min_fall**, **max_rise**, and **max_fall**. If any of these four values are missing and there are paths from or to the port related to the missing value(s) which are not disabled or false paths, the delays will be left unconstrained.

For example, if only **-rise** is specified, all falling signals will be unconstrained.

Risk Associated with Violation

If the port input or output delays do not include **min_rise**, **min_fall**, **max_rise**, and **max_fall**, the setup and hold violation produced may be incorrect.

One Possible Scenario

Consider a scenario where a clock is created on port CLK, and a partial input delay **-rise** is specified with respect to clock "CLK" on port "A" as shown below:

```
create_clock -period 10 CLK
set_input_delay -rise -clock CLK 1 A
```


The missing **-fall** specification for the input delay may lead to incorrect result in the setup and hold check performed at the registers receiving data from port “A”.

What Next

The “Violation Details” section located in the GUI reports the input port and the respective input delay specification. You can also use the [report_port](#) command as shown below to identify what data is missing from your constraint specification. In this example, the minimum and maximum fall delay is missing.

```
ptc_shell> report_port -input_delay [get_ports A]

*****
Report : port
        -input_delay
Design : EXD_0004
Scenario: default
Version: ...
Date   : ...
*****
```

Input Port	Input Delay				Related Clock	Related Pin		
	Min		Max					
	Rise	Fall	Rise	Fall				
A	1.00	--	1.00	--	CLK	--		

You can see that no input delay value is available for the **max_fall** and **min_fall** sections.

Fix Suggestion

To resolve this issue, consider specifying complete input and output delay for each port.

For the scenario described above, modify the input delay specification as shown below:

```
set_input_delay -clock CLK 1 [get_ports A]
```

This will address the missing **fall** input delay as can be seen with the [report_port](#) command below:

```
ptc_shell> report_port -input_delay [get_ports A]

*****
Report : port
        -input_delay
Design : EXD_0004
Scenario: default
Version: ...
Date   : ...
*****
```

Input Port	Input Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		
A	1.00	1.00	1.00	1.00	CLK	--

See Also

[report_port](#)

[set_input_delay](#)

[set_output_delay](#)

Rule: EXD_0007

No clock signals exist at reference pin *reference_pin* for *input_or_output* delay on *object_type object_name*.

Severity: Warning

Description

The reference pin should specify the clock pin or port that refers to the specified delay. If no clock signal can be found for the given reference pin the delay will be ignored.

Risk Associated with Violation

Realistic input or output delay values relative to a clock are required to perform accurate timing analysis in the design. Without proper input or output delays, the timing analysis can lead to unconstrained paths or incorrect setup and hold check results at the path endpoints.

One Possible Scenario

Consider the scenario shown in Figure 1 below; a clock is created on port CLK. An input delay is specified with respect to the reference port B. However, no clock signal is defined on port B.

```
create_clock -period 10 CLK
set_input_delay -reference_pin B 1 A
```

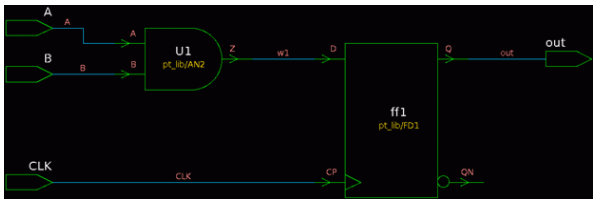


Figure 1: No Clock Signal Reaching Reference Pin or Port

Since no clock is defined for reference port B, the input delay on port A will be ignored or assigned a value relative to the “default” clock depending on the value of the [timing_input_port_default_clock](#) variable. In either case, the timing of the design will be incorrect. The violation will be flagged with the **EXD_0007** rule.

What Next

The violation message reports the pin or port on which the input or output delay is specified. The “Violation Details” section reports the reference pin and the details of the input or output delay.

The pin specified with the **-reference_pin** option should be a leaf pin or port in a clock network and should be in the direct or transitive fanout of a clock source specified with the **-clock** option. If multiple clocks reach the port or pin where you are setting the input delay, and if the **-clock** option is not used, the command considers all the clocks.

To investigate why there is no clock signal reaching the reference pin, use one of the following methods:

- If the reference pin is a design port, there is a missing clock definition. This can be verified by using the following command:

```
ptc_shell > report_clock
```

- If the reference pin is an internal design pin, there are several possible reasons why it is not receiving a clock. For example, there is a missing clock or an exception that prevents the clock from propagating to the reference pin. In the latter case, the following command will help to investigate the issue:

```
analyze_clock_networks -traverse_disabled -through ref_pin
```

Fix Suggestion

Verify that a valid clock reaches the reference pin associated with the [set_input_delay](#) or [set_output_delay](#) commands. You might also specify the input or output delay related to a known clock in the design rather than using a reference pin.

See Also

[analyze_clock_networks](#)

[report_clock](#)

[set_input_delay](#)

[set_output_delay](#)

[timing_input_port_default_clock](#)

Rule: EXD_0008

The **input_or_output** delay at pin **object_name** is forcing a **start_or_end** point that blocks paths through this pin.

Severity: Warning

Description

Use the [set_input_delay](#) or [set_output_delay](#) command to set the input or output delay on the reported pin location. This forces the reported pin to be either a timing startpoint or a timing endpoint. All paths through this pin are now blocked.

In the event no valid paths are going through this pin, this warning will not be issued.

Risk Associated with Violation

Paths which should be crossing this pin will now be segmented. The path delays are now calculated based on the [set_input_delay](#) or [set_output_delay](#) command specifications. This calculation might result in incorrect timing analysis of the path.

One Possible Scenario

Consider a scenario shown in Figure 1, where a clock is created on port CLK, while input delay is specified with respect to clock CLK on pin U1/Z:

```
create_clock -period 10 [get_ports CLK]
set_input_delay -clock CLK 1 [get_ports A]
set_input_delay -clock CLK 1 [get_ports B]
set_input_delay -clock CLK 3 [get_pins U1/Z]
```

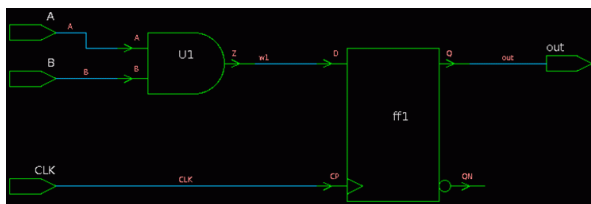


Figure 1: Forcing Startpoint

The [set_input_delay](#) command breaks any timing paths going through pin U1/Z as shown in Figure 1. The pin, U1/Z, becomes a startpoint for the rest of the design. Any path originating at pin U1/Z will now use “3” as the input delay rather than the actual propagated delay between ports “A” or “B” and pin U1/Z.

What Next

The “Violation Details” section located in the GUI reports the pin that is forced to become a path startpoint due to the input delay specification. Use the [analyze_paths](#) command

to obtain the Summary of Paths report that details what timing paths are broken by this constraint.

```
ptc_shell> analyze_paths -traverse_disabled \  
-through [get_pins U1/Z] -path_type summary
```

```
*****
```

```
Report : analyze_paths
```

```
Design : EXD_0008
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Summary of Paths:
```

```
Dominant Overridden Path Clocks
```

```
Startpoint Endpoint Constraint Constraints Count Launch/Capture
```

```
-----
```

```
U1/Z (AN2) ff1/D (FD1) 2 CLK/CLK
```

```
B (in) ff1/D (FD1) ID#0 2 CLK/CLK
```

```
A (in) ff1/D (FD1) ID#0 2 CLK/CLK
```

```
Disabled Object Information:
```

```
ID#0 = set_input_delay
```

```
at pin: U1/Z
```

Fix Suggestion

Remove the [set_input_delay](#) command specification and use the propagated delay instead.

See Also

[analyze_paths](#)

[set_input_delay](#)

[set_output_delay](#)

Rule: EXD_0009

The input delay at ***object_type object_name*** has values larger than ***max_percent*** % of the relative clock's period.

Severity: Warning

Description

The input delay contains at least one value that is large compared to the period of the related clock. If the value associated with the rule property **max_property** is overly pessimistic, the paths related to this object may be over constrained.

The percentage threshold for this rule is controlled by the value associated with the rule property **max_property**. The threshold can be adjusted using the [set_rule_property](#) command.

There are also rule properties for this rule that consider exceptions that have been set on paths from the object before a violation is issued.

The **suppress_violations_for_multicycle_paths** rule property controls whether a violation is issued or suppressed based on whether all paths from the object have [set_multicycle_path](#) exceptions specified on them. The default value for this rule property is **true**, which means a violation will not be issued if all paths from the object have multicycle path constraints.

The **suppress_violations_for_false_paths** rule property controls whether a violation is issued or suppressed based on whether all of the paths from the object are false. The default value for this rule property is **true**, which means a violation will not be issued if all paths from the object are false.

The **suppress_violations_for_min_max_delays** rule property controls whether a violation is issued or suppressed based on whether all paths from the object have [set_min_delay](#) and [set_max_delay](#) constraints. The default value for this rule property is **false**, which means a violation will be issued irrespective of whether or not [set_min_delay](#) and [set_max_delay](#) constraints are set on paths originating from the object.

Risk Associated with Violation

The input delay might be larger than necessary. This can cause over constraining of the paths starting from the reported port and result in excessive runtime or not meet slack requirements.

One Possible Scenario

Consider a scenario where a clock is created on the clock port and a large input delay is specified on port "A":

```
create_clock -period 10 CLK
set_input_delay 8.0 -clock CLK A
```

The input delay in itself is consuming 80% of the clock period. This seems quite a high input delay and could flag an erroneous constraint.

What Next

The violation message will report the pin or port input delay over the threshold specification. The “Violation Details” section located in the GUI reports the input delay details. Use the [report_port](#) command to obtain the current input delay value on a port.

Fix Suggestion

Make sure the input delay is correct for the design.

Rule Property: max_percent

By default, the **max_percent** rule property is set to 50. Any input delay over 50% of the related clock period will trigger a **EXD_0009** violation. That threshold is fully under user control and can be adjusted through the [set_rule_property](#) command as demonstrated below:

```
set_rule_property max_percent 40 EXD_0009
```

Next time [analyze_design](#) is run, **EXD_0009** violations will be triggered for any input delay value over 40% of their related clock period.

Rule Property: suppress_violations_for_multicycle_paths

This rule property considers whether paths from the object are constrained by [set_multicycle_path](#) commands when performing its check. The default value for this rule property is **true**, which means the violation is issued if the object fails the check and has paths that do not have [set_multicycle_path](#) exceptions. The number of violations for this rule can increase when the rule property is set to its non-default value, since objects will be reported that have all multicycle paths from them.

Rule Property: suppress_violations_for_false_paths

The default value for this rule property is **true**, which means a violation is only issued if the port is not a clock signal, is missing a [set_output_delay](#) or clock-related [set_output_delay](#) constraint, and has paths that are non-disabled and non-false. When this rule property is set to **false**, a violation is issued if the object fails the check and has paths from it that are false or non-disabled. The number of violations for this rule can increase when the rule property is set to its non-default value, since ports will be reported that have all false paths from them.

Rule Property: suppress_violations_for_min_max_delays

This rule property considers whether paths from the object are constrained by [set_min_delay](#) and [set_max_delay](#) commands when performing its check. The default value for this rule property is **false**, which means violations are issued irrespective of whether or not paths from the object are constrained by [set_min_delay](#) and [set_max_delay](#) commands. When this rule property is set to **true**, a violation is issued if the object fails the check and has paths that do not have [set_min_delay](#) and [set_max_delay](#) exceptions.

See Also

[set_input_delay](#)

[set_input_delay](#)

[set_max_delay](#)

[set_min_delay](#)

[set_rule_property](#)

[analyze_design](#)

[report_port](#)

Rule: EXD_0010

The output delay at ***object_type object_name*** has values larger than ***max_percent*** % of the relative clock's period.

Severity: Warning

Description

The output delay contains at least one value that is large compared to the period of the related clock. This rule warns the user to check that the values are correct. Large output delays on this object might result in an over-constrained design. The percentage threshold is user controlled and adjusted using the **max_percent** rule property.

There are also rule properties for this rule that consider exceptions which have been set on paths to the object before a violation is issued.

The **suppress_violations_for_multicycle_paths** rule property controls whether a violation is issued or suppressed based on whether all paths to the object have [set_multicycle_path](#) exceptions specified on them. The default value for this rule property is **true**, which means a violation will not be issued if all paths to the object have multicycle path constraints.

The **suppress_violations_for_false_paths** rule property controls whether a violation is issued or suppressed based on whether all of the paths to the object are **false**. The default

value for this rule property is **true**, which means a violation will not be issued if all paths to the object are **false**.

The **suppress_violations_for_min_max_delays** rule property controls whether a violation is issued or suppressed based on whether all paths to the object have [set_min_delay](#) and [set_max_delay](#) constraints. The default value for this rule property is **false**, which means a violation will be issued irrespective of whether [set_min_delay](#) and [set_max_delay](#) constraints are set on paths terminating at the object.

Risk Associated with Violation

Over constraining the design could result in long runtimes and issues with timing closure. Design size may be affected since cells with stronger drive are required to achieve correct timing transitions.

One Possible Scenario

Consider a scenario where a clock is created on the clock port “CLK” and a large output delay is specified on port “A” with respect to that clock as shown below:

```
create_clock -period 10 CLK
set_output_delay 8.0 -clock CLK A
```

The output delay is consuming 80% of the clock period which will result in difficulties meeting timing for paths starting at this point.

What Next

The violation message reports the pin or port with a large output delay. The “Violation Details” section located in the GUI reports the details of the clock period and the output delays.

The output delay value is specified with the **max_percent** rule property and is modified using the [set_rule_property](#) command shown below:

```
ptc_shell> set_rule_property max_percent 30 [get_rules EXD_0010]
```

The properties associated with this rule are listed in the “Rule Property” section below.

Fix Suggestion

Make sure the output delay has the correct value for the design. If the output delay is incorrect, it needs to be modified in the constraints file. If the output delay value is correct, you can waive the violation or change the rule property to be larger than 50 percent to avoid false **EXD_0010** violations on your design.

Rule Properties:

max_percent

By default, the **max_percent** rule property is set to **50**. This property allows you to adjust the threshold at which an **EXD_0010** violation is trigger. This property is specified as a percentage of the clock period.

`suppress_violations_for_multicycle_paths`

This rule property considers whether paths to the object are constrained by **set_multicycle_path** commands when performing its check. The default value for this rule property is **true**, which means the violation is issued if the object fails the check and has paths that do not have `set_multicycle_path` exceptions. The number of violations for this rule can increase when the rule property is set to its non-default value, since objects will be reported that have all multicycle paths to them.

`suppress_violations_for_false_paths`

The default value for this rule property is **true**, which means a violation is issued only if the object has paths to it that are non-disabled and non-false. When this rule property is set to **false**, a violation is issued if the object fails the check and has paths to it that are false or non-disabled. The number of violations for this rule can increase when the rule property is set to its non-default value, since objects will be reported that have all false paths to them.

`suppress_violations_for_min_max_delays`

This rule property considers whether paths to the object are constrained by `set_min_delay` and `set_max_delay` commands when performing its check. The default value for this rule property is **false**, which means violations are issued irrespective of whether or not paths to the object are constrained by `set_min_delay` and `set_max_delay` commands. When this rule property is set to **true**, the violation is issued if the object fails the check and has paths that do not have `set_min_delay` and `set_max_delay` exceptions

See Also

[set_output_delay](#)

[set_max_delay](#)

[set_min_delay](#)

[set_rule_property](#)

[get_rule_property](#)

Rule: EXD_0011

Some ports have no input delay or input delay without a relative clock. A default clock will be assumed for such ports because the variable **timing_input_port_default_clock** is set to **true**.

Severity: Warning

Description

Some ports in the designs have incomplete input delay specification. There is no delay or the delay is not specified to a known clock in the design. Delay values relative to clock source in the design are a requirement to getting accurate timing analysis.

Risk Associated with Violation

1. There is no input delay relative to a particular input or inout port that causing incorrect setup or hold checks at the path endpoints.
2. Any input delay needs to be relative to a clock source in the design. When no clock information is provided, the delay calculation tool will assume one of the following:
 - For combinational designs, the delay will be considered relative to time 0
 - For sequential designs, the delay will be relative to a created clock. The period of this clock will be determined by the sequential cells in the transitive fanout of the reported port

One Possible Scenario

Whenever an **EXD_0011** violation is present, you should also see at least one of the two following violations (or possibly both):

- **EXD_0001**: if your issue is a missing input delay value
- **EXD_0002**: if your issue is a missing clock information

What Next

Examine **EXD_0001** and **EXD_0002** violations for more details and follow the instructions provided for those rules.

Fix Suggestion

Examine **EXD_0001** and **EXD_0002** violations for more details and follow the instructions provided for those rules.

See Also

[EXD_0001](#)

[EXD_0002](#)

Rule: EXD_0012

The input delay at ***object_type object_name*** has zero arrival window for min and max values.

Severity: Error

Status: Disabled by default

Description

The minimum and maximum input delay values for either the rise or fall (or both) are equal. This creates a zero arrival window and could result in an optimistic signal integrity timing analysis.

NOTE: This rule is disabled by default and is part of the **primetime_si** ruleset in constraint consistency. To enable the rule, you can either:

- Selectively enable it using

```
ptc_shell> enable_rule [get_rule EXD_0012]
```

Or

- Enable the rule through the **primetime_si** ruleset

```
ptc_shell> enable_rule [get_ruleset primetime_si]
```

Risk Associated with Violation

A zero arrival window is not representative of the real conditions in the design. During signal integrity calculations, this can lead to over-optimistic timing results. The input delay settings should include the **-min** and the **-max** options to specify a realistic window, so the aggressor or victim nets can be properly computed.

One Possible Scenario

Consider the following scenario, where an input delay is specified on port A with **min_*** values equal to **max_*** values:

```
create_clock -period 10 CLK  
set_input_delay 8.0 -clock CLK A
```

Because **-min** and **-max** are not used, constraint consistency will use the same value for both. This will result in a zero width arrival window, and constraint consistency will flag an **EXD_0012** violation.

What Next

The "Violation Details" section reports the current input delay values on the flagged object, the file, and the line number. The table helps identify what values contribute to the zero width arrival delay window.

Fix Suggestion

The minimum value should be smaller than its corresponding maximum value. In addition, they should closely represent the delay in the final design.

For the above scenario, modify the specification to resolve the **EXD_0012** violation as shown below:

```
set_input_delay -max 8.0 -clock CLK A  
set_input_delay -min 4.0 -clock CLK A
```

See Also

[set_input_delay](#)

[set_output_delay](#)

Rule: EXD_0013

The output delay at **object_type object_name** has inconsistent values.

Severity: Information

Status: Disabled by default

Description

The object has minimum output delay greater than its corresponding maximum. This is not common practice and should be verified to ensure the correctness of the constraint. The **EXD_0013** violation will be issued if:

- The **-min -fall** output delay value is greater than the **-max -fall** value
- The **-min -rise** output delay value is greater than the **-max -rise** value

Risk Associated with Violation

Minimum delay that is greater than the maximum delay is not common and may not reflect the actual timing of the design. Timing analysis tools that work with unrealistic values lead to incorrect results.

One Possible Scenario

Consider the following scenario, where an output delay is specified on the port **out** where the specified minimum value is greater than the specified maximum:

```
create_clock -period 10 CLK  
set_output_delay -min -clock CLK 8.0 out  
set_output_delay -max -clock CLK 4.0 out
```

These constraints specify a design where both **rise** and **fall** minimum output delays are larger than the corresponding maximum delays. Constraint consistency will issue the **EXD_0013** violation to alert the user to this situation.

What Next

The "Violation Details" section will display a table containing the various minimum or maximum, and rise or fall parameters for the port. The source file and line number where those values have been set in the file will also be available.

Alternatively, you can use the [report_port](#) command to review the current settings related to a particular port. An example of the report is shown below:

```
ptc_shell> report_port -output_delay [get_ports out]
*****
Report :  port
        -output_delay
Design :  EXD_0013
Scenario: default
Version: ...
Date    : ...
*****
```

Output Port	Output Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		
out	8.00	8.00	4.00	4.00	CLK	--

Fix Suggestion

The minimum delay should always be smaller than its corresponding maximum. For the example scenario discussed above, a possible solution is to adjust the specified minimum delay to be smaller than the maximum delay as shown below:

```
set_output_delay -max -clock CLK 8.0 out
set_output_delay -min -clock CLK 4.0 out
```

See Also

[set_input_delay](#)

[set_output_delay](#)

[report_port](#)

Rule: EXD_0014

The input/output delay set on the combinational path from **'from_object_type'** **'from_object'** to **'to_object_type'** **'to_object'** is incorrect.

Severity: Warning

Status: Disabled by default

Description

The combinational path reported has one or more of the following issues with its input and output delays:

- The sum of the input delay and output delay for the combinational path is greater than or equal to the related clock period.
- The sum of the input delay and output delay for the combinational path is greater than or equal to the maximum delay value specified with the [set_max_delay](#) constraint on the path.
- The input and output delay constraints have been set with the **-min** and **-max** options, and the sum of the minimum input delay and output delay is greater than the sum of the maximum input delay and output delay.

There are rule properties for this rule that consider exceptions that have been set on the combinational path before a violation is issued.

The **suppress_violations_for_multicycle_paths** rule property controls whether a violation is issued or suppressed based on whether the combinational path has [set_multicycle_path](#) exceptions specified on it. The default value for this rule property is **true**, which means a violation will not be issued if the combinational path has a multicycle path constraint.

The **suppress_violations_for_false_paths** rule property controls whether a violation is issued or suppressed based on whether the combinational path is false. The default value for this rule property is **true**, which means a violation will not be issued if the combinational path is false.

Risk Associated with Violation

If the sum of the input delay and output delay on a combinational path is greater than or equal to the related clock period or maximum delay value on the path, it will cause a timing violation in a timing analysis tool.

For the case where the sum of the minimum input delay and output delay is greater than the sum of the maximum input delay and output delay, the constraints might not reflect the actual timing of the design. Timing analysis tools that work with unrealistic values will lead to incorrect results.

Possible Scenario

Consider the combinational paths pictured in Figure 1 and the constraints that follow. These constraints demonstrate the first type of reason that can cause an EXD_0014 violation.

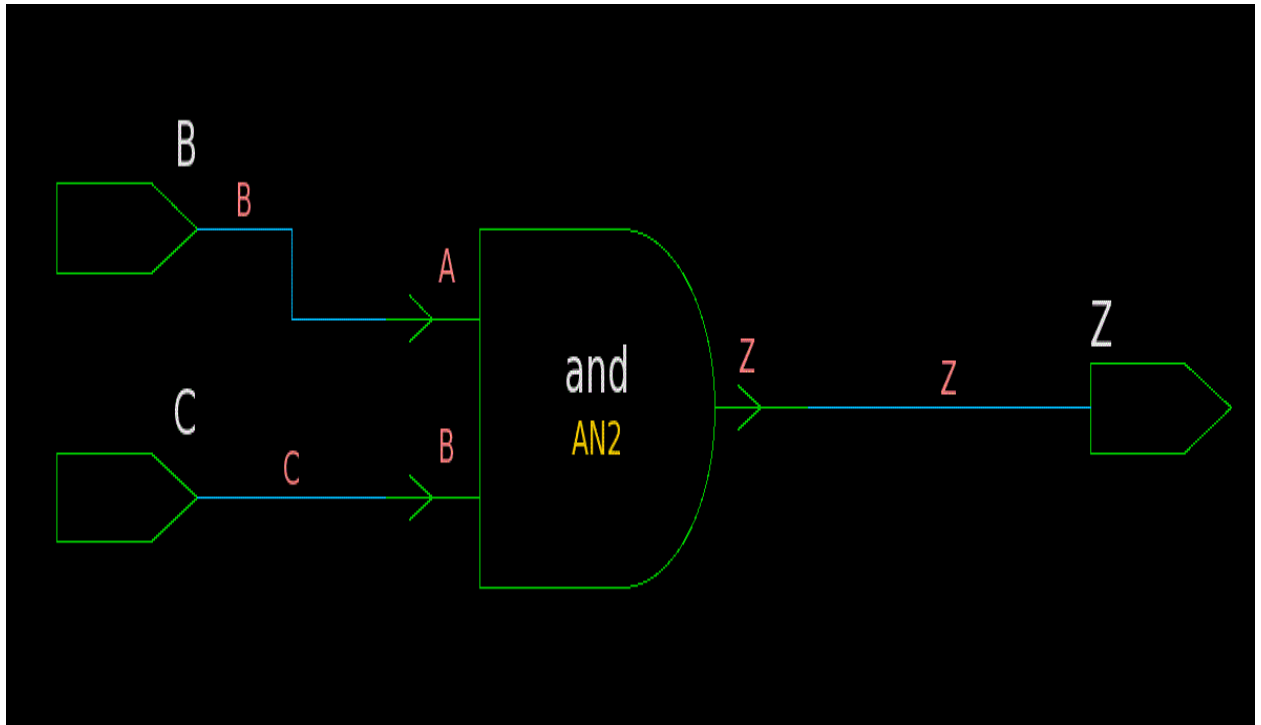


Figure 1

Constraint set:

```
create_clock -name CLK -period 10 -waveform {0 5} [get_ports CLK]
set_input_delay 8 -clock CLK [get_ports B]
set_output_delay 3 -clock CLK [get_ports Z]
```

The sum of the input delay (8 time units) specified on Port B and the output delay (3 time units) specified on Port Z of the combinational path is 11 time units, which is greater than the clock period of 10 time units. As a result, an EXD_0014 rule violation is issued.

What Next

The Violation Details section of the GUI includes a table for the input delay and output delay values for the combinational path. The file and line number for the input delay and output delay constraints are also included. You can click the hyperlink to open the Constraint Browser to view the relevant command. Below is what is shown for the example scenario:

You can use the [report_port](#) command to review the current delay settings related to a particular port. An set of reports is shown below for the example scenario.

```
ptc_shell> report_port -input_delay [get_ports B]
```

```
*****
Report : port
        -input_delay
Design : EXD_0014
Scenario: default
Version: ...
Date   : ...
*****
```

Input Port	Input Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		
B	8.00	8.00	8.00	8.00	CLK	--

```
ptc_shell> report_port -output_delay [get_ports Z]
```

```
*****
Report : port
        -output_delay
Design : EXD_0014
Scenario: default
Version: ...
Date   : ...
*****
```

Output Port	Output Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		
Z	3.00	3.00	3.00	3.00	CLK	--

Fix Suggestion

The [set_input_delay](#) and [set_output_delay](#) constraints should be adjusted so that a timing violation is not caused in a timing analysis tool or to reflect the actual timing of the design.

Rule Properties

`suppress_violations_for_multicycle_paths`

This rule property considers whether the combinational path is constrained by [set_multicycle_path](#) commands when performing its check. The default value for this rule property is **true**, which means the violation is issued if the combinational path fails the check and does not have a [set_multicycle_path](#) exception. The number of violations

for this rule can increase when the rule property is set to its non-default value, since combinational paths will be reported that are multicycle paths.

`suppress_violations_for_false_paths`

The default value for this rule property is **true**, which means a violation is issued only if the combinational path is non-disabled and non-false. When this rule property is set to **false**, a violation is issued if the combinational path fails the check and is false or non-disabled. The number of violations for this rule can increase when the rule property is set to its non-default value, since combinational paths will be reported that are false paths.

See Also

[set_input_delay](#)

[set_output_delay](#)

[set_multicycle_path](#)

[report_port](#)

Rule: EXD_0015

The input delay at ***object_type object*** has inconsistent values.

Severity:Warning

Description

The object has a minimum input delay that is greater than its corresponding maximum. This is not common practice and should be verified to ensure the correctness of the constraint. The **EXD_0015** violation is issued if at least one of the following is true:

- The **-min -fall** input delay value is greater than the **-max -fall** value
- The **-min -rise** input delay value is greater than the **-max -rise** value

Risk Associated with Violation

Minimum delay that is greater than the maximum delay is not common and might not reflect the actual timing of the design. Unrealistic values can lead to incorrect timing results.

One Possible Scenario

In the following scenario, an input delay is specified on the port **in** where the specified minimum value is greater than the specified maximum:

```
create_clock -period 10 CLK
set_input_delay -min -clock CLK 8.0 [get_ports in]
```

```
set_input_delay -max -clock CLK 4.0 [get_ports in]
```

These constraints specify a design where both **rise** and **fall** minimum input delays are larger than the corresponding maximum delays. The tool issues the **EXD_0015** violation to alert you to this situation.

What Next

The "Violation Details" section displays a table containing the minimum or maximum and rise or fall parameters for the port. The source file and line number where those values have been set are also shown.

Alternatively, you can use the [report_port](#) command to review the current settings related to a particular port, as shown in the following example:

```
ptc_shell> report_port -input_delay [get_ports in]
*****
Report : port
        -input_delay
Design : EXD_0015
Scenario: default
...
*****
```

Input Port	Input Delay		Max		Related Clock	Related Pin
	Rise	Fall	Rise	Fall		
in	8.00	8.00	4.00	4.00	CLK	--

Fix Suggestion

The minimum delay should always be smaller than its corresponding maximum. For the previous scenario example, a possible solution is to adjust the specified minimum delay to be smaller than the maximum delay as shown below:

```
set_input_delay -max -clock CLK 8.0 [get_ports in]
set_input_delay -min -clock CLK 4.0 [get_ports in]
```

See Also

[report_port](#)

[set_input_delay](#)

[set_output_delay](#)

Rule: HIER_001

Hierarchical pin *pin* has constraint **constraint** defined on *flr*.

Severity: Warning

Description

One of the following constraints is defined on a hierarchical pin: `set_false_path`, `set_multicycle_path`, `set_min_delay`, `set_max_delay`, `set_disable_timing`, `set_case_analysis`, `set_annotated_transition`, `set_max_transition`, or `set_max_capacitance`. This constraint will be correctly applied on the current design but if a change in the design hierarchy results in the removal of the hierarchical pin, the constraint will be ignored and will not be applied on the design.

Risk Associated with Violation

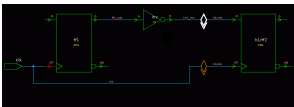
If the pin is removed during changes in hierarchy, the constraint will be ignored, resulting in an undesired timing analysis.

One Possible Scenario

Consider a scenario where an exception is specified on a hierarchical pin.

```
set_false_path -from ff1/CP -through b1/in1
```

Figure 40 Figure 1: Exception Specified on Hierarchical Pin



In such a case, the path from the register **ff1** to the register **ff2** will not be considered during timing analysis, and the tool issues a **HIER_001** rule violation. If a change in hierarchy results in removal of the hierarchical pin **b1/in1**, then the tool will issue an error for this constraint when it is read and not apply it to the design. In such a case, the false path exception will not be applied.

What Next

The “Violation Details” section will list all constraints and their types set on this hierarchical pin, and as well as the constraint file and line number where each constraint is specified.

Constraint Type	Constraint Source File
false_path	constraint.tcl, line 23

Fix Suggestion

Consider moving the constraint to the leaf pins related to the hierarchical pin. The leaf pins could either be the driver side pins or the load side pins, depending on the intent of the original constraint. In the above example, moving the constraint definition to the load pin will remove this violation.

```
set_false_path -from ff1/CP -through b1/ff2/D
```

See Also

[set_false_path](#)

[set_multicycle_path](#)

[set_min_delay](#)

[set_max_delay](#)

[set_case_analysis](#)

[set_disable_timing](#)

[set_annotated_transition](#)

[set_max_transition](#)

[set_max_capacitance](#)

Rule: LOOP_001

Arc from pin *pin* to *pin* is disabled due to combinational loop.

Severity: Warning

Description

There is a combinational feedback loop in the design that was broken by the tool. The "Violation Details" will give the pins that make up the combinational feedback loop.

Risk Associated with Violation

This may not be the best location to disable this combinational feedback loop. Other tools in the flow may break the loop at a different location and produce different results. It is best to add an explicit [set_disable_timing](#) or [set_case_analysis](#) command to the constraints to break the loop at the appropriate location.

One Possible Scenario

Consider a scenario like the combinational feedback loop in the schematic below. If the constraints do not disable/break the combinational feedback loop, the **LOOP_001** violation will be issued and the timing arc that was disabled by the tool to break the loop will be reported.

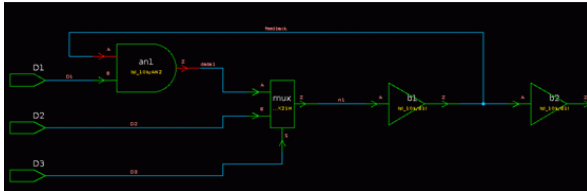


Figure 1: Combinational Feedback Loop

For the example scenario, the rule violation message specifies that the arc from pin **an1/A** to pin **an1/Z** was disabled, and the "Violation Details" will list the pins in the loop:

- an1/Z
- mux/A
- mux/Z
- b1/A
- b1/Z
- an1/A

What Next

Look at the combinational loop and determine the best location for it to be broken for this scenario. The [report_disable_timing](#) command can be used to see the disabled timing arcs in the design. Items in the report with the attribute **I** are disabled due to loop breaking.

```
ptc_shell> report_disable_timing [get_cells an1]
```

Attributes

```
c - case-analysis
C - Conditional arc
d - default conditional arc
f - false net-arc
l - loop breaking
L - db inherited loopbreaking
m - mode
p - propagated constant
u - user-defined
U - User-defined library arcs
```

Cell or Port	From	To	Sense	Flag	Reason
--------------	------	----	-------	------	--------

```
an1                                A      Z      positive_unate  1
```

Fix Suggestion

Add a [set_disable_timing](#) or [set_case_analysis](#) command to your constraints to disable the combinational timing loop for this scenario. For the example scenario, the following command can be used to see the disabled arcs for the cell where the loop was broken:

```
set_disable_timing [get_cells an1] -from A -to Z
```

Rule Property: `display_violation_details_per_rule_limit`

This rule property limits the number of violations that will display loop pins in the violation details in the GUI. The value is set to 100 by default. If the number of violations is greater than the display limit for a design, a message will be included in the information pane stating that the loop details will not be displayed because the rule's display limit has been reached. To see additional loop details, increase the limit and rerun the [analyze_design](#) command. Increasing this limit will increase the memory usage for your session. An example of how to change the limit is shown below.

```
set_rule_property display_violation_details_per_rule_limit 50 [get_rules LOOP_001]
```

See Also

[set_disable_timing](#)

[set_case_analysis](#)

[report_disable_timing](#)

Rule: LOOP_002

Arc from pin 'from_pin' to 'to_pin' belongs to a sequential loop.

Severity: Warning

Status: Disabled by default

Description

A loop involving sequential arcs is detected. You should ensure that this loop is intended in the design.

Risk Associated with Violation

This situation might not be intended. Other tools in the design flow might break this loop, resulting in a different timing analysis of the design. Constraint consistency does not break

the timing loop, it only reports it. If you need to break the timing loop, you can use the [set_disable_timing](#) command.

One Possible Scenario

Consider the scenario in Figure 1 below where a sequential feedback loop exists. If the constraints do not disable or break the sequential feedback loop, the tool issues a LOOP_002 violation and reports the pins involved in that feedback loop.

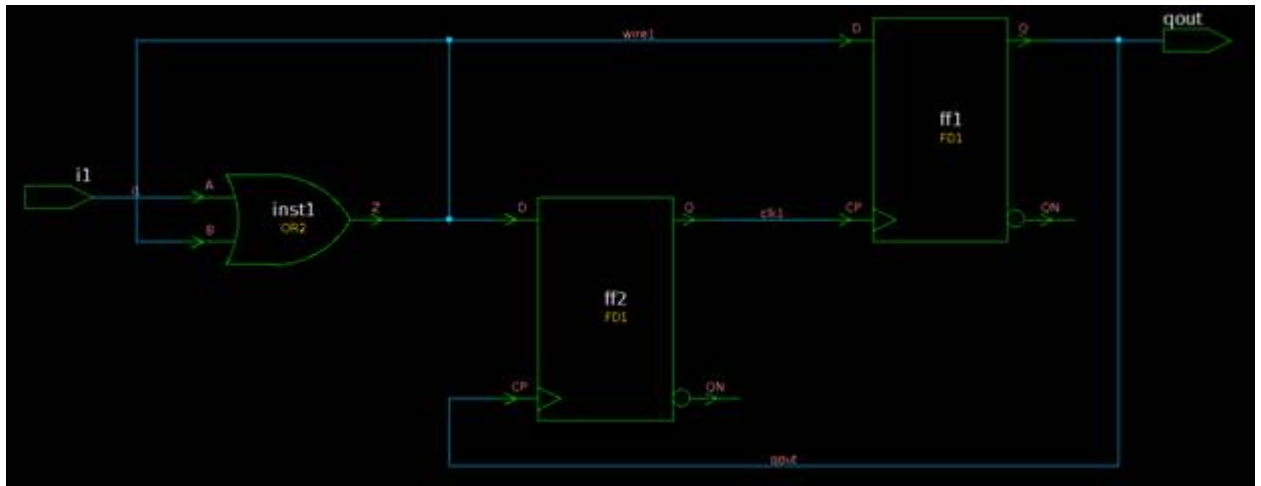


Figure 1

For the example scenario, the rule violation message specifies that the arc from pin **ff2/CP** to pin **ff2/Q** belongs to a sequential loop, and the 'Violation Details' section lists the pins in the loop as follows:

- * ff1/CP
- * ff1/Q
- * ff2/CP
- * ff2/Q

What Next

Look at the sequential loop and determine whether it is intended for this scenario. Check the behavior of other tools in your flow and adjust constraint consistency accordingly. By default, constraint consistency behaves like PrimeTime and it does not break the reported arc.

The command [report_disable_timing](#) can be used to see the disabled timing arcs in the design. Items in the report with the attribute **I** are disabled because of loop breaking.

```
ptc_shell> report_disable_timing
```


Attributes

c - case-analysis
C - Conditional arc
d - default conditional arc
f - false net-arc
l - loop breaking
L - db inherited loopbreaking
m - mode
p - propagated constant
u - user-defined
U - User-defined library arcs

Cell or Port From To Sense Flag Reason

inst1 B Z positive_unate l

You can break the timing loop using the following command:

```
ptc_shell> disable_timing -from CP -to Q [get_cells ff2]  
ptc_shell> report_disable_timing
```

Report : disable_timing

Design : top

Version: ...

Date : Tue ...

Attributes

c - case-analysis
C - Conditional arc
d - default conditional arc

f - false net-arc
l - loop breaking
L - db inherited loopbreaking
m - mode
p - propagated constant
u - user-defined
U - User-defined library arcs

Cell or Port From To Sense Flag Reason

inst1	B	Z	positive_unate	l
ff2	CP	Q	rising_edge	u

Fix Suggestion

Add a [set_disable_timing](#) command to your script if you need to disable the sequential timing loop.

Rule Property: display_violation_details_per_rule_limit

This rule property limits the number of violations that will display loop pins in the violation details in the GUI. The value is set to 100 by default. If the number of violations is greater than the display limit for a design, a message will be included in the information pane stating that the loop details will not be displayed because the rule's display limit has been reached. To see additional loop details, increase the limit and rerun the [analyze_design](#) command. Increasing this limit will increase the memory usage for your session. An example of how to change the limit is shown below.

```
set_rule_property display_violation_details_per_rule_limit 50 [get_rules LOOP_002]
```

See Also

[set_disable_timing](#)

[report_disable_timing](#)

Rule: NTL_0001

Output port **port** is unbuffered.

Severity: Warning

Description

The net for this output port is connected to input pins or to other output ports. Changing the capacitive load on this output will impact the performance of paths to the other load pins.

The "Violation Details" section gives a list of the other ports and pins driven by the net connected to this unbuffered port.

Risk Associated with Violation

If the capacitive load is changed on this output, it will impact the performance of paths to the other load pins.

One Possible Scenario

Consider the scenario shown in the diagram below. The output port **out2** is unbuffered, and the net connected to it also reaches **ff2**. Any capacitive load connected to the port **out2** also acts as a load for the path to **ff2**. When this condition exists in the netlist, the tool will issue a NTL_0001 violation.

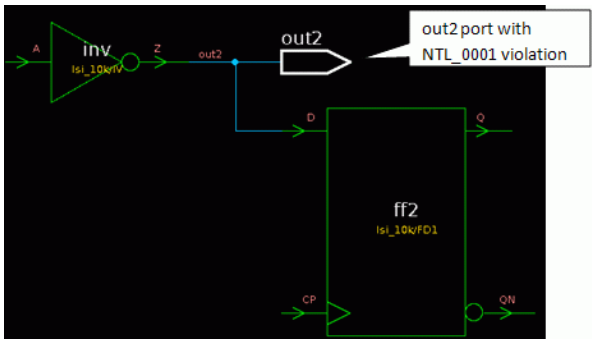


Figure 1: Unbuffered Output Port

What Next

The "Violation Details" section gives a list of the other ports and pins driven by the net connected to the unbuffered port. You can use the [report_port](#) command to view the capacitive loading defined on the output port. For the example scenario, the capacitive loading defined on the **out2** port is found using this command:

```
ptc_shell> report_ports [get_ports out2]
```

Port	Dir	Pin Cap		Wire Cap	
		Min	Max	Min	Max
		rise/fall	rise/fall	rise/fall	rise/fall
Attributes					
out2	out	0.43/0.43	0.50/0.50	0.00/0.00	0.00/0.00

The report shows that there is a min capacitance of 0.43 library capacitance units and a max capacitance of 0.50 library capacitance units on the **out2**port. If the value of these change, the timing of paths related to the **ff2/D** pin will be affected.

Fix Suggestion

No change to the design may be needed as long as the designer is aware of the timing implications of keeping the existing implementation. If the dependency is undesired, buffering the port from the other load pins can be done to remove this violation.

See Also

[report_port](#)

[set_load](#)

Rule: NTL_0002

Net **net** has both strong and three-state drivers.

Severity: Error

Description

The reported net has a mixture of strong drivers and three-state drivers. This is unusual and could be a logic error.

The violation details section gives the list of the drivers and the three-state drivers.

Risk Associated with Violation

The strong drivers will dominate and hence will always determine the logic output of the net. The three-state drivers will not have any impact on the design function.

One Possible Scenario

Consider the scenario shown in the schematic below. Both the three-state driver **tri_state** and the inverter **inv** are driving the net **inout1**. In such a scenario, the tool will give out an **NTL_0002** violation.

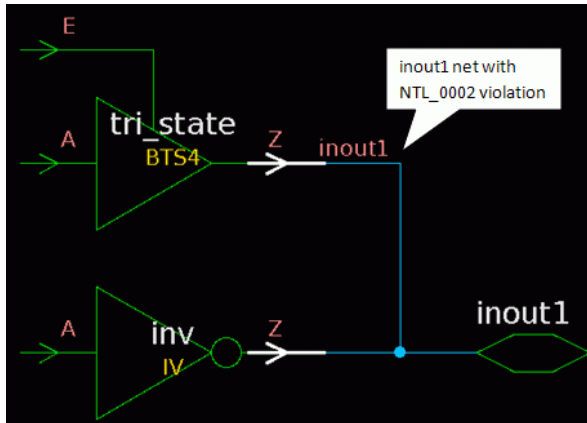


Figure 1: Three-State Driver and Inverter Driving the Net

What Next

You can use the [all_fanin](#) command to get a collection of all the drivers of the reported net.

```
ptc_shell> all_fanin -to inout1 -pin_level 1 -flat \  
                  {"inout1", "tri_state/Z", "inv/Z"}
```

Fix Suggestion

Consider changing the netlist to remove this violation.

See Also

[all_fanin](#)

Rule: NTL_0003

Net **net** has potentially mismatched multiple strong drivers.

Severity: Warning

Description

Multiple strong drivers on a net can be mismatched if the drivers are of different types or if their cells are not connected in parallel. Depending on the logic and delay through the cells, the drivers might try to drive the net to both logic one and zero simultaneously.

The "Violation Details" section will list the inconsistent driver types or the cells which are not connected in parallel.

Risk Associated with Violation

Generally, the intent of driving a net with multiple drivers is to give a higher drive strength to the net. If the driving cells are of inconsistent types or are the same cells, but driven by different nets, problems may occur in the circuit. In some cases, this configuration could lead to glitches in the signal due to timing differences between the drivers. In more serious cases, the output net can potentially have conflicting values driving it, resulting in larger than intended power consumption.

Possible Scenario for 'Inconsistent Driver Types'

Consider the scenario related to the **out1** net shown in the Figure 1 below. The net **out1** is driven by two cells - an inverter and an AND gate. In this case, all but one combination of the inputs (a logic 1 for **in1** and a logic 0 for **in2**) produces conflicting logic on the **out1** net. Because the cell types driving the net are inconsistent, the tool will issue an **NTL_0003** rule violation.

Possible Scenario for 'Cells Not Connected in Parallel'

Consider the scenario related to the **out2** net shown in the Figure 1 below. The net **out2** is driven by cells of the same type, but they are driven by different nets. It is possible for the **in2** and **in3** nets to have different logic values that will result in net **out2** being driven by conflicting logic. Because the cells may have conflicting input values, the tool will issue an **NTL_0003** rule violation.

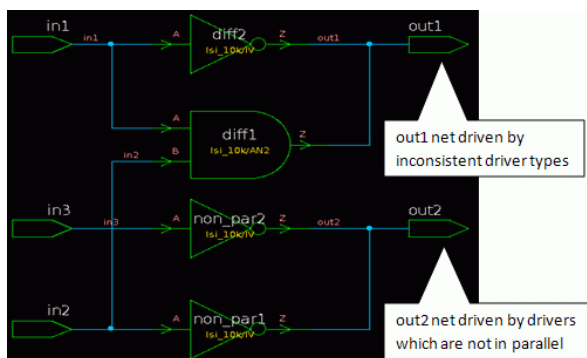


Figure 1: Cells Not Connected in Parallel

Fix Suggestion

If the "Violation Details" section shows that the drivers are of inconsistent type, then making these consistent will fix this violation. If the "Violation Details" section shows that the drivers are not in parallel, then the timing and sense of the input signals to the cells should be examined closely to ensure that no conflicts will occur on the output of the cells. If the timing and sense of the signals cannot be guaranteed, then changes to the netlist will be required to remove this violation.

Rule: NTL_0004

Net **net** is a top-level feedthrough.

Severity: Info

Description

The reported net directly connects at least one input port to at least one output port. Nets attached to inout ports are not included in this check.

Risk Associated with Violation

There are specific considerations for feedthrough nets during place and route. They can be used to connect circuitry through hierarchical blocks in a design as a result of the design floorplan. The timing report for a feedthrough net will report the net delay associated with it.

One Possible Scenario

Consider the scenario with the schematic shown in the diagram below. The input port **in** is directly connected to the output port **out** through a hierarchical block. For nets that are directly connected to input and output ports, the tool will issue a **NTL_0004** violation.

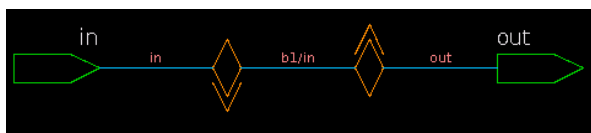


Figure 1: Input Port Directly Connected to Output Port Through Hierarchical Block

What Next

You can verify the connectivity of the net using the [report_net-connections](#) command. For the example scenario, the command and report are shown below.

```
ptc_shell> report_net -connections [get_net in]
```

Connections for net 'in':

Driver Pins	Type
-----	-----
in	Input Port
Load Pins	Type
-----	-----
out	Output Port

Alternatively, the [analyze_paths](#) command can be used to report the paths between the ports. For the example scenario, the command and report are shown below.

```
ptc_shell> analyze_paths -from [get_ports in] -to [get_ports out] \
               -unconstrained -traverse_disabled -path_type full
```

Summary of Paths:

Startpoint Capture	Endpoint	Dominant Constraint	Overridden Constraints	Path Count	Clocks Launch/
in (in) unclocked/	out (out)			2	

Full report of all pins in the paths

Level	Pin Constraints	Attr
1	in (in)	Start
2	out (out)	End

Both reports verify that the net is connected to the input port **in** and the output port **out**.

Fix Suggestion

Examine your design to verify whether a feedthrough net is part of extra metal tracks for potential use during silicon debug. If you are checking constraints for a hierarchical block, you may want to verify that the reported net has connectivity at the design level.

See Also

[report_net](#)

[analyze_paths](#)

Rule: NTL_0005

Unresolved reference to **reference_name**.

Severity: Warning

Description

No match was found for the named reference.

Note: A **LNK-005** user error is also issued during the linking phase for this case.

Risk Associated with Violation

The reference cell specified for the instance in the design could not be resolved to a library cell or design module. The instances using this reference will be treated as black boxes, and any timing analysis involving paths through this black box will be incomplete.

Possible Scenarios

Unresolved references generally occur when the linked libraries are either incorrect or the list of link libraries is incomplete. When a reference cannot be resolved during linking, the tool issues an **NTL_0005** rule violation.

What Next

The "Violation Details" section lists the instances for which black boxes were created due to the unresolved reference. Examining these could help identify which reference designs could not be resolved. It is best to query the [link_path](#) and [search_path](#) variables to see what libraries are being used:

```
ptc_shell> printvar link_path  
  
ptc_shell> printvar search_path
```

The [link_path](#) variable specifies a list of libraries, design files and library files used during linking to resolve references. The [search_path](#) variable specifies the locations to search for library and design files. It is possible either the [search_path](#) does not contain the correct location for the library, or the [link_path](#) does not have the correct library name.

Fix Suggestion

Linking the correct libraries, updating the locations to search for libraries, or removing any typographical errors from these variables will fix this violation.

See Also

[printvar](#)

[link_path](#)

[search_path](#)

Rule: NTL_0006

Net *net* has high fanout; fanout count is *fanout_count*.

Severity: Warning

Status: Disabled by default

Description

Nets with high fanout are not expected in post-layout analysis. They should ideally be adequately buffered to meet realistic timing constraints. To find the default fanout limit used during rule checking, you can use the `get_rule_property fanout_limit NTL_0006` syntax. To change the maximum allowable fanout value used during rule checking, you can use the `set_rule_property fanout_limit new_value NTL_0006` syntax.

The **NTL_0006** rule is disabled by default and can be enabled by issuing the `enable_rule NTL_0006` syntax before running `analyze_design`.

Risk Associated with Violation

High fanout nets result in higher loads for driving cells, potentially producing max_transition violations which affect both the power and timing of the design. The possible cause could be that this net was not properly constrained and therefore left unoptimized.

One Possible Scenario

The design rules for a project do not allow nets with a fanout greater than 5. The **NTL_0006** rule's parameter should be changed as:

```
ptc_shell> set_rule_property fanout_limit 5 NTL_0006
```

Additionally since this rule is disabled it should be enabled as follows:

```
ptc_shell> enable_rule NTL_0006
```

With the **NTL_0006** rule enabled and the maximum fanout limit set to 5, for all nets in the design with a fanout of greater than 5, the tool will issue a **NTL_0006** violation.

What Next

Use the `report_net-connections` command to see the driving cell(s) and loads of the net with the **NTL_0006** violation.

```
ptc_shell> report_net -connections [get_nets high_fanout]
```

Connections for net 'high_fanout':

Driver Pins	Type
-----	-----
high_fanout	Input Port
Load Pins	Type
-----	-----
buf3/A	Input Pin (B1I)
buf2/A	Input Pin (B1I)
buf1/A	Input Pin (B1I)
inv3/A	Input Pin (IV)

inv2/A	Input Pin (IV)
inv1/A	Input Pin (IV)

The report shows that the net **high_fanout** has 6 receiver cells, which is over the specified fanout limit of 5.

Fix Suggestion

If the net violates the max_transition requirement, and there are a large number of nets that fall in this category, the design may need to be re-optimized in synthesis. If there are a small number of nets that fall in this category, the violation may be addressed through targeted ECO changes. If the net does not have a max_transition violation, no change may be necessary.

Rule Property: fanout_limit

The default value of the rule property, **fanout_limit**, can be queried using the following syntax:

```
get_rule_property fanout_limit NTL_0006
```

It can be set to a different number using the [set_rule_property](#) syntax:

```
set_rule_property fanout_limit 100 NTL_0006
```

The number of **NTL_0006** violations is dependent on the setting of this property. If it is set to a large limit, then fewer **NTL_0006** violations will be issued if there are nets with high fanout in the design.

See Also

[get_rule_property](#)

[set_rule_property](#)

[enable_rule](#)

[disable_rule](#)

[report_net](#)

Rule: OPN_0001

The constraint **constraint** set on **object_typeobject** has incomplete *options* options.

Severity: Warning

Description

The reported constraint is incompletely defined. It is missing some options. For example, a constraint could be defined with a max but not a min parameter or a fall without a rise. This rule targets the following commands:

- set_clock_gating_check
- set_annotated_transition
- set_clock_latency
- set_timing_derate

Risk Associated with Violation

These commands affect the results of static timing analysis. In order to get accurate timing results, these commands need to be applied with the relevant options. This rule flags the missing options and brings this to the attention of the user.

One Possible Scenario

Consider a scenario (Fig 1 below) where a clock is defined on port M1 and it is gated at MUX U1. You can specify a value for the clock gating check using the set_clock_gating_check command as follows:

```
create_clock -name m1 -period 2 [get_ports M1]
set_clock_gating_check -hold 0.3 -low [get_pins U1/S]
```

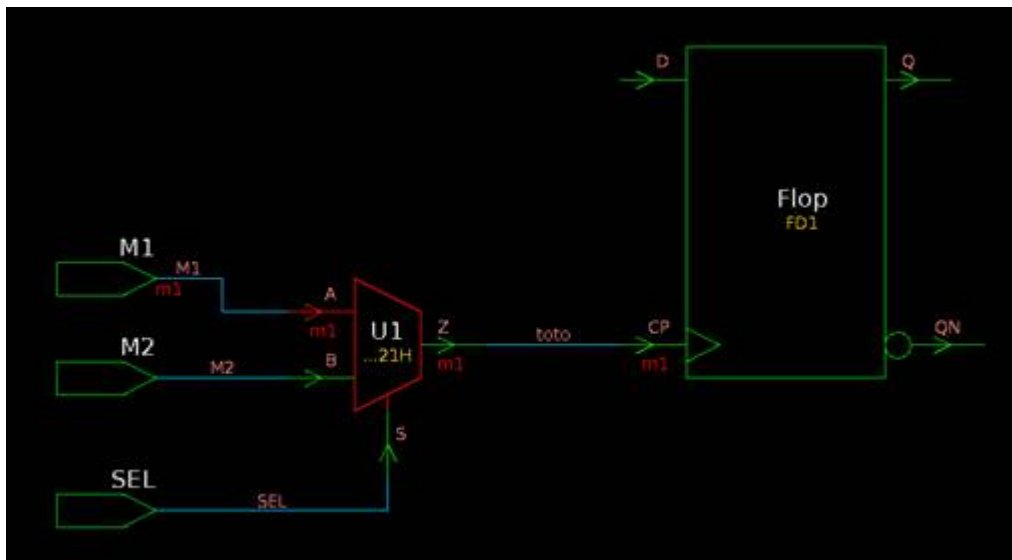


Fig 1: Clock gating example

It is expected, in this case, that you will provide values for both hold and setup. If you do not fully specify the clock gating check values, constraint consistency generates an OPN_0001 violation.

What Next

The Info Pane window associated with the violation reports the type of constraint responsible for this violation (`set_clock_gating_check` in the example above). An HTML table is also displayed, filled with the values currently defined. Missing values can be identified in this table; they are represented with "--".

You can open the SDC browser window and look at the command by clicking on the provided hyperlink.

Fix Suggestion

Modify the reported command by adding the missing parameters.

See Also

[set_clock_gating_check](#)

[set_timing_derate](#)

[set_annotated_transition](#)

[set_clock_latency](#)

Rule: PRF_0001

There are **count** clock pins with more than **limit** clocks.

Severity: Warning

Description

The performance of many tools is negatively impacted by having many clocks propagating to a single clock pin. The number of clocks allowed without triggering a violation is set with the **max_clocks_per_register** property on this rule. The more clock pins there are in a design with large numbers of clocks, the more likely there will be a negative impact on tool runtime and memory.

The [CLK_0024](#) rule is disabled by default, but will give one violation per clock pin with more than the **max_clocks_per_register** property of this rule.

Risk Associated with Violation

There is a risk that the timing analysis will run slower and take more memory for a set of constraints that have many clocks propagating to the same registers. To improve

performance, the user may need to consider separating functional modes into more than one constraint scenario.

One Possible Scenario

A small IP block has 8 functional modes merged into a single set of best/worst case constraints. With these constraints, 70% of the clock pins in the design are clocked by six different clocks. A **PRF_0001** violation is issued, because the **max_clocks_per_register** property on this rule has been set to 4:



Figure 1: PRF_0001 Violation

In this case, because the block is small, the performance impact of having so many clocking combinations for the tool to analyze at once is not a concern.

What Next

If the number of clock pins with a large number of clocks is unexpected, you may want to enable rule [CLK_0024](#) to see the first 1000 pins with more clocks than the value of the **max_clocks_per_register** property for this rule. Then you can use the [analyze_clock_networks](#) command to find the clocks at one of those clock network pins.

To enable this rule and rerun the analysis targeting this rule only, you can use the following commands:

```
enable_rule CLK_0024
analyze_design -rules CLK_0024 -output_dir /mypath/multiclockrule
```

Fix Suggestion

If the number of clocks is incorrect for the functional modes being analyzed, you will need to add additional case analysis constraints so that only the desired clocks propagate through various clock selectors in the clock network.

You may also wish to increase the maximum number of clocks per pin property to a larger value; see the “Rule Property” section below.

You may also consider not merging as many functional modes together into a single analysis run to improve analysis performance.

For the IP block example, you can simply disable this rule check for the design before issuing the [analyze_design](#) command:

```
disable_rule PRF_0001
```

Rule Property: max_clocks_per_register

The default value of the rule property, **max_clocks_per_register**, can be queried using the following command:

```
get_rule_property max_clocks_per_register PRF_0001
```

It can be set to a different value using the [set_rule_property](#) command:

```
set_rule_property max_clocks_per_register 10 PRF_0001
```

The number of clock pins reported by this rule is dependent on the setting of this property. If it is set to a large limit, then pins below that limit will not be included in the clock pin count in the **PRF_0001** violation.

See Also

[CLK_0024](#)

[analyze_clock_networks](#)

[get_rule_property](#)

[set_rule_property](#)

[analyze_design](#)

[disable_rule](#)

[enable_rule](#)

Rule: TRN_0001

The transition constraint on ***object_type object*** is larger than the maximum transition design rule.

Severity: Warning

Status: Disabled by default

Description

A transition constraint is being specified via the [set_clock_transition](#), [set_input_transition](#), or [set_annotated_transition](#) commands. However, the value specified in the constraint is larger than the limit specified by [set_max_transition](#) constraint or the applicable library level max transition or pin level max transition attribute. The most restrictive limit is used to perform the check.

If no limit has been specified with the [set_max_transition](#) constraint or in the library, the constraint is compared to the **max_transition** property associated with this rule.

This rule does not perform design rule checking (DRC) of transition times in the design; instead, only the value in the constraint is checked against the design rule limit. A [set_clock_transition](#), [set_input_transition](#), or [set_annotated_transition](#) constraint can pass this check, but the design may have max transition violations when design rule checking is performed in another tool.

The TRN_0001 rule is disabled by default and can be enabled by issuing the command

```
enable_rule TRN_0001
```

before the [analyze_design](#) command.

Risk Associated with Violation

The value in the specified constraint can directly lead to a design rule check (DRC) violation for max transition in another tool. If a transition time is larger than intended, this could lead to higher power consumption in the design.

One Possible Scenario

Consider a library where the max transition specified is 1.3 nanoseconds.

```
library (example_lib) {  
...  
  
time_unit : "1ns";  
...  
  
default_max_transition : 1.100000;  
...  
  
cell (AND2) {  
pin (A) {  
direction : "input";  
max_transition : 1.300000;  
...  
}}
```


The constraint file for the design specifies a transition on a pin of 1.5 library units (in this case nanoseconds) on an input pin of a cell in the design.

```
set_annotated_transition 1.5 [get_pin u1/A]
```

The annotated transition set on the pin is larger than the maximum allowed transition of 1.3 nanoseconds from the library. This will cause a TRN_0001 violation to be issued.

What Next

The 'Violation Details' section provides a table of the transition values for rise/fall and min/max analysis, as well as the file name and line number of the applicable constraint. For the example scenario, the table below is created in the Information Pane.

	Min	Max
Rise	1.500000	1.500000
Fall	1.500000	1.500000

Fix Suggestion

It is recommended to specify transition constraints that are smaller than the maximum transition design rule to avoid max transition design rule check (DRC) violations in another tool.

Rule Property: max_transition

By default, the **max_transition** rule property is set to 300 picoseconds (ps). Any [set_clock_transition](#), [set_input_transition](#), or [set_annotated_transition](#) constraint that is over this value will cause a TRN_0001 violation to be issued if there are no transition limits specified through the [set_max_transition](#) constraint or in the library. The threshold used for checking can be changed by the user with the [set_rule_property](#) command as show below. If no units are specified with the command, seconds are assumed by the tool.

```
set_rule_property max_transition 500ps TRN_0001
```

For the rule property value to have an effect, [analyze_design](#) must be run after the [set_rule_property](#) command is issued.

See Also

[set_clock_transition](#)

[set_input_transition](#)

[set_annotated_transition](#)

[set_max_transition](#)

[set_rule_property](#)

[analyze_design](#)

[enable_rule](#)

[disable_rule](#)

Rule: TRN_0002

The `set_max_transition` constraint on ***object_type object*** is larger than the maximum transition design rule in the library.

Severity: Warning

Status: Disabled by default

Description

A [set_max_transition](#) constraint has a value larger than the applicable max transition attribute in the related library. When design rule checking is performed in another tool, the more restrictive value is used. The value specified in the [set_max_transition](#) constraint will be redundant. This rule finds [set_max_transition](#) constraints with this issue.

If no limit has been specified in the library, the constraint is compared to the **max_transition** property associated with this rule.

The TRN_0002 rule is disabled by default and can be enabled by issuing the following command:

```
enable_rule TRN_0002
```

before the [analyze_design](#) command.

Risk Associated with Violation

The [set_max_transition](#) constraint can be used to specify a transition value to be used during design rule checking in another tool. If the transition obtained during delay calculation in the tool is larger than the limit specified, it is considered to have failed the design rule check. The most restrictive limit from the constraint or applicable library level max transition or pin level max transition is used to perform the check. If the intent of the constraint was to set a value more restrictive than the library, transition times in the design might be larger than intended, resulting in slower signal transitions and higher power consumption.

One Possible Scenario

Consider a library where the max transition specified is 1.4 nanoseconds, as shown below.

```
library (example_lib) {
```

```
...  
  
time_unit : "1ns";  
  
...  
  
default_max_transition : 1.300000;  
  
...  
  
cell (AND2) {  
  pin (A) {  
    direction : "input";  
    max_transition : 1.400000;  
  }  
}
```

The constraint file for the design specifies a maximum transition of 1.8 library units (in this case nanoseconds) on an input pin of a cell in the design, as shown below.

```
set_max_transition 1.8 [get_pin u1/A]
```

The maximum allowed transition specified on the pin by the constraint is larger than the maximum allowed transition of 1.4 nanoseconds from the library. This will cause a TRN_0002 violation to be issued.

What Next

The 'Violation Details' section provides the file name and line number of the [set_max_transition](#) constraint. You can also use the [get_attribute](#) command to verify the related max_transition attribute on the object specified in the constraint. For the example scenario, you can verify the attribute using the following command:

```
ptc_shell> get_attribute [get_pins u1/A] max_transition  
1.400000
```

Fix Suggestion

The [set_max_transition](#) constraint will only have an effect in design rule checking if it is more conservative than the related library max transition attribute. It is recommended to review the value specified for correctness, and either modify the value if it is incorrect or remove the constraint.

Rule Property: max_transition

By default, the **max_transition** rule property is set to 300 picoseconds (ps). Any [set_max_transition](#) constraint that is over this value will cause a TRN_0002 violation to be issued if there are no transition limits specified in the library. If no units are specified with

the command, seconds are assumed by the tool. You can change the threshold used for checking with the [set_rule_property](#) command as show below.

```
set_rule_property max_transition 200ps TRN_0002
```

For the rule property value to have an effect, [analyze_design](#) must be run after the [set_rule_property](#) command is issued.

See Also

[set_clock_transition](#)

[set_input_transition](#)

[set_annotated_transition](#)

[set_max_transition](#)

[get_attribute](#)

[set_rule_property](#)

[analyze_design](#)

[enable_rule](#)

[disable_rule](#)

Rule: UNC_0001

Output delays are defined relative to clock **clock**. This clock has incomplete uncertainty values required for those output delays even though its source ports and pins have more complete uncertainties defined on them.

Severity: Warning

Description

The [set_clock_uncertainty](#) constraint does not fully specify the uncertainty values for a given clock.

For path delay calculation, clock object uncertainties are used. However, if these are missing or incomplete, a clock uncertainty value of 0 will be assumed. Paths ending at ports with output delay related to that clock will be timed using a clock uncertainty of 0.

Risk Associated with Violation

The incomplete specification can lead to inaccurate timing analysis if the user assumes the pin/port uncertainty values will be used instead.

One Possible Scenario

Consider the following scenario where a clock is defined on an input port

```
create_clock -period 10 CLK
```

Clock uncertainty values are defined on both the clock object and the clock source port.

```
set_clock_uncertainty -setup 0.6 [get_clock CLK]  
set_clock_uncertainty 0.8 [get_port CLK]
```

An input delay is specified to a port relative to this clock:

```
set_output_delay 4 -clock CLK -add [get_ports OUT]
```

The external delay set on the “OUT” port has been defined with respect to clock CLK. Since no uncertainty is defined on the clock object for the hold condition, the clock uncertainty for minimum delay calculations will be zero (the clock uncertainty for setup will be 0.6 as specified through the command).

The uncertainty applied on the port object ‘CLK’ will not be used here and a **UNC_0001** violation is issued.

What Next

The “Violation Details” section shows a table of the uncertainty values for both the clock and its sources. For the above scenario, the table is shown as follows:

Uncertainty values

Object Type	Object Name	Setup	Hold
Clock	CLK	0.6000	--
Port	CLK	0.8000	0.8000

The current value applied on the clock can also be obtained using the [report_clock](#) command. This would generate the following report where one can check the clock uncertainty data:

```
ptc_shell> report_clock CLK -skew  
  
*****  
Report : clock_skew  
Design : UNC_0001  
Scenario: default  
Version: ...  
Date    : ...
```

	Min Rise	Min Fall	Max Rise	Max Fall	Hold
Setup					
Related					
Object	Delay		Delay	Delay	Delay
Uncertainty	Uncertainty				
Clock					

CLK		-	-	-	-
	-		-	0.60	

Fix Suggestion

You should always specify the clock uncertainty data fully.

For the example scenario above, specifying a clock uncertainty value for the hold condition is shown below:

```
set_clock_uncertainty -hold 0.4 [get_clock CLK]
```

See Also

[set_clock_uncertainty](#)

[remove_clock_uncertainty](#)

[set_output_delay](#)

[report_clock](#)

Rule: UNC_0002

Clock uncertainty is specified on ***object_type object*** but that object is not in any clock network.

Severity: Warning

Description

The clock uncertainty has been specified on a valid object; however, this object is not part of a clock network. The clock uncertainty command needs to refer to an object that is part of a clock network. Valid objects for clock uncertainty are clocks, ports, pins, or cells.

Note: The [set_clock_latency](#) command does not check if the clock uncertainty is specified on an object receiving a clock. As a consequence, applying a clock uncertainty on an object that is not part of a clock network will not generate an error message.

Risk Associated with Violation

Correct modeling of clock uncertainties on clock paths is very important in order to get accurate timing analysis results. Underestimating clock uncertainties can result in optimistic timing results.

One Possible Scenario

Consider the design shown below:

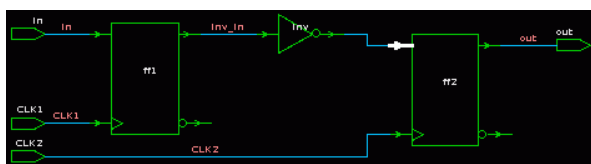


Figure 1: Separate Clocks Defined from Separate Ports

This design has two separate clocks defined from ports CLK1 and CLK2.

```
create_clock -name CLK1 -period 1 [get_ports CLK1]
create_clock -name CLK2 -period 2 [get_ports CLK2]
```

If you want to model uncertainty on the setup check of ff2, you can set the uncertainty as follows:

```
set_clock_uncertainty 0.6 -setup [get_pins ff2/D]
```

No error message will be issued in the log file but this uncertainty will be ignored. The ff2/D pin is not part of a clock network and, as a consequence, the clock uncertainty will have no effect.

What Next

The "Violation Details" section reports the location (file name and line number) of the constraints where the clock uncertainty is being applied.

You can analyze clock paths through the pin or port using the [analyze_clock_networks](#) command.

Using the **-traverse_disabled** option with the [analyze_clock_networks](#) command allows constraint consistency to trace through disabled timing arcs along the clock network. This allows you to understand whether a clock has been blocked due to a constraint in the design or if there is no clock reaching this pin.

In the above example, the **ff1/D** pin is not part of any potential clock network associated with **CLK1**. This information is available from the [analyze_clock_networks](#) command as shown below:

```
ptc_shell> analyze_clock_network -through ff1/D -traverse_disabled
```

```
Error: None of the -through objects are in a potential clock network.  
(ACN-003)
```

Fix Suggestion

Verify that the object provided in the clock uncertainty constraint is correctly specified.

In the example scenario, if the intended behavior was to specify a clock uncertainty for the setup check on **ff2**, the specification should be either:

- On the clock pin of flip-flop **ff2**
- or
- Directly on the clock source **CLK2**

A correct constraint example for that scenario would be:

```
ptc_shell> set_clock_uncertainty -setup 0.6 [get_port CLK2]
```

See Also

[set_clock_uncertainty](#)

[remove_clock_uncertainty](#)

[analyze_clock_networks](#)

Rule: UNC_0003

Clock uncertainty is incomplete or incorrect between two interacting clocks **clock1** and **clock2**

Severity: Warning

Status: Disabled by default

Description

It is desirable to specify uncertainty values between interacting clocks if a valid timing path exists between the two clocks. These uncertainty values can be used to properly model clock skew, clock jitter, and clock margin. The [set_clock_uncertainty](#) command is missing, incomplete, or incorrect between a launching clock and a capturing clock.

Risk Associated with Violation

Clock uncertainties are used for timing paths that are launched by one clock and captured by another clock. However, missing or incomplete clock uncertainties lead to optimistic timing results for some or all timing paths. Incorrect clock uncertainty value creates

a larger hold margin compared to setup, and can lead to increased efforts in timing optimization and timing closure.

One Possible Scenario

Consider a valid timing path launched by clock **clk1** and captured by clock **clk2** with the following clock uncertainty values:

```
create_clock -period 10 clk1
create_clock -period 10 clk2
set_clock_uncertainty 1.1 -setup -rise_from [get_clocks clk1] \
    -fall_to [get_clocks clk2]
set_clock_uncertainty 1.5 -hold [get_clocks clk2]
```

There are two problems with the clock uncertainty values from clock **clk1** to clock **clk2**:

1. Clock uncertainty values are incomplete because setup values are not defined for rise-to-rise, fall-to-rise, and fall-to-fall analysis.
2. Clock uncertainty value is incorrect because the hold value is larger than the setup value for the rise-to-fall analysis.

Because of the incomplete and incorrect uncertainty values between two interacting clocks, the **UNC_0003** violation will be issued.

What Next

The "Violation Details" section shows a table of uncertainty values for hold and setup between the two interacting clocks:

	Hold	Setup
Rise-Rise	1.5000	--
Rise-Fall	1.5000	1.1000
Fall-Rise	1.5000	--
Fall-Fall	1.5000	--

This table makes it easy to check which uncertainty values are missing or incorrect.

To verify the uncertainty values, use the [report_clock](#) command with the `-skew` option, as shown in the following example:

```
ptc_shell> report_clock -skew [get_clock clk2]
*****
Report : clock_skew
Design : UNC_0003
```

```
*****
```

	Min Rise	Min Fall	Max Rise	Max Fall	Hold	Setup
Related						
Object	Delay	Delay	Delay	Delay	Uncertainty	Uncertainty
Clock						

clk12	-	-	-	-	1.50	-
--						

	From	To	Hold	Setup
	Object	Object	Uncertainty	Uncertainty

	clk1 (r)	clk12 (f)	1.50	1.50
--	----------	-----------	------	------

Fix Suggestion

It is recommended to always specify complete clock uncertainty values between two interacting clocks. For missing clock uncertainty values, define them using the `set_clock_uncertainty` command. For incorrect clock uncertainty values, make the setup uncertainty value larger than the hold uncertainty value or make the hold uncertainty value smaller than the setup uncertainty value.

Rule Property: `suppress_violations_for_incorrect_uncertainty`

This rule property specifies whether violations are issued for incorrect clock uncertainty values. When this rule property is set to `false` (the default), a violation is issued if the uncertainty values are incorrect. When you set this rule property to `true`, constraint consistency does not check uncertainty values, so violations are not issued for incorrect values. The number of violations for this rule can decrease when the rule property is set to `true`.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: UNC_0004

Inter-clock uncertainty is smaller than simple uncertainty for clock X.

Severity: Warning

Status: Disabled by default

Description

This violation is issued when inter-clock uncertainty is smaller than clock-based uncertainty.

Risk Associated with Violation

You can traverse through all of the inter-clock uncertainties if:

- the inter-clock uncertainty is set on the same launch and capture clock, that is, it is of the form, `set_clock_uncertainty val1 -from [get_clock X] -to [get_clock X]`.
 - Check if the value is smaller than simple uncertainty on the clock set via `set_clock_uncertainty val2 [get_clock X]`
 - If *val1* is smaller than *val2* for either setup or hold, then issue a UNC_0004 violation.
- For inter-clock uncertainty that are not specified on the same launch and capture clock, no check is performed.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: UNC_0005

Inter-clock uncertainty is smaller than object based uncertainty at *pin/port* for clock *X*.

Severity: Warning

Status: Disabled by default

Description

This violation is issued when inter-clock uncertainty is smaller than netlist-object based uncertainty.

Risk Associated with Violation

You can traverse through all inter-clock uncertainties if:

- The inter-clock uncertainty is set on the same launch and capture clock, that is, it is of the form, `set_clock_uncertainty val1 -from [get_clock X] -to [get_clock X]`
 - Check if the value is smaller than simple uncertainty set on a netlist object (set via `set_clock_uncertainty val3 [get_pin/port YY]`) which is in the clock network of clock *X*.
 - If *val1* is smaller than *val3* for either setup or hold, then issue a UNC_0005 violation.

- For inter-clock uncertainty that are not specified on the same launch and capture clock, no check is performed.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: UNC_0006

Object-based uncertainty at *pin/port* is smaller than simple uncertainty for clock *X*.

Severity: Warning

Status: Disabled by default

Description

This violation is issued when netlist object-based uncertainty is smaller than clock based uncertainty.

Risk Associated with Violation

You can traverse through all object-based clock uncertainties (set via `set_clock_uncertainty val3 [get_pin/port YY]`)

- If the netlist object is in the clock network of clock *X*,
 - Check if the value is smaller than simple uncertainty on the clock *X* (set via `set_clock_uncertainty val2 [get_clock X]`)
 - If *val3* is smaller than *val2* for either setup or hold, then issue a UNC_0006 violation.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: UNT_0001

Library ***library*** has no units defined.

Severity: Error

Description

The specified library has cells instantiated in the design, but does not have any units defined that are used during timing or power analysis in other tools. All cell libraries should have explicit units for time, resistance, capacitance, voltage, current, and leakage power to ensure correct delay and power calculations.

Risk Associated with Violation

The units used to characterize the library can be different from those assumed by the tool during delay or power calculation, leading to incorrect timing or power analysis results.

What Next

The "Violation Details" section provides the list of missing units.

Quantity	Unit Value
time	--
resistance	--
capacitance	--
voltage	--
current	--
leakage_power	--

Fix Suggestion

Verify the units that were used to characterize the library and add them to the library group section as shown below. The library should be recompiled to create an updated .db file.

```
library (lib1) {  
    time_unit : "1ps";  
    pulling_resistance_unit : "1kohm";  
    capacitive_load_unit (1,ff);  
    voltage_unit : "1V";  
    current_unit : "1uA";  
    leakage_power_unit : "1mW";  
    ...  
}
```

Rule: UNT_0002

Library *library* has incomplete units defined.

Severity: Error

Description

The specified library has cells instantiated in the design, but it does not have all the units defined that are used during timing or power analysis in other tools. All cell libraries should have explicit units for time, resistance, capacitance, voltage, current, and leakage power to ensure correct delay and power calculations.

Risk Associated with Violation

The units used to characterize the library can be different from those assumed by the tool during delay or power calculation, leading to incorrect timing or power analysis results.

One Possible Scenario

The "Violation Details" section provides the list of missing units. The example below shows that the library has the time unit defined (1 ps), but the remaining library units are undefined:

Quantity	Unit Value
time	--
resistance	--
capacitance	--
voltage	--
current	--
leakage_power	--

Fix Suggestion

Verify the units that were used to characterize the library and add them to the library group section as shown below. The library should be recompiled to create an updated .db file.

```
library (lib1) {  
    time_unit : "1ps";  
    pulling_resistance_unit : "1kohm";  
    capacitive_load_unit (1,ff);  
    voltage_unit : "1V";  
    current_unit : "1uA";  
    leakage_power_unit : "1mW";  
    ...  
}
```

Rule: UNT_0003

The libraries used by this design have inconsistent units.

Severity: Info

Description

The libraries that have cells instantiated in the design have inconsistent units. Inconsistent units can cause constraints to be interpreted incorrectly in tools that choose design units based on the units of one library. For example, a set input delay value of 1.0 may be interpreted as 1.0 ps or 10.0 ps if one library has a time unit of 1 ps and the other has a time unit of 10 ps.

Risk Associated with Violation

For tools that choose design units based on a single library, the constraints will be interpreted with the units chosen for the tool session. If these do not match the units with which the constraints were created, the constraints will be interpreted differently than expected. This will affect the timing results of the design.

What Next

The "Violation Details" section provides a table that details the units in each library. The inconsistent units between libraries can be found by examining the table. For example, if two libraries, **lib1** and **lib2**, have inconsistent time, resistance, and capacitance units, a table will be created in the Information Pane of the GUI as shown below. By examining each column of the table for differences, all unit mismatches between the libraries can be identified.

Library	time	resistance	capacitance	voltage	current	leakage_power
lib1	10.00ps	10.00Ohm	1.00pF	1.00V	1.00mA	1.00mW
lib2	1.00ps	1.00kOhm	1.00fF	1.00V	1.00mA	1.00mW

Fix Suggestion

Use libraries with consistent units for your design. If consistent units between libraries are not possible, ensure that the **set_units** command is in each constraint file that is used for analysis. This command will issue errors during the session in other tools for any unit mismatches between those specified in the **set_units** command and the units chosen for the tool analysis session.

2

Block-To-Top Rules

Rules highlighted in gray are disabled by default.

Rule ID	Severity	Message
B2T_CAS_0001	Error	Pin top_pin in top instance has case value that is missing in block design.
B2T_CAS_0002	Error	Pin or port blk_pin_port in block design has case value that is missing in top instance.
B2T_CAS_0003	Error	Pin top_pin in top instance and corresponding pin or port blk_pin_port in block design have different case values.
B2T_CLK_0001	Error	Clock clock in top instance has no matching clock in block design.
B2T_CLK_0002	Error	Clock clock defined in block design has no matching clock in top instance.
B2T_CLK_0003	Error	Clock top_clock in top instance does not match with clock blk_clock in block design.
B2T_CLK_0004	Error	Pin top_pin has clock sense applied in top instance but is missing in block design.
B2T_CLK_0005	Error	Pin block_pin has clock sense applied in block design but is missing in top instance.
B2T_CLK_0006	Error	Clock sense applied to pin top_pin mismatches with clock sense applied to pin block_pin .
B2T_CLK_0007	Error	arc_sense arc from from_top_pin to to_top_pin has clock sense applied in top instance but is missing in block design.
B2T_CLK_0008	Error	arc_sense arc from from_block_pin to to_block_pin has clock sense applied in block design but is missing in top instance.
B2T_CLK_0010	Error	Clock gating check applied to object_type object in top instance is missing in block design.
B2T_CLK_0011	Error	Clock gating check applied to object_type object in block design is missing in top instance.

Rule ID	Severity	Message
B2T_CLK_0012	Error	Clock gating check applied to top_object_type top_object in top instance mismatched with clock gating check applied to blk_object_type blk_object in block design.
B2T_CLK_0013	Warning	Clock types of top_clock in top instance and clock blk_clock in block design do not match.
B2T_CLT_0001	Error	' constraint ' applied to ' object_type ' object ' in top design is missing in block design.
B2T_CLT_0002	Error	' constraint ' applied to ' object_type ' object ' in block design is missing in top design.
B2T_CLT_0003	Error	'constraint' applied to 'top_object_type' 'top_object' in the top design does not match the one applied to 'blk_object_type' 'blk_object' in the block design.
B2T_DIS_0001	Error	object_type object is disabled in top instance but not in block design.
B2T_DIS_0002	Error	object_type object is disabled in block design but not in top instance.
B2T_DIS_0003	Error	Timing arc from from_object to to_object is disabled in top instance but not in block design.
B2T_DIS_0004	Error	Timing arc from from_object to to_object is disabled in block design but not in top instance.
B2T_EXC_0001	Error	Exception exception in top instance is missing in block design.
B2T_EXC_0002	Error	Exception exception in block design is missing in top instance.
B2T_EXC_0003	Error	Exception top_exception in top instance does not match with exception blk_exception in block design.
B2T_EXD_0001	Error	top_pin in top instance has input_or_output delay constraints that are missing in the block level.
B2T_EXD_0002	Error	blk_port in block design has input_or_output delay constraints that are missing in the top instance pin.
B2T_EXD_0003	Error	top_pin in top instance and blk_port in block design have different input_or_output delay constraints.
B2T_OPC_0001	Error	An operating condition is specified on instance object in top instance but not in block design.
B2T_OPC_0002	Error	An operating condition is specified on instance object in block design but not in top instance.

Rule ID	Severity	Message
B2T_OPC_0003	Error	The <i>object_type object</i> has incompatible operating condition specification in top instance and block design.
B2T_OPC_0004	Error	The top design and block design have incompatible operating condition analysis types.
B2T_TRL_0001	Warning	Port in the top design has input_transition/load constraint that is missing in the block-level port connected to it.
B2T_TRL_0002	Warning	Port in the block design has transition/load constraint that is missing in the top-level port connected to it.
B2T_TRL_0003	Info	The port in the top design and the port in the block design have different transition/load constraint.
B2T_UNC_0001	Error	Clock uncertainty applied to ' object_type"object ' in top instance is missing in block instance.
B2T_UNC_0002	Error	Clock uncertainty applied to 'object_type' object in block instance is missing in the top instance.
B2T_UNC_0003	Error	Clock uncertainty applied to ' top_object_type ' ' top_object ' in the top instance does not match the clock uncertainty setting on ' blk_object_type ' ' blk_object ' in the block instance.
B2T_UNC_0004	Error	Clock uncertainty applied between ' from_clock ' and ' to_clock ' in top instance is missing in block instance.
B2T_UNC_0005	Error	Clock uncertainty applied between ' from_clock ' and ' to_clock ' in block instance is missing in top instance.
B2T_UNC_0006	Error	Clock uncertainty applied between ' top_from_clock ' and ' top_to_clock ' in top instance does not match the clock uncertainty applied between ' blk_from_clock ' and ' blk_to_clock ' in block instance.

Rule: B2T_CAS_0001

Pin ***pin*** in top instance has case value that is missing in block design

Severity: Error

Description

The given pin has a case analysis value propagated or defined in the top-level constraints. However, there is no corresponding case value defined in the block-level constraints for that particular pin.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where a case value of 0 is set on the input pin of the inverter **inv4** at the top level.

```
set_case_analysis 0 [get_pins inv4/A]
```

This case value at the top level gets propagated to the downstream logic and eventually, a constant value of 1 is set to port **in3** of the block **b1**. Hence, the select pin of the MUX inside the block **b1** also gets a value of 1. However, there is no corresponding case value defined for the select pin of the MUX in the block-level constraints. A **B2T_CAS_0001** violation is issued due to the missing case value at the select pin of the MUX in the block-level constraints.

Top Level Netlist	Block Level Netlist
----------------------	---------------------

What Next

The "Violations Details" section will show the top-level pin name along with the propagated or the user-defined case value. To see the sources of the case value that propagated to the top-level pin, use the `report_case_details -to top_level_pin` command. For the example scenario, the command can be issued as follows:

```
## First you need to focus on the top level block
ptc_shell> current_design top
```

```
## Then use the debugging feature
ptc_shell> report_case_details -to b1/mux/S
```

```
*****
```

```
Report : case_details
```

```
Design : top
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

Properties	Value	Pin/Port
from user case	1	b1/mux/S (MUX21H)

Case fanin report:

Verbose Source Trace for pin/port b1/mux/S:

Path number: 1

Path	Ref #	Value	Properties	Pin/Port
------	-------	-------	------------	----------

	1			b1/mux/S (MUX21H)
	1	F()=A'		inv4/Z (IV)
	0	user case		inv4/A (IV)

The above `report_case_details` report for the top design shows that the case value 0 was specified by you at the pin **inv4/A** in the top-level constraints. This case value 0 propagates to the select pin of the MUX inside the block **b1** and a value of 1 is set on the pin **b1/mux/S**.

To check the missing case value from the block-level constraints, use the `report_case_details -to block_pin` command. For the example scenario, the command can be issued as follows:

```
## Next you need to focus on the block level instantiated inside top
ptc_shell> current_design block
```

```
ptc_shell> report_case_details -to mux/S
*****
```

```
Report : case_details
      -to
      -max_objects = 1000
```

```
Design : block
Scenario: default
Version: ...
Date    : ...
*****
```

Properties	Value	Pin/Port
------------	-------	----------

	x	mux/S (MUX21H)
--	---	----------------

Case fanin report:

Verbose Source Trace for pin/port mux/S: --

The above `report_case_details` for the block shows that there is no case value defined on the select pin of the MUX in the block-level constraints.

Fix Suggestion

To resolve the issue, add the proper case value for the particular pin in the block-level constraints or remove the case value setting from the top-level constraints. For the example scenario, the case value at the block level can be added as follows:

```
set_case_analysis 1 mux/S
```

See Also

[set_case_analysis](#)

[report_case_details](#)

[remove_case_analysis](#)

Rule: B2T_CAS_0002

Pin or port ***pin*** in block design has a case value that is missing in top instance.

Severity: Error

Description

The given pin or port in the block design has a case analysis value defined in the block-level constraints. However, there is no corresponding case value defined in the top-level constraints for that particular pin.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block instance in full chip timing analysis.

Possible Scenarios

Consider a scenario where a case value of 0 is set on the input port **in3** of the block **b1** in the block-level constraints.

```
set_case_analysis 0 [get_ports in3]
```

This case value at the block level gets propagated to the downstream logic and eventually, a constant value of 0 is set to the select pin of the MUX. However, there is no corresponding case value defined for the pin **b1/mux/S** in the top-level constraints. A **B2T_CAS_0002** violation is issued due to the missing case value at the select pin of the MUX in the top-level constraints.

Top Level Netlist	Block Level Netlist
----------------------	---------------------

What Next

The "Violations Details" section shows the block-level pin or port name along with the propagated or the user-defined case value. To see the sources of the case value that propagated to the block-level pin, use the `report_case_details -to block_level_pin` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> report_case_details -to mux/S

*****
Report : case_details
Design : block
Scenario: default
Version: ...
Date   : ...
*****

Properties          Value   Pin/Port
-----
from user case      0      mux/S (MUX21H)

Case fanin report:
Verbose Source Trace for pin/port mux/S:
Path number: 1
Path Ref #   Value   Properties          Pin/Port
-----
              0      mux/S (MUX21H)
              0      user case      in3 (in)
```

The above `report_case_details` report for the block design shows that the case value of 0 was specified by the user at the input port **in3** in the block level constraints. This case value of 0 propagates to the select pin of the MUX and a case value of 0 is set on the pin **mux/S**.

To check the missing case value from the top-level constraints, use the `report_case_details -to top_pin` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> current_design top

ptc_shell> report_case_details -to b1/mux/S
*****
Report : case_details
Design : top
Scenario: default
Version: ...
Date   : ...
*****
```

Properties	Value	Pin/Port
	x	b1/mux/S (MUX21H)

Case fanin report:
Verbose Source Trace for pin/port b1/mux/S: --

The above `report_case_details` report for the top design shows that there is no case value defined on the select pin of the MUX in the top-level constraints.

Fix Suggestion

To resolve the issue, add the proper case value for the particular pin in the top-level constraints or remove the case value setting from the block-level constraints. For the example scenario, the case value at the top level can be added as follows:

```
set_case_analysis 0 b1/mux/S
```

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[set_case_analysis](#)

[report_case_details](#)

[remove_case_analysis](#)

Rule: B2T_CAS_0003

Pin *pin* in top instance and corresponding pin or port *pin* in block design have different case values.

Severity: Error

Description

The given pin in the top instance has a case analysis value propagated or defined on it. However, the corresponding pin in the block design has a different case value.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in inaccurate analysis coverage or even incorrect behavior for the block or the full chip.

Possible Scenarios

Consider a scenario where a case value of 0 is set on the input pin of the inverter **inv4** at the top instance and a case value of 0 is set on the input port **in3** at the block level.

```
current_design top
set_case_analysis 0 [get_pins inv4/A]
current_design block
set_case_analysis 0 [get_ports in3]
```

The case value of 0 at the top level gets propagated to the downstream logic and eventually, a constant value of 1 is set to port **in3** of the block **b1**. Hence, a case value of 1 is set on the select pin of the MUX at the top level. However, the case value of 0 at the block level gets propagated to the downstream logic and a value of 0 is set on the select pin of the MUX. A **B2T_CAS_0003** violation is issued due to the mismatch in case values on the select pin of the MUX at the top instance and block design.

Top Level Netlist	Block Level Netlist
----------------------	---------------------

What Next

The "Violation Details" section will show the top-level pin name and the block-level pin name along with the propagated or the user-defined case values that do not match. To see the sources of the case value that propagated to the top-level pin, use the `report_case_details -to top_level_pin` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> current_design top

ptc_shell> report_case_details -to b1/mux/S

*****
Report : case_details
Design : top
Scenario: default
Version: ...
Date   : ...
*****
```


Properties	Value	Pin/Port
from user case	1	b1/mux/S (MUX21H)

Case fanin report:

Verbose Source Trace for pin/port b1/mux/S:

Path number: 1

Path Ref #	Value	Properties	Pin/Port
1			b1/mux/S (MUX21H)
1	F()=A'		inv4/Z (IV)
0	user case		inv4/A (IV)

The above `report_case_details` report for the top design shows that the case value of 0 was specified by the user at the pin **inv4/A** in the top-level constraints. This case value of 0 propagates through the logic and a case value of 1 is set on the pin **b1/mux/S**.

To check the sources of the case value to the block-level pin, use the `report_case_details -to block_pin` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> current_design block
```

```
ptc_shell> report_case_details -to mux/S
```

```
*****
Report : case_details
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

Properties	Value	Pin/Port
from user case	0	mux/S (MUX21H)

Case fanin report:

Verbose Source Trace for pin/port mux/S:

Path number: 1

Path Ref #	Value	Properties	Pin/Port
0			mux/S (MUX21H)
0	user case		in3 (in)

The above `report_case_details` report for the block shows that there is a case value of 0 set on the pin **mux/S**. The case value 0 set on the block-level pin **mux/S** does not match with the case value 1 set on the top-level pin **b1/mux/S**.

Fix Suggestion

To resolve the issue, the case values for both the top level design and block level design should be reviewed. The case values should be identical for the pin of interest both at the top level and block level.

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[set_case_analysis](#)

[report_case_details](#)

[remove_case_analysis](#)

Rule: B2T_CLK_0001

Clock **clock** in top instance has no matching clock in block design.

Severity: Error

Description

A clock signal originating from the top level is reaching the block. However, there is no corresponding clock defined in the block-level constraints matching the top-level clock.

Risk Associated with Violation

The block-level timing analysis results will be inconsistent from the top-level results. Some portions of the block level design might be unconstrained. Block-level timing closure might not lead to less effort at top-level timing closure and vice versa.

Possible Scenarios

There could be several cases leading to a B2T_CLK_0001 violation:

1. There is missing clock definition at the block level.

2. A top-level clock is launching data that reaches at least one block input. This input does not have an input delay relative to the top-level clock.
3. There is a related clock defined at the block level, but the path to some or all its register is blocked by an exception.

What Next

To match two clocks, constraint consistency not only looks at the clock themselves (period, waveform, sense, and so on.) but it also checks the objects driven by these clocks. The following objects are directly involved in the clock matching capabilities:

- Register clock pins
- Input delays
- Output delays

The "Violations Details" section reports the top-level clock name. If block register clock pins are identified, they are also reported in this section.

The possible scenarios leading to a **B2T_CLK_0001** violation is listed in the previous section. Here is how you can get more information in each case.

Scenario 1

In the first scenario, the mismatch is related to a missing clock definition in the block design. The following steps discuss a way to find out where a clock should be defined.

Top Clock	Period	Waveform	Missing at Pins
clk2	10.00	{0.00 5.00}	b1/zInv5/A, b1/zInv3/A

The easiest way to find out about a missing clock definition in the block constraints is to use the [analyze_unclocked_pins](#) command on the block itself.

```
## First you need to focus on the block itself
ptc_shell> current_design block

## Then use the debugging feature
ptc_shell> analyze_unclocked_pins
*****
Report : analyze_unclocked_pins
Design : block
Version: ...
Date   : ...
*****
Analyze Register Clock Pins that do not have clocks assigned
```

Summary	
Register Clock Pin Status	Number of Pins
Clocked	0
Unclocked	2
Case disabled clock pin	0
Register behavior disabled	0
Total register clock pins	2

Possible Startpoints for Missing Clock Definitions		
Pin/Port	Startpoints	Number unclocked Reason a Startpoint
clk	2	input port

The `analyze_unclocked_pins` report indicates that the reason for those two pins not being clocked is a missing clock source on port **clk**. This is also the reason for the clock mismatch in `compare_block_to_top`.

Scenario 2

If the input delay clock information is missing on the block, a **B2T_CLK_0003** violation appears. See the [B2T_CLK_0003](#) rule reference document to debug this case.

Scenario 3

Consider a block design where a clock is created on the relevant ports, and a related clock is propagating from the top-level set of constraints. If the clock network is blocked inside the block (in the block constraints), and if this constraint does not exist at the top level, some clock mismatches are expected. There are two cases to consider:

- Some registers relative to this particular clock are affected inside the block
- All registers relative to this particular clock are affected inside the block

If only some registers are affected, constraint consistency reports a **B2T_CLK_0001** violation:

Top Clock	Period	Waveform	Missing at Pins
clk2	10.00	{0.00 5.00}	b1/zInv3/A

Clocks at the top and block levels will not match unless all the endpoints register pin match. You can use the `analyze_unclocked_pins` command at the block level to see why

the clock is not matching. This is equivalent to understanding why the clock is blocked at pin **b1/zInv3/A** here. The following is an example report:

```
ptc_shell> analyze_unclocked_pins -verbose
*****
```

```
Report : analyze_unclocked_pins
        -verbose
```

```
Design : block
```

```
Version: ...
```

```
Date   : ...
```

```
*****
```

```
Analyze Register Clock Pins that do not have clocks assigned
```

Summary

Register Clock Pin Status	Number of Pins
Clocked	1
Unclocked	1
Case disabled clock pin	0
Register behavior disabled	0
Total register clock pins	2

Possible Startpoints for Missing Clock Definitions

Pin/Port Startpoints	Number unclocked	Reason a Startpoint
zInv3/A	1	set_clock_sense

```
zInv3/A
```

```
- fans out to 1 unclocked pins
  In fanin of ffg/CP
```

Clocks are blocked from register clock pins at the following locations:

Clock	Blocking Pin	Number unclocked	Reason
clk2	zInv3/A	1	
set_clock_sense			

```
Clock clk2 Blocking Locations (verbose)
```

Blocking Pin	Reason
zInv3/A	set_clock_sense

Verbose Output for Blocked Clocks:

Blocking Pin	Clock Pin
zInv3/A	ffg/CP

If all registers are affected in the block, constraint consistency reports two different clock violations. In addition to the **B2T_CLK_0001** violation, you also get a **B2T_CLK_0002** violation. See the [B2T_CLK_0002](#) rule reference document to get more information about debugging this violation.

Top Clock	Period	Waveform Missing at Pins
clk2	10.00	{0.00 5.00} b1/zInv5/A, b1/zInv3/A

Block Clock	Period	Waveform Missing at Pins
clk2	10.00	{0.00 5.00}

Fix Suggestion

Clock definitions and constraints affecting the clock network propagation should be reviewed and aligned between top-level constraints and block-level constraints.

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[analyze_unclocked_pins](#)

[B2T_CLK_0002](#)

[B2T_CLK_0003](#)

[get_ports](#)

Rule: B2T_CLK_0002

Clock **clock** defined in block design has no matching clock in top instance.

Severity: Error

Description

A clock signal is specified on a block port. However, no clock signal is reaching this block port through the top-level constraints.

Risk Associated with Violation

The block-level timing analysis results will be inconsistent from the top-level results. Some portions of top level design might be unconstrained. Block-level timing closure might not lead to less effort at top level timing closure and vice versa.

Possible Scenarios

Several scenarios can lead to a **B2T_CLK_0002** violation:

1. A clock exists in the top instance, but its propagation is blocked by a constraint before it reaches the block level port.
2. A clock definition is missing in the top level constraints.

What Next

To match two clocks, constraint consistency not only looks at the clock themselves (period, waveform, sense, and so on) but it also checks the objects driven by these clocks. These objects are directly involved in the clock matching capabilities:

- Register clock pins
- Input delays
- Output delays

The "Violations Details" section reports the block-level clock name. In the GUI, you can click on the clock name hyperlink and constraint consistency opens a SDC browser window, showing the clock definition itself. Another way to understand where the clock is defined at the block level is to use the `report_clocks` command.

```
ptc_shell> report_clock [get_clocks clk2_bis]
```

```
*****
Report   : clock
Design   : block
Scenario: default
Version  : ...
Date     : ...
*****
```

```
Attributes:
  p - Propagated clock
```

G - Generated clock
I - Inactive clock

Clock	Period	Waveform	Attrs	Sources
clk2_bis	10.00	{0 5}		{clk2}

From this report, you can see that the clock in the block constraints is called **clk2_bis** and it is defined on port **clk2**. Let us look at debugging leads for the various violation scenarios described in the previous section.

Scenario 1

In this scenario, the clock exists at the top level but its propagation is blocked before it can reach the relevant block port. In this case, the most appropriate technique is to use the [analyze_clock_networks](#) command on the top-level design.

```
ptc_shell> analyze_clock_networks -through [get_pins b1/clk2] \
          -traverse_disabled -style full -text_only
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
        -traverse_disabled
Design : top
Scenario: default
Version: ...
Date   : ...
*****
```

```
...
Full report for clock: clk2
```

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	clk2 (in)
1		P		ck2_i2/A (IV)
2		[N]	STOP#0	ck2_i2/Z (IV)
3		[N]		ck2_i22/A (IV)
4		[P]		ck2_i22/Z (IV)
5		[P]		b1/zInv3/A (IV)
6		[N]		b1/zInv3/Z (IV)
7		[N]		b1/zInv4/A (IV)
8		[P]		b1/zInv4/Z (IV)
9		[P]	REG	b1/ffg/CP (FD1)

```
Clock Blocking Constraints:
STOP#0 = set_clock_sense -stop
```



```
at pin: ck2_i2/Z
run.tcl, line 26
```

The report shows the clock propagation is blocked due to a `set_clock_sense -stop_propagation` command applied on pin **ck2_i2**.

Scenario 2

In the second scenario, there is no clock definition in the top-level constraint and the goal is to try to understand where the missing clock definition responsible for the current **B2T_CLK_0002** violation is. The most appropriate command in this case is the [analyze_unclocked_pins](#) command.

There could be several missing clock definitions in the top level constraints leading to several **B2T_CLK_0002** violations. The `analyze_unclocked_pins` command supports specifying a set of register clock pins to analyze. Because the rule itself does not report the register clock pins affected, we first have to deduce that from the design.

At the block level, we know the clock is defined on pin **clk2** and we can use this information as follow:

```
ptc_shell> set zPins [all_fanout -from [get_ports clk2]]
{"clk2", "zInv3/A", "zInv3/Z", "zInv4/A", "zInv4/Z", "ffg/CP"}

ptc_shell> set zClockPins [filter_collection ${zPins} -regexp
{is_clock_pin == true}]
{"ffg/CP"}
```

Now that we understand what register clock pins are affected by clock **clk2** at the block level, we can use this information in the top level design to narrow down the unclocked pins analysis.

```
ptc_shell> analyze_unclocked_pins [list b1/ffg/CP] -verbose

*****
Report : analyze_unclocked_pins
        -verbose
Design : top
Version: ...
Date   : ...
*****
Analyze Register Clock Pins that do not have clocks assigned
```

Summary	
Register Clock Pin Status	Number of Pins
Clocked	0
Unclocked	1
Case disabled clock pin	0

Register behavior disabled	0	

Total register clock pins	1	
Possible Startpoints for Missing Clock Definitions		
Pin/Port Startpoints	Number	Reason a Startpoint

clk2	1	input port
clk2		
- fans out to 1 unclocked pins		
In fanin of b1/ffg/CP		

The report above points to a missing clock definition located on port **clk2** in the top-level design.

Fix Suggestion

Clock definitions and constraints affecting the clock network propagation should be reviewed and aligned between top-level constraints and block-level constraints.

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[analyze_clock_networks](#)

[analyze_unclocked_pins](#)

[all_fanout](#)

[filter_collection](#)

Rule: B2T_CLK_0003

Clock **top_clock** in top instance does not match with clock **blk_clock** in block design.

Severity: Error

Description

A clock signal originating from the top-level constraints is reaching a block port with a clock definition. However, those two clocks have different periods or waveforms.

Risk Associated with Violation

The block-level timing analysis results will be inconsistent from the top-level results. Block-level timing closure might not lead to less effort at top-level timing closure and vice versa.

Possible Scenarios

These scenarios can lead to a **B2T_CLK_0003** violation:

1. The top-level and block-level clocks have different periods.
2. The top-level and block-level clocks have identical periods but their edges are different.

What Next

To match two clocks, constraint consistency looks at how the register clock pins are sensitized by each of the clocks (the top-level clock and the block-level clock). If the period, the sense, or the edge definitions do not match, a **B2T_CLK_0003** violation is issued.

Both scenarios can be analyzed using the same technique. When debugging the violation using the PrimeTime constraint consistency GUI, you can click on the hyperlinks for each of the clocks. This opens a dual constraint browser window allowing you to compare the two clock definitions.

The markers in each SDC file highlight the clock definition. The top-level constraints appear on the left-hand side of the dual constraint browser; the block-level constraints on the right-hand side of the dual constraint browser.

It is also possible to use the `report_clocks` command to achieve similar results. The command can be applied on both the top-level design and the block-level design. You can then compare the two clock definitions. Here is how to do this:

```
ptc_shell> current_design top
{"top"}
ptc_shell> report_clocks -attributes [get_clocks clk2]
*****
Report : clock
Design : top
Scenario: default
Version: ...
Date   : ...
*****
```

```
Attributes:
  p - Propagated clock
  G - Generated  clock
  I - Inactive   clock
```

Clock	Period	Waveform	Attrs	Sources
-------	--------	----------	-------	---------

```
-----
clk2                10.00    {0 7}                                {clk2}

ptc_shell> current_design block
{"block"}
ptc_shell> report_clocks -attributes [get_clocks clk2]
*****
Report : clock
Design : block
Scenario: default
Version: ...
Date   : ...
*****

Attributes:
  p - Propagated clock
  G - Generated  clock
  I - Inactive   clock

Clock      Period    Waveform      Attrs      Sources
-----
clk2       10.00     {0 5}                                {clk}
```

Both the period and the waveform are displayed in the report. It is easy to spot any differences in the clock definition.

Note: The presence of a **B2T_CLK_0003** violation can also trigger a [B2T_EXD_0001](#) violation under certain conditions. Fixing the **B2T_CLK_0003** violation should also address the related **B2T_EXD_0001** issues.

Fix Suggestion

Clock definitions for both top-level design and block-level design should be reviewed. They need to match for constraint consistency to declare them correct.

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[report_clock](#)

[B2T_EXD_0001](#)

Rule: B2T_CLK_0004

Pin **top_pin** has clock sense applied in top instance but the clock sense is not applied in block design.

Severity: Error

Description

The clock sense of the reported pin differs between top instance and block instance.

Risk Associated with Violation

The block level will not be analyzed under the conditions it was created. The clock sense applied in the top constraints will modify the clock networks in the sub-module.

Possible Scenarios

Consider a design where a clock with multiple senses (positive and negative) propagates into the block design (Figure 1). Such a scenario can lead to a **B2T_CLK_0004** violation if one of the clock senses is restricted in the top-level constraints.

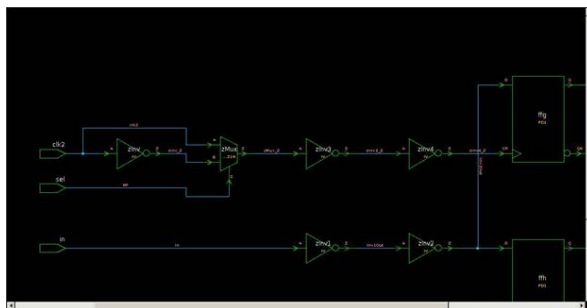


Figure 1: Multiple Clock Senses on the Output of zMux

Block constraints

```
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports
clk]
```

Top constraints

```
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
set_clock_sense -negative -clock [get_clocks clk2] [get_pins b1/zMux/Z]
```

What Next

At the block level, both the positive and negative sense of clock **clk2** propagate through the output of the multiplexer **zMux/Z**. On the top instance however, the positive sense is

blocked by the [set_clock_sense](#) command. As a consequence, only the negative clock sense of clock **clk2** propagates through the output of the multiplexer. This causes a **B2T_CLK_0004** violation to be issued.

You can get more information on the problem with the [analyze_clock_networks](#) command. Use the reported pin to tailor your report. The relevant information is highlighted in red in the following report.

```
ptc_shell> current_design block
{"block"}
ptc_shell> analyze_clock_networks -through zMux/Z -style full
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : blockScenario: default
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
  P,N - positive, negative
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: clk2_bis - 2 partial path branches

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0	S1	P	source	clk2 (in)
1		P		zInv/A (IV)
2		N		zInv/Z (IV)
3		N		zMux/B (MUX21H)
4	E1	P,N		zMux/Z (MUX21H)
5		P,N		zInv3/A (IV)
6		P,N		zInv3/Z (IV)
7		P,N		zInv4/A (IV)
8		P,N		zInv4/Z (IV)
9		P,N	REG	ffg/CP (FD1)

Branch 1: from branch 0 reconverges to branch 0

Level	Branch Info	Sense	Notes	Port/Pin
1		P		zMux/A (MUX21H)

```
ptc_shell> current_design B2T_CLK_0004
{"B2T_CLK_0004"}
ptc_shell> analyze_clock_networks -through b1/zMux/Z -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : B2T_CLK_0004
Scenario: default
Version: ...
Date    : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: clk2 - 2 partial path branches

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	clk2 (in)
1		P		ck2_i2/A (IV)
2		N		ck2_i2/Z (IV)
3		N		ck2_i22/A (IV)
4	S1	P		ck2_i22/Z (IV)
5		P		b1/zInv/A (IV)
6		N		b1/zInv/Z (IV)
7		N		b1/zMux/B (MUX21H)
8	E1	N	SENSE#0	b1/zMux/Z (MUX21H)
9		N		b1/zInv3/A (IV)
10		P		b1/zInv3/Z (IV)
11		P		b1/zInv4/A (IV)
12		N		b1/zInv4/Z (IV)
13		N	REG	b1/ffg/CP (FD1)

Branch 1: from branch 0 reconverges to branch 0

Level	Branch Info	Sense	Notes	Port/Pin
5		P		b1/zMux/A (MUX21H)

Fix Suggestion

Constraints affecting the clock propagation for both top-level design and block-level design should be reviewed. In the example discussed above, there are two possible ways to solve the problem:

- Remove the clock sense restriction in the top design.
- Add the same restriction on the block design.

See Also

[analyze_clock_networks](#)

[set_clock_sense](#)

[B2T_CLK_0005](#)

Rule: B2T_CLK_0005

Pin **block_pin** has clock sense applied in block design but the clock sense is not applied in top instance.

Severity: Error

Description

The clock sense of the reported pin differs between block design and top instance.

Risk Associated with Violation

The block level will not be analyzed under the conditions it was created. The missing clock sense restriction from the top constraints will modify the clock networks analysis in the block design.

Possible Scenarios

Consider a design where a clock with multiple senses (positive and negative) propagates into the block design (Figure 1). Such a scenario can lead to a **B2T_CLK_0005** violation if one of the clock senses is restricted in the block design constraints.

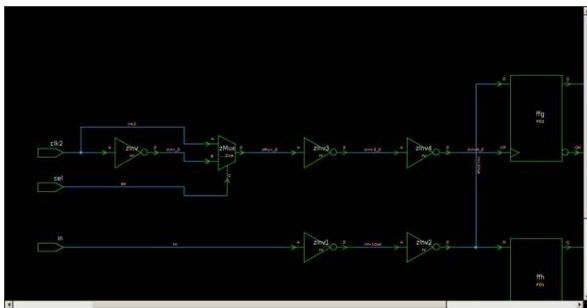


Figure 1: Multiple Clock Senses on the Output of zMux

Block constraints


```
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports
  clk]
set_clock_sense -negative -clock [get_clocks clk2] [get_pins zMux/Z]
```

Top constraints

```
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
```

What Next

At the top level, both the positive and negative sense of clock **clk2** propagate through the output of the multiplexer **b1/zMux/Z**. In the block design however, the positive sense is blocked by the [set_clock_sense](#) command. As a consequence, only the negative clock sense of clock **clk2** propagates through the output of the multiplexer. This causes a **B2T_CLK_0005** violation to be issued.

You can get more information on the problem with the [analyze_clock_networks](#) command. Use the reported pin to tailor your report. The relevant information is highlighted in red in the following report.

```
ptc_shell> ptc_shell> current_design block
{"block"}
ptc_shell> analyze_clock_networks -through [get_pins zMux/Z] -style full

*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : block
Scenario: default
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
  P,N - positive, negative
Clock Network End Type Abbreviations:
  REG - register

Full report for clock: clk2_bis - 2 partial path branches

Branch 0:
      Branch
Level  Info      Sense Notes      Port/Pin
-----
      0  S1       P      source      clk2 (in)
      1              P              zInv/A (IV)
```

Chapter 2: Block-To-Top Rules

Rule: B2T_CLK_0005

```

      2          N          zInv/Z (IV)
      3          N          zMux/B (MUX21H)
      4  E1        P,N      SENSE#0  zMux/Z (MUX21H)
      5          P,N          zInv3/A (IV)
      6          P,N          zInv3/Z (IV)
      7          P,N          zInv4/A (IV)
      8          P,N          zInv4/Z (IV)
      9          P,N      REG      ffg/CP (FD1)

```

Branch 1: from branch 0 reconverges to branch 0

Branch

Level	Info	Sense	Notes	Port/Pin
-------	------	-------	-------	----------

1		P		zMux/A (MUX21H)
---	--	---	--	-----------------

Clock Blocking Constraints:

SENSE#0 = set_clock_sense

at pin: zMux/Z

run.tcl, line 40

```
ptc_shell> current_design B2T_CLK_0005
```

```
{"B2T_CLK_0005"}
```

```
ptc_shell> analyze_clock_networks -through [get_pins b1/zMux/Z] -style
full
```

```
*****
```

Report : analyze_clock_networks

-max_endpoints=1000

-style full

-end_types {register port clock_source}

Design : B2T_CLK_0005

Scenario: default

Version: ...

Date : ...

```
*****
```

Clock Sense Abbreviations:

P - positive

N - negative

P,N - positive, negative

Clock Network End Type Abbreviations:

REG - register

Full report for clock: clk2 - 2 partial path branches

Branch 0:

Branch

Level	Info	Sense	Notes	Port/Pin
-------	------	-------	-------	----------

0		P	source	clk2 (in)
1		P		ck2_i2/A (IV)
2		N		ck2_i2/Z (IV)
3		N		ck2_i22/A (IV)
4	S1	P		ck2_i22/Z (IV)

5		P		b1/zInv/A (IV)
6		N		b1/zInv/Z (IV)
7		N		b1/zMux/B (MUX21H)
8	E1	P,N		b1/zMux/Z (MUX21H)
9		P,N		b1/zInv3/A (IV)
10		P,N		b1/zInv3/Z (IV)
11		P,N		b1/zInv4/A (IV)
12		P,N		b1/zInv4/Z (IV)
13		P,N	REG	b1/ffg/CP (FD1)

Branch 1: from branch 0 reconverges to branch 0

Level	Branch	Info	Sense	Notes	Port/Pin
5			P		b1/zMux/A (MUX21H)

Fix Suggestion

Constraints affecting the clock propagation for both top-level design and block-level design should be reviewed. In the example discussed above, there are two possible ways to solve the problem:

- Remove the clock sense restriction in the block design.
- Add the same restriction on the top instance.

See Also

[analyze_clock_networks](#)

[set_clock_sense](#)

[B2T_CLK_0004](#)

Rule: B2T_CLK_0006

Clock sense applied to pin **top_pin** mismatches with clock sense applied to pin **block_pin**.

Severity: Error

Description

The clock sense of the reported pin differs between block design and top instance.

Risk Associated with Violation

The block level will not be analyzed under the conditions it was created. The differing clock senses between top and block constraints will modify the clock networks analysis in the block design. This can result in incorrect timing analysis.

Possible Scenarios

Consider a design where a clock with multiple senses (positive and negative) propagates inside the block design (Figure 1). Such a scenario can lead to a **B2T_CLK_0006** violation if the clock sense on the output of the multiplexer is set differently between the top- and block-level constraints.

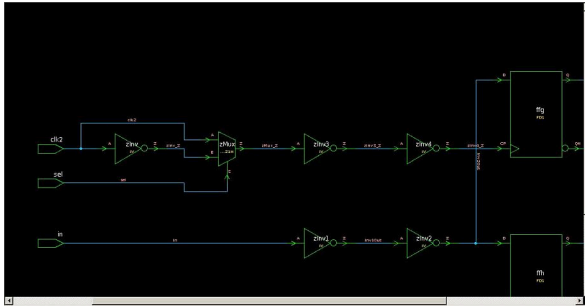


Figure 1: multiple clock senses on the output of zMux

The constraints for this example have been set, as shown in Figure 2. The two markers show the difference in constraints leading to the generation of a **B2T_CLK_0006** violation.

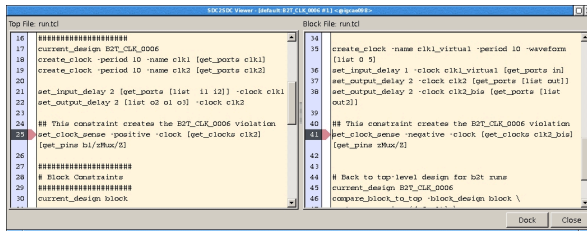


Figure 2: Constraints set

What Next

At the top level, only the positive sense of clock **clk2** is allowed to propagate through the output of the multiplexer **b1/zMux/Z**. In the block design however, the positive sense is blocked by the [set_clock_sense](#) command and only the negative sense is allowed to go through. This creates a discrepancy in the clock network between top and block designs and results in a **B2T_CLK_0006** violation to be issued.

You can get more information on the problem with the [analyze_clock_networks](#) command. Use the reported pin to tailor your report. The relevant information is highlighted in red in the following report.

```
ptc_shell> current_design block
{"block"}
ptc_shell> analyze_clock_networks -through [get_pins zMux/Z] -style full
```

```

*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : block
Scenario: default
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
  P,N - positive, negative
Clock Network End Type Abbreviations:
  REG - register

Full report for clock: clk2_bis - 2 partial path branches

Branch 0:
  Branch
Level  Info      Sense Notes      Port/Pin
-----
   0  S1         P      source      clk2 (in)
   1             P              zInv/A (IV)
   2             N              zInv/Z (IV)
   3             N              zMux/B (MUX21H)
   4  E1         N      SENSE#0      zMux/Z (MUX21H)
   5             N              zInv3/A (IV)
   6             P              zInv3/Z (IV)
   7             P              zInv4/A (IV)
   8             N              zInv4/Z (IV)
   9             N      REG          ffg/CP (FD1)
Branch 1: from branch 0 reconverges to branch 0
  Branch
Level  Info      Sense Notes      Port/Pin
-----
   1             P              zMux/A (MUX21H)

Clock Blocking Constraints:
  SENSE#0 = set_clock_sense
    at pin: zMux/Z
    run.tcl, line 41

ptc_shell> current_design B2T_CLK_0006
{"B2T_CLK_0006"}

ptc_shell> analyze_clock_networks -through [get_pins b1/zMux/Z] -style
full
*****

```

```

Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : B2T_CLK_0006
Scenario: default
Version: ...
Date : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
  P,N - positive, negative
Clock Network End Type Abbreviations:
  REG - register

Full report for clock: clk2 - 2 partial path branches
Branch 0:
  Branch
Level  Info      Sense Notes      Port/Pin
-----
   0             P      source      clk2 (in)
   1             P              ck2_i2/A (IV)
   2             N              ck2_i2/Z (IV)
   3             N             ck2_i22/A (IV)
   4  S1          P             ck2_i22/Z (IV)
   5             P             b1/zInv/A (IV)
   6             N             b1/zInv/Z (IV)
   7             N             b1/zMux/B (MUX21H)
   8  E1          P      SENSE#0      b1/zMux/Z (MUX21H)
   9             P             b1/zInv3/A (IV)
  10             N             b1/zInv3/Z (IV)
  11             N             b1/zInv4/A (IV)
  12             P             b1/zInv4/Z (IV)
  13             P      REG           b1/ffg/CP (FD1)

Branch 1: from branch 0 reconverges to branch 0
  Branch
Level  Info      Sense Notes      Port/Pin
-----
   5             P             b1/zMux/A (MUX21H)

Clock Blocking Constraints:
  SENSE#0 = set_clock_sense
            at pin: b1/zMux/Z
            run.tcl, line 25

```

Fix Suggestion

Constraints affecting the clock propagation for both top-level design and block-level design should be reviewed. In the example discussed above the clock sense should be consistent between block and top constraints. Failing to fix this might result in incorrect timing results.

See Also

[analyze_clock_networks](#)

[set_clock_sense](#)

Rule: B2T_CLK_0007

arc_sense arc from *from_top_pin* to *to_top_pin* has clock sense applied in top instance but is missing in block design.

Severity: Error

Description

The clock sense associated with the reported top instance timing arc does not exist in the block design.

Risk Associated with Violation

The block level will not be analyzed under the conditions it was created. The clock sense applied in the top constraints will modify the clock networks in the sub-module.

Possible Scenarios

Consider a design where a clock with multiple senses (positive and negative) propagates into the block design (Figure 1) at the output of the multiplexer **zMux**. If the clock propagation is blocked on one the multiplexer timing arcs in the top-level constraints, a **B2T_CLK_0007** violation is created.

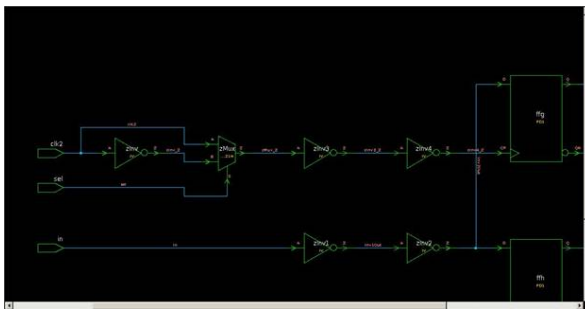


Figure 1: multiple clock senses on the output of zMux

The constraints set on the top level are shown in Figure 2. At the block level, no restriction on clock sense propagation is set.

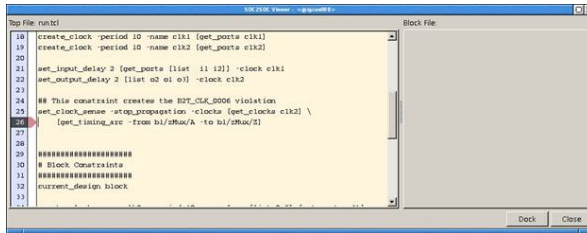


Figure 2: Constraints set

What Next

At the block level, both the positive sense and the negative sense of clock **clk2** propagate through the output of the multiplexer **zMux/Z**. On the top instance however, the clock propagation is blocked by the [set_clock_sense](#) command affecting the timing arc **A->Z** of the MUX gate. As a consequence, only the negative clock sense of clock **clk2** propagates through the output of the multiplexer (through timing arc **B->Z**). This causes a **B2T_CLK_0007** violation to be issued.

You can get more information on the problem with the [analyze_clock_networks](#) command. Use the reported **-to** pin to tailor your report. The relevant information is highlighted in red in the following report.

```
ptc_shell> current_design block
{"block"}
ptc_shell> analyze_clock_networks -through [get_pins zMux/Z] -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : block
Scenario: default
Version: ...
Date : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
  P,N - positive, negative
Clock Network End Type Abbreviations:
  REG - register

Full report for clock: clk2_bis - 2 partial path branches
Branch 0:
      Branch
Level  Info      Sense Notes      Port/Pin
-----
```


Chapter 2: Block-To-Top Rules

Rule: B2T_CLK_0007

```

0  S1      P      source      clk2 (in)
1          P          zInv/A (IV)
2          N          zInv/Z (IV)
3          N          zMux/B (MUX21H)
4  E1      P,N      zMux/Z (MUX21H)
5          P,N      zInv3/A (IV)
6          P,N      zInv3/Z (IV)
7          P,N      zInv4/A (IV)
8          P,N      zInv4/Z (IV)
9          P,N      REG      ffg/CP (FD1)

```

Branch 1: from branch 0 reconverges to branch 0

Level	Info	Sense	Notes	Port/Pin
1		P		zMux/A (MUX21H)

```

ptc_shell> current_design B2T_CLK_0007
{"B2T_CLK_0007"}
ptc_shell> analyze_clock_networks -through [get_pins b1/zMux/Z] -style
full

```

```

*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : B2T_CLK_0007
Scenario: default
Version: ...
Date : ...
*****

```

Clock Sense Abbreviations:

P - positive
N - negative

Clock Network End Type Abbreviations:

REG - register

Full report for clock: clk2 - 2 partial path branches

Branch 0:

Level	Info	Sense	Notes	Port/Pin
0		P	source	clk2 (in)
1		P		ck2_i2/A (IV)
2		N		ck2_i2/Z (IV)
3		N		ck2_i22/A (IV)
4	S1	P		ck2_i22/Z (IV)
5		P		b1/zInv/A (IV)
6		N		b1/zInv/Z (IV)
7		N		b1/zMux/B (MUX21H)

8	N		b1/zMux/Z (MUX21H)
9	N		b1/zInv3/A (IV)
10	P		b1/zInv3/Z (IV)
11	P		b1/zInv4/A (IV)
12	N		b1/zInv4/Z (IV)
13	N	REG	b1/ffg/CP (FD1)

Branch 1: from branch 0

Level	Info	Sense	Notes	Port/Pin
5		P		b1/zMux/A (MUX21H)

Note how the sections highlighted in red differ. At the block level, you can see that both the positive and negative clock senses propagate through **zMux/A**. Also note how at the block level, the clock network diverges (S1 label) and then reconverges (E1 label) on the output of the multiplexer. With the top-level constraint however, the clock network cannot reconverge (not E1 label) because the timing arc has a `clock_sense` restriction applied.

Fix Suggestion

Constraints affecting the clock propagation for both the top-level design and block-level design should be reviewed. In the example discussed above, there are two possible solutions:

- Remove the clock sense restriction in the top design
- Add the same restriction on the block design

See Also

[analyze_clock_networks](#)

[set_clock_sense](#)

Rule: B2T_CLK_0008

arc_sense arc from *from_block_pin* to *to_block_pin* has clock sense applied in block design but is missing in top instance.

Severity: Error

Description

The clock sense associated with the reported block design timing arc does not exist in the top instance constraints.

Risk Associated with Violation

The block level will not be analyzed under the conditions it was created in the full chip context. The clock sense applied in the block design constraints will modify the clock networks in the sub-module.

Possible Scenarios

Consider a design where a clock with multiple senses (positive and negative) propagates into the block design (Figure 1) at the output of the MUX **zMux**. If the clock propagation is blocked on one the MUX timing arcs in the block-level constraints, a **B2T_CLK_0008** violation is created.

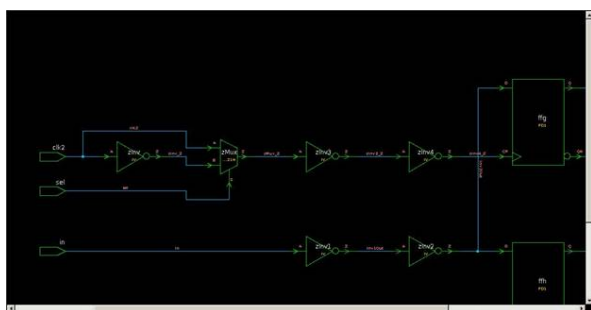


Figure 1: multiple clock senses on the output of zMux

The constraints applied at the block level are shown in Figure 2. In the top instance, no restriction on clock sense propagation is set.

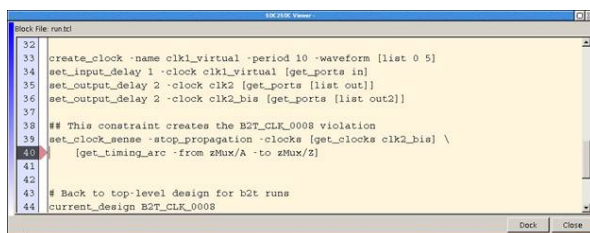


Figure 2: Rule-Violating Block-Level Constraint

What Next

In the top instance, both the positive and negative sense of clock **clk2** propagate through the output of the MUX **zMux/Z**. At the block level however, the clock propagation is blocked by the **set_clock_sense** command affecting the timing arc **A->Z** of the MUX gate. As a consequence, only the negative clock sense of clock **clk2** propagates through the output of the MUX (through timing arc **B->Z**). This causes a **B2T_CLK_0008** violation to be issued.

You can get more information on the problem with the **analyze_clock_networks** command. Use the reported **-to** pin to tailor your report. The relevant information is highlighted in red in the following report.

```
ptc_shell> current_design block
{"block"}
ptc_shell> analyze_clock_networks -through [get_pins zMux/Z] -style full

*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : block
Scenario: default
Version: ...
Date   : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
  P,N - positive, negative
Clock Network End Type Abbreviations:
  REG - register

Full report for clock: clk2_bis - 2 partial path branches
Branch 0:
  Branch
Level  Info      Sense Notes      Port/Pin
-----
  0  S1          P      source      clk2 (in)
  1          P              zInv/A (IV)
  2          N              zInv/Z (IV)
  3          N              zMux/B (MUX21H)
  4          N              zMux/Z (MUX21H)
  5          N              zInv3/A (IV)
  6          P              zInv3/Z (IV)
  7          P              zInv4/A (IV)
  8          N              zInv4/Z (IV)
  9          N      REG      ffg/CP (FD1)

Branch 1: from branch 0 reconverges to branch 0
  Branch
Level  Info      Sense Notes      Port/Pin
-----
  1          P              zMux/A (MUX21H)

ptc_shell> current_design B2T_CLK_0008
{"B2T_CLK_0007"}
```

```
ptc_shell> analyze_clock_networks -through [get_pins b1/zMux/Z] -style
full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : B2T_CLK_0007
Scenario: default
Version: ...
Date   : ...
*****
```

```
Clock Sense Abbreviations:
  P - positive
  N - negative
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: clk2 - 2 partial path branches

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	clk2 (in)
1		P		ck2_i2/A (IV)
2		N		ck2_i2/Z (IV)
3		N		ck2_i22/A (IV)
4	S1	P		ck2_i22/Z (IV)
5		P		b1/zInv/A (IV)
6		N		b1/zInv/Z (IV)
7		N		b1/zMux/B (MUX21H)
8	E1	P,N		b1/zMux/Z (MUX21H)
9		P,N		b1/zInv3/A (IV)
10		P,N		b1/zInv3/Z (IV)
11		P,N		b1/zInv4/A (IV)
12		P,N		b1/zInv4/Z (IV)
13		P,N	REG	b1/ffg/CP (FD1)

Branch 1: from branch 0

Level	Branch Info	Sense	Notes	Port/Pin
5		P		b1/zMux/A (MUX21H)

Note how the sections highlighted in red color differ. At the top level, you can see that both the positive and negative clock senses propagate through **zMux/A**. Also note how at the top level, the clock network diverges (S1 label) and then reconverges (E1 label) on the output of the MUX. With the block constraints however, the clock network cannot reconverge (not E1 label) because the timing arc has a **clock_sense** restriction applied.

Constraints affecting the clock propagation for both top-level design and block-level design should be reviewed. In the example discussed above, there are two possible solutions:

- ## See Also

set clock sense

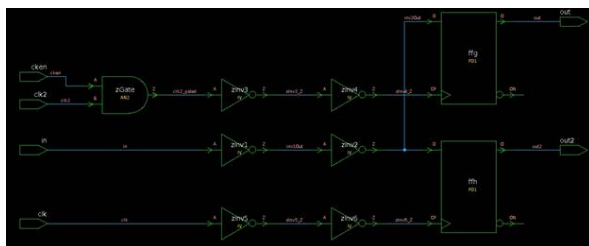
Clock gating check applied to **object_type object** in top instance is missing in block design.

Description

Risk Associated with Violation

Possible Scenarios

Case 1: Consider a design where a clock gating structure (**zGate**) is defined in a block as shown in Figure 1:



PrimeTime Constraint Consistency Rules

R-2020.09

A clock gating check is applied on the output **Z** of gate **zGate**. If in the block design a constraint is forcing a constant value on the port **cken**, the clock gating check will not exist anymore in the block. However, if that same constraint does not propagate through the top instance logic, a discrepancy between the clock gating analysis at the block design and the one at the top instance exists and is reported through a **B2T_CLK_0010** violation.

Case 2: Manually forcing the creation of clock gating checks on complex cells using the [set_clock_gating_check](#) command can also lead to this violation. For example, if the top design specifies a clock gating check to be applied on a particular cell, but the block design does not also have the same constraint, there will be a discrepancy. Such a difference is reported through a **B2T_CLK_0010** instance violation.

Example of top instance constraint leading to this violation:

```
set_clock_gating_check -setup 0.2 -hold 0.4 [get_cells b1/zGate]
report_clock_gating_check
```

Case 3: Manually removing clock gating checks from the block level design without removing the equivalent check in the top instance with the [remove_clock_gating_check](#) command can create a **B2T_CLK_0010** violation.

What Next

The "Violation Details" section provides more information related to the affected checks. Here is an example of such a report for Case 2:

Violation Details					
Clock Gating Setting			Source	Impacted checks	
	Setup	Hold	run.tcl, line 31	enable pin	clock pin
Rise	0.20	0.40		b1/zGate/A	b1/zGate/B
Fall	0.20	0.40			
check level: --					

Figure 2: Violation Details for Case 2

The various clock gating checks is reported using the [report_clock_gating_check](#) command. An example report is shown below; it corresponds to Case 1.

Clock gating checks inferred in the top instance:

```
ptc_shell> current_design B2T_CLK_0010
{"B2T_CLK_0010"}

ptc_shell> report_clock_gating_check -nosplit
*****
Report : clock_gating_check
```

```
Design : B2T_CLK_0010
Scenario: default
Version: ...
Date : ...
*****
```

Clock gating checks:

Cell	EnablePin	ClockPin	High/Low	Clock	Attr	User
Settings						

b1/zGate	A	B	High		I	
----------	---	---	------	--	---	--

Attr: I:auto inferred, L:library cell defined

Clocks gating checks inferred at the block level:

```
ptc_shell> current_design block
{"block"}
ptc_shell> report_clock_gating_check -nosplit
*****
Report : clock_gating_check
Design : block
Scenario: default
Version: ...
Date : ...
*****
```

Clock gating checks:

Cell	EnablePin	ClockPin	High/Low	Clock	Attr	User
Settings						

Attr: I:auto inferred, L:library cell defined

Note that in Case 1, a [B2T_CAS_0002](#) violation would also be present in the design. This should be investigated first, before working on any **B2T_CLK_0010** violation.

Fix Suggestion

Constraints affecting the clock gating checks for both top-level design and block-level design should be reviewed. In the various cases presented, here is how to fix the issue:

Case 1: Address the case value discrepancy first (the **B2T_CAS_0002** violation) and it will automatically fix the clock gating check problem (the **B2T_CLK_0010** violation).

Case 2: Review the clock gating check definition at the top instance and make sure it is correct and intended. If it is intended, apply the same constraint on the block level. If it is not intended, remove this constraint from the top instance constraints set.

Case 3: Review the reason behind removing the clock gating check constraint in the block design. If it is justified, perform the same at the top instance. If the clock gating check

See Also

report_clock_gating_check

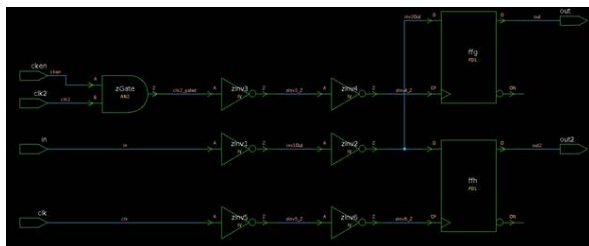
Clock gating check applied to ***object_type object*** in block design is missing in top instance.

Description

Risk Associated with Violation

Possible Scenarios

Case 1: Consider a design where a clock gating structure (zGate) is defined in a block as shown in Figure 1:



A clock gating check is applied on the output **Z** of gate **zGate** at the block level. If, in the top instance, a constraint is forcing a constant value on the block pin **cken**, the clock gating check will not exist anymore on the output of **zGate**. A discrepancy between the

clock gating analysis at the block design and the one at the top instance exists and is reported through a **B2T_CLK_0011** violation.

Case 2: Manually forcing the creation of clock gating checks on complex cells using the [set_clock_gating_check](#) command can also lead to this violation. For example, if the block design specifies a clock gating check to be applied on a particular cell, but the top instance does not also have the same constraint, there will be a discrepancy. Such a difference is reported through a **B2T_CLK_0011** violation.

Case 3: Manually removing clock gating checks from the top instance design without removing the equivalent check in the block design with the [remove_clock_gating_check](#) command can create a **B2T_CLK_0011** violation.

What Next

The "Violation Details" section describes the checks affected by this violation. Figure 2 is an example for Case 2.

Violation Details					
Clock Gating Setting			Source	Impacted checks	
	Setup	Hold	run.tcl, line 49	enable pin	clock pin
Rise	0.20	0.40		zGate/A	zGate/B
Fall	0.20	0.40			
check level: --					

Figure 2: Violation Details for Case 2

The various clock gating checks can be reported using the [report_clock_gating_check](#) command. An example report is shown below; it corresponds to Case 1.

Clock gating checks inferred in the top instance:

```
ptc_shell> current_design B2T_CLK_0010
{"B2T_CLK_0010"}

ptc_shell> report_clock_gating_check -nosplit
*****
Report : clock_gating_check
Design : B2T_CLK_0010
Scenario: default
Version: ...
Date   : ...
*****

Clock gating checks:
Cell      EnablePin ClockPin  High/Low  Clock      Attr  User
Settings
-----
```

Attr: I:auto inferred, L:library cell defined

Clocks gating checks inferred at the block level:

```
ptc_shell> current_design block
{"block"}
ptc_shell> report_clock_gating_check -nosplit
*****
Report : clock_gating_check
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

```
Clock gating checks:
Cell          EnablePin ClockPin  High/Low  Clock          Attr  User
Settings
-----
zGate          A          B          High          I
```

Attr: I:auto inferred, L:library cell defined

Note that in the Case 1, a [B2T_CAS_0001](#) violation would also be present in the design. This should be investigated first, before working on any **B2T_CLK_0011** violation.

Fix Suggestion

Constraints affecting the clock gating checks for both top-level design and block-level design should be reviewed. In the various cases presented, here is how to fix the issue:

Case 1: Address the case value discrepancy first (the **B2T_CAS_0001** violation) and it will automatically fix the clock gating check problem (the **B2T_CLK_0011** violation).

Case 2: Review the clock gating check definition at the top instance and make sure it is correct and intended. If it is intended, apply the same constraint on the block level. If it is not intended, remove this constraint from the top instance constraints set.

Case 3: Review the reason behind removing the clock gating check constraint in the block design. If it is justified, perform the same at the top instance. If the clock gating check should be in place, remove the command affecting the clock gating check from the block design.

See Also

[set_clock_gating_check](#)

[remove_clock_gating_check](#)

[report_clock_gating_check](#)

Rule: B2T_CLK_0012

Clock gating check applied to **top_object_type top_object** in top instance mismatched with clock gating check applied to **blk_object_type blk_object** in block design.

Severity: Error

Description

The clock gating check occurs in both the top instance and in block design but the values mismatch.

Risk Associated with Violation

The block level will not be analyzed under the conditions it was created in the full chip context. The clock gating check which is performed at the top instance will be different than the one performed in the block level context.

Possible Scenarios

Consider a design with a clock gating structure. Designers can decide/override the various parameters for the clock gating check using the [set_clock_gating_check](#) command. Using different values at the top instance and at the block level might trigger a **B2T_CLK_0012** violation. There are two cases to consider:

Case 1: Manually setting different values of setup/hold between top instance and block design. Below is an example of such settings:

```
current_design B2T_CLK_0012
set_clock_gating_check -setup 0.2 -hold 0.4 [get_cells b1/zGate]
current_design block
set_clock_gating_check -setup 0.3 -hold 0.4 [get_cells zGate]
```

Violation 1: Clock gating check applied to 'pin' 'b1/zGateA' in top instance does not match with clock gating check applied to 'pin' 'zGateA' in block design.

Top Clock Gating Setting			Block Clock Gating Setting			Impacted Top and Block Checks			
Top Source			Block Source			Top		Block	
Setup	Hold	pin	Setup	Hold	pin	enable	clock	enable	clock
Rise	0.20	0.40	Rise	0.30	0.50	b1/zGateA	b1/zGateB	zGateA	zGateB
Fall	0.20	0.40	Fall	0.30	0.50				
check level: --			check level: --						

Case 2: Manually setting different level values for the clock gating checks between the block-level design and the top instance. Below is an example of such setting:

```
current_design B2T_CLK_0012
set_clock_gating_check -high [get_cells b1/zGate]
current_design block
set_clock_gating_check -low [get_cells zGate]
```

Violation 1: Clock gating check applied to 'pin' 'b1/zGateA' in top instance does not match with clock gating check applied to 'pin' 'zGateA' in block design.

Violation Details

Top Clock Gating Setting	Top Source	Block Clock Gating Setting	Block Source	Impacted Top and Block Checks																												
<table><tr><th>Setup</th><th>Hold</th></tr><tr><td>Rise</td><td>--</td></tr><tr><td>Fall</td><td>--</td></tr><tr><td>check level: low</td><td></td></tr></table>	Setup	Hold	Rise	--	Fall	--	check level: low		run.tcl, line 30	<table><tr><th>Setup</th><th>Hold</th></tr><tr><td>Rise</td><td>--</td></tr><tr><td>Fall</td><td>--</td></tr><tr><td>check level: high</td><td></td></tr></table>	Setup	Hold	Rise	--	Fall	--	check level: high		run.tcl, line 49	<table><tr><th colspan="2">Top</th><th colspan="2">Block</th></tr><tr><td>enable pin</td><td>clock pin</td><td>enable pin</td><td>clock pin</td></tr><tr><td>b1/zGateA</td><td>b1/zGateB</td><td>zGateA</td><td>zGateB</td></tr></table>	Top		Block		enable pin	clock pin	enable pin	clock pin	b1/zGateA	b1/zGateB	zGateA	zGateB
Setup	Hold																															
Rise	--																															
Fall	--																															
check level: low																																
Setup	Hold																															
Rise	--																															
Fall	--																															
check level: high																																
Top		Block																														
enable pin	clock pin	enable pin	clock pin																													
b1/zGateA	b1/zGateB	zGateA	zGateB																													

What Next

The "Violation Details" section of the Info Pane window reports the current differences leading to the **B2T_CLK_0012** violation. In addition, you can get some information related to the various clock gating checks in the design using the [report_clock_gating_check](#) command. An example report is shown below; it corresponds to Case 1.

Clock gating checks inferred in the top instance:

```
ptc_shell> current_design B2T_CLK_0012
{"B2T_CLK_0012"}
ptc_shell> report_clock_gating_check
*****
Report : clock_gating_check
Design : B2T_CLK_0010
Scenario: default
Version: ...
Date : ...
*****
```

```
Clock gating checks:
Cell          EnablePin ClockPin  High/Low  Clock          Attr  User
Settings
-----
b1/zGate      A          B          High          I      1
```

Attr: I:auto inferred, L:library cell defined

```
User clock gating settings:
          Rise          Fall
ID  Object      Setup  Hold      Setup  Hold      High/Low      File/Line
-----
1   b1/zGate/A  0.20   0.40   0.20   0.40   --          run.tcl, line 30
```

Clocks gating checks inferred at the block level:

```
ptc_shell> current_design block
{"block"}
ptc_shell> report_clock_gating_check
*****
Report : clock_gating_check
Design : block
Scenario: default
```

```
Version: ...
Date   : ...
*****

Clock gating checks:
Cell      EnablePin ClockPin  High/Low  Clock      Attr      User
Settings
-----
zGate      A          B          High      Clock      I         1

Attr: I:auto inferred, L:library cell defined

User clock gating settings:
              Rise      Fall
ID  Object    Setup  Hold  Setup  Hold  High/Low  File/Line
-----
1   zGate/A   0.30   0.50   0.30   0.50   --        run.tcl, line 49
```

Fix Suggestion

Constraints affecting the clock gating checks for both top-level design and block-level design should be reviewed. In general, they need to be identical.

See Also

[set_clock_gating_check](#)

[remove_clock_gating_check](#)

[report_clock_gating_check](#)

Rule: B2T_CLK_0013

Clock types of **top_clock** in top instance and clock **blk_clock** in block design do not match.

Severity: Warning

Description

A clock signal originating from the top-level constraints is reaching a block pin with a clock definition. However, the type of these two clocks are different.

Risk Associated with Violation

The block-level timing analysis results will be inconsistent from the top-level results. Block-level timing closure might not lead to less effort at the top-level timing closure or vice versa.

Possible Scenarios

The following scenarios can lead to a **B2T_CLK_0013** violation.

The top-level and block-level clocks (defined at block level pins only) might have the same periods and waveforms, but two clocks are different types.

Example:

Top Scenario - no clocks

```
current_design top
{"top"}
create_clock -name clk -period 2.0 -waveform {0.0 1.0} [get_ports
{t_CLK}]
1
create_generated_clock -source [get_ports {t_CLK}] -divide_by 2 [get_pins
{BLOCK/F_clk1/q}]
1
```

Block Scenario - no clocks

```
current_design btop
{"btop"}
create_clock -name b_clk -period 2.0 -waveform {0.0 1.0} [get_ports
{t_CLK}]
1
create_clock -period 4.0 -waveform {0.0 2.0} [get_pins {F_clk1/q}]
1
```

Type of Clock defined at top level pin *BLOCK/F_clk1/q* is different than type of Clock defined at block level pin *F_clk1/q*. which leads to **B2T_CLK_0013** violation as shown here.

Violation 1: Clock types of clock '*BLOCK/F_clk1/q*' in top instance and clock '*F_clk1/q*' in block design do not match.

Violation Details

Top clock *BLOCK/F_clk1/q* reaches where clock *F_clk1/q* is defined in the block design, but these two clocks are different types.

Top clock	Top clock type	Block clock	Block clock type
<i>BLOCK/F_clk1/q</i>	Generated	<i>F_clk1/q</i>	Primary/Master

What Next

The "Violation Details" section of the Info Pane window reports the current differences leading to the **B2T_CLK_0013** violation. In addition, you can get some information related to clock definitions using the [report_clock](#) command at both the top and block level.

Example:

```
ptc_shell> report_clock BLOCK/F_clk1/q
*****
Report : clock
```

```
Design : top
Scenario: default
Version: ...
Date   : ...
*****
```

```
Attributes:
p - Propogated clock
G - Generated clock
I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
BLOCK/F_clk1/q	4.00	{0 2}	G	{BLOCK/F_clk1/q}

Generated Clock	Master Source	Generated Source	Master Clock	Waveform Modification
BLOCK/F_clk1/q	t_CLK	BLOCK/F_clk1/q_clk	div(2)	

1

```
ptc_shell> current_design block
{"block"}
ptc_shell> report_clock F_clk1/q
*****
Report : clock
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

```
Attributes:
p - Propogated clock
G - Generated clock
I - Inactive clock
```

Clock	Period	Waveform	Attrs	Sources
F_clk1/q	4.00	{0 2}		{F_clk1/q}

1

Fix Suggestion

Clock definitions for both top-level design and block-level designs should be reviewed. Their type needs to match for constraint consistency to declare them correct.

See Also

[report_clock](#)

Rule: B2T_CLT_0001

'**constraint**' applied to '**object_type**' '**object**' in the top design is missing in the block design.

Severity: Error

Description

A clock latency constraint is specified in the top design but not in the block design. The *constraint* parameter lets you know which type of constraint is missing. It can be one of the following:

- Clock source latency
- Clock network latency

Risk Associated With Violation

The clock latency applied in the top constraints will affect timing results. The delay calculation involving cells in the block will not be consistent between block timing analysis and top timing analysis.

Possible Scenarios

Consider a design depicted in Figure 1 below. If specific clock latency is declared in the top level constraints, it also needs to be present in the block level constraints. Failure to do so can result in different timing numbers between block timing analysis and top timing analysis.

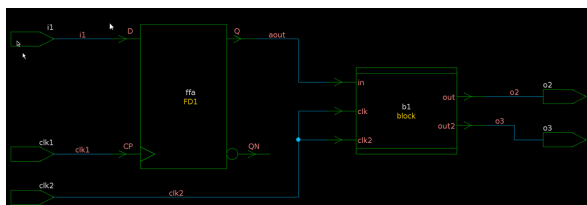


Figure 1: Clock Latencies

Block constraints

```
create_clock -name clk -period 10 -waveform [list 0 5] [get_ports clk]
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports
clk2]
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
set_clock_latency 0.3 -max [get_clocks clk2]
set_clock_latency 0.05 -source [get_clocks clk2]
```

These constraints generate two different **B2T_CLT_0001** violations:

- One related to the clock network latency of 0.3 on clk2
- One related to source clock latency of 0.05 on clk2

What Next

The file name and line number where the clock latency is specified is displayed in the Info Pane window. Clicking the hyperlink opens the SDC Browser and points to the relevant command.

You can also review the current clock latency settings using the `report_clock` command.

```
ptc_shell> current_design B2T_CLT_0001
{"B2T_CLT_0001"}

ptc_shell> report_clock -skew
*****
Report : clock skew
Design : B2T_CLT_0001
Scenario: default
Version: ...
Date   : ...
*****
```

Object	Setup Delay Uncertainty	Min Rise Delay	Rise Related Delay Clock	Min Fall Delay	Max Rise Delay	Max Fall Delay	Hold
clk2		-			-	0.30	--
0.30	-				-		

Object	Late_r	Late_f	Early_r Rel_clk	Early_f	Late_r	Late_f	Early_r	Early_f	Max
clk2			0.05		0.05		0.05		0.05
0.05	0.05		0.05	--					

Fix Suggestion

Constraints affecting the clock latency for both the top-level design and the block-level design should be reviewed. In the example above, there are two possible ways to solve the problem:

- Remove both the clock source and clock network latencies at the top level
- Add the same clock latencies on the block design

For the above example, adding the clock latencies at the block level would be done as follows (top-level clock is split into two different block-level clocks...):

```
set_clock_latency 0.3 -max [get_clocks clk2]
set_clock_latency 0.3 -max [get_clocks clk]
set_clock_latency 0.05 -source [get_clocks clk2]
set_clock_latency 0.05 -source [get_clocks clk]
```

See Also

[set_clock_latency](#)

[report_clock](#)

Rule: B2T_CLT_0002

‘**constraint**’ applied to ‘**object_type**’‘**object**’ in block design is missing in top design.

Severity: Error

Description

A clock latency constraint is specified in the block design but not in the top design. The *constraint* parameter lets you know which type of constraint is missing. It can be one of the following:

- Clock source latency
- Clock network latency

Risk Associated With Violation

The clock latency applied in the block constraints will affect timing results. Delay calculation involving cells in the block will not be consistent between block timing analysis and top timing analysis.

Possible Scenarios

Consider a design depicted in Figure 1 below. If specific clock latency is declared in the block level constraints, it also needs to be present in the top level constraints. Failure to do so can result in different timing numbers between block timing analysis and top timing analysis.

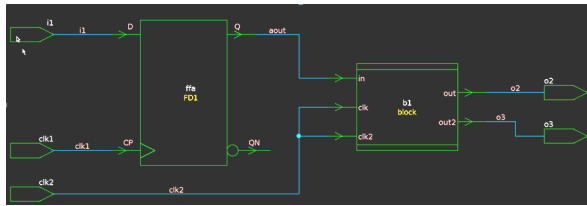


Figure 1: Clock Latencies

Block constraints

```
create_clock -name clk -period 10 -waveform [list 0 5] [get_ports clk]
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports clk2]
set_clock_latency 0.3 -max [get_clocks clk2]
set_clock_latency 0.3 -max [get_clocks clk]
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
```

These constraints generate two **B2T_CLT_0002** violations:

- One for the block clk clock
- One for the block clk2 clock

What Next

The file name and line number where the clock latency is specified is displayed in the Info Pane window. Clicking the hyperlink opens the SDC Browser and points to the relevant command.

You can also review the current clock latency settings using the `report_clock` command.

```
ptc_shell> current_design block
{"block"}

ptc_shell> report_clock -skew

*****

Report : clock_skew

Design : block

Scenario: default
```

Version: ...

Date : ...

Min Rise Min Fall Max Rise Max Fall Hold Setup Related

Object Delay Delay Delay Delay Uncertainty Uncertainty Clock

clk1 - - 0.30 0.30 - - --

clk2 - - 0.30 0.30 - - --

Fix Suggestion

Constraints affecting the clock latency for both the top-level design and the block-level design should be reviewed. For the example above, there are two possible ways to solve the problem:

- Remove both the clock network latencies at the block level
- Add the same clock latencies to the top design

In this example, adding the clock latencies at the top level is done as follows:

```
set_clock_latency 0.3 -max [get_clock clk2]
```

See Also

[set_clock_latency](#)

[report_clock](#)

Rule: B2T_CLT_0003

'**constraint**' applied to '**top_object_type**' '**top_object**' in the top design does not match the one applied to '**blk_object_type**' '**blk_object**' in the block design.

Severity: Error

Description

The values of a clock latency specified in both the top design constraints and the block design constraints are inconsistent. The '**constraint**' parameter lets you know the type of constraint mismatching. It can be one of the following:

- Clock source latency
- Clock network latency

Risk Associated With Violation

The clock latency applied in both top and block constraints will affect timing results. Because these values are different, the delay calculation involving cells in the block will not be similar between block timing analysis and top timing analysis.

Possible Scenarios

Consider the design depicted in Figure 1. Any simple clock latency setting should match both the top and block constraints. Failure to do so can result in different timing numbers between block timing analysis and top timing analysis.

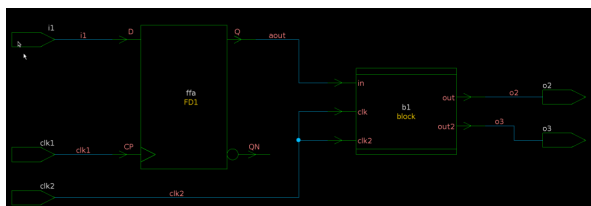


Figure 1: Clock Latencies

Block constraints

```
create_clock -name clk1 -period 10 -waveform [list 0 5] [get_ports clk]
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports clk2]
set_clock_latency -max 0.3 [get_clocks clk1]
set_clock_latency -max 0.3 [get_clocks clk2]
```

Top constraints

```
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
set_clock_latency 0.5 -max [get_clocks clk2]
```

What Next

The file name and line number where the clock latency is specified is displayed in the Info Pane window for both constraint sets. Clicking the hyperlink opens the dual SDC Browser and points to the relevant command.

You can review the current clock latency settings using the **report_clocks** command.

```
ptc_shell> current_design B2T_CLT_0003

{"B2T_CLT_0003"}

ptc_shell> report_clocks -skew
```

Chapter 2: Block-To-Top Rules

Rule: B2T_CLT_0003

Report : clock_skew

Design : B2T_CLT_0003

Scenario: default

Version: ...

Date : ...

Min Rise Min Fall Max Rise Max Fall Hold Setup Related

Object Delay Delay Delay Delay Uncertainty Uncertainty Clock

clk2 - - 0.50 0.50 - - --

ptc_shell> current_design block

{"block"}

ptc_shell> report_clocks -skew

Report : clock_skew

Design : block

Scenario: default

Version: ...

Date : Thu ...

Min Rise Min Fall Max Rise Max Fall Hold Setup Related

Object Delay Delay Delay Delay Uncertainty Uncertainty Clock

clk1 - - 0.30 0.30 - - --

clk2 - - 0.30 0.30 - - --

Fix Suggestion

Constraints affecting the clock latencies for both top-level design and block-level design should be reviewed and made identical.

Rule Property: Tolerance

This rule allows tolerance to be taken into consideration when performing simple clock latency matching. This tolerance is specified as a percentage of the smallest value compared.

By default, this tolerance is set at 1e-5% which is equivalent to performing exact matches. You have full control of setting the tolerance and can be changed through the **set_rule_property** command as demonstrated here:

```
set_rule_property tolerance 20 [get_rules B2T_CLT_0003]
```

See Also

[set_clock_latency](#)

[report_clock](#)

Rule: B2T_DIS_0001

Pin *pin* is disabled in top instance but not in block design.

Severity: Error

Description

A [set_disable_timing](#) constraint disables the given pin or port in the top instance. However, there is no corresponding [set_disable_timing](#) constraint set on the corresponding pin in the block level constraints.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where a [set_disable_timing](#) constraint is set on the input pin of the inverter **b1/zInv2** at the top level.

```
set_disable_timing [get_pins b1/zInv2/A]
```


This constraint disables all the timing paths through the pin **b1/zInv2/A** at the top level. As a result, the timing paths to pins **b1/ffg/D** and **b1/ffh/D** are disabled at the top level. However, the timing paths through pin **zInv2/A** is not disabled in the block level constraints. A B2T_DIS_0001 violation will be issued due to the missing `set_disable_timing` constraint at the pin **zInv2/A** in the block level constraints.

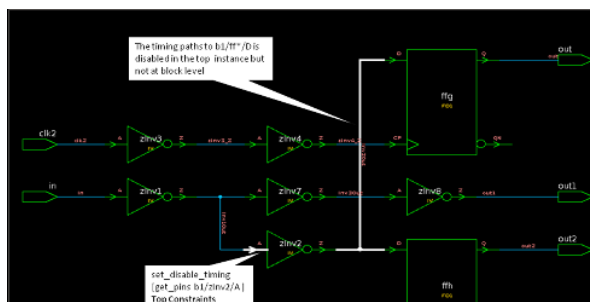


Figure 1: Block Level Netlist

What Next

The "Violations Details" section will show the link to top level SDC constraint, where the top level pin has been disabled. To see the list of timing paths that have been disabled at the top level, use the `analyze_paths -through top_level_pin` command along with the `traverse_disabled` option. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design top
```

```
ptc_shell> analyze_paths -through [get_pins b1/zInv2/A]
-traverse_disabled
```

```
*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : top
Version: ...
Date   : ...
*****
```

Path Endpoints:

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/ Capture
b1/ffg/D (FD1)	DT#0		(2,2,2,2)	clk1/clk2
b1/ffh/D (FD1)	DT#0		(2,2,2,2)	clk1/clk2

```
Disabled Object Information:
  DT#0 = set_disable_timing
        at pin: b1/zInv2/A
        run.tcl, line 25
```

The above `analyze_paths` report shows that the timing paths through the pin **b1/Inv2/A** and to the endpoints **b1/ff*/D** are disabled at the top level. The dominant constraint `DT#0` in the above report shows that a `set_disable_timing` constraint is set on the top level pin **b1/Inv2/A**, which disables the timing paths at the top level.

To check the missing constraint from the block level constraints, use the `analyze_paths -through block_pin` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -through [get_pins zInv2/A]

*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****

Path Endpoints:
```

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
ffg/D (FD1)			(1,1,1,1)	clk1/clk2
ffh/D (FD1)			(1,1,1,1)	clk1/clk2

The above `analyze_paths` report for the block shows that the timing paths to endpoints **ff*/D** are valid and there is no disable timing constraint defined at the block level.

Fix Suggestion

To resolve the issue, add the proper disable constraint for the particular pin in the block level constraints or remove the disable constraint setting from the top level constraints. For the example scenario, the disable constraint at the block level can be added as follows:

```
set_disable_timing [get_pins zInv2/A]
```

See Also

[set_disable_timing](#)

[report_disable_timing](#)

[remove_disable_timing](#)

[B2T_DIS_0002](#)

Rule: B2T_DIS_0002

Pin **pin** is disabled in block design but not in top instance.

Severity: Error

Description

A [set_disable_timing](#) constraint disables the given pin in the block design. However, there is no corresponding [set_disable_timing](#) constraint set on the corresponding pin in the top level constraints.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where a [set_disable_timing](#) constraint is set on the input pin of the inverter **zInv2** in the block design.

```
set_disable_timing [get_pins zInv2/A]
```

This constraint disables all the timing paths through the pin **zInv2/A** in the block design. As a result, the timing paths to pins **ffg/D** and **ffh/D** are disabled at the block level. However, the timing paths through pin **b1/zInv2/A** are not disabled at the top level. A B2T_DIS_0002 violation will be issued due to the missing [set_disable_timing](#) constraint at the pin **b1/zInv2/A** in the top level constraints.

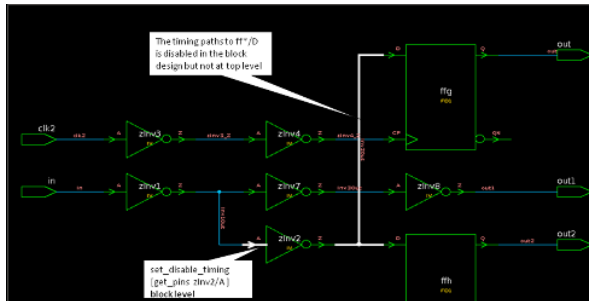


Figure 1: Block Level Netlist

What Next

The "Violations Details" section will show the link to block level SDC constraint, where the pin at the block level has been disabled. To see the list of timing paths that have been disabled at the block level, use the [analyze_paths](#) -through *block_level_pin* command along with the *traverse_disabled* option. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -through [get_pins zInv2/A] -traverse_disabled

*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****

Path Endpoints:

Endpoint          Dominant   Overridden   Count      Clocks
Capture          Constraint Constraints (r,f,R,F)  Launch/

-----
ffg/D (FD1)      DT#0      (1,,1,1)    clk1/clk2
ffh/D (FD1)      DT#0      (1,1,1,1)    clk1/clk2

Disabled Object Information:
DT#0 = set_disable_timing
      at pin: zInv2/A
      run.tcl, line 37
```

The above `analyze_paths` report shows that the timing paths through the pin **zInv2/A** and to the endpoints **ff*/D** are disabled in the block design. The dominant constraint **DT#0** in the above report shows that a `set_disable_timing` constraint is set on the top level pin **zInv2/A**, which disables the timing paths at the block level.

To check the missing constraint from the top level constraints, use the `analyze_paths -through top_pin` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design top

ptc_shell> analyze_paths -through [get_pins b1/zInv2/A]

*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : Top
Version: ...
Date   : ...
*****

Path Endpoints:

Endpoint          Dominant   Overridden   Count        Clocks
Constraint         Constraints (r,f,R,F) Launch/
Capture
-----
b1/ffg/D (FD1)                (2,2,2,2)    clk1/clk2
b1/ffh/D (FD1)                (2,2,2,2)    clk1/clk2
```

The above `analyze_paths` report for the top level shows that the timing paths to endpoints **ff*/D** are valid and there is no disable timing constraint defined at the block level.

Fix Suggestion

To resolve the issue, add the proper disable constraint for the particular pin in the top level constraints or remove the disable constraint setting from the block level constraints. For the example scenario, the disable constraint at the top level can be added as follows:

```
set_disable_timing [get_pins b1/zInv2/A]
```

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an

equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[set_disable_timing](#)

[report_disable_timing](#)

[remove_disable_timing](#)

[B2T_DIS_0001](#)

Rule: B2T_DIS_0003

Timing arc from **from_pin** to **to_pin** is disabled in top instance but not in block design.

Severity: Error

Description

A [set_disable_timing](#) constraint disables the timing arc in the top instance. However, there is no [set_disable_timing](#) constraint defined on the corresponding timing arc in the block level constraints.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where a [set_disable_timing](#) constraint is set on the cell timing arc of the inverter **b1/zInv9** at the top level.

```
set_disable_timing -from A -to Z [get_cells b1/zInv9]
```

This constraint disables all the arcs between the pins **A** and **Z** of the cell **b1/zInv9** at the top level. As a result, the timing path to pin **b1/ffh/D** is disabled at the top level. However, the timing arc from **A** to **Z** of the cell **zInv9** is not disabled in the block level constraints. A B2T_DIS_0003 violation will be issued due to the missing [set_disable_timing](#) constraint on the cell timing arc in the block level constraints.

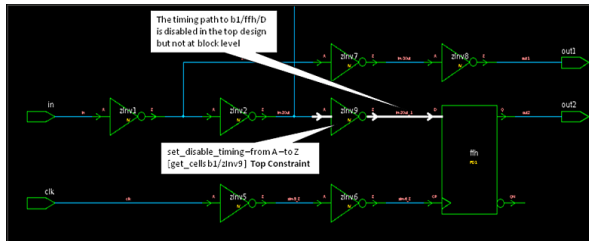


Figure 1: Block Level Netlist

What Next

The "Violations Details" section will show the link to top level SDC constraint, where the cell timing arc at the top level has been disabled. To see the list of timing paths that have been disabled at the top level, use the [analyze_paths](#) -through *cell* command along with the *traverse_disabled* option. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design top
```

```
ptc_shell> analyze_paths -through [get_cells b1/zInv9]
-traverse_disabled
```

```
*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : top
Version: ...
Date   : ...
*****
```

Path Endpoints:

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/ Capture
b1/ffh/D (FD1)	DA#0		(2,2,2,2)	clk1/clk2

Disabled Object Information:

```
DA#0 = set_disable_timing
       at negative_unate arc from: b1/zInv9/A
       to: b1/zInv9/Z
       run.tcl, line 25
```

The above `analyze_paths` report shows that the timing path through the cell **b1/zInv9** and to the endpoint **b1/ffh/D** is disabled at the top level. The dominant constraint **DA#0**

in the above report shows that a `set_disable_timing` constraint is set on the negative unate arc from **b1/zInv9/A** to **b1/zInv9/Z**, which disables the timing path at the top level.

To check the missing constraint from the block level constraints, use the `analyze_paths -through cell` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -through [get_cells zInv9]

*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****

Path Endpoints:

Endpoint          Dominant   Overridden   Count      Clocks
Capture          Constraint Constraints (r,f,R,F)  Launch/
-----
ffh/D (FD1)                                (1,1,1,1)  clk1/clk2
```

The above `analyze_paths` report for the block shows that the timing path to the endpoint **ffh/D** is valid and there is no disable timing constraint defined at the block level.

Fix Suggestion

To resolve the issue, add the proper disable constraint for the particular timing arc in the block level constraints or remove the disable constraint setting from the top level constraints. For the example scenario, the disable constraint at the block level can be added as follows:

```
set_disable_timing -from A -to Z [get_cells zInv9]
```

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[set_disable_timing](#)

[report_disable_timing](#)

[remove_disable_timing](#)

[B2T_DIS_0004](#)

Rule: B2T_DIS_0004

Timing arc from **from_pin** to **to_pin** is disabled in block design but not in top instance.

Severity: Error

Description

A [set_disable_timing](#) constraint disables the timing arc in the block design. However, there is no [set_disable_timing](#) constraint defined on the corresponding timing arc in the top level constraints.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where a [set_disable_timing](#) constraint is set on the timing arc of the inverter **zInv9** at the block level.

```
set_disable_timing -from A -to Z [get_cells zInv9]
```

This constraint disables all the arcs between the pins **A** and **Z** of the cell **zInv9** at the block design. As a result, the timing path to pin **ffh/D** is disabled in the block design. However, the timing arc from **A** to **Z** of the cell **b1/zInv9** is not disabled in the top level constraints. A B2T_DIS_0004 violation will be issued due to the missing [set_disable_timing](#) constraint for the cell timing arc in the top level constraints.

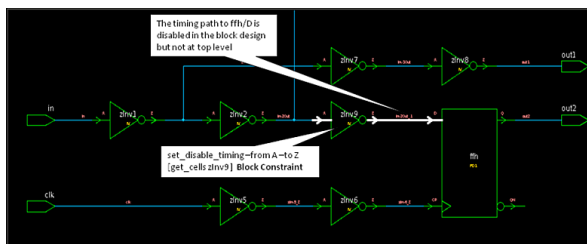


Figure 1: Block Level Netlist

What Next

The "Violations Details" section shows the link to block-level constraint, where the cell timing arc at the block level has been disabled. To see the list of timing paths that have been disabled in the block design, use the `analyze_paths -through cell` command along with the `traverse_disabled` option. For the example scenario, the command in the block design can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -through [get_cells zInv9] -traverse_disabled
```

```
*****
Report : analyze_paths
        -path_type end
        -traverse_disabled
        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****
```

Path Endpoints:

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
ffh/D (FD1)	DA#0		(1,1,1,1)	clk1/clk2

Disabled Object Information:

```
DA#0 = set_disable_timing
      at negative_unate arc from: zInv9/A
      to: zInv9/Z
      run.tcl, line 37
```

The above `analyze_paths` report shows that the timing path through the cell **zInv9** and to the endpoint **ffh/D** is disabled in the block design. The dominant constraint **DA#0** in the above report shows that a `set_disable_timing` constraint is set on the negative unate arc from **zInv9/A** to **zInv9/Z**, which disables the timing path in the block design. To check the missing constraint from the top level constraints, use the `analyze_paths -through cell` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design top

ptc_shell> analyze_paths -through [get_cells b1/zInv9]
```

```
*****
Report : analyze_paths
        -path_type end
        -max_endpoints = 1000
Design : Top
Version: ...
Date   : ...
*****
```

```
Path Endpoints:
                Dominant   Overridden   Count       Clocks
Endpoint       Constraint Constraints (r,f,R,F)   Launch/
Capture
-----
b1/ffh/D (FD1)                (2,2,2,2)    clk1/clk2
```

The above `analyze_paths` report for the top instance shows that the timing path to the endpoint **b1/ffh/D** is valid and there is no disable timing constraint defined in the top level constraints.

Fix Suggestion

To resolve the issue, add the proper disable constraint for the particular timing arc in the top level constraints or remove the disable constraint setting from the block level constraints. For the example scenario, the disable constraint at the top level can be added as follows:

```
set_disable_timing -from A -to Z [get_cells b1/zInv9]
```

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[set_disable_timing](#)

[report_disable_timing](#)

[remove_disable_timing](#)

[B2T_DIS_0003](#)

Rule: B2T_EXC_0001

Exception **exception** in top instance is missing in block design.

Severity: Error

Description

A timing exception such as `set_false_path`, `set_multicycle_path`, etc. has been defined on a particular object in the top instance. However, there is no timing exception defined on the corresponding object in the block level constraints.

Risk Associated with Violation

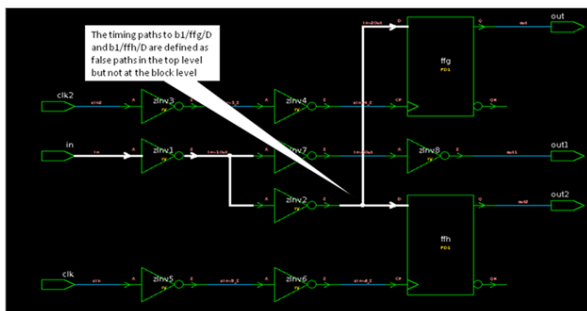
The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

One Possible Scenario

Consider a scenario where a `set_false_path` constraint is set on the input pin of the flops **b1/ff*/D** at the top level.

```
set_false_path -to [get_pins b1/ff*/D]
```

This timing exception identifies the timing paths to the pin **b1/ff*/D** as false paths at the top level. As a result, the timing paths to pins **b1/ffg/D** and **b1/ffh/D** are not considered during timing analysis at the top level. However, the timing paths to pin **ff*/D** are not defined as false paths in the block level constraints and will be analyzed during timing analysis. A **B2T_EXC_0001** violation will be issued due to the missing `set_false_path` constraint at the pin **ff*/D** in the block level constraints.



Block Level Netlist

What Next

The "Violations Details" section shows the link to top-level constraint, where the timing exception is defined. Further, it also shows a table with the type of exception, values

specified for rise/fall and setup/hold timing, and the objects impacted by the exception. To see the list of timing paths that have been defined as false paths at the top level, use the `analyze_paths -to top_level_pin` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design top

ptc_shell> analyze_paths -to [get_pins b1/ff*/D]

*****
Report : analyze_paths
        -path_type end
        -max_endpoints = 1000
Design : Top
Version: ...
Date   : ...
*****

Path Endpoints:
```

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
b1/ffg/D (FD1)	F0		(2,2,2,2)	clk1/clk2
b1/ffh/D (FD1)	F0		(2,2,2,2)	clk1/clk2

```

Exception Information:

ID      Type              Setup Hold  Source
-----
F0      false_path         FALSE FALSE  run.tcl, line 24
```

The above `analyze_paths` report shows that the timing paths to pins **b1/ff*/D** are defined as false paths at the top level. The dominant constraint "**F0**" in the above report shows that a `set_false_path` constraint is set for setup and hold on the top level pins **b1/ff*/D**, and these paths will not be considered for timing analysis.

To check the missing constraint from the block level constraints, use the `analyze_paths -to block_pin` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -to [get_pins ff*/D]

*****
Report : analyze_paths
        -path_type end
```

```

        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****

```

Path Endpoints:

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
ffg/D (FD1)			(1,1,1,1)	clk1/clk2
ffh/D (FD1)			(1,1,1,1)	clk1/clk2

The above `analyze_paths` report for the block shows that the timing paths to endpoints **ff*/D** are valid and there is no false path constraint defined at the block level.

Fix Suggestion

To resolve the issue, add the proper timing exception constraint for the particular pin in the block level constraints or remove the timing exception constraint setting from the top level constraints. For the example scenario, the false path constraint at the block level can be added as follows:

```
set_false_path -to [get_pins ff*/D]
```

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

```

        If suppress_violations_for_exceptions_outside_block = true,
        skip the violation that is outside of the block exception.

```

See Also

[set_false_path](#)

[analyze_paths](#)

[B2T_EXC_0002](#)

Rule: B2T_EXC_0002

Exception **exception** in block design is missing in top instance.

Severity: Error

Description

A timing exception such as `set_false_path`, `set_multicycle_path`, etc. has been defined on a particular object at the block level. However, there is no timing exception defined on the corresponding object in the top level constraints.

Risk Associated with Violation

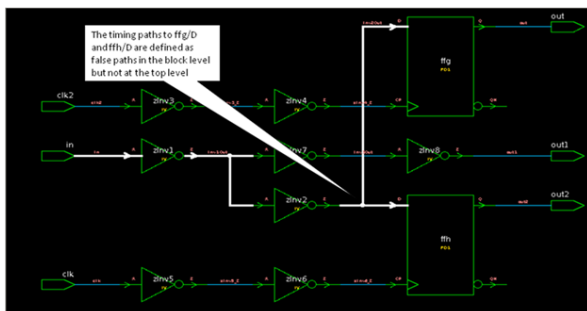
The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

One Possible Scenario

Consider a scenario where a `set_false_path` constraint is set on the input pin of the flops **ff*/D** in the block level constraints.

```
set_false_path -to [get_pins ff*/D]
```

This timing exception identifies the timing paths to the pin **ff*/D** as false paths at the block instance. As a result, the timing paths to pins **ffg/D** and **ffh/D** are not considered during timing analysis at the block level. However, the timing paths to pin **b1/ff*/D** are not defined as false paths in the top level constraints and will be analyzed during timing analysis. A **B2T_EXC_0002** violation will be issued due to the missing `set_false_path` constraint at the pin **b1/ff*/D** in the top level constraints.



Block Level Netlist

What Next

The "Violations Details" section shows the link to block-level constraint, where the timing exception is defined. Further, it will also show a table with the type of exception,

values specified for rise/fall and setup/hold timing, and the block objects impacted by the exception. To see the list of timing paths that have been defined as false paths at the block instance, use the `analyze_paths -to block_level_pin` command. For the example scenario, the command at the block instance can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -to [get_pins ff*/D]

*****
Report : analyze_paths
        -path_type end
        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****

Path Endpoints:
```

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
ffg/D (FD1)	F0		(1,1,1,1)	clk1/clk2
ffh/D (FD1)	F0		(1,1,1,1)	clk1/clk2

```
Exception Information:
```

ID	Type	Setup	Hold	Source
F0	false_path	FALSE	FALSE	run.tcl, line 37

The above `analyze_paths` report shows that the timing paths to pins **ff*/D** are defined as false paths in the block level constraints. The dominant constraint "**F0**" in the above report shows that a `set_false_path` constraint is set for setup and hold on the block level pin **ff*/D**, and these paths will not be analyzed for timing analysis.

To check the missing constraint from the top level constraints, use the `analyze_paths -to top_level_pin` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design Top

ptc_shell> analyze_paths -to [get_pins b1/ff*/D]

*****
Report : analyze_paths
        -path_type end
```



```

-max_endpoints = 1000
Design : Top
Version: -----
Date   : -----
*****

```

Path Endpoints:

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r, f, R, F)	Clocks Launch/
b1/ffg/D (FD1)			(2, 2, 2, 2)	clk1/clk2
b1/ffh/D (FD1)			(2, 2, 2, 2)	clk1/clk2

The above `analyze_paths` report at the top level shows that the timing paths to endpoints **b1/ff*/D** are valid and there is no false path constraint defined at the top level.

Fix Suggestion

To resolve the issue, add the proper timing exception constraint for the particular pin in the top level constraints or remove the timing exception constraint setting from the block level constraints. For the example scenario, the false path constraint at the top level can be added as follows:

```
set_false_path -to [get_pins b1/ff*/D]
```

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

See Also

[set_false_path](#)

[analyze_paths](#)

[B2T_EXC_0001](#)

Rule: B2T_EXC_0003

Exception **top_exception** in top instance does not match with exception **blk_exception** in block design.

Severity: Error

Description

A timing exception such as `set_false_path`, `set_multicycle_path`, etc. has been defined on a particular object in the top instance. However, the timing exception defined on corresponding pin in the block design has a different exception value (-setup, -hold, -rise, -fall or multicycle path multiplier).

Risk Associated with Violation

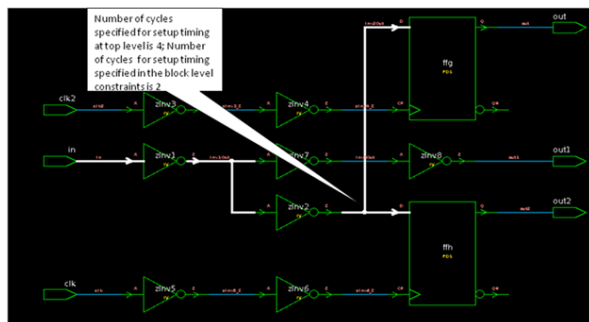
The block level will not be analyzed under the same conditions as it was created. This might result in inaccurate analysis coverage or even incorrect behavior for the block or the full chip.

One Possible Scenario

Consider a scenario where a multicycle path exception of setup value 4 is set on the timing paths to the endpoints **b1/ff*/D** at the top instance. And, a multicycle path exception of setup value 2 is set on the timing paths to the endpoints **ff*/D** at the block level.

```
current_design top
set_multicycle_path -setup 4 -to [get_pins b1/ff*/D]
current_design block
set_multicycle_path -setup 2 -to [get_pins ff*/D]
```

The multicycle path definition at the top level specifies that the number of cycles the data path must have for setup is 4. Hence, the timing paths to **b1/ffg/D** and **b1/ffh/D** will be analyzed with 4 clock cycles for max timing analysis at the top level. However, the multicycle path definition in the block level constraints specified that the number of cycles the data path must have is 2. And, the timing paths to **ffg/D** and **ffh/D** will be analyzed with 2 clock cycles for max timing analysis at the block level. A **B2T_EXC_0003** violation will be issued due to the mismatch in the number of cycles specified for setup timing analysis to the **D** pin of the flops **ffg** and **ffh** at the top instance and block design.



Block Level Netlist

What Next

The "Violation Details" section shows the top-level exception details and the block level exception details along with the exception values specified that do not match. This includes a table for the exception at the top level and block design including the type of exception, value specified, objects specified, and the link to SDC file where the exception is defined. To see list of timing paths for which the exception is defined and the exception details at the top level, use the `analyze_paths -to top_level_pin` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design Top

ptc_shell> analyze_paths -to [get_pins b1/ff*/D]

*****
Report : analyze_paths
        -path_type end
        -max_endpoints = 1000
Design : Top
Version: ...
Date   : ...
*****

Path Endpoints:
```

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
b1/ffg/D (FD1)	M0		(2,2,2,2)	clk1/clk2
b1/ffh/D (FD1)	M0		(2,2,2,2)	clk1/clk2

```
Exception Information:
```

ID	Type	Setup	Hold	Source
M0	multicycle_path	4	--	run.tcl, line 24

The above `analyze_paths` report shows that a multicycle path of value 4 for setup timing is defined for the timing paths to pins **b1/ff*/D** at the top level. The dominant constraint "M0" in the above report shows that a `set_multicycle_path` constraint of value 4 is defined on the top level pins **b1/ff*/D**.

To see list of timing paths for which the exception is defined and the exception details at the block instance, use the `analyze_paths -to block_level_pin` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> analyze_paths -to [get_pins ff*/D]
```

```
*****
Report : analyze_paths
        -path_type end
        -max_endpoints = 1000
Design : block
Version: ...
Date   : ...
*****
```

Path Endpoints:

Endpoint Capture	Dominant Constraint	Overridden Constraints	Count (r,f,R,F)	Clocks Launch/
ffg/D (FD1)	M0		(1,1,1,1)	clk1/clk2
ffh/D (FD1)	M0		(1,1,1,1)	clk1/clk2

Exception Information:

ID	Type	Setup	Hold	Source
M0	multicycle_path	2	--	run.tcl, line

The above `analyze_path` report shows that a multicycle path of value 2 for setup timing is defined for the timing paths to pins **ff*/D** in the block level constraints. The dominant constraint "**M0**" in the above report shows that a `set_multicycle_path` constraint of value 2 is defined on the block level pins **ff*/D**.

Fix Suggestion

To resolve the issue, the exception values for both the top level design and block level design should be reviewed. The exception values should be identical for the pin of interest both at the top level and block level.

Clock matching is the basis of other constraint matching in block-to-top comparison. For more information

Clock Matching

Clock matching is the basis of constraint matching in block-to-top comparison. The correspondence of top-level and block-level clocks are checked by the **B2T_CLK_0001**, **B2T_CLK_0002**, and **B2T_CLK_0003** rules. This also implies that, during hierarchical constraint checking, if a block clock cannot be extracted within the block context, an equivalent virtual clock should be created. The virtual clocks are used by block-to-top comparison to establish the mapping between top and block clocks.

If `suppress_violations_for_exceptions_outside_block = true`, skip the violation that is outside of the block exception.

See Also

[set_multicycle_path](#)

[analyze_paths](#)

[B2T_EXC_0001](#)

[B2T_EXC_0002](#)

Rule: B2T_EXD_0001

top_pin in the top instance has *input_or_output* delay constraints that are missing in the block level.

Severity: Error

Description

The hierarchical pin in the top level has signals that do not have corresponding **set_input_delay** or **set_output_delay** constraints on ports at the block level. The registers feeding the input of the block or receiving the output signals from the block have clock periods, edges, and/or min/max path specifications that are missing at the block level. If the block level port is directly connected to a top level port, then the signals used for comparison are obtained by the **set_input_delay** or **set_output_delay** constraints at the top level

Risk Associated with Violation

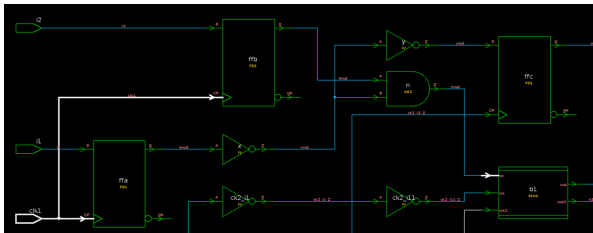
The block level will not be analyzed under the same conditions as the top level. This might result in less analysis coverage or even incorrect behavior of the block.

One Possible Scenario

Consider a scenario where clk1 at the top level is clocking a register whose output feeds the **b1/in** pin at the top level. A **set_input_delay** constraint for max analysis is set on the input port in **in** the block-level constraints:

```
current_design top
create_clock -period 10 -name clk1 [get_ports clk1]
current_design block
create_clock -period 10 -name clk3 [get_ports clk]
set_input_delay 1 -max -clock clk3 [get_ports in]
```

The input delay constraint at the block level has a max rise/fall specification. However, the clock specification at the top is for min/max rise/fall. The block level delay constraint for the **in** port is missing a min rise/fall specification. A **B2T_EXD_0001** violation is issued due to the missing signals between the top level pin **b1/in** and the **set_input_delay** constraint on the input port **in** at the block level.



Top Level Netlist

What Next

The "Violation Details" section shows the details about the top level propagated clock. Use the **analyze_paths** command with the *-through* and *-path_type* summary options at the top level to get a summary of the paths through the pin. For the example scenario, the path summary can be generated with the following commands:

```
ptc_shell> current_design top

ptc_shell> analyze_paths -path_type summary -through [get_pins b1/in]

*****
Report : analyze_paths
        -path_type summary
        -max_endpoints = 1000
Design : top
Version: ...
Date   : ...
*****
Summary of Paths:
```

Clocks		Dominant	Overridden	Count
Startpoint	Endpoint	Constraint	Constraints	(r,f,R,F)
Launch/Capture				
ffa/CP (FD1)	b1/ffg/D (FD1)			(1,1,1,1)
clk1/clk2				
ffb/CP (FD1)	b1/ffg/D (FD1)			(1,1,1,1)
clk1/clk2				
ffa/CP (FD1)	b1/ffh/D (FD1)			(1,1,1,1)
clk1/clk2				
ffb/CP (FD1)	b1/ffh/D (FD1)			(1,1,1,1)
clk1/clk2				

This shows that the launch clock for all paths is clk1. You can get more details about any of these paths and a corresponding schematic by using the the following command: For the example scenario, one path can be fully expanded by using the following command.

```
analyze_paths -path_type full -from <startpoint>
               -to <endpoint>
```

For the example scenario, one path can be fully expanded by using the following command.

```
ptc_shell> analyze_paths -path_type full -from [get_pins ffa/CP] -to
[get_pins b1/ffg/D]
```

```
*****
```

```
Report : analyze_paths
        -path_type full
        -max_endpoints = 1000
```

```
Design : top
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Summary of Paths:
```

Clocks	Endpoint	Dominant	Overridden	Count
Startpoint		Constraint	Constraints	(r,f,R,F)
Launch/Capture				
ffa/CP (FD1)	b1/ffg/D (FD1)			(1,1,1,1)
clk1/clk2				

Full report of all pins in the paths

Level	Pin	Attr
	Constraints	
1	ffa/CP (FD1)	Start
2	ffa/Q (FD1)	
3	x/A (IV)	
4	x/Z (IV)	
5	n/B (AN2)	
6	n/Z (AN2)	
7	b1/zInv1/A (IV)	
8	b1/zInv1/Z (IV)	
9	b1/zInv2/A (IV)	
10	b1/zInv2/Z (IV)	
11	b1/ffg/D (FD1)	End

You can then use the **all_connected** command to obtain the name of the hierarchical pin in the path by providing the first pin in the hierarchy boundary. For the example scenario, the following command can be used.

```
ptc_shell> all_connected [get_pins b1/zInv1/A]
{"b1/in"}
```

To see the delay definitions related to the block level port, you can use the **report_port -verbose** command. For the example scenario, the report is as follows:

```
ptc_shell> current_design block

ptc_shell> report_port -verbose [get_ports in]
*****
Report : port
        -input_delay
        -drive
        -output_delay
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

		Pin Cap		Wire Cap	
Port	Dir	Min	Max	Min	Max
Attributes		rise/fall	rise/fall	rise/fall	rise/fall
in	in	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00

		Input Delay		Related		
Input Port		Min	Max	Related	Related	
		Rise	Fall	Clock	Pin	
in	--	--	1.00	1.00	clk3	--

		Resistance (min/max)	
Input Port		Rise	Fall
in	--	--	--

The reports show that clk1 is the launching clock at the top level, and the set_input_delay command at the block level does not contain a min rise/fall specification.

Fix Suggestion

To resolve the issue, the signals for both the top level design and block level should be reviewed. The signals should be defined with consistent clock periods, edges, and min/max specifications.

See Also

[set_input_delay](#)

[set_output_delay](#)

[analyze_paths](#)

[report_port](#)

Rule: B2T_EXD_0002

blk_port in block design has input_or_output delay constraints that are missing in the top instance pin.

Severity: Error

Description

The port in the block design has set_input_delay or set_output_delay constraints that do not have signals in the corresponding hierarchical pin in the top instance. The registers feeding the input of the block or receiving the output signals from the block have no clock specifications that match the clocks in the constraint definition. If the block level port is directly connected to a top level port, then the signals used for comparison are obtained by the set_input_delay or set_output_delay constraints at the top level.

Risk Associated with Violation

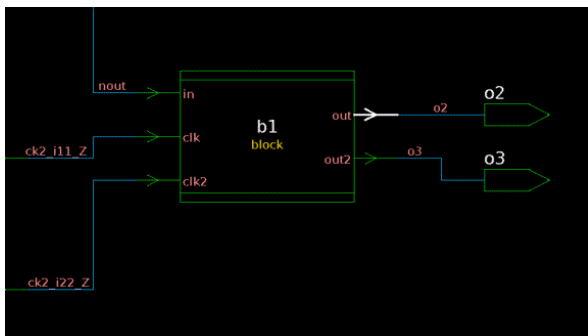
The top level will not be analyzed under the same conditions as the block level. This might result in less analysis coverage or even incorrect behavior of the block in the design.

One Possible Scenario

Consider a scenario where a set_output_delay constraint is set on the output port out at the block level:

```
current_design block  
  
set_output_delay 2 -clock clk2 [get_ports out]
```

The set_output_delay constraint at the block level sets an output path delay on the port out. At the top level, the **b1/out** pin is connected to the output port **o2**. However, no corresponding output delay has been specified on the design. A B2T_EXD_0002 violation is issued due to the missing set_output_delay constraint for the output port o2 at the top level.



Top Level Schematic

What Next

The "Violation Details" section shows the details about the block level output delay constraint. To see the output delay constraint specified on the output port at the block level, use the `report_port` command with the `-verbose` option. For the example scenario, the command can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> report_port -verbose [get_ports out]
*****
Report : port
        -input_delay
        -drive
        -output_delay
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

Port	Dir	Pin Cap		Wire Cap	
		Min rise/fall	Max rise/fall	Min rise/fall	Max rise/fall
out	out	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00

Output Port	Output Delay				Related Clock	Related Pin
	Min		Max			
	Rise	Fall	Rise	Fall		
out	2.00	2.00	2.00	2.00	clk2	--

The report shows that an output delay of 2 time units has been specified for max rise, max fall, min rise, and min fall analysis relative to **clk2** on the output port out in the block level constraints. You can use the `analyze_paths` command with the **-through** and **-path_type** summary options at the top level to get a summary of the paths through the pin at the top level to investigate a potentially missing capture clock or the `set_output_delay` command for this scenario.

Fix Suggestion

To resolve the issue, the signals for both the top level design and block level should be reviewed. The signals should be defined with consistent clock periods, edges, and min/max specifications.

See Also

[set_input_delay](#)
[set_output_delay](#)
[report_port](#)
[analyze_paths](#)

Rule: B2T_EXD_0003

top_pin in top instance and blk_port in block design have different input_or_output delay constraints.

Severity: Error

Description

The port in the block design has set_input_delay or set_output_delay constraints that do not match the signals in the corresponding hierarchical pin in the top instance. The registers feeding the input of the block or receiving the output signals from the block have clock periods, edges, and/or min/max path specifications that do not match the clocks in the constraint definition. If the block-level port is directly connected to a top-level port, the signals used for comparison are obtained by the set_input_delay or set_output_delay constraints at the top level.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as the top level. This might result in less analysis coverage or even incorrect behavior of the block.

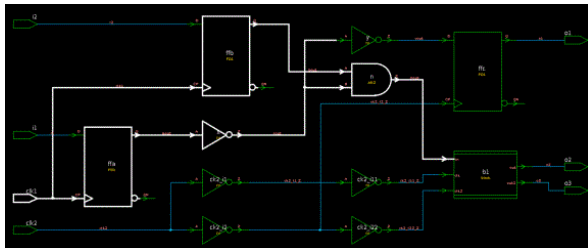
One Possible Scenario

Consider a scenario where a set_input_delay constraint is set on the input port in the block-level constraints:

```
current_design top
create_clock -period 10 -name clk1 [get_ports clk1]

current_design block
create_clock -period 20 -name clk3 [get_ports clk3]
set_input_delay 1 -clock clk3 [get_ports in]
```

The registers feeding the input port in of the block are clocked by **clk1**, which has a period of 10 time units at the top level. However, the input port in of the block is clocked by **clk3**, which has a period of 20 time units. A B2T_EXD_0003 violation is issued due to the mismatch in signals between the top level b1/in pin and the block level input port in.



Top Level Schematic

What Next

The "Violation Details" section shows the details about the block level input delay constraint and the related top level propagated clock. Use the `analyze_paths` command with the `-through` and `-path_type` summary options at the top level to get a summary of the paths through the pin. For the example scenario, the path summary can be generated with the following commands:

```
ptc_shell> current_design top

ptc_shell> analyze_paths -path_type summary -through [get_pins b1/in]

*****
Report : analyze_paths
-path_type summary
-max_endpoints = 1000
Design : top
Version: ...
Date : ...
*****
Summary of Paths:
```

Clocks	Endpoint	Dominant	Overridden	Count
Startpoint		Constraint	Constraints	(r,f,R,F)
Launch/Capture				
ffa/CP (FD1)	b1/ffg/D (FD1)			(1,1,1,1)
clk1/clk2				
ffb/CP (FD1)	b1/ffg/D (FD1)			(1,1,1,1)
clk1/clk2				
ffa/CP (FD1)	b1/ffh/D (FD1)			(1,1,1,1)
clk1/clk2				
ffb/CP (FD1)	b1/ffh/D (FD1)			(1,1,1,1)
clk1/clk2				

This shows that the launch clock for all paths through the b1/in pin is clk1. To see the delay definitions related to the block level port, use the **report_port -verbose** command. For the example scenario, the report is as follows:

```
ptc_shell> current_design block

ptc_shell> report_port -verbose
*****
Report : port
        -input_delay
        -drive
        -output_delay
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

Port	Pin Dir	Pin Cap		Wire Cap		rise/fall
		Min	Max	Min	Max	
in	in	0.00	0.00	0.00	0.00	0.00/0.00

```
Attributes
```

Input Port	Input Delay		Related		Clock	Pin	Related
	Rise	Fall	Rise	Fall			
in	1.00	1.00	1.00	1.00	clk3	--	

Input Port	Resistance (min/max)		Fall
	Rise	Fall	
in	--	--	--

The reports show that clk1 is the launching clock at the top level, and clk3 is the clock used in the set_input_delay command at the block level. The clk1 clock at the top level has a period of 10 time units, and the set_input_delay constraint at the block level is defined relative to clk3 which has a clock period of 20 time units.

Fix Suggestion

To resolve the issue, the signals for both the top level design and block level should be reviewed. The signals should be defined with consistent clock periods, edges, and min/max specifications.

See Also

[set_input_delay](#)

[set_output_delay](#)

[report_port](#)

[analyze_paths](#)

Rule: B2T_OPC_0001

An operating condition is specified on instance **inst** in top instance but not in block design.

Severity: Error

Description

An operating condition is created and set on a particular instance at the top level. However, there is no operating condition defined and set on the corresponding instance in the block level constraints.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where an operating condition is created and set on the instance **b1/zInv3** at the top level.

```
create_operating_conditions -name user_oc_max -library lsi_10k \  
    -process 1 -voltage 1.5 -temperature 125  
create_operating_conditions -name user_oc_min -library lsi_10k \  
    -process 1 -voltage 10 -temperature -40  
set_operating_conditions -max user_oc_max -min user_oc_min \  
    -object [get_cells b1/zInv3]
```

This creates a new set of operating conditions called **user_oc_max** and **user_oc_min** in the library **lsi_10k** for the instance **b1/zInv3** at the top level. As a result, these operating conditions specify new values for voltage and temperature for max and min scenario on the instance **b1/zInv3** at the top level. However, there is no corresponding max and min operating conditions defined on the instance **zInv3** in the block level constraints. A B2T_OPC_0001 violation will be issued due to the missing user-defined operating conditions on the instance **zInv3** in the block level constraints.

What Next

The "Violations Details" section will show the operating condition details for max and min scenario at the top level. This includes the name of the operating condition, the process, voltage, and temperature values defined for max and min respectively at the top level. To view the operating conditions defined in the specified library, use the `report_lib library_name` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> report_lib lsi_10k

*****
Report : library
Library: lsi_10k
Version: ...
Date   : ...
*****
```

Operating Conditions:

Name	Process	Temp	Voltage	Tree Type
<lib_default>	1.00	25.00	5.00	balanced_case
user_oc_max	1.00	125.00	1.50	balanced_case
user_oc_min	1.00	-40.00	10.00	balanced_case

The above report shows the list of operating conditions defined for the library **lsi_10k**. This includes the default operating condition `lib_default`, which already exists in the library, and the user-defined operating conditions such as `user_oc_max` and `user_oc_min`.

To view the operating conditions defined on the instance at the top level, use the `report_cell top_instance` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design Top

ptc_shell> report_cell [get_cells b1/zInv3]
*****
Report : cell
Design : Top
Version: ...
Date   : ...
*****
```

Cell	Reference	Library	Attributes
zInv3	IV	lsi_10k	
Total 1 cells			

Cell specific operating conditions

Cell	Min OC	Max OC
b1/zInv3	lsi_10k/user_oc_min	lsi_10k/
user_oc_max		

The above report for the top-level instance shows the cell specific operating conditions called `user_oc_min` and `user_oc_max` defined for min and max scenario respectively.

To check the missing cell specific operating conditions from the block level constraints, use the `report_cell block_instance` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> report_cell [get_cells zInv3]

*****
Report : cell
Design : block
Version: ...
Date   : ...
*****
```

Cell	Reference	Library	Attributes
zInv3	IV	lsi_10k	
Total 1 cells			

The above report shows that there are no cell specific operating conditions defined for the cell **zInv3** in the block level constraints.

Fix Suggestion

To resolve the issue, create and set proper operating condition for the particular instance in the block level constraints or remove the operating condition definition from the top level constraints. For the example scenario, the max and min operating conditions can be created and defined at the block level as follows:

```
create_operating_conditions -name user_oc_max -library lsi_10k \
    -process 1 -voltage 1.5 -temperature 125
create_operating_conditions -name user_oc_min -library lsi_10k \
    -process 1 -voltage 10 -temperature -40
set_operating_conditions -max user_oc_max -min user_oc_min -object
zInv3
```


See Also

[create_operating_conditions](#)

[set_operating_conditions](#)

[remove_operating_conditions](#)

[report_lib](#)

[report_cell](#)

[B2T_OPC_0002](#)

Rule: B2T_OPC_0002

An operating condition is specified on instance **inst** in block design but not in top instance.

Severity: Error

Description

An operating condition is created and set on a particular instance in the block design. However, there is no operating condition defined and set on the corresponding instance at the top level.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where an operating condition is created and set on the instance **zInv3** at the block level.

```
create_operating_conditions -name user_oc_max -library lsi_10k \  
    -process 1 -voltage 1.5 -temperature 125  
create_operating_conditions -name user_oc_min -library lsi_10k \  
    -process 1 -voltage 10 -temperature -40  
set_operating_conditions -max user_oc_max -min user_oc_min \  
    -object [get_cells zInv3]
```

This creates a new set of operating conditions called **user_oc_max** and **user_oc_min** in the library **lsi_10k** for the instance **zInv3** in the block design. As a result, these operating conditions specify new values for voltage and temperature for max and min scenario on the instance **zInv3** in the block design. However, there is no corresponding max and min operating conditions defined on the instance **b1/zInv3** in the top level constraints.

A B2T_OPC_0002 violation will be issued due to the missing user-defined operating conditions on the instance **b1/zInv3** at the top level.

What Next

The "Violations Details" section will show the operating condition details for max and min scenario in the block design. This includes the name of the operating condition, the process, voltage, and temperature values defined for max and min respectively at the block level. To view the operating conditions defined in the specified library, use the `report_lib library_name` command. For the example scenario, the command can be issued as follows:

```
ptc_shell> report_lib lsi_10k

*****
Report : library
Library: lsi_10k
Version: ...
Date   : ...
*****
```

Operating Conditions:

Name	Process	Temp	Voltage	Tree Type
<lib_default>	1.00	25.00	5.00	balanced_case
user_oc_max	1.00	125.00	1.50	balanced_case
user_oc_min	1.00	-40.00	10.00	balanced_case

The above report shows the list of operating conditions defined for the library **lsi_10k**. This includes the default operating condition `lib_default`, which already exists in the library, and the user-defined operating conditions such as `user_oc_max` and `user_oc_min`.

To view the operating conditions defined on the instance in the block design, use the `report_cell block_instance` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block

ptc_shell> report_cell [get_cells zInv3]

*****
Report : cell
Design : block
Version: ...
Date   : ...
*****
```

Cell	Reference	Library	Attributes
zInv3	IV	lsi_10k	

Total 1 cells

Cell specific operating conditions

Cell	Min OC	Max OC
zInv3	lsi_10k/user_oc_min	lsi_10k/
user_oc_max		

The above report for the block-level instance shows the cell specific operating conditions called `user_oc_min` and `user_oc_max` defined for min and max scenario respectively.

To check the missing cell specific operating conditions from the top level constraints, use the `report_cell top_instance` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design Top
report_cell [get_cells b1/zInv3]

*****
Report : cell
Design : Top
Version: ...
Date   : ...
*****
```

Cell	Reference	Library	Attributes
zInv3	IV	lsi_10k	

Total 1 cells

The above report shows that there are no cell specific operating conditions defined for the cell **b1/zInv3** in the top level constraints.

Fix Suggestion

To resolve the issue, create and set proper operating condition for the particular instance in the top level constraints or remove the operating condition definition from the block level constraints. For the example scenario, the max and min operating conditions can be created and defined at the top level as follows:

```
create_operating_conditions -name user_oc_max -library lsi_10k \
    -process 1 -voltage 1.5 -temperature 125
```

```
create_operating_conditions -name user_oc_min -library lsi_10k \  
    -process 1 -voltage 10 -temperature -40  
set_operating_conditions -max user_oc_max -min user_oc_min -object  
b1/zInv3
```

See Also

[create_operating_conditions](#)

[set_operating_conditions](#)

[remove_operating_conditions](#)

[report_lib](#)

[report_cell](#)

[B2T_OPC_0001](#)

Rule: B2T_OPC_0003

The ***object_type object*** has incompatible operating condition specification in top instance and block design.

Severity: Error

Description

An operating condition is created and set on a particular instance at the top level. However, the operating condition defined and set on the corresponding instance in the block level constraints is different.

Risk Associated with Violation

The block level will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where an operating condition is defined on the instance **b1/zInv3** at the top level and an operating condition with a different voltage value is defined on the instance **zInv3** in the block design.

```
current_design top  
create_operating_conditions -name user_max_top -library lsi_10k \  
    -process 1 -voltage 3.0 -temperature 125  
create_operating_conditions -name user_min_top -library lsi_10k \  
    -process 1 -voltage 10.0 -temperature -40  
set_operating_conditions -max user_max_top -min user_min_top \  
    -object [get_cells b1/zInv3]
```

```

current_design block
create_operating_conditions -name user_max_block -library lsi_10k
\
    -process 1 -voltage 1.0 -temperature 125
create_operating_conditions -name user_min_block -library lsi_10k
\
    -process 1 -voltage 8.0 -temperature -40
set_operating_conditions -max user_max_block -min user_min_block \
    -object [get_cells zInv3]

```

This creates a new set of operating conditions called `user_max_top` and `user_min_top` in the library `lsi_10k` for the instance `b1/zInv3` at the top level. It also creates a new set of operating conditions called `user_max_block` and `user_min_block` in the library `lsi_10k` for the instance `zInv3` in the block design. But the operating condition voltage values for max and min scenario are different at the top level and in the block design. A B2T_OPC_0003 violation will be issued due to the mismatch in voltage values on the instance `b1/zInv3` at the top level and instance `zInv3` in the block level constraints.

What Next

The "Violations Details" section will show the operating condition details for max and min scenario at the top level and in the block design. This includes the name of the operating condition, the process, voltage, and temperature values defined for max and min respectively at the top level. To view the operating conditions defined in the specified library, use the `report_lib library_name` command. For the example scenario, the command can be issued as follows:

```

ptc_shell> report_lib lsi_10k

*****
Report : library
Library: lsi_10k
Version: ...
Date   : ...
*****

```

Operating Conditions:

Name	Process	Temp	Voltage	Tree Type
<lib_default>	1.00	25.00	5.00	balanced_case
user_max_top	1.00	125.00	3.00	balanced_case
user_min_top	1.00	-40.00	10.00	balanced_case
user_max_block	1.00	125.00	1.00	balanced_case
user_min_block	1.00	-40.00	8.00	balanced_case

The above report shows the list of operating conditions defined for the library `lsi_10k`. This includes the default operating condition `lib_default`, which already exists in the

library, and the user-defined operating conditions. The user defined operating conditions include `user_max_top` and `user_min_top` at the top level and `user_max_block` and `user_min_block` in the block design.

To view the operating conditions defined on the instance at the top level, use the `report_cell top_instance` command. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design Top
```

```
ptc_shell> report_cell [get_cells b1/zInv3]
```

```
*****
```

```
Report : cell
```

```
Design : Top
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

Cell	Reference	Library	Attributes
------	-----------	---------	------------

zInv3	IV	lsi_10k	
-------	----	---------	--

Total 1 cells

Cell specific operating conditions

Cell	Min OC	Max OC
------	--------	--------

b1/zInv3	lsi_10k/user_min_top	lsi_10k/
----------	----------------------	----------

user_max_top

The above report for the top-level instance shows the cell specific operating conditions called `user_min_top` and `user_max_top` defined for min and max scenario respectively.

To check the cell specific operating conditions from the block level constraints, use the `report_cell block_instance` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block
```

```
ptc_shell> report_cell [get_cells zInv3]
```

```
*****
```

```
Report : cell
```

```
Design : block
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

Cell	Reference	Library	Attributes
------	-----------	---------	------------

zInv3	IV	lsi_10k

Total 1 cells		
Cell specific operating conditions		
Cell	Min OC	Max OC

zInv3	lsi_10k/user_min_block	lsi_10k/
user_max_block		

The above report of the instance in the block design shows the cell specific operating conditions called `user_min_block` and `user_max_block` defined for min and max scenario respectively.

Fix Suggestion

To resolve the issue, the operating condition values for both the top level design and block level design should be reviewed. The operating condition values should be identical for the instance of interest both at the top level and block level. For the example scenario, modify the voltage values in the block design as follows:

```

create_operating_conditions -name user_max_block -library lsi_10k
\
    -process 1 -voltage 3 -temperature 125
create_operating_conditions -name user_min_block -library lsi_10k
\
    -process 1 -voltage 10 -temperature -40
set_operating_conditions -max user_max_block -min user_min_block \
    -object zInv3

```

See Also

[create_operating_conditions](#)

[set_operating_conditions](#)

[remove_operating_conditions](#)

[report_lib](#)

[report_cell](#)

Rule: B2T_OPC_0004

The top design and block design have incompatible operating condition analysis types.

Severity: Error

Description

An operating condition with a particular analysis type is set at the top level. However, the analysis type in the operating condition set for the block level is different.

Risk Associated with Violation

The block will not be analyzed under the same conditions as it was created. This might result in less analysis coverage or even incorrect behavior of the block.

Possible Scenarios

Consider a scenario where an operating condition with analysis type "bc_wc" is set at the top level and an operating condition with analysis type "on_chip_variation" is set on the block design.

```
current_design top
create_operating_conditions -name user_max_top -library lsi_10k \
    -process 1 -voltage 5.0 -temperature 125
create_operating_conditions -name user_min_top -library lsi_10k \
    -process 1 -voltage 10.0 -temperature -40
set_operating_conditions -max user_max_top -min user_min_top \
    -analysis_type bc_wc

current_design block
create_operating_conditions -name user_max_block -library lsi_10k \
    -process 1 -voltage 5.0 -temperature 125
create_operating_conditions -name user_min_block -library lsi_10k \
    -process 1 -voltage 10.0 -temperature -40
set_operating_conditions -max user_max_block -min user_min_block \
    -analysis_type on_chip_variation
```

This creates a new set of operating conditions called `user_max_top` and `user_min_top` in the library **lsi_10k** for the top level design. These operating conditions at the top level will be analyzed as the "bc_wc" mode. Similarly, a new set of operating conditions called `user_max_block` and `user_min_block` is created in the library **lsi_10k** for the block design. But these operating conditions at the block design will be analyzed as "on_chip_variation" mode. A B2T_OPC_0004 violation will be issued due to the mismatch in the type of analysis set at the top level and at the block level.

What Next

The "Violations Details" section will show the operating condition details along with the type of analysis specified at the top level and in the block design. To view the operating conditions with analysis type defined in the specified library, use the `report_design` command at the top and block level. For the example scenario, the command at the top level can be issued as follows:

```
ptc_shell> current_design Top
```



```
ptc_shell> report_design
```

```
*****
Report : design
Design : Top
Version: ...
Date   : ...
*****
```

Design Attribute	Value

Operating Conditions:	
analysis_type	bc_wc
operating_condition_min_name	user_min_top
process_min	1.00
temperature_min	-40.00
voltage_min	10.00
tree_type_min	balanced_case
operating_condition_max_name	user_max_top
process_max	1.00
temperature_max	125.00
voltage_max	5.00
tree_type_max	balanced_case

The above report for the top-level design shows the details of the operating conditions and the type of analysis as "bc_wc".

Similarly, to view the operating conditions with analysis type defined for the block, use the `report_design` command. For the example scenario, the command at the block level can be issued as follows:

```
ptc_shell> current_design block
```

```
ptc_shell> report_design
```

```
*****
Report : design
Design : block
Version: ...
Date   : ...
*****
```

Design Attribute	Value

Operating Conditions:	
analysis_type	on_chip_variation
operating_condition_min_name	user_min_block

```
process_min                1.00
temperature_min            -40.00
voltage_min                10.00
tree_type_min              balanced_case

operating_condition_max_name user_max_block
process_max                1.00
temperature_max            125.00
voltage_max                5.00
tree_type_max              balanced_case
```

The above report for the block level design shows the details of the operating conditions defined and the type of analysis as "on_chip_variation".

Fix Suggestion

To resolve the issue, the analysis type specification for both the top level design and block level design should be reviewed. The type of analysis should be identical for both at the top level and block level. For the example scenario, modify the analysis type in the block design as follows:

```
create_operating_conditions -name user_max_block -library lsi_10k
\
    -process 1 -voltage 5 -temperature 125
create_operating_conditions -name user_min_block -library lsi_10k
\
    -process 1 -voltage 10 -temperature -40
set_operating_conditions -max user_max_block -min user_min_block \
    -analysis_type bc_wc
```

See Also

[create_operating_conditions](#)

[set_operating_conditions](#)

[remove_operating_conditions](#)

[report_design](#)

Rule: B2T_TRL_0001

Port in the top design has input_transition/load constraint that is missing in the block-level port connected to it.

Severity: Warning

Description

The given port in the top design has a transition/load constraint defined on it. However, the corresponding port in the block design has no transition/load constraint defined on it.

Risk Associated With Violation

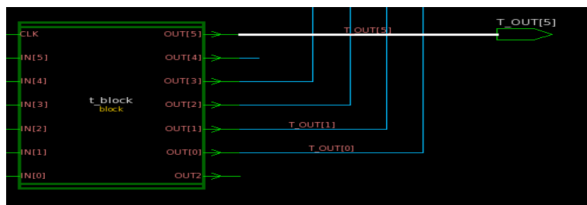
The block level will not be analyzed under the same conditions as it was created. This might result in inaccurate analysis coverage or even incorrect behavior for the block or the full chip.

Possible Scenarios

Consider a scenario where a load value of 10 is set on the **T_OUT[5]** output port of the **Top** and there is no load value is set on the **OUT[5]** output port in **block** which is directly connected to the **T_OUT[5]** of top.

```
current_design top
set_load 10 [get_ports T_OUT[5]]
current_design block
# set_load 10 [get_ports OUT[5]] (no load has been set because line
has been commented)
```

As shown in the constraints, constraint consistency issues a **B2T_TRL_0001** violation for the **T_OUT[5]** output port.



What Next

The "Violation Details" section shows the top port name and the block port name along with the user-defined load(capacitance)/transition values on the top port. To see transition present on the port, use 'report_port -drive' on the top port. to see the pin capacitance present on the port, use 'report_port -verbose' on the top port.

For the example scenario, the command can be issued as follows:

```
ptc_shell> report_port T_OUT[5] -verbose
```

```
*****
Report : port
        -input_delay
        -drive
        -output_delay
Design : top
```

Scenario: default
Version: ...
Date : ...

Port	Dir	Pin Cap		Wire Cap	
		Min	Max	Min	Max
Attributes		rise/fall	rise/fall	rise/fall	rise/fall
T_OUT[5]	out	10.00/10.00	10.00/10.00	0.00/0.00	0.00/0.00

Output Port	Output Delay				Related Clock	Related Pin
	Min	Max	Related	Related		
	Rise	Fall	Rise	Fall		
T_OUT[5]	--	--	--	--	--	--

To see the pin capacitance present on the block port:

```
ptc_shell> current_design block
{"block"}
```

```
ptc_shell> report_port OUT[5] -verbose
```

```
Report : port
-input_delay
-drive
-output_delay
Design : block
Scenario: default
Version: ...
Date : ...
```

Port	Dir	Pin Cap		Wire Cap	
		Min	Max	Min	Max
Attributes		rise/fall	rise/fall	rise/fall	rise/fall
OUT[5]	out	0.00/0.00	0.00/0.00	0.00/0.00	0.00/0.00

Output Port	Output Delay				Related Clock	Related Pin
	Min	Max	Related	Related		
	Rise	Fall	Rise	Fall		
OUT[5]	--	--	--	--	--	--

Fix Suggestion

To resolve the issue, the load/input_transition values for both the top port and block port should be reviewed. And the load/input_transition values should be identical for the top port and block port which are directly connected without any intermediate cells.

See Also

[set_load](#)

[set_input_transition](#)

Rule: B2T_TRL_0002

The port in the block design has transition/load constraint that is missing in the top-level port connected to it.

Severity: Warning

Description

The port in the block design has transition/load constraint which is missing from the corresponding top level port connected to it with respect to the min/max path type.

Risk Associated With Violation

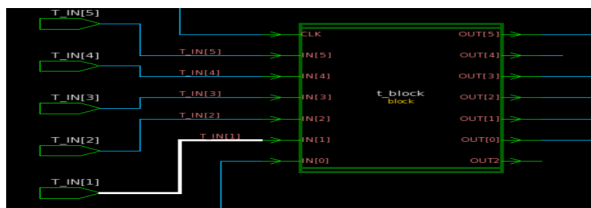
The block level will not be analyzed under the same conditions as it was created. This might result in inaccurate analysis coverage or even incorrect behavior for the block or the full chip.

Possible Scenarios

Consider a scenario where an input_transition value of 0.5 is set on the **IN[1]** input port of the **block** and there is no input_transition value set on the **T_IN[1]** input port in **top** which is directly connected to the **IN[1]** of top.

```
current_design top
set_input_transition 0.5 [get_ports T_IN[1]]
current_design block
set_input_transition 0.5 [get_ports IN[1]]
```

As shown in the constraints, constraint consistency issues a **B2T_TRL_0002** violation for the **IN[1]** port.



What Next

The "Violation Details" section will show the top port name and the block port name along with the user-defined load(capacitance)/transition values on block port. To see transition present on the port, use 'report_port -drive' on the top port. to see the Pin capacitance present on the port, use 'report_port -verbose' on the top port.

For the example scenario, the command can be issued as follows:

```
ptc_shell> current_design block
{"block"}

ptc_shell> report_port IN[1] -drive
*****
Report : port
        -drive
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

Resistance (min/max)			
Input Port	Rise	Fall	
IN[1]	--	--	

Transition (min/max)			
Input Port	Rise	Fall	Related Clock
IN[1]	0.50/0.50	0.50/0.50	
1			

To see the input_transition present on the top port:

```
ptc_shell> current_design top
```

```

{"top"}

ptc_shell> report_port T_IN[1] -drive
*****
Report : port
        -drive
Design : top
Scenario: default
Version: ...
Date   : ...
*****

```

Resistance (min/max)		
Input Port	Rise	Fall
T_IN[1]	--	--
1		

Fix Suggestion

To resolve the issue, the load/input_transition values for both the top port and block port should be reviewed. And the load/input_transition values should be identical for the top port and block port which are directly connected without any intermediate cells.

See Also

[set_load](#)

[set_input_transition](#)

Rule: B2T_TRL_0003

The port in the top design and the port in the block design have different transition/load constraint.

Severity: Info

Description

The port in the block design has transition/load constraint which do not match in the corresponding top level port connected to it with respect to the min/max path type.

Risk Associated With Violation

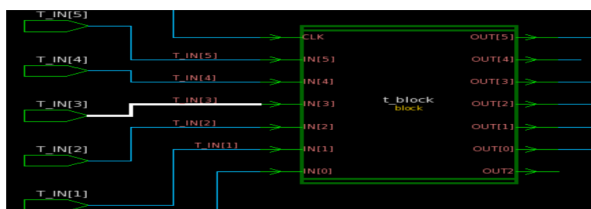
The block level will not be analyzed under the same conditions as it was created. This might result in inaccurate analysis coverage or even incorrect behavior for the block or the full chip.

Possible Scenarios

Consider a scenario where an input_transition value of 1.0 is set on the input port **T_IN[3]** of the **Top** and where an input_transition value of 0.5 is set on the input port **IN[3]** in **block** which is directly connected to the **T_IN[3]** of top.

```
current_design top
set_input_transition 1.0 [get_ports T_IN[3]]
current_design block
set_input_transition 0.5 [get_ports IN[3]]
```

As shown in the constraints, constraint consistency issues a B2T_TRL_0002 violation for the ports IN[3] and T_IN[3].



What Next

The "Violation Details" section will show the top port name and the block port name along with the user defined load(capacitance)/transition values on block port as well as top port which are directly connected to each other. To see transition present on the port, use 'report_port -drive' on the top port. to see the Pin capacitance present on the port, use 'report_port -verbose' on the top port.

For the example scenario, the command can be issued as follows:

```
ptc_shell> report_port T_IN[3] -drive
*****
Report : port
        -drive
Design : top
Scenario: default
Version: ...
Date   : ...
*****
```

Resistance (min/max)		
Input Port	Rise	Fall
T_IN[3]	--	--

Input Port	Transition (min/max)		Related Clock
	Rise	Fall	
IN[3]	1.00/1.00	1.00/1.00	
1			

To see the input_transition present on the block port:

```
ptc_shell> current_design block

{"block"}

ptc_shell> report_port IN[3] -drive
*****
Report : port
        -drive
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

Input Port	Resistance (min/max)	
	Rise	Fall
IN[3]	--	--

Input Port	Transition (min/max)		Related Clock
	Rise	Fall	
IN[3]	0.50/0.50	0.50/0.50	
1			

Fix Suggestion

To resolve the issue, the load/input_transition values for both the top port and block port should be reviewed. And the load/input_transition values should be identical for the top port and block port which are directly connected without any intermediate cells.

See Also

[set_load](#)

[set_input_transition](#)

Rule: B2T_UNC_0001

Clock uncertainty applied to '**object_type**"**object**' in top instance is missing in block instance.

Severity: Error

Description

A simple clock uncertainty setting is specified in the top design but not in the block design.

Risk Associated With Violation

The clock uncertainty applied in the top constraints will affect timing results. The delay calculation involving cells in the block will not be consistent between block timing analysis and top timing analysis.

Possible Scenarios

Consider a design where data entering a block is triggered by clock clk1, and clock clk2 captures inside the block b1 (see Figure 1 below). If a simple clock uncertainty is declared in the top-level constraints, it also needs to be present in the block-level constraints. Failure to do so can result in different timing numbers between block timing analysis and top timing analysis.

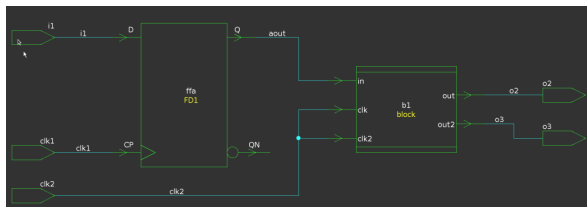


Figure 1: Clock Uncertainty

Block constraints

```
create_clock -name clk -period 10 -waveform [list 0 5] [get_ports clk]
```

```
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports clk2]
```

Top constraints

```
create_clock -period 10 -name clk1 – waveform [list 0 5] [get_ports clk1]
```

```
create_clock – period 10 – name clk2 – waveform [list 0 5] [get_ports clk2]
```

```
set_clock_uncertainty 0.65 -setup [get_clocks clk1]
```

What Next

The file name and line number where the clock uncertainty is specified is displayed in the Info Pane window. Clicking the hyperlink opens the SDC Browser and points to the relevant command. In addition, an html table summarizes the current clock uncertainty related to this violation.

You can review the current clock uncertainty settings using the `report_clocks` command.

```
ptc_shell> current_design B2T_UNC_0001
```

```
{"B2T_UNC_0001"}
```

```
ptc_shell> report_clock -skew
```

```
*****
```

```
Report : clock_skew
```

```
Design : B2T_UNC_0001
```

```
Scenario: default
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Min Rise Min Fall Max Rise Max Fall Hold Setup Related
```

```
Object Delay Delay Delay Delay Uncertainty Uncertainty Clock
```

```
-----
```

```
clk1 - - - - 0.65 --
```

```
Fix Suggestion
```

Constraints affecting the clock uncertainty for both top level design and block level design should be reviewed. In the example above, there are two possible ways to solve the problem:

- Remove the simple clock uncertainty at the top level
- Add the same clock uncertainty on the block design

In this example, adding the clock uncertainty at the block level should be done with a virtual clock. Here is an example:

```
create_clock - name clk1_virtual - period 10 - waveform [list 0 5]
```

```
set_clock_uncertainty 0.65 -setup [get_clocks clk1_virtual]
```

See Also

[set_clock_uncertainty](#)

[report_clock](#)

Rule: B2T_UNC_0002

Clock uncertainty applied to '**object_type**' '**object**' in the top instance is missing in the block instance.

Severity: Error

Description

A simple clock uncertainty setting is specified in the block design but not in the top design.

Risk Associated With Violation

The clock uncertainty applied in the block constraints will affect timing results. The delay calculation involving cells in the block will not be consistent between block timing analysis and top timing analysis.

Possible Scenarios

Consider a design where data entering a block is triggered by clock clk1, and clock clk2 is used to capture inside the block b1 (see Figure 1 below). If a simple clock uncertainty is declared in the block level constraints, it also needs to be present in the top level constraints. Failure to do so can result in different timing numbers between block timing analysis and top timing analysis.

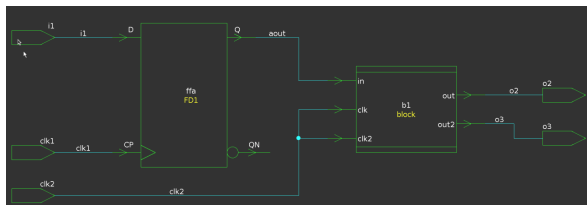


Figure 1: Clock Uncertainty

Block constraints

```
create_clock -name clk1_virtual -period 10 -waveform [list 0 5]  
set_clock_uncertainty 0.65 -setup [get_clocks clk1_virtual]
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]
```

What Next

The file name and line number where the clock uncertainty is specified is displayed in the Info Pane window. Clicking on the hyperlink opens the SDC Browser and points to the relevant command. In addition, a HTML table summarizes the current clock uncertainty related to this violation.

You can review the current clock uncertainty settings using the **report_clock** command.

```
ptc_shell> current_design block
{"block"}

ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : B2T_UNC_0002
Scenario: default
Version: ...
Date   : ...
*****
```

Hold	Setup	Min Rise	Min Fall	Max Rise	Max Fall
Object	Delay	Related			
Uncertainty	Uncertainty	Delay	Delay	Delay	Delay
Clock					
clk1_virtual	-	-	-	-	-
0.65	--				

Fix Suggestion

Constraints affecting the clock uncertainty for both top-level design and block-level design should be reviewed. In the previous example, there are two possible ways to solve the problem:

- Remove the simple clock uncertainty at the block level
- Add the same clock uncertainty on the top design

In this example, adding the clock uncertainty at the top level can be done as follows:

```
set_clock_uncertainty 0.65 -setup [get_clocks clk1]
```

See Also

[set_clock_uncertainty](#)

[analyze_clock_networks](#)

Rule: B2T_UNC_0003

Clock uncertainty applied to '**top_object_type**' '**top_object**' in the top instance does not match the clock uncertainty setting on '**blk_object_type**' '**blk_object**' in the block instance.

Severity: Error

Description

The values of a simple clock uncertainty specified in both the top design constraints and the block design constraints are inconsistent.

Risk Associated With Violation

The clock uncertainty applied in both top and block constraints will affect timing results. Because these values are different, the delay calculation involving cells in the block will not be similar between block timing analysis and top timing analysis.

Possible Scenarios

Consider a design where data entering a block is triggered by clock clk1, and clock clk2 is used to capture inside the block b1 (see Figure 1 below). Any simple clock uncertainty setting should match between the top and block constraints. Failure to do so can result in different timing numbers between block timing analysis and top timing analysis.

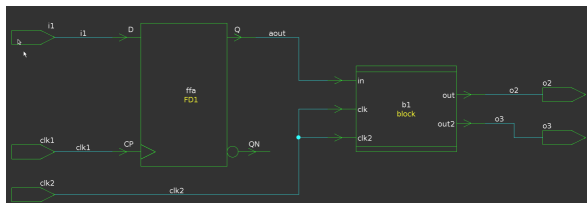


Figure 1: Clock Uncertainty

Block constraints

```
create_clock -name clk1_virtual -period 10 -waveform [list 0 5]  
set_clock_uncertainty 0.65 -setup [get_clocks clk1_virtual]
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]  
set_clock_uncertainty 0.80 -setup [get_clocks clk1]
```

What Next

The file name and line number where the clock uncertainty is specified is displayed in the Info Pane window for both constraints sets. Clicking the hyperlink opens the dual SDC Browser and points to the relevant command.

You can review the current clock uncertainty settings using the report_clock command.

```
ptc_shell> current_design B2T_UNC_0003
{"B2T_UNC_0003"}
```

```
ptc_shell> report_clock -skew
*****
```

```
Report : clock_skew
Design : B2T_UNC_0003
Scenario: default
Version: ...
Date : ...
*****
```

Setup	Min Rise	Min Fall	Max Rise	Max Fall	Hold
Object	Related				
Uncertainty	Delay	Delay	Delay	Delay	Uncertainty
	Clock				
clk1	-	-	-	-	-
0.80	--				

```
ptc_shell> current_design block
{"block"}
```

```
ptc_shell> report_clock -skew
*****
```

```
Report : clock_skew
Design : block
Scenario: default
Version: ...
Date : ...
*****
```

Setup	Min Rise	Min Fall	Max Rise	Max Fall	Hold
Object	Related				
Uncertainty	Delay	Delay	Delay	Delay	Uncertainty
	Clock				
clk1_virtual	-	-	-	-	-
0.65	--				

Fix Suggestion

Constraints affecting the clock uncertainty for both top level design and block level design should be reviewed and made identical.

Rule Property: Tolerance

This rule allows tolerance to be taken into consideration when performing simple clock uncertainty matching. This tolerance is specified as a percentage of the smallest value compared.

By default, this tolerance is set at 1e-5% which is equivalent to performing exact matches. This is fully under user control and can be changed through the `set_rule_property` command as demonstrated below:

```
set_rule_property tolerance 20 [get_rules B2T_UNC_0003]
```

See Also

[set_clock_uncertainty](#)

[report_clock](#)

Rule: B2T_UNC_0004

Clock uncertainty applied between **'from_clock'** and **'to_clock'** in top instance is missing in block instance.

Severity: Error

Description

An inter-clock uncertainty setting is specified at the top level but cannot be found in the block.

Risk Associated With Violation

The block level will not be analyzed under the conditions it was created. The inter clock uncertainty applied in the top constraints will affect timing results.

Possible Scenarios

Consider a design where data entering a block is triggered by clock `clk1`, and clock `clk2` is used to capture inside the block `b1` (see Figure 1 below). If an inter-clock uncertainty is specified at the top-level, it should also be specified at the block level. Failing to do so results in a B2T_UNC_0004 violation and, most likely, unexpected timing results on the paths ending inside the block.

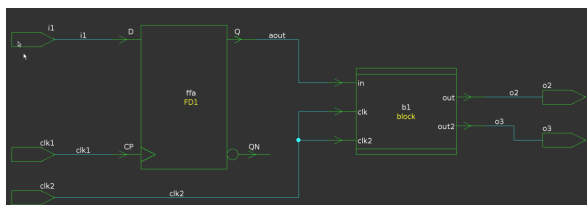


Figure 1: Inter-clock uncertainty

Block constraints

```
create_clock -name clk -period 10 -waveform [list 0 5] [get_ports clk]
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports
clk2]
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
set_clock_uncertainty -from [get_clocks clk1] -to [get_clocks clk2] 1.2
```

What Next

The file name and line number where the clock uncertainty is specified is displayed in the Info Pane window. Clicking the hyperlink opens the SDC Browser and points to the relevant command.

You can review the current clock uncertainty settings using the **report_clocks** command.

```
ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : B2T_UNC_0004
Scenario: default
Version: ...
Date : ...
*****
```

	From	To	Hold
Setup	Object	Object	Uncertainty
-----	-----	-----	-----
1.20	clk1	clk2	1.20

Fix Suggestion

Constraints affecting the clock uncertainty for both top-level design and block-level design should be reviewed. In the example above, there are two possible ways to solve the problem:

- Remove the inter clock uncertainty at the top level
- Add the same clock uncertainty on the block design

In this example, adding the clock uncertainty at the block level should be done with a virtual clock. Here is an example:

```
create_clock -name clk1_virtual -period 10 -waveform [list 0 5]
set_clock_uncertainty -from [get_clocks clk1_virtual] \
```

```
-to [get_clocks clk] 1.2
set_clock_uncertainty -from [get_clocks clk1_virtual] \
-to [get_clocks clk2] 1.2
```

See Also

[set_clock_uncertainty](#)

[report_clock](#)

Rule: B2T_UNC_0005

Clock uncertainty applied between ‘**from_clock**’ and ‘**to_clock**’ in block instance is missing in top instance.

Severity: Error

Description

An inter-clock uncertainty setting is specified at the block level but cannot be found in the top instance.

Risk Associated With Violation

The inter clock uncertainty applied in the block constraints will affect timing results.

Possible Scenarios

Consider a design where data entering a block is triggered by clock clk1, and clock clk2 is used to capture inside the block b1 (see Figure 1 below). If an inter-clock uncertainty is specified at the block level, it should also be specified at the top level. Failure to do so results in a B2T_UNC_0005 violation and, most likely, unexpected timing results on the paths ending inside the block.

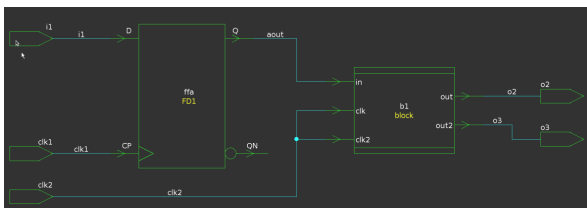


Figure 1: Inter-clock uncertainty

Block constraints

```
create_clock -name clk -period 10 -waveform [list 0 5] [get_ports clk]
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports
clk2]
create_clock -name clk1_virtual -period 10 -waveform [list 0 5]
```

```
set_clock_uncertainty -from [get_clocks clk1_virtual] \
-to [get_clocks clk2] 1.2
set_clock_uncertainty -from [get_clocks clk1_virtual] \
-to [get_clocks clk] 1.2
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
```

What Next

The file name and line number where the clock uncertainty is specified is displayed in the Info Pane window. Clicking the hyperlink opens the SDC Browser and points to the relevant command.

You can review the current clock uncertainty settings using the **report_clock** command.

```
ptc_shell> current_design block
{"block"}

ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : block
Scenario: default
Version: ...
Date : ...
*****
```

From Object	To Object	Hold Uncertainty	Setup Uncertainty
clk1_virtual	clk	1.20	1.20
clk1_virtual	clk2	1.20	1.20

Fix Suggestion

Constraints affecting the clock uncertainty for both top level design and block level design should be reviewed. In the example above, there are two possible ways to solve the problem:

- Add the inter-clock uncertainty at the top level
- Remove the clock uncertainty at the block design

In this example, adding the clock uncertainty at the level can be done as follows:

```
set_clock_uncertainty -from [get_clocks clk1] \
-to [get_clocks clk2] 1.2
```

See Also

[set_clock_uncertainty](#)

[report_clock](#)

Rule: B2T_UNC_0006

Clock uncertainty applied between **'top_from_clock'** and **'top_to_clock'** in top instance does not match the clock uncertainty applied between **'blk_from_clock'** and **'blk_to_clock'** in block instance.

Severity: Error

Description

The values of an inter-clock uncertainty setting specified both at the top design and block design are inconsistent.

Risk Associated With Violation

The inter clock uncertainty is applied with different values at the top level and block level. This will result in differences in delay calculation between the block and top designs.

Possible Scenarios

Consider a design where data entering a block is triggered by clock clk1, and clock clk2 is used to capture inside the block b1 (see Figure 1 below). If inter-clock uncertainty is required, it needs to be defined both in the top constraints and in the block constraints. Constraint consistency performs some checks related to the uncertainty values. If they do not match, a B2T_UNC_0006 violation is issued.

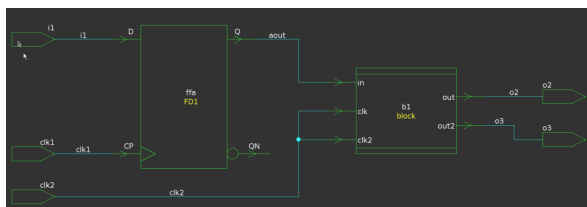


Figure 1: Inter-Clock Uncertainty

Block constraints

```
create_clock -name clk -period 10 -waveform [list 0 5] [get_ports clk]
create_clock -name clk2 -period 10 -waveform [list 0 5] [get_ports
  clk2]
create_clock -name clk1_virtual -period 10 -waveform [list 0 5]
set_clock_uncertainty -from [get_clocks clk1_virtual] \
-to [get_clocks clk2] 1.0
set_clock_uncertainty -from [get_clocks clk1_virtual] \
-to [get_clocks clk] 1.2
```

Top constraints

```
create_clock -period 10 -name clk1 -waveform [list 0 5] [get_ports clk1]
create_clock -period 10 -name clk2 -waveform [list 0 5] [get_ports clk2]
set_clock_uncertainty -from [get_clocks clk1] \
-to [get_clocks clk2] 1.2
```

What Next

The file name and line number where the clock uncertainty is specified is displayed in the Info Pane window. Clicking the hyperlink opens the dual SDC Browser and points to the relevant commands in each constraint file.

You can review the current clock uncertainty settings using the `report_clocks` command.

```
ptc_shell> current_design block
{"block"}
```

```
ptc_shell> report_clocks -skew
*****
Report : clock_skew
Design : block
Scenario: default
Version: ...
Date   : ...
*****
```

From Setup Uncertainty	Object	To Object	Hold Uncertainty
1.20	clk1_virtual	clk	1.20
1.20	clk1_virtual	clk2	1.20

```
ptc_shell> current_design B2T_UNC_0006
{"B2T_UNC_0006"}
```

```
ptc_shell> report_clocks -skew
*****
Report : clock_skew
Design : B2T_UNC_0006
Scenario: default
Version: ...
Date   : ...
*****
```

From Object	To Object	Hold Uncertainty	Setup Uncertainty
clk1	clk2	1.20	1.20

Fix Suggestion

Constraints affecting the clock uncertainty for both top level design and block level design should be reviewed and adjusted. In the example above, one way of solving the problem is to modify the block-level clock uncertainty to match the clock uncertainty from the top. Here is how to do this:

```
set_clock_uncertainty -from [get_clocks clk1_virtual] \  
-to [get_clocks clk2] 1.2  
set_clock_uncertainty -from [get_clocks clk1_virtual] \  
-to [get_clocks clk1] 1.2
```

Rule Property: Tolerance

This rule allows tolerance to be taken into consideration when performing inter-clock uncertainty matching. This tolerance is specified as a percentage of the smallest value compared.

By default, this tolerance is set at 1e-5% which is equivalent to performing exact matches. This is fully under user control and can be changed through the `set_rule_property` command as demonstrated below:

```
set_rule_property tolerance 20 [get_rules B2T_UNC_0006]
```

See Also

[set_clock_uncertainty](#)

[report_clock](#)

3

Constraint Comparison Rules

Rules highlighted in gray are disabled by default.

Rule ID	Severity	Message
S2S_CAS_0001	Error	Pin/Port pin in constraint set scenario1 has case value that is missing in constraint set scenario2 .
S2S_CAS_0003	Error	Pin/Port pin_port has different case values between the constraint set scenario1 and constraint set scenario2 .
S2S_CLK_0001	Error	Clock clock in constraint set scenario1 has no matching clock in constraint set scenario2 .
S2S_CLK_0003	Error	Clock clock1 in constraint set scenario1 does not match with clock clock2 in constraint set scenario2 .
S2S_CLK_0004	Error	Pin pin has clock sense applied in constraint set set1 that is missing in constraint set set2 .
S2S_CLK_0006	Error	Clock sense applied to pin pin in constraint set set1 does not match the clock sense applied in constraint set set2 .
S2S_CLK_0007	Error	arc_sense arc from from_pin to to_pin has clock sense applied in constraint set set1 but is missing in constraint set set2 .
S2S_CLK_0010	Error	Clock gating check applied to object_typeobject in constraint set scenario1 is missing in constraint set scenario2 .
S2S_CLK_0012	Error	Clock gating check applied to object_type object1 in constraint set scenario1 does not match the clock gating check applied to object2 in the constraint set scenario2 .
S2S_CLT_0001	Error	The constraint applied to object_typeobject in constraint set scenario1 is missing in constraint set scenario2 .
S2S_CLT_0003	Error	The constraint applied to object_type1object1 in constraint set scenario1 does not match the object_type2object2 applied in constraint set scenario2 .
S2S_CST_0001	Error	The constraint applied to object_type object in constraint set scenario1 is missing in constraint set scenario2 .

Rule ID	Severity	Message
S2S_CST_0003	Error	The constraint applied to object_type1object1 in constraint set scenario1 does not match the one applied on object_type2 object in constraint set scenario2 .
S2S_DIS_0001	Error	object_typeobject is disabled in constraint set scenario1 but not in constraint set scenario2 .
S2S_DIS_0003	Error	Arc objectobject is disabled in constraint set scenario1 but not in constraint set scenario2 .
S2S_EXC_0001	Error	Exception exception in constraint set scenario1 is missing in constraint set scenario2 .
S2S_EXC_0003	Error	Exception exception in constraint set scenario1 does not match exception exception2 in constraint set scenario2 .
S2S_EXD_0001	Error	pin_port has input_output delay constraints in constraint set scenario1 that are missing in constraint set scenario2 .
S2S_EXD_0003	Error	pin_port has different input_output delay constraints in constraint set scenario1 and constraint set scenario2 .
S2S_UNC_0001	Error	Clock uncertainty applied to object_typeobject in constraint set scenario1 is missing in constraint set scenario2 .
S2S_UNC_0003	Error	Clock uncertainty applied to object_type object1 in constraint set scenario1 does not match the clock uncertainty applied to object2 in the constraint set scenario2 .
S2S_UNC_0004	Error	Clock uncertainty applied between from_clock and to_clock in constraint set scenario1 is missing in constraint set scenario2 .
S2S_UNC_0006	Error	Clock uncertainty applied between from_clock1 and to_clock1 in constraint set scenario1 does not match the clock uncertainty applied between from_clock2 and to_clock2 in the constraint set scenario2 .

Rule: S2S_CAS_0001

Pin/Port '**pin**' in constraint set '**scenario1**' has a case value that is missing in constraint set '**scenario2**'.

Severity: Error

Description

A constant value is specified on the reported pin by one of the scenarios but it is not in the other. This results in a different set of paths to be timed between scenario1 and scenario2.

Risk Associated with Violation

The two sets of constraints do not have the same behavior on the design. This could lead to incorrect timing results.

Possible Scenarios

Below is an example of a set of constraints which would lead to an **S2S_CAS_0001** violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different.

Constraints set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

Constraints set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk2
set_output_delay 2 [list o2 o1 o3] -clock clk2_bis
set_case_analysis 0 [get_pins b1/ffg/D]
```

What Next

The Violation Details section of the GUI provides more information related to the case value itself as well as where it was defined in the constraint file. A table provides this information for the set of constraints where the constant is defined. Below is an example of such table:

Case	0
pin	b1/ffg/D
Source File	run.tcl, line 29

A schematic displaying the constant pin can be created by clicking either on the schematic link at the top of the Info Pane window or clicking on the [execute] hyperlink.

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this case, one of the following steps should be taken:

- Add the missing case value in the scenario2 constraints set.
- Remove the command from the scenario1 constraints set.

See Also

[report_case_details](#)

[set_case_analysis](#)

[set_false_path](#)

[set_disable_timing](#)

Rule: S2S_CAS_0003

Pin/Port '**pin_port**' has different case values between the constraint set 'scenario1' and constraint set 'scenario2'.

Severity: Error

Description

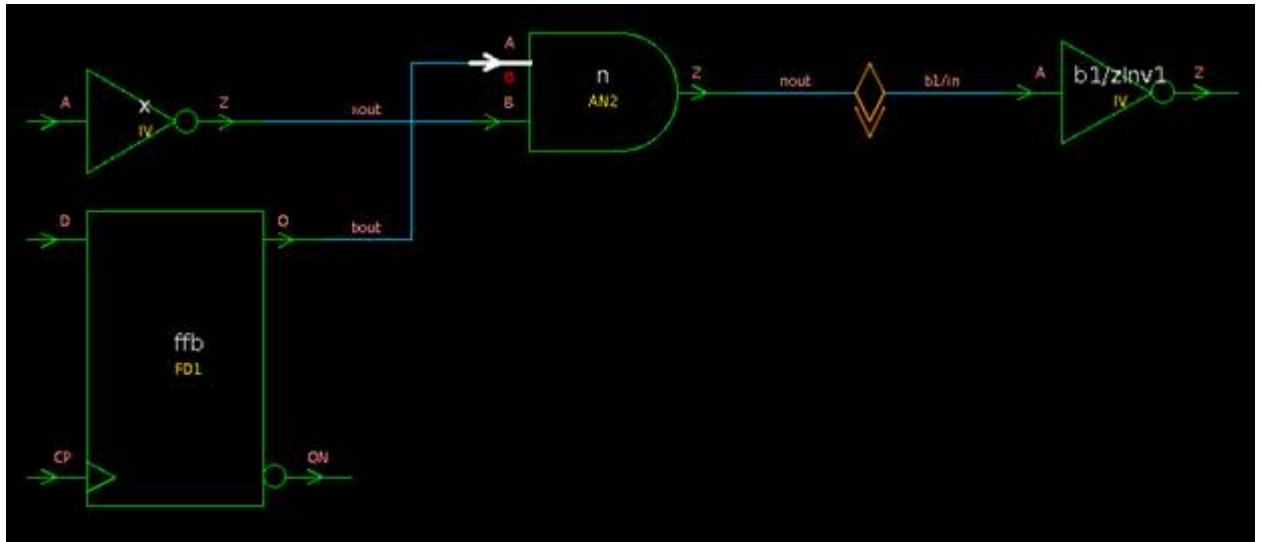
The reported pin is set to different constant values in the two scenarios. This results in a different set of paths to be timed between scenario1 and scenario2.

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This could lead to incorrect timing results.

Possible Scenarios

Below is an example of a set of constraints which would lead to a S2S_CAS_0003 violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different. The case value is applied to the input of an AND gate, disabling extra paths in the scenario where the case value of '0' is applied.



```
create_clock -name clk1 -period 12 [get_ports clk1]
create_clock -name clk2 -period 12 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
set_case_analysis 0 [get_pins n/A]
```

```
create_scenario set2
create_clock -name clk1 -period 12 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk2
set_output_delay 2 [list o2 o1 o3] -clock clk2_bis
set_case_analysis 1 [get_pins n/A]
```

The Violation Details section of the GUI provides more information related to the case values for each of the scenarios. HTML tables provide this data as can be seen below:

User Case Value In Scenario 1

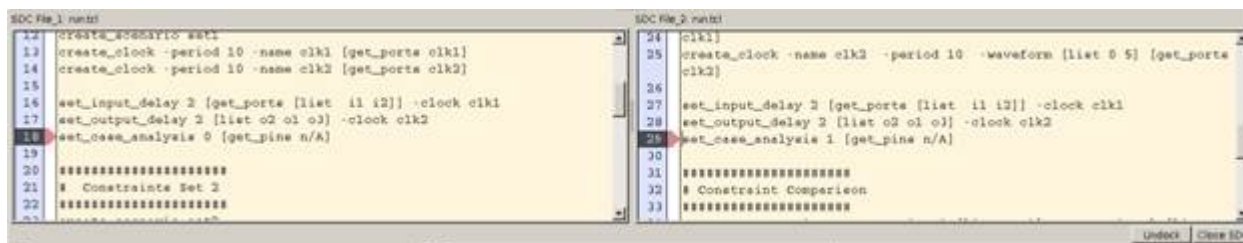
Case	0
pin	n/A
Source File	run.tcl, line 18

User Case Value In Scenario 2

Case	1
pin	n/A
Source File	run.tcl, line 29

A side-by-side schematic displaying logic affected by the constant pins in each scenario can be created by clicking on the schematic link at the top of the Info Pane window.

Clicking on the source file hyperlink in the HTML tables, a side-by-side SDC viewer is opened enabling a very easy constraint comparison:



Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this example, the constant values should be reviewed and the correct one should be applied to both scenarios.

See Also

[report_case_details](#)

[set_case_analysis](#)

[set_disable_timing](#)

[set_false_path](#)

Rule: S2S_CLK_0001

Clock '**clock**' in constraint set '**scenario1**' has no matching clock in constraint set '**scenario2**'.

Severity: Error

Description

The reported clock cannot be found in the second set of constraints. Constraint consistency indicates some of the clock pins where a clock is expected but cannot be found.

Risk Associated with Violation

The design timing will be different between the two sets of constraints. The violation should be reviewed and the constraints should be modified to match. There can be several reasons why the clock is found missing in the second set of constraints. Some possible examples include:

- Missing clock definition
- Clock propagation affected by another constraint

Possible Scenarios

Below are a couple of examples of constraints which can lead to a S2S_CLK_0001 violation. When performing clock matching, constraint consistency looks at all the register clock pins in the design for each scenario to compare. For each register clock pin in the design, it expects to see the same clock (same sense, same period, same waveform, name can be different) reaching the register.

Case 1: In the second set of constraints, one of the clock definitions is missing. constraint consistency reports in the "Violation Details" section a list of pins with missing clock definition. If you click on the schematic link, the GUI displays a schematic showing how the clock propagates to the register pins (for the scenario where the clock is not missing). Below is an example of such a schematic; clock names are annotated on the nets:

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

Constraint set 2:

```
create_scenario set2
```

```
create_clock -name clk1 -period 10 [get_ports clk1]
```

```
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
```

```
set_output_delay 2 [list o2 o1 o3] -clock clk1
```



Case 2: In the second set of constraints, an exception could be blocking the clock propagation to the reported register clock pins. This is reported as a missing clock on the register clock pins. This situation is very likely to also trigger a violation related to case value or exception in the design.

What Next

The Violation Details section of the GUI provides more information related to the affected register clock pins. The [analyze_clock_networks](#) provides insights as to how the clock is propagating to the register clock pins. Clicking on the schematic link allows you to graphically see how the reported clock propagates in the design. Understanding how the clock propagates to the reported register clock pin in one of the scenarios will help you find out what is the difference compared to the other scenario.

Other commands can also give some insights into what is happening. For example, for case 1 above, if a clock definition is entirely missing, you might be able to find this using the [report_clock](#) command as seen below:

```
ptc_shell> current_scenario set1;
```

```
report_clocks
```

Report : clockDesign : S2S_CLK_0001Scenario: set1Version: ...Date : ...

Attributes:

p - Propagated clock

G - Generated clock

I - Inactive clock

Clock Period Waveform Attrs Sources

clk1 10.00 {0 5} {clk1}

clk2 10.00 {0 5} {clk2}

1

ptc_shell> current_scenario set2 ;

report_clocks

Report : clock

Design : S2S_CLK_0001

Scenario: set2

Version: ...

Date : ...

Attributes:

p - Propagated clock

G - Generated clock

I - Inactive clock

Clock Period Waveform Attrs Sources

```
clk1 10.00 {0 5} {clk1}
```

If the clock definitions in [report_clock](#) seem right, maybe clock propagation is blocked along the way (case 2 above), resulting in an unclocked register pin in one of the scenarios. You will also be able to see this in [analyze_clock_networks](#) reports. Below is an example of a clock path affected by a [set_disable_timing](#) exception:

```
ptc_shell> current_design S2S_CLK_0001

ptc_shell> current_scenario set2

ptc_shell> analyze_clock_networks -to [get_pins {b1/ffg/CP}] \
-style full -end_types {register} \
-nosplit -traverse

*****

Report : analyze_clock_networks

-style full

-end_types {register}

-traverse_disabled

-no_split

Design : S2S_CLK_0001

Scenario: set2

Version: ...

Date : ...

*****

Clock Sense Abbreviations:

P - positive

Potential senses detected with -traverse_disabled:

[P] - potential positive

[N] - potential negative

Clock Network End Type Abbreviations:

REG - register

Full report for clock: clk2
```


Branch 0:

Branch

Level Info Sense Notes Port/Pin

```
-----  
0 P source clk2 (in)  
1 P ck2_i2/A (IV)  
2 [N] DT#0 ck2_i2/Z (IV)  
3 [N] ck2_i22/A (IV)  
4 [P] ck2_i22/Z (IV)  
5 [P] b1/zInv3/A (IV)  
6 [N] b1/zInv3/Z (IV)  
7 [N] b1/zInv4/A (IV)  
8 [P] b1/zInv4/Z (IV)  
9 [P] REG b1/ffg/CP (FD1)
```

Clock Blocking Constraints:

DT#0 = set_disable_timing

at pin: ck2_i2/Z

run.tcl, line 31

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed.

See Also

[analyze_clock_networks](#)

[report_clock](#)

[set_disable_timing](#)

Rule: S2S_CLK_0003

Clock '**clock1**' in constraint set '**scenario1**' does not match with clock '**clock2**' in constraint set '**scenario2**'.

Severity: Error

Description

Even though both clocks are reaching the same set of register clock pins, they do not have the same characteristics.

Risk Associated with Violation

Because the reported clocks have different waveforms in scenario1 and scenario2, the design timing will be different between the two sets of constraints. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

This violation occurs because of mismatched clock or generated clock definitions. Below is an example set of constraints which creates a **S2S_CLK_0003** violation.

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]

set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 -waveform [list 0 5] \ [get_ports
clk1]
create_clock -name clk2 -period 10 -waveform [list 0 7] \ [get_ports
clk2]

set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

What Next

The Violation Details section of the GUI provides more information related to each clock and the affected register clock pins. A table displays the various parameters used for matching, such as clock sense (with a tilde following the clock name) and waveform. The list of timing endpoints where mismatches occur is also displayed in that table. Below is an example:

First clock (sense)	First clock waveform	Second clock (sense)	Second clock waveform	Mismatching at pins
clk2'	{5.0000 10.0000}	clk2'	{7.0000 10.0000}	ffc/CP

Alternatively, you can also report the clock characteristics using the [report_clock](#) command. Examples of such reports are shown below:

```
ptc_shell> current_scenario set1; report_clocks
```

```
*****
```

```
Report : clock
```

```
Design : S2S_CLK_0003
```

```
Scenario: set1
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Attributes:
```

```
p - Propagated clock
```

```
G - Generated clock
```

```
I - Inactive clock
```

```
Clock Period Waveform Attrs Sources
```

```
-----
```

```
clk1 10.00 {0 5} {clk1}
```

```
clk2 10.00 {0 5} {clk2}
```

```
ptc_shell> current_scenario set2; report_clocks
```

```
*****
```

```
Report : clock
```

Design : S2S_CLK_0003

Scenario: set2

Version: ...

Date : ...

Attributes:

p - Propagated clock

G - Generated clock

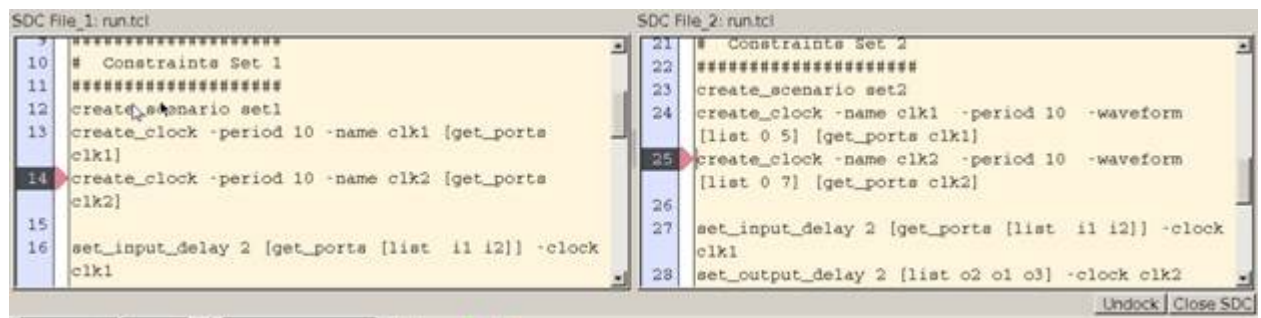
I - Inactive clock

Clock Period Waveform Attrs Sources

clk1 10.00 {0 5} {clk1}

clk2 10.00 {0 7} {clk2}

You can also click on the clock names hyperlink to open a side-by-side SDC viewer showing the two clock definitions.



Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed.

See Also

[report_clock](#)

Rule: S2S_CLK_0004

Pin 'pin' has clock sense applied in constraint set 'set1' that is missing in constraint set 'set2'.

Severity: Error

Description

A [set_clock_sense](#) command is affecting how clocks propagate through the reported pin in constraint set set1. The same pin is not affected by the constraints in set2.

Risk Associated With Violation

The clock propagation through the reported pin is not identical in both constraint sets. As a consequence, the timing results will be different between the two sets of constraints. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLK_0004 violation occurs because of mismatched use of the [set_clock_sense](#) command. Below is an example set of constraints which create a S2S_CLK_0004 violation.

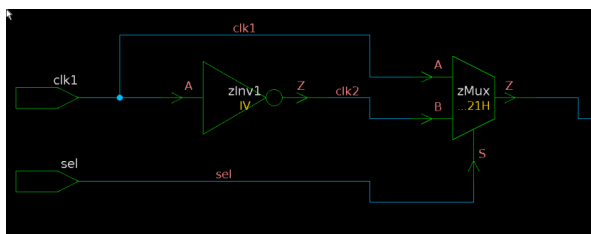


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 12 [get_ports clk1] -add
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_sense -stop_propagation zMux/B -clock clk2
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 12 [get_ports clk1] -add
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
```

What Next

The Violation Details section of the GUI provides more information related to which clock is affected and the location of the constraint responsible for the mismatch. A table displays the parameters used for matching, such as the type of clock sense, if it is restricted to a particular clock (or affects all the clocks, in which case the clock column will show “None”), and the file and line number where the constraint can be found. Below is an example:

Alternatively, you can use the [analyze_clock_networks](#) command to report and visualize the clock propagation through the reported pin in each mode, as shown below. The relevant information in the reports is highlighted in red.

```
ptc_shell> current_scenario set1;
ptc_shell> analyze_clock_networks -through zMux/B -style full

*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : S2S_CLK_0004
Scenario: set1
Version: ...
Date    : ...
*****
Clock Sense Abbreviations:
    P - positive
    N - negative
Clock Network End Type Abbreviations:
    REG - register

Full report for clock: clk1

Branch 0:
      Branch
Level  Info      Sense Notes      Port/Pin
-----
      0          P      source      clk1 (in)
      1          P              zInv1/A (IV)
      2          N              zInv1/Z (IV)
      3          N      STOP#0      zMux/B (MUX21H)
      4          N              zMux/Z (MUX21H)
      5          N      REG          ffa/CP (FD1)

Clock Blocking Constraints:
  STOP#0 = set_clock_sense -stop
          at pin: zMux/B
          /remote/caell08/lionel/Product_testing/S2S_Rules/S2S_CLK_0004/
run.tcl, line 22

ptc_shell> current_scenario set2
```

```
ptc_shell> analyze_clock_networks -through zMux/B -style full
```

```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
Design : S2S_CLK_0004
Scenario: set2
Version: ...
Date   : ...
*****
```

```
Clock Sense Abbreviations:
  P - positive
  N - negative
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: clk1

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	clk1 (in)
1		P		zInv1/A (IV)
2		N		zInv1/Z (IV)
3		N		zMux/B (MUX21H)
4		N		zMux/Z (MUX21H)
5		N	REG	ffa/CP (FD1)

Full report for clock: clk2

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	clk1 (in)
1		P		zInv1/A (IV)
2		N		zInv1/Z (IV)
3		N		zMux/B (MUX21H)
4		N		zMux/Z (MUX21H)
5		N	REG	ffa/CP (FD1)

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In the above example, the `set_clock_sense` command can be removed from the constraint set 1 or added to the constraint set 2.

See Also

[set_clock_sense](#)

[analyze_clock_networks](#)

Rule: S2S_CLK_0006

Clock sense applied to pin '**pin**' in constraint set '**set1**' does not match the clock sense applied in constraint set '**set2**'.

Severity: Error

Description

A [set_clock_sense](#) command is affecting how clocks propagate through the reported pin in both constraint sets. However, the clock propagation is different between the two sets of constraints.

Risk Associated With Violation

The clock propagation through the reported pin is not identical in both constraint sets. As a consequence, the timing results could be different between the two sets of constraints. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLK_0006 violation occurs because of the mismatched use of the [set_clock_sense](#) command. Below is an example set of constraints which create a S2S_CLK_0006 violation.

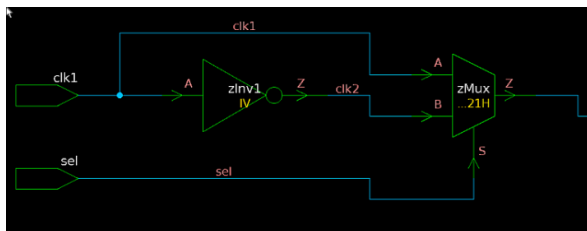


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 12 [get_ports clk1] -add
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_sense -stop_propagation zMux/B -clock clk2
```

Constraint set 2:

```
create_scenario set2
```



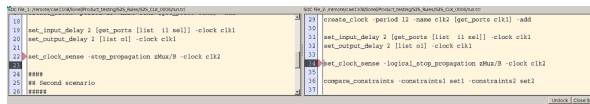
```
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 12 [get_ports clk1] -add
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1

set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_sense -logical_stop_propagation zMux/B -clock clk2
```

What Next

The Violation Details section of the GUI provides more information related to which clock is affected and the location of the constraint responsible for the mismatch. Two tables display how the reported pin is affected in each constraint set. Below is an example:

The easiest way to check for differences between the two constraints is to click the Source hyperlinks. This will bring up the dual SDC browser and you can compare the commands.



Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In the above example, the `set_clock_sense` command in the constraint set 1 can be changed to use the `-logical_stop_propagation` option or the `set_clock_command` in the constraint set 2 can be changed to use the `-stop_propagation` option.

See Also

[set_clock_sense](#)

Rule: S2S_CLK_0007

'**arc_sense**' arc from '**from_pin**' to '**to_pin**' has clock sense applied in constraint set '**set1**' but is missing in constraint set '**set2**'.

Severity: Error

Description

A [set_clock_sense](#) command is affecting how clocks propagate through the reported timing arc in constraint set set1. The same timing arc is not affected by the constraints in set2.

Risk Associated With Violation

The clock propagation through the reported timing arc is not identical in both constraint sets. As a consequence, the timing results could be different between the two sets of constraints. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLK_0007 violation occurs because of mismatched use of the [set_clock_sense](#) command. Below is an example set of constraints which create a S2S_CLK_0007 violation.

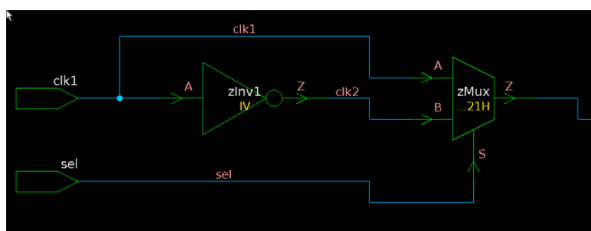


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_sense -stop_propagation [get_timing_arc \
    -from zMux/B -to zMux/Z]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
```

What Next

The Violation Details section of the GUI provides more information related to which clock is affected and the location of the constraint responsible for the mismatch. A table displays the various parameters used for matching such as the type of clock sense, if it is restricted to a particular clock (or affects all the clocks, in which case the clock column will show “None”) and the file and line number where the constraint can be found. Below is an example:

Alternatively, you can also use the [analyze_clock_networks](#) command to report and visualize the clock propagation through the reported pin, in each mode, as shown below.

Note that because this command reports pins, you will see that the propagation is stopped but the command will not report that the cause is related to a timing arc.

```
ptc_shell> current_scenario set1;
ptc_shell> analyze_clock_network -through zMux/B \
-style full -traverse
```

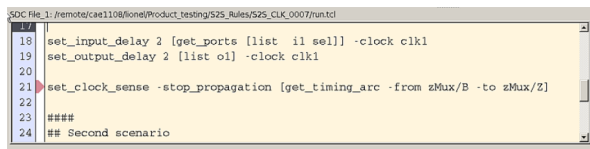
```
*****
Report : analyze_clock_networks
        -max_endpoints=1000
        -style full
        -end_types {register port clock_source}
        -traverse_disabled
Design : S2S_CLK_0007
Scenario: set1
Version: ...
Date : ...
*****
Clock Sense Abbreviations:
  P - positive
  N - negative
Potential senses detected with -traverse_disabled:
  [N] - potential negative
Clock Network End Type Abbreviations:
  REG - register
```

Full report for clock: clk1

Branch 0:

Level	Branch Info	Sense	Notes	Port/Pin
0		P	source	clk1 (in)
1		P		zInv1/A (IV)
2		N		zInv1/Z (IV)
3		N		zMux/B (MUX21H)
4		[N]		zMux/Z (MUX21H)
5		[N]	REG	ffa/CP (FD1)

The easiest way to find out exactly what is causing the mismatch is to use the SDC browser. Simply click on the source hyperlink in the table and the SDC browser will open, pointing to the relevant constraint.



Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In the above example, the `set_clock_sense` command can be removed from the constraint set 1 or be added to the constraint set 2.

See Also

[set_clock_sense](#)

[analyze_clock_networks](#)

Rule: S2S_CLK_0010

Clock gating check applied to '**object_type**' '**object**' in constraint set '**scenario1**' is missing in constraint set '**scenario2**'.

Severity: Error

Description

A [set_clock_gating_check](#) command is present in constraint set scenario1. A similar constraint could not be found for the same objects in constraint set scenario2.

Risk Associated With Violation

The clock gating check specifies setup and hold checks and will affect timing analysis of the design. Because the constraints are different for a particular object between the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLK_0010 violation occurs because a [set_clock_gating_check](#) command is specified in one set of constraints, but not the other. A clock-gating check for the clock gating signal, **cken**, has been specified in the first set of constraints, but not the second in the constraint sets shown below. As a result, an S2S_CLK_0010 violation is issued.

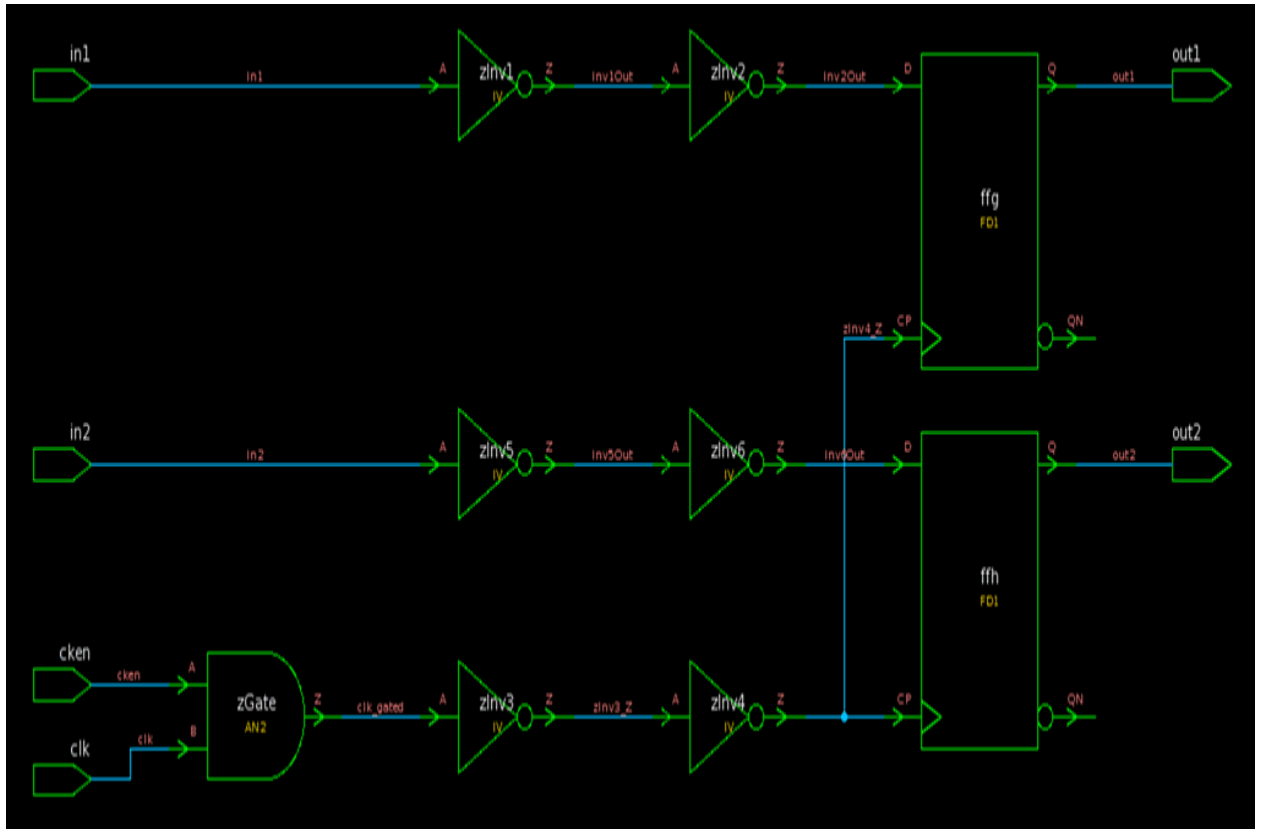


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 [get_ports [list in1 in2]] -clock clk
set_output_delay 2 [list out1 out2] -clock clk
create_clock -name ckv -period 5
set_input_delay 1 [get_ports cken] -clock ckv
set_clock_gating_check -setup 0.2 -hold 0.4 [get_pins zGate/A]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 [get_ports [list in1 in2]] -clock clk
set_output_delay 2 [list out1 out2] -clock clk
create_clock -name ckv -period 5
set_input_delay 1 [get_ports cken] -clock ckv
```

What Next

The Violation Details section of the GUI will include a table for the setup and hold clock gating check values for the constraint set that contains the clock gating check specification. The file and line number where the clock gating check is specified is also included. You can click the hyperlink to open the Constraint Browser to view the relevant command. Below is what is shown for our example scenario:

You can use the [report_clock_gating_check](#) command to report the inferred and user-specified clock gating checks in each constraint set. An example is shown below for our scenario.

```
ptc_shell> current_scenario set2;
ptc_shell> report_clock_gating_check [get_pins zGate/A]
```

```
*****
Report : clock_gating_check
Design : S2S_CLK_0010
Scenario: set1
Version: ...
Date   : ...
*****
```

Clock gating checks:

Cell	EnablePin	ClockPin	High/Low	Clock	Attr	User
Settings						

zGate	A	B	High		I	1
-------	---	---	------	--	---	---

Attr: I:auto inferred, L:library cell defined

User clock gating settings:

ID	Object	Rise		Fall		High/Low	File/Line
		Setup	Hold	Setup	Hold		
1	zGate/A	0.20	0.40	0.20	0.40	--	./S2S_CLK_0010/ run.tcl, line 26

```
ptc_shell> current_scenario set1;
ptc_shell> report_clock_gating_check [get_pins zGate/A]
```

```
*****
Report : clock_gating_check
Design : S2S_CLK_0010
Scenario: set2
Version: ...
Date   : ...
*****
```

Clock gating checks:

Cell	EnablePin	ClockPin	High/Low	Clock	Attr	User
Settings						

zGate	A	B	High		I	1
-------	---	---	------	--	---	---

Attr: I:auto inferred, L:library cell defined

Fix Suggestion

Constraints affecting clock gating checks should be reviewed for both scenarios, and the cause for the discrepancy should be addressed. For the above example, either the setup and hold user-specified clock gating check in scenario set1 should be removed, or an equivalent clock gating check should be added in scenario set2.

See Also

[set_clock_gating_check](#)

[report_clock_gating_check](#)

[remove_clock_gating_check](#)

Rule: S2S_CLK_0012

Clock gating check applied to '**object_type**'**object1**' in constraint set '**scenario1**' does not match the clock gating check applied to '**object2**' in the constraint set '**scenario2**'.

Severity: Error

Description

A [set_clock_gating_check](#) command is defined in both constraint sets, but with different values.

Risk Associated With Violation

The clock gating check specifies setup and hold checks and will affect timing analysis of the design. Because the constraints are different for the reported objects in the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLK_0012 violation occurs because the [set_clock_gating_check](#) commands in the constraint sets have different values. A clock-gating check for the clock-gating signal, **cken**, has been specified in the first set of constraints shown below, but the values are different from those specified in the second set of constraints. As a result, an S2S_CLK_0012 violation is issued.

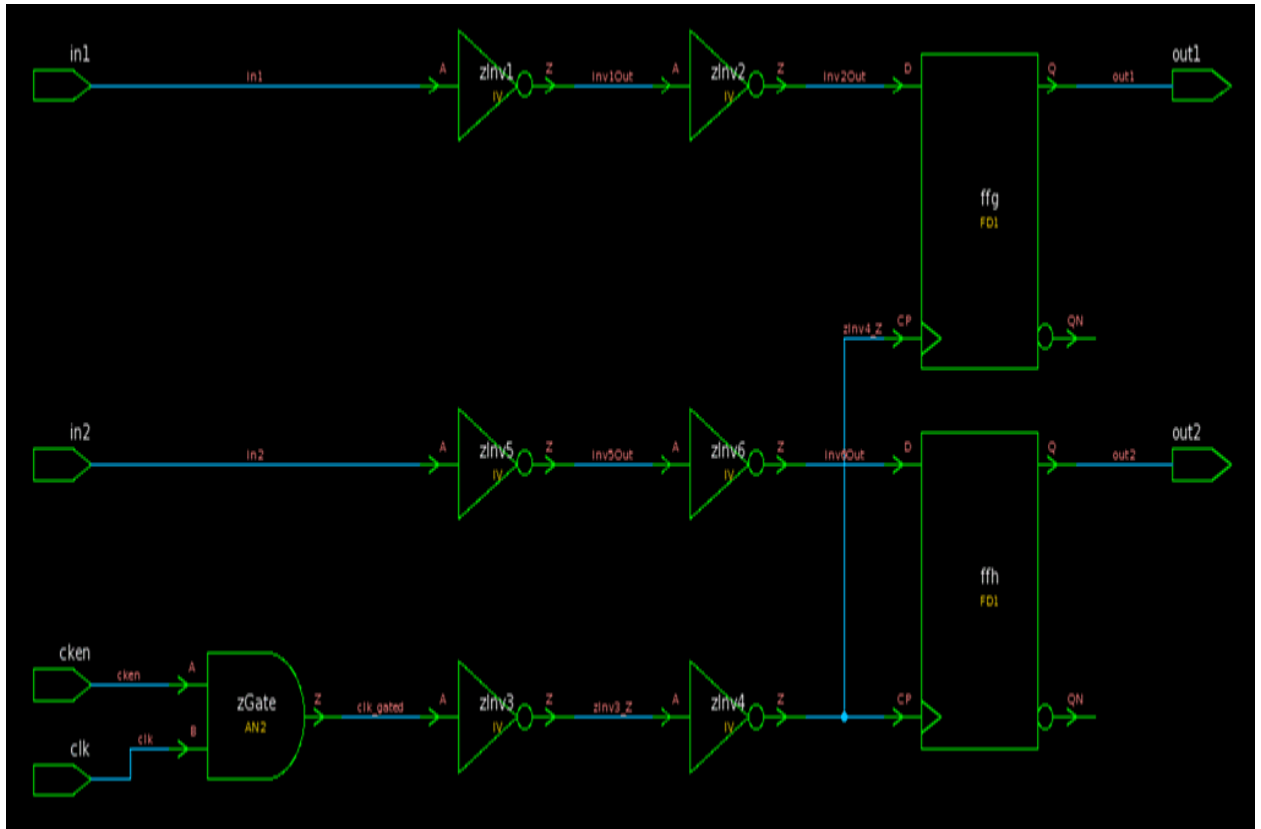


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 [get_ports [list in1 in2]] -clock clk
set_output_delay 2 [list out1 out2] -clock clk
create_clock -name ckv -period 5
set_input_delay 1 [get_ports cken] -clock ckv
set_clock_gating_check -setup 0.2 -hold 0.4 [get_pins zGate/A]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk -period 10 [get_ports clk]
set_input_delay 2 [get_ports [list in1 in2]] -clock clk
set_output_delay 2 [list out1 out2] -clock clk
create_clock -name ckv -period 5
set_input_delay 1 [get_ports cken] -clock ckv
set_clock_gating_check -setup 0.3 [get_pins zGate/A]
```

What Next

The Violation Details section of the GUI will include tables for the setup and hold clock gating check values for each constraint set. The file and line number where the clock gating check is specified is also included for each constraint set. Below is what is shown for our example scenario:

You can click the file reference hyperlink to open the SDC browser and look at the constraints themselves. Because we are looking at a discrepancy between two existing constraints, the dual SDC browser will open and point to the constraints.

You can use the [report_clock_gating_check](#) command to report the inferred and user-specified clock gating checks in each constraint set. An example is shown below for our scenario.

```
ptc_shell> current_scenario set1;
ptc_shell> report_clock_gating_check [get_pins zGate/A]

*****
Report : clock_gating_check
Design : S2S_CLK_0010
Scenario: set1
Version: ...
Date   : ...
*****

Clock gating checks:
Cell      EnablePin ClockPin  High/Low  Clock      Attr      User
Settings
-----
zGate      A          B          High      Clock      I         1

Attr: I:auto inferred, L:library cell defined

User clock gating settings:
          Rise      Fall
ID  Object  Setup  Hold  Setup  Hold  High/Low  File/Line
-----
1   zGate/A  0.20   0.40   0.20   0.40   --        ./S2S_CLK_0010/
run.tcl, line 26

ptc_shell> current_scenario set2;
ptc_shell> report_clock_gating_check [get_pins zGate/A]

*****
Report : clock_gating_check
Design : S2S_CLK_0010
Scenario: set2
Version: ...
Date   : ...
*****

Clock gating checks:
```

```

Cell          EnablePin ClockPin  High/Low  Clock          Attr  User
Settings
-----
zGate         A          B          High          I      1

Attr: I:auto inferred, L:library cell defined

User clock gating settings:
      Rise      Fall
ID  Object  Setup Hold  Setup Hold  High/Low  File/Line
-----
1   zGate/A  0.30  --    0.30  --    --      ./S2S_CLK_0012/
run.tcl, line 42

```

Fix Suggestion

Constraints affecting clock gating checks should be reviewed for both scenarios, and the cause for the discrepancy should be addressed. For the above example, the clock gating check values should be consistent between the two constraint sets.

Rule Property: Tolerance

This rule allows tolerance to be taken into consideration when performing clock gating check matching. This tolerance is specified as a percentage of the smallest value compared. By default, this tolerance is set to 1e-05%, which is equivalent to performing exact matches. The tolerance can be changed through the `set_rule_property` command as shown below.

```
set_rule_property tolerance 15 [get_rules S2S_CLK_0012]
```

See Also

[set_clock_gating_check](#)

[report_clock_gating_check](#)

[remove_clock_gating_check](#)

Rule: S2S_CLT_0001

'**constraint**' applied to '**object_type**'**object**' in constraint set '**scenario1**' is missing in constraint set '**scenario2**'.

Severity: Error

Description

A [set_clock_latency](#) command is present in constraint set scenario1. A similar constraint could not be found for the same objects in constraint set scenario2.

Risk Associated With Violation

The clock network latency or clock source latency will affect timing analysis of the design. Because the constraints are different for a particular object between the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLT_0001 violation occurs because a [set_clock_latency](#) command is specified in one set of constraints, but not the other. A clock source latency for the clock, **clk1**, has been specified in the first set of constraints shown below. As a result, an S2S_CLT_0001 violation is issued.

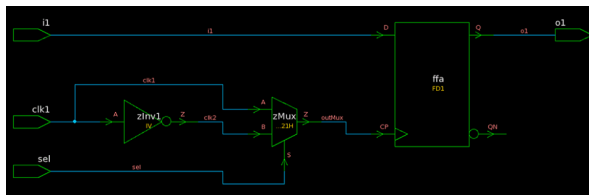


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
set_clock_latency 0.05 -source [get_clocks clk1]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
```

What Next

The Violation Details section of the GUI will include a table for the clock source latency and/or the clock network latency values for the constraint set that contains the clock latency specification. The file and line number where the clock latency is specified is also included. You can click the hyperlink to open the Constraint Browser to view the relevant command. Below is what is shown for our example scenario:

You can also review the current clock latency settings using the `report_clock -skew` command.

```
ptc_shell> current_scenario set1;
ptc_shell> report_clock -skew [get_clocks clk1]
```

```
*****
Report : clock_skew
Design : S2S_CLT_0001
Scenario: set1
Version: ...
Date   : ...
*****
```

		Min Condition Source Latency				Max
Condition Source Latency						
Object		Early_r	Early_f	Late_r	Late_f	
Early_r	Early_f	Late_r	Late_f	Rel_clk		
clk1		0.05	0.05		0.05	0.05
0.05	0.05	0.05	0.05	--		

Fix Suggestion

Constraints affecting the clock latency should be reviewed for both scenarios, and the cause for the discrepancy should be addressed. For the above example, either the clock source latency in scenario set1 should be removed, or an equivalent latency should be added in scenario set2.

See Also

[set_clock_latency](#)

[report_clock](#)

Rule: S2S_CLT_0003

'constraint' applied to 'object_type1' 'object1' in constraint set 'scenario1' does not match the 'object_type2' 'object2' applied in constraint set 'scenario2'.

Severity: Error

Description

A [set_clock_latency](#) command is defined in both constraint sets, but with different values.

Risk Associated with Violation

The clock network latency or clock source latency will affect timing analysis of the design. Because the constraints are different between the reported objects in the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CLT_0003 violation occurs because the [set_clock_latency](#) commands specified in the constraint sets have different values. A clock source latency for the clock, **clk1**, has been specified in the first set of constraints shown below, but the value is different from that specified for the corresponding clock in the second set of constraints. As a result, an S2S_CLT_0003 violation is issued.

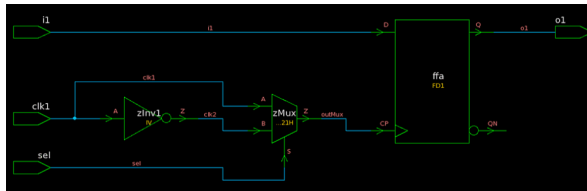


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
set_clock_latency 0.05 -source [get_clocks clk1]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
set_clock_latency 0.07 -source [get_clocks clk1]
```

What Next

The Violation Details section of the GUI will include a table for the clock source latency and/or the clock network latency values for the clock latency specification in each constraint set. The file and line number where the clock latency is specified in each constraint set is also included. Below is what is shown for our example scenario:

You can click the file reference hyperlink to open the SDC browser and look at the constraints themselves. Because you are looking at a discrepancy between two existing constraints, the dual SDC browser will open and point to the constraints.

You can also review the clock latency settings in each scenario using the `report_clock -skew` command.

```
ptc_shell> current_scenario set1;
ptc_shell> report_clock -skew [get_clocks clk1]
```

```
*****
```

```
Report : clock_skew
Design : S2S_CLT_0003
Scenario: set1
Version: ...
Date   : ...
*****
```

Latency		Min Condition Source Latency				Max Condition Source		
Object		Early_r	Early_f	Late_r	Late_f	Early_r	Early_f	Late_r
Late_f	Rel_clk							
clk1		0.05	0.05	0.05	0.05	0.05	0.05	0.05
0.05	--							

```
ptc_shell> current_scenario set2;
ptc_shell> report_clock -skew [get_clocks clk1]
```

```
*****
Report : clock_skew
Design : S2S_CLT_0003
Scenario: set2
Version: ...
Date   : ...
*****
```

		Min Condition Source Latency				Max Condition		
Source Latency								
Object		Early_r	Early_f	Late_r	Late_f	Early_r	Early_f	Late_r
Late_f	Rel_clk							
clk1		0.07	0.07	0.07	0.07	0.07	0.07	0.07
0.07	--							

Fix Suggestion

Constraints affecting the clock latency should be reviewed for both scenarios, and the cause for the discrepancy should be addressed. For the above example, the clock source latency values should be consistent between the two constraint sets.

Rule Property: Tolerance

This rule allows tolerance to be taken into consideration when performing clock latency matching. This tolerance is specified as a percentage of the smallest value compared. By default, this tolerance is set to 1e-05%, which is equivalent to performing exact matches. The tolerance can be changed through the `set_rule_property` command as shown below.

```
set_rule_property tolerance 10 [get_rules S2S_CLT_0003]
```

See Also

[set_clock_latency](#)

[report_clock](#)

[set_rule_property](#)

Rule: S2S_CST_0001

‘Constraint’ applied to **‘object_type’ ‘object’** in constraint set **‘scenario1’** is missing in constraint set **‘scenario2’**.

Severity: Error

Description

The SDC constraint applied on the reported object in constraint set senario1 could not be found for the same object in the constraint set scenario2.

Risk Associated With Violation

This SDC constraint will affect timing analysis of the design. Because the constraints are different for a particular object between the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CST_0001 violation can occur if clock uncertainties are not applied in a similar fashion in both constraints sets. Below is an example set of constraints creating a S2S_CST_0001 violation.

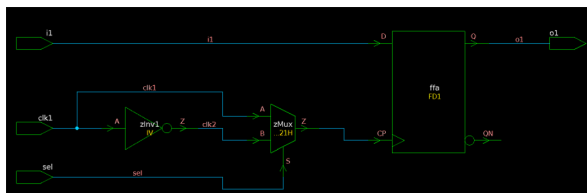


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
set_clock_transition -rise 0.05 [get_clocks clk1]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
```

What Next

The Violation Details section of the GUI provides more information related to which object is affected, the location of the constraint responsible for the mismatch, and some information related to the constraint itself. A table displays the various parameters. Below is an example:

You can click the file reference hyperlink to open the SDC browser and look at the constraint. Alternatively, you can use the [report_clock](#) command to gather information about this problem. To see the clock transitions applied on clock clk1 in both scenarios, do the following:

```
ptc_shell> current_scenario set1
{"set1"}

ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_CST_0001
Scenario: set1
Version: ...
Date   : ...
*****
```

Object	Min Rise Transition	Min Fall Transition	Max Rise Transition	Max Fall Transition
clk1	0.05	-	0.05	-

```
1

ptc_shell> current_scenario set2
{"set2"}

ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_CST_0001
Scenario: set2
Version: ...
Date   : ...
*****

1
```

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this particular case, either the clock transition in scenario set1 should be removed, or an equivalent constraint should be added in scenario set2.

See Also

[report_clock](#)

[set_clock_transition](#)

Rule: S2S_CST_0003

'Constraint' applied to **'object_type1'** **'object1'** in constraint set **'scenario1'** does not match the one applied on **'object_type2'** **'object'** in constraint set **'scenario2'**.

Severity: Error

Description

The SDC constraint applied on the reported object in constraint set senario1 exists in scenario2. However, the values for this constraint differ between the two scenarios.

Risk Associated With Violation

This SDC constraint will affect timing analysis of the design. Since the constraints are different for a particular object between the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_CST_0003 violation can occur if clock uncertainties are not applied in a similar fashion in both constraints sets. Below is an example set of constraints creating a S2S_CST_0003 violation.

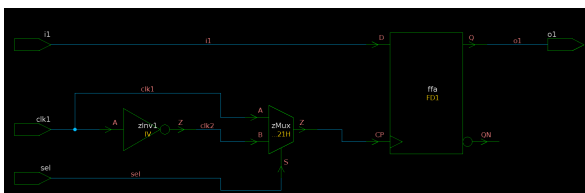


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
```

```
set_output_delay 2 [list o1] -clock clk1
set_clock_transition -rise 0.05 [get_clocks clk1]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk1
set_clock_transition -fall 0.05 [get_clocks clk1]
```

What Next

The Violation Details section of the GUI provides more information related to which object is affected, the location of the constraint responsible for the mismatch, and some information related to the constraint itself. HTML tables display the various parameters for each of the constraints involved in this violation. Below is an example:

You can click the file reference hyperlink to open the SDC browser and look at the constraint. Alternatively, you can use the [report_clock](#) command to gather information about this problem. To see the clock transitions applied on clock clk1 in both scenarios, do the following:

```
ptc_shell> current_scenario set1
{"set1"}
```

```
ptc_shell> report_clocks -skew
*****
Report : clock_skew
Design : S2S_CST_0003
Scenario: set1
Version: ...
Date   : ...
*****
```

Object	Min Rise Transition	Min Fall Transition	Max Rise Transition	Max Fall Transition
clk1	0.05	-	0.05	-

1

```
ptc_shell> current_scenario set2
{"set2"}
```

```
ptc_shell> report_clocks -skew
*****
Report : clock_skew
Design : S2S_CST_0003
Scenario: set2
Version: ...
Date   : ...
```

Object	Min Rise Transition	Min Fall Transition	Max Rise Transition	Max Fall Transition
clk1	-	0.05	-	0.05

1

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this particular case, one of the constraints defines a rise transition whereas the other describes a fall. Both should be consistent and probably define both the rise and fall values.

Rule Property: Tolerance

This rule supports the tolerance property. By default, constraint consistency expects the constraints to match exactly. If they don't, a violation is generated. Using the tolerance property, this behavior can be relaxed. The tolerance property specifies an acceptable percentage difference between the two constraints. If the difference is within this percentage, no violation is generated.

By default, the tolerance is set to a very small percentage (1e-5) which forces exact constraint comparison. The tolerance property can be modified with the [set_rule_property](#) command.

See Also

[report_clock](#)

[set_clock_transition](#)

[set_rule_property](#)

Rule: S2S_DIS_0001

'object_type' 'object' is disabled in constraint set 'scenario1' but not in constraint set 'scenario2'.

Severity: Error

Description

A [set_disable_timing](#) constraint is applied on one of the constraints sets but it does not exist in the second one.

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This most likely will result in a different set of paths being timed and will produce inconsistent timing results between the two sets of constraints.

Possible Scenarios

Below is an example of a set of constraints which would lead to an **S2S_DIS_0001** violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different.

Constraints set 1:

```
create_scenario set1  
  
create_clock -name clk1 -period 10 [get_ports clk1]  
create_clock -name clk2 -period 10 [get_ports clk2]  
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1  
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

Constraints set 2:

```
create_scenario set2  
  
create_clock -name clk1 -period 10 [get_ports clk1]  
create_clock -name clk2 -period 10 [get_ports clk2]  
set_input_delay 2 [get_ports [list i1 i2]] -clock clk2  
set_output_delay 2 [list o2 o1 o3] -clock clk2_bis  
set_disable_timing [get_pins b1/zInv2/Z]
```

What Next

The Violation Details section of the GUI provides more information related to where the `set_disable_timing` constraint was defined in the constraint file. The file name and line number corresponding to the exception are reported in the form of a hyperlink. Clicking on the hyperlink will automatically open the SDC browser and you can see the command responsible for the violation. An example can be found below:

Clicking on the Schematic link at the top of the Info Pane window creates a schematic displaying the various gates involved with this violation. The pin at which the timing is disabled is selected and highlighted in white.



```
SDC File_2: run.tcl
26 clk2]
27 set_input_delay 2 [get_ports [list i1 i2]] -clock clk2
28 set_output_delay 2 [list o2 o1 o3] -clock clk2_bis
29
30 set_disable_timing [get_pins b1/zInv2/Z]
31
32 #####
33 # Constraint Comparison
34 #####
```

Alternatively, disabled timing arcs can be reported using the `report_disable_timing` command in constraint consistency. Below is an example of such a report:

```
ptc_shell> report_disable_timing

*****

Report : disable_timing
Design : S2S_DIS_0001
Version: ...
Date : ...

*****

Attributes

c - case-analysis
C - Conditional arc
d - default conditional arc
f - false net-arc
l - loop breaking
L - db inherited loopbreaking
m - mode
p - propagated constant
u - user-defined
```

U - User-defined library arcs

Cell or Port From To Sense Flag Reason

b1/zInv2 * Z * u

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this case, one of the following steps should be taken:

- Add the missing disable timing exception in the set1 constraints set.
- Remove the disable timing exception from the set2 constraints set.

See Also

[set_disable_timing](#)

[report_disable_timing](#)

Rule: S2S_DIS_0003

Arc '**object'****object'** is disabled in constraint set '**scenario1'**but not in constraint set '**scenario2'**.

Severity: Error

Description

A timing arc is disabled in scenario1 but is still enabled in scenario2. The two '**object'** parameters in the violation message correspond to the source and destination of the timing arc.

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This most likely will result in a different set of paths being timed and will produce inconsistent timing results between the two sets of constraints.

Possible Scenarios

Below is an example of a set of constraints which would lead to an **S2S_DIS_0003** violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different.

Constraints set 1:

```
create_scenario set1

create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

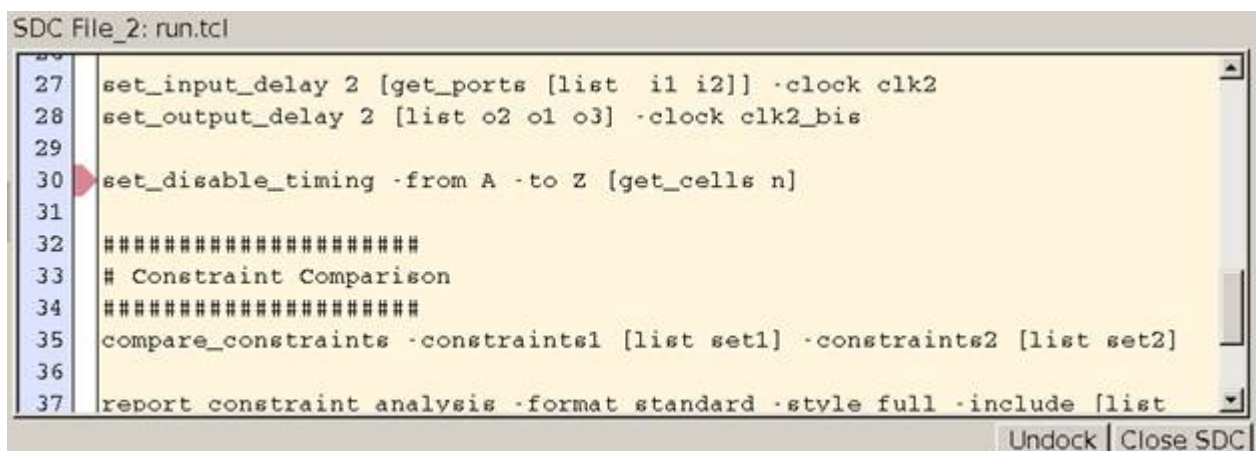
Constraints set 2:

```
create_scenario set2

create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk2
set_output_delay 2 [list o2 o1 o3] -clock clk2_bis
set_disable_timing -from A -to Z [get_cells n]
```

What Next

The Violation Details section of the GUI provides more information related to where it was defined in the constraint file. The file name and line number corresponding to the exception are reported in the form of a hyperlink. Clicking on the hyperlink will automatically open the SDC browser and you can see the command responsible for the violation. An example can be found below:



```
SDC File_2: run.tcl
27 set_input_delay 2 [get_ports [list i1 i2]] -clock clk2
28 set_output_delay 2 [list o2 o1 o3] -clock clk2_bis
29
30 set_disable_timing -from A -to Z [get_cells n]
31
32 #####
33 # Constraint Comparison
34 #####
35 compare_constraints -constraints1 [list set1] -constraints2 [list set2]
36
37 report constraint analysis -format standard -style full -include [list
```

Undock Close SDC

A schematic can also be generated using the schematic button located at the top of the Info Pane window. The cell with the disabled timing arc is displayed and the “from” and “to” pins of the disabled arc are highlighted in white.

Alternatively, disabled timing arcs can be reported using the `report_disable_timing` command in constraint consistency. Below is an example of such a report:

```
ptc_shell> report_disable_timing

*****

Report : disable_timing

Design : S2S_DIS_0003

Version: ...

Date : ...

*****

Attributes

c - case-analysis
C - Conditional arc
d - default conditional arc
f - false net-arc
l - loop breaking
L - db inherited loopbreaking
m - mode
p - propagated constant
u - user-defined
U - User-defined library arcs
```

Cell or Port From To Sense Flag Reason

n A Z positive_unate u

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this case, one of the following steps should be taken:

- Add the missing disable timing exception in the set1 constraints set.
- Remove the disable timing exception from the set2 constraints set.

See Also

[set_disable_timing](#)

[report_disable_timing](#)

Rule: S2S_EXC_0001

Exception '**exception**' in constraint set '**scenario1**' is missing in constraint set '**scenario2**'.

Severity: Error

Description

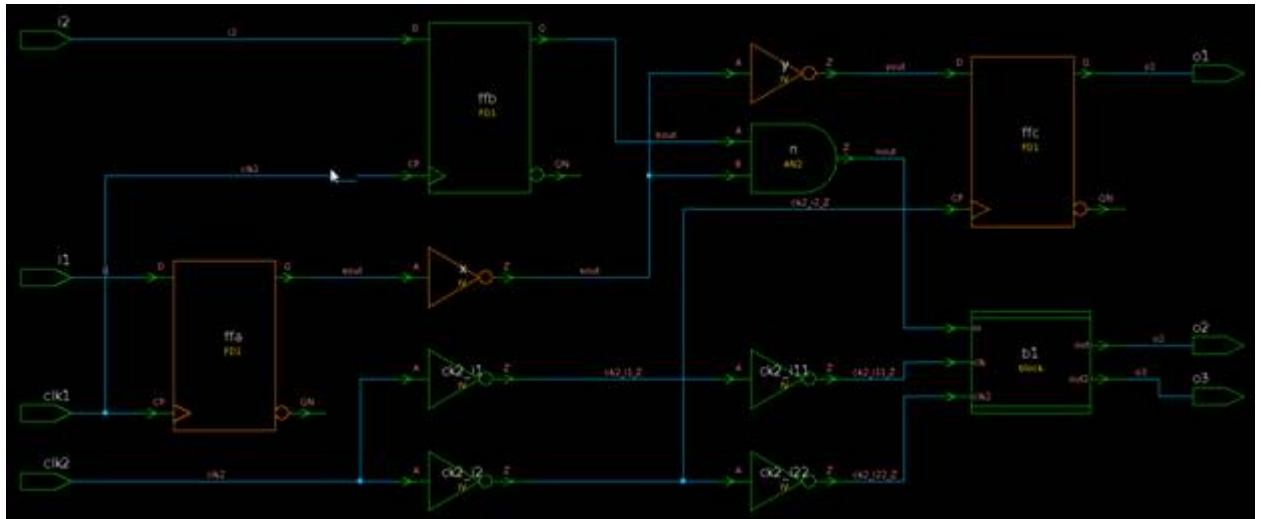
A timing exception is applied on one of the constraints set but it does not exist in the second one.

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This will result in a different set of paths being timed and will produce inconsistent timing results between the two sets of constraints.

Possible Scenarios

Consider an example of a design and a set of constraints which would lead to an **S2S_EXC_0001** violation (snapshot below).



If the following two sets of constraints are applied, the timing behavior of the design will be different. The path in orange in the netlist above will be affected by the constraints.

Constraints set 1:

```
create_scenario set1

create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
set_false_path -from [get_pins ffa/CP] -to [get_pins ffc/D]
```

Constraints set 2:

```
create_scenario set2

create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk2
set_output_delay 2 [list o2 o1 o3] -clock clk2_bis
```

What Next

The Violation Details section of the GUI provides more information related to the exception which is not found in the scenario2 constraints. An HTML table details this exception and

clicking on the source file hyperlink brings up the SDC viewer. Here is an example of such a table:

Exception			
Type	false_path		
Values		Hold	Setup
	Rise	yes	yes
	Fall	yes	yes
From	ffa/CP		
To	ffc/D		
Source File	run.tcl, line 19		

In addition to the exception itself, the paths directly affected are also reported. This helps you understand the impact of that exception on the design. This is how the paths are reported:

Exception is missing on the following paths			
-from	-through	-to	analyze_paths command for first constraint set
*	*	ffc/D	echo command , execute

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this case, one of the following steps should be taken:

- Add the missing [set_false_path](#) exception in the scenario2 constraints set.
- Remove the [set_false_path](#) exception from the scenario1 constraints set.

See Also

[set_false_path](#)

[report_exceptions](#)

[set_disable_timing](#)

[set_case_analysis](#)

Rule: S2S_EXC_0003

Exception '**exception**' in constraint set '**scenario1**' does not match exception '**exception2**' in constraint set '**scenario2**'.

Severity: Error

Description

The exception found in scenario1 does not match the exception found in scenario2. The differences could be due to

- Exceptions are of different types (false path vs. multicycle paths).
- Values to the corresponding exceptions are different (early/late/rise/fall).

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This could lead to incorrect timing results.

Possible Scenarios

Below is an example of a set of constraints which would lead to an **S2S_EXC_0003** violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different.

Constraints set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
set_false_path -from [get_pins ffa/CP] -to [get_pins ffc/D]
```

Constraints set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_output_delay 2 [list o2 o1 o3] -clock clk2
set_multicycle_path 2 -from [get_pins ffa/CP] -to [get_pins ffc/D]
```

What Next

The Violation Details section of the GUI provides more information related to the mismatching exceptions. For each of the scenarios, the exception is detailed in a table which contains all the relevant fields. The SDC browser can be brought up by clicking on the source file hyperlink in the table.

Exception			
Type	false_path		
Values		Hold	Setup
	Rise	yes	yes
	Fall	yes	yes
From	ffa/CP		
To	ffc/D		
Source File	run.tcl, line 19		

Exception			
Type	multicycle_path		
Values		Hold	Setup
	rise	0	2
	fall	0	2
From	ffa/CP		
To	ffc/D		
Source File	run.tcl, line 31		

The various paths affected by these exceptions can also be displayed in a schematic. In the Debugging Help section, clicking on the “execute” hyperlink for each of the two constraint sets will display the paths currently affected by each exception.

Exception is missing on the following paths				
-from	-through	-to	analyze_paths command for first constraint set	analyze_paths command for second constraint set
*	*	ffc/D	echo command, execute	echo command, execute

In addition to creating the schematic, some text reports are available in the Console window. Below is the example for the above scenario1. This allows you to see what gates are affected in a textual way rather than using the schematics.

```
ptc_shell> current_design S2S_EXC_0003; current_scenario set1
ptc_shell> analyze_paths -through [get_pins {ffc/D}] -max_endpoints 1 \
-path_type full -traverse_disabled -unconstrained
```

Report : analyze_paths

-path_type full

-traverse_disabled

-max_endpoints = 1

-unconstrained

Design : S2S_EXC_0003

Version: ...

Date : ...

Summary of Paths:

Dominant Overridden Count Clocks

Startpoint Endpoint Constraint Constraints (r,f,R,F) Launch/Capture

ffa/CP (FD1) ffc/D (FD1) F0 (1,1,1,1) clk1/clk2'

Exception Information:

ID Type Setup Hold Source

F0 false_path FALSE FALSE run.tcl, line 19

Full report of all pins in the paths

Level Pin Attr Constraints

1 ffa/CP (FD1) Start F0

2 ffa/Q (FD1)

3 x/A (IV)

4 x/Z (IV)

5 y/A (IV)

6 y/Z (IV)

7 ffc/D (FD1) End F0

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. The constraints need to be modified in order to produce the same timing behavior in both scenarios.

See Also

[set_false_path](#)

[set_multicycle_path](#)

[set_min_delay](#)

[set_max_delay](#)

[set_clock_groups](#)

Rule: S2S_EXD_0001

'pin_port' has 'input_output'delay constraints in constraint set '**scenario1**' that are missing in constraint set '**scenario2**'.

Severity: Error

Description

An input or output delay is specified for the reported object in scenario1. However, this input or output delay is not specified in scenario2.

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This could lead to incorrect timing results.

Possible Scenarios

Below is an example of a set of constraints which would lead to an **S2S_EXD_0001** violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different.

Constraints set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

Constraints set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

What Next

The Violation Details section of the GUI provides more information related to the input or output delays. A table provides this information for the set of constraints where the delay values are defined. Below is an example of such a table:

Input Delay			
Clock	clk1		
Clock Period	10.0000		
Values		Min	Max
	Rise	2.0000	2.0000
	Fall	2.0000	2.0000
Source File	run.tcl, line 16		

All the data related to the input delay is displayed in the table (related clock, delay values, location of the input delay command). Clicking on the Source File hyperlink brings the SDC viewer and places the marker to the relevant exception.

Alternatively, you can also report the delay information of a particular port using the `report_ports` command as demonstrated below:

```
ptc_shell> current_scenario set2; report_ports i1 -input_delay
```

```
*****
```



```
Report : port
-input_delay
Design : S2S_EXD_0001
Scenario: set2
Version: ...
Date : ...

*****

Input Delay
Min Max Related Related
Input Port
Rise Fall Rise Fall Clock Pin
-----

i1 -- -- -- -- --

ptc_shell> current_scenario set1; report_ports i1 -input_delay
*****

Report : port
-input_delay
Design : S2S_EXD_0001
Scenario: set1
Version: ...
Date : Thu ...

*****

Input Delay
Min Max Related Related
Input Port
Rise Fall Rise Fall Clock Pin
-----
```

```
i1 2.00 2.00 2.00 2.00 clk1 --
```

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this case, the following step should be taken:

-Add the missing input delay value in the scenario2 constraints set.

See Also

[set_input_delay](#)

[set_output_delay](#)

[report_port](#)

Rule: S2S_EXD_0003

'pin_port' has different 'input_output' delay constraints in constraint set 'scenario1' and constraint set 'scenario2'.

Severity: Error

Description

For the reported pin or port, the input or output delay values found in scenario1 do not match the input or output delay values found in scenario2. The differences could be due to

- Different reference clocks
- Different clock periods
- Different min/max/rise/fall values

Risk Associated with Violation

The two sets of constraints do not have the same timing behavior on the design. This could lead to incorrect timing results.

Possible Scenarios

Below is an example of a set of constraints which would lead to an **S2S_EXD_0003** violation. Given a netlist, if the following two sets of constraints are applied, the timing behavior of the design will be different.

Constraints set 1:

```
create_scenario set1  
  
create_clock -name clk1 -period 10 [get_ports clk1]
```

```
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

Constraints set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
create_clock -name clk2 -period 10 [get_ports clk2]
set_input_delay 2 -min [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk2
```

What Next

The Violation Details section of the GUI provides more information related to the mismatching delays. For each scenario, the input or output delay is detailed in a table which contains all the relevant fields. The SDC browser can be brought up by clicking on the source file hyperlink in the table.

Input Delay			
Clock	clk1		
Clock Period	10.0000		
Values		Min	Max
	Rise	2.0000	2.0000
	Fall	2.0000	2.0000
Source File	run.tcl, line 16		

Input Delay			
Clock	clk1		
Clock Period	10.0000		
Values		Min	Max
	Rise	2.0000	--
	Fall	2.0000	--
Source File	run.tcl, line 27		

If the violation refers to ports, you can alternatively use the [report_ports](#) command to get more information on the exact input/output delays specified on those ports.

```
ptc_shell> current_scenario set1; report_ports -input_delay [get_ports i1]
```

```
*****
```

```
Report : port
```

```
-input_delay
```

```
-output_delay
```

```
Design : S2S_EXD_0003
```

```
Scenario: set1
```

```
Version: ...
```

```
Date : ...
```

```
*****
```

```
Input Delay
```

```
Min Max Related Related
```

```
Input Port
```

```
Rise Fall Rise Fall Clock Pin
```

```
-----
```

```
i1 2.00 2.00 2.00 2.00 clk1 --
```

Chapter 3: Constraint Comparison Rules

Rule: S2S_EXD_0003

```
ptc_shell> current_scenario set2; report_ports -input_delay [get_ports
i1]

*****

Report : port

-input_delay

Design : S2S_EXD_0003

Scenario: set2

Version: ...

Date : ...

*****

Input Delay

Min Max Related Related

Input Port

Rise Fall Rise Fall Clock Pin

-----

i1 2.00 2.00 -- -- clk1 --
```

If the violation refers to pins, you can use the [analyze_paths](#) command to help you understand the differences between the two scenarios.

Clicking on the source files hyperlinks will bring side-by-side SDC browser to facilitate debugging of the constraints. Below is an example:



Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. The constraints need to be modified in order to produce the same timing behavior in both scenarios.

See Also

[report_port](#)

[analyze_paths](#)

Rule: S2S_UNC_0001

Clock uncertainty applied to '**object_type**' '**object**' in constraint set '**scenario1**' that is missing in constraint set '**scenario2**'.

Severity: Error

Description

A simple clock uncertainty constraint is applied to the reported object in constraint set senario1. A similar constraint could not be found for the same object in the constraint set scenario2.

Risk Associated With Violation

The clock uncertainty will affect timing analysis of the design. Because the constraints are different for a particular object between the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_UNC_0001 violation can occur if clock uncertainties are not applied in a similar fashion in both constraints sets. Below is an example set of constraints which create a S2S_UNC_0001 violation.

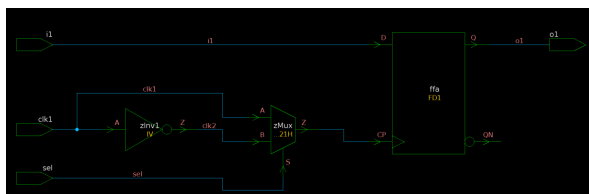


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_uncertainty 0.65 -setup [get_clocks clk1]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports [list i1 i2]] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
```

What Next

The Violation Details section of the GUI provides more information related to which object is affected, the location of the constraint responsible for the mismatch, and some information related to the constraint itself (setup/hold data). A table displays the various parameters. Below is an example:

You can click on the file reference hyperlink to open the SDC browser and look at the constraint itself. Alternatively, you can also use the [report_clock](#) command to gather information about this problem. To see the clock uncertainty applied on clock clk1 in the scenario set1, do the following:

```
ptc_shell> current_scenario set1
{"set1"}
```

```
ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_UNC_0001
Scenario: set1
Version: ...
Date : ...
*****
```

Setup Object Uncertainty	Min Rise Related Delay Clock	Min Fall Delay	Max Rise Delay	Max Fall Delay	Hold Uncertainty
clk1 0.65	--	-	-	-	-

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this particular case, either the clock uncertainty in scenario set1 should be removed, or an equivalent uncertainty should be added in scenario set2.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: S2S_UNC_0003

Clock uncertainty applied to 'object_type' 'object1' in constraint set 'scenario1' does not match the clock uncertainty applied to 'object2' in the constraint set 'scenario2'.

Severity: Error

Description

A simple clock uncertainty constraint is defined in both constraint sets but with different values.

Risk Associated With Violation

The clock uncertainty will affect timing analysis of the design. Because the constraints are different for a particular object between the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_UNC_0003 violation can occur if clock uncertainties are not applied in a similar fashion in both constraints sets. Below is an example set of constraints which creates a S2S_UNC_0003 violation.

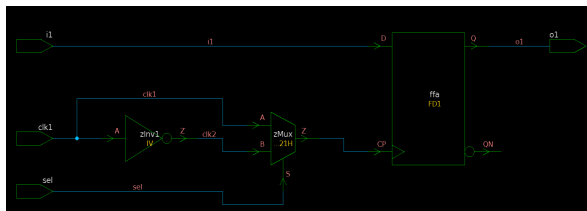


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports {list i1 i2}] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_uncertainty 0.65 -setup [get_clocks clk1]
```

Constraint set 2:

```
create_scenario set2
create_clock -name clk1 -period 10 [get_ports clk1]
set_input_delay 2 [get_ports {list i1 i2}] -clock clk1
set_output_delay 2 [list o2 o1 o3] -clock clk1
set_clock_uncertainty 0.65 -hold [get_clocks clk1]
```


What Next

The Violation Details section of the GUI provides more information related to which object is affected, the location of the constraint responsible for the mismatch, and some information related to the constraint itself (setup/hold data). Two tables display the various parameters; one the scenario1 and one for scenario2. Below is an example:

You can click on the file reference hyperlink to open the SDC browser and look at the constraints themselves. Since we are looking at a discrepancy between two existing constraints, the dual SDC browser will open and point to the constraints.

Alternatively, you can also use the [report_clock](#) command to gather information about this problem. To see the clock uncertainty applied on clock clk1 in the scenario set1, do the following:

```
ptc_shell> current_scenario set1
{"set1"}
```

```
ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_UNC_0001
Scenario: set1
Version: ...
Date   : ...
*****
```

Setup	Min Rise	Min Fall	Max Rise	Max Fall	Hold
Object	Related				
Uncertainty	Delay	Delay	Delay	Delay	Uncertainty
	Clock				
clk1	--	-	-	-	-
0.65	--				

You can contrast this to the clocks defined for scenario set2 as seen below:

```
ptc_shell> current_scenario set2
{"set2"}
```

```
ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_UNC_0001
Scenario: set2
Version: ...
Date   : ...
*****
```

Setup	Min Rise	Min Fall	Max Rise	Max Fall	Hold
	Related				

Object Uncertainty	Delay Clock	Delay	Delay	Delay	Uncertainty
clk1	-	-	-	-	0.65
-	--				

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this particular case, the `set_clock_uncertainty` command in the constraint set 1 can be changed to use the `-hold` option, or the `set_clock_uncertainty` command in the constraint set 2 can be changed to use the `-setup` option.

Rule Property: Tolerance

This rule supports the tolerance property. By default, constraint consistency expects the constraints to match exactly. If they don't, a violation is generated. Using the tolerance property, this behavior can be relaxed somewhat. The tolerance property specifies an acceptable percentage difference between the two constraints. If the difference is within this percentage, no violation is generated.

By default, the tolerance is set to a very small percentage (1e-5) which forces exact constraint comparison. The tolerance property can be modified with the [set_rule_property](#) command.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: S2S_UNC_0004

Clock uncertainty applied between **'from_clock'** and **'to_clock'** in constraint set **'scenario1'** is missing in constraint set **'scenario2'**.

Severity: Error

Description

An inter-clock uncertainty setting is applied between the reported objects in constraint set `senario1`. A similar constraint could not be found for the same objects in the constraint set `scenario2`.

Risk Associated With Violation

The inter-clock uncertainty will affect timing analysis of the design. Because the constraints are different for a particular object between the two scenarios, the timing

results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_UNC_0004 violation can occur if clock uncertainties are not applied in a similar fashion in both constraints sets. Below is an example set of constraints creating a S2S_UNC_0004 violation.

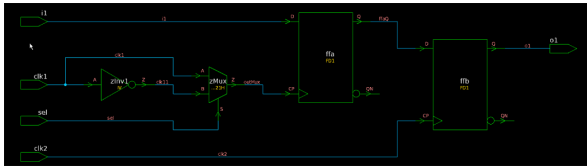


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -period 10 -name clk1 [get_ports clk1]
create_clock -period 10 -name clk2 [get_ports clk2]

set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk2

set_clock_uncertainty -setup 0.45 -from [get_clocks clk1] -to [get_clocks
clk2]
```

Constraint set 2:

```
create_scenario set2
create_clock -period 10 -name clk1 [get_ports clk1]
create_clock -period 10 -name clk2 [get_ports clk2]

set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk2
```

What Next

The Violation Details section of the GUI provides more information related to which object is affected, the location of the constraint responsible for the mismatch, and some information related to the constraint itself (setup/hold data). A table displays the various parameters. Below is an example:

You can click on the file reference hyperlink to open the SDC browser and look at the constraint itself. Alternatively, you can use the [report_clock](#) command to gather information about this problem. To see the clock uncertainty applied on clock clk1 in the scenario set1, do the following:

```
ptc_shell> current_scenario set1
{"set1"}

ptc_shell> report_clocks -skew
*****
Report : clock_skew
Design : S2S_UNC_0004
Scenario: set1
Version: ...
Date   : ...
*****
```

From Object	To Object	Hold Uncertainty	Setup Uncertainty
clk1	clk2	-	0.45

```
1

ptc_shell> current_scenario set2
{"set2"}

ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_UNC_0004
Scenario: set2
Version: ...
Date   : ...
*****

1
```

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this particular case, either the inter-clock uncertainty in scenario set1 should be removed, or an equivalent uncertainty should be added in scenario set2.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

Rule: S2S_UNC_0006

Clock uncertainty applied between **'from_clock1'** and **'to_clock1'** in constraint set **'scenario1'** does not match the clock uncertainty applied between **'from_clock2'** and **'to_clock2'** in the constraint set **'scenario2'**.

Severity: Error

Description

An inter-clock uncertainty constraint is defined in both constraint sets but with different values.

Risk Associated With Violation

The inter-clock uncertainty will affect timing analysis of the design. Because the constraints are different between the reported objects in the two scenarios, the timing results will also be different. The violation should be reviewed and the constraints should be modified to match.

Possible Scenarios

Consider the design pictured in Figure 1 below. The S2S_UNC_0006 violation can occur if inter-clock uncertainties are not applied in a similar fashion in both constraints sets. Below is an example set of constraints creating a S2S_UNC_0006 violation.

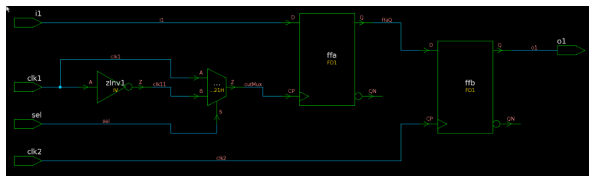


Figure 1

Constraint set 1:

```
create_scenario set1
create_clock -period 10 -name clk1 [get_ports clk1]
create_clock -period 10 -name clk2 [get_ports clk2]

set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk2

set_clock_uncertainty -setup 0.45 -from [get_clocks clk1] \
-to [get_clocks clk2]
```

Constraint set 2:

```
create_scenario set2
create_clock -period 10 -name clk1 [get_ports clk1]
create_clock -period 10 -name clk2 [get_ports clk2]

set_input_delay 2 [get_ports [list i1 sel]] -clock clk1
set_output_delay 2 [list o1] -clock clk2

set_clock_uncertainty -hold 0.25 -from [get_clocks clk1] \
-to [get_clocks clk2]
```

What Next

The Violation Details section of the GUI provides more information related to which objects are affected, the location of the constraint responsible for the mismatch, and some information related to the constraints themselves (setup/hold data). Two tables display the various parameters; one for scenario1 and one for scenario2. Below is an example.

You can click on the file reference hyperlink to open the SDC browser and look at the constraints. Because you are looking at a discrepancy between two existing constraints, the dual SDC browser will open and point to the constraints.

Alternatively, you can also use the [report_clock](#) command to gather information about this problem. To see the clock uncertainty applied on clock clk1 in the scenario set1, do the following:

```
ptc_shell> current_scenario set1
{"set1"}
```

```
ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_UNC_0006
Scenario: set1
Version: ...
Date   : ...
*****
```

	From Object	To Object	Hold Uncertainty	Setup Uncertainty
-----	clk1	clk2	-	0.45

1

```
ptc_shell> current_scenario set2
{"set2"}
```

```
ptc_shell> report_clock -skew
*****
Report : clock_skew
Design : S2S_UNC_0006
Scenario: set2
Version: ...
Date   : ...
*****
```

	From Object	To Object	Hold Uncertainty	Setup Uncertainty
-----	clk1	clk2	0.25	-

1

Fix Suggestion

Constraints should be reviewed for both scenarios and the cause for the discrepancy should be addressed. In this particular example, both hold and setup values should probably be set in both constraints sets. In addition, the respective values for hold and setup should be consistent between the two sets of constraints.

Rule Property: Tolerance

This rule supports the tolerance property. By default, constraint consistency expects the constraints to match exactly. If they don't, a violation is generated. Using the tolerance property, this behavior can be relaxed somewhat. The tolerance property specifies an acceptable percentage difference between the two constraints. If the difference is within this percentage, no violation is generated.

By default, the tolerance is set to a very small percentage (1e-5) which forces exact constraint comparison. The tolerance property can be modified with the [set_rule_property](#) command.

See Also

[report_clock](#)

[set_clock_uncertainty](#)

[set_rule_property](#)